

Edition 6.0

Squeak By Example

Christoph Thiede Patrick Rein

based on previous editions by
Andrew P. Black, Stéphane Ducasse,
Oscar Nierstrasz, Damien Pollet,
Damien Cassou, Marcus Denker

Squeak by Example

Version for Squeak 6.0 of July 20, 2023

Christoph Thiede, Patrick Rein

based on previous versions by
Andrew P. Black, Stéphane Ducasse, Oscar Nierstrasz,
Damien Pollet, Damien Cassou, and Marcus Denker

Copyright of original editions © 2007, 2008, 2009 by Andrew P. Black, Stéphane Ducasse, Oscar Nierstrasz, Damien Pollet, Damien Cassou, and Marcus Denker.

Copyright of the 5.3 and 6.0 editions as derivatives of original editions through corrections and extensions © 2023 by Patrick Rein and Christoph Thiede.

The contents of this book are protected under the Creative Commons Attribution-ShareAlike 3.0 Unported license.

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution. You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike. If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar, or a compatible license.

- For any reuse or distribution, you must make clear to others the license terms of this work. The best way to do this is with a link to this web page: creativecommons.org/licenses/by-sa/3.0/
- Any of the above conditions can be waived if you get permission from the copyright holder.
- Nothing in this license impairs or restricts the author's moral rights.



Your fair dealing and other rights are in no way affected by the above. This is a human-readable summary of the Legal Code (the full license): creativecommons.org/licenses/by-sa/3.0/legalcode

First Edition, September 2007.
Revised editions with corrections, March 2008, May 2008, September 2009.
Published by Square Bracket Associates, Switzerland.
ISBN 978-3-9523341-0-2 (First Edition)

6.0 Edition, June 2023.
Published by Software Architecture Group, Hasso Plattner Institute, Germany. hpi.de/swa
ISBN 978-1-4476-2948-1 (6.0 Edition)

Color of cover is Color moss.

Figures generated on July 20, 2023.

Contributors

This book is only kept up-to-date thanks to the help of many members of the Smalltalk community, who have contributed small and large additions over the years[1]. Thanks goes to all of them (in alphabetical order):

Nick Ager, Tarik Alnawa, Tom Beckmann, Joana Bergsiek, Luca Bettosini, Andrew P. Black, Tom Braun, Ken G. Brown, Damien Cassou, Jordi Delgado, Stéphane Ducasse, Doug Edmunds, Hilaire Fernandes, Tim Garrels, Orla Greevy, Henrik Guhl, Jonathan Hankinds, Pascal Hecker, Robert Hirschfeld, Theresa Hradilak, Lea Hänsenberger, Uwe Hübner, Chris Kassopulo, Has van der Krieken, Leon Matthes, Oscar Nierstrasz, Frederica Nierstrasz, Damien Pollet, Patrick Rein, Lukas Renggli, Mike Roberts, Victor Rodriguez, Tim Rowledge, Laurent Sebag, Serge Stinckwich, Marcel Taeumel, Christoph Thiede, Silvan Verhoeven, Mark Volkmann, and Kira Weinlein.

Acknowledgments (2009 edition)

We would like to thank Hilaire Fernandes and Serge Stinckwich who allowed us to translate parts of their columns on Smalltalk, and Damien Cassou for contributing the chapter on streams. We also thank Tim Rowledge for the use of the Squeak logo, and Frederica Nierstrasz for the original cover art.

We especially thank Lukas Renggli and Orla Greevy for their comments on drafts of the first release.

We thank the University of Bern, Switzerland, for graciously supporting this open-source project and for hosting the web site of this book.

[1]The list was assembled more than ten years after the first edition was published. During this time, we probably have lost sight of contributors. If you feel you should be on this list, let us know at https://github.com/hpi-swa-lab/SqueakByExample-english.

Contents

Preface **xi**

I Getting started

1 A quick tour of Squeak **3**

1.1 Getting started 3
1.2 The world menu 8
1.3 Saving, quitting, and restarting a Squeak session 8
1.4 Workspaces and transcripts 11
1.5 Keyboard shortcuts 12
1.6 The system browser 14
1.7 Finding classes 16
1.8 Finding methods 17
1.9 Defining a new method 19
1.10 Chapter summary 24

2 Your first interactive object: the Quinto game **25**

2.1 The Quinto game 25
2.2 Creating a new class category 26
2.3 Defining the class SBECell 26
2.4 Adding methods to a class. 29
2.5 Inspecting an object 31
2.6 Defining the class SBEGame 32

2.7 Organizing methods into protocols 35
2.8 Let's try our code 39
2.9 Saving and sharing Smalltalk code. 41
2.10 Chapter summary. 45

3 Syntax in a nutshell 47
3.1 Syntactic elements 47
3.2 Pseudo-variables 51
3.3 Message sends 52
3.4 Method syntax 53
3.5 Block syntax. 54
3.6 Conditionals and loops in a nutshell 55
3.7 Primitives and pragmas 57
3.8 Chapter summary. 58

4 Understanding message syntax 61
4.1 Identifying messages 61
4.2 Three kinds of messages 63
4.3 Message composition 65
4.4 Hints for identifying keyword messages 71
4.5 Expression sequences 73
4.6 Cascaded messages 73
4.7 Chapter summary. 74

II Developing in Squeak

5 The Smalltalk object model 77
5.1 The rules of the model 77
5.2 Everything is an object 78
5.3 Every object is an instance of a class 78
5.4 Every class has a superclass 86
5.5 Everything happens by sending messages 89
5.6 Method lookup follows the inheritance chain 91

5.7 Shared variables 97
5.8 Chapter summary 102

6 The Squeak programming environment 105
6.1 Overview 105
6.2 The system browser 106
6.3 The inspector and the explorer 120
6.4 The debugger 124
6.5 The process browser 134
6.6 Finding methods 135
6.7 Other interesting tools 136
6.8 Chapter summary 138

7 Storing and sharing source code 141
7.1 Packages and Monticello 142
7.2 Change sets and the change sorter 151
7.3 The file list browser 154
7.4 In Smalltalk, you can't lose code 156
7.5 Git tools 157
7.6 Chapter summary 157

8 SUnit 159
8.1 Introduction 159
8.2 Why testing is important 160
8.3 What makes a good test? 161
8.4 SUnit by example 162
8.5 The SUnit cook book 166
8.6 The SUnit framework 168
8.7 Advanced features of SUnit 170
8.8 The implementation of SUnit 171
8.9 Some advice on testing 174
8.10 Chapter summary 176

9 Basic Classes 177
9.1 Object . . . 177
9.2 Numbers . . . 187
9.3 Characters . . . 191
9.4 Strings . . . 192
9.5 Booleans . . . 194
9.6 Exceptions . . . 196
9.7 Chapter summary . . . 199

10 Collections 201
10.1 Introduction . . . 201
10.2 Implementations of collections . . . 205
10.3 Examples of key classes . . . 206
10.4 Collection iterators . . . 217
10.5 Some hints for using collections . . . 222
10.6 Sorting collections . . . 223
10.7 Chapter summary . . . 225

11 Streams 227
11.1 Two sequences of elements . . . 227
11.2 Streams vs. collections . . . 228
11.3 Streaming over collections . . . 229
11.4 Using streams for file access . . . 237
11.5 Chapter summary . . . 240

12 Morphic 243
12.1 The history of Morphic . . . 243
12.2 Manipulating morphs . . . 245
12.3 Composing morphs . . . 246
12.4 Creating and drawing your own morphs . . . 247
12.5 Interaction and animation . . . 251
12.6 Dialog windows . . . 256
12.7 Drag-and-drop . . . 257

12.8 A complete example 259
12.9 More about the canvas 263
12.10 Chapter summary 265

III Advanced Squeak

13 Classes and metaclasses 269
13.1 Rules for classes and metaclasses 269
13.2 Revisiting the Smalltalk object model 270
13.3 Every class is an instance of a metaclass 272
13.4 The metaclass hierarchy parallels the class hierarchy . . . 273
13.5 Every metaclass inherits from Class and Behavior 276
13.6 Every metaclass is an instance of Metaclass 278
13.7 The metaclass of Metaclass is an instance of Metaclass . . . 279
13.8 Chapter summary 280

IV Appendices

A Frequently asked questions 285
A.1 Getting started 285
A.2 Collections 286
A.3 Browsing the system 287
A.4 Morphic 289
A.5 System 290
A.6 Using Monticello and SqueakSource 290
A.7 Tools 292
A.8 Additional packages 293

Bibliography 295

Preface

What is Squeak?

Squeak is a modern, open-source, fully-featured implementation of the Smalltalk programming language and environment.

Squeak is highly portable — even its virtual machine is written almost entirely in Smalltalk, making it easy to debug, analyze, and change. Squeak is the vehicle for a wide range of innovative projects, from multimedia applications and educational platforms to commercial web development environments.

Who should read this book?

This book presents the various aspects of Squeak, starting with the basics and proceeding to more advanced topics.

This book will not teach you how to program. The reader should have some familiarity with programming languages. Some background in object-oriented programming is also helpful.

This book will introduce the Squeak programming environment, the language, and the associated tools. You will be exposed to common idioms and practices, but the focus is on the technology, not on object-oriented design. Wherever possible, we will show you lots of examples. (We have been inspired by Alec Sharp's excellent book on Smalltalk[2].)

[2] Alec Sharp, *Smalltalk by Example*. McGraw-Hill, 1997.

A word of advice

Do not be frustrated by parts of Smalltalk that you do not immediately understand. You do not have to know everything! Alan Knight expresses this principle as follows[3]:

> **Try not to care.** Beginning Smalltalk programmers often have trouble because they think they need to understand all the details of how a thing works before they can use it. This means it takes quite a while before they can master `Transcript show: 'Hello World'`. One of the great leaps in OO is to be able to answer the question "How does this work?" with "I don't care".

An open book

This book is an open book in the following senses:

- The content of this book is released under the Creative Commons Attribution-ShareAlike (by-sa) license. In short, you are allowed to freely share and adapt this book, as long as you respect the conditions of the license available at the following URL: creativecommons.org/licenses/by-sa/3.0/.
- This book just describes the core of Squeak. Ideally, we would like to encourage others to contribute chapters on the parts of Squeak that we have not described. If you would like to participate in this effort, take a look at the repository at https://github.com/hpi-swa-lab/SqueakByExample-english. We would like to see this book grow!

The Squeak community

The Squeak community is friendly and active. Here is a short list of resources that you may find useful:

- www.squeak.org is the main web site of Squeak.

[3] https://web.archive.org/web/20191012144419/http://alanknightsblog.blogspot.com/2011/10/principles-of-oo-design-or-everything-i.html

- www.squeaksource.com is the equivalent of GitHub for Squeak projects.
- wiki.squeak.org/squeak is a wiki with all kinds of information about Squeak.
- The *help browser* inside Squeak gives you access to a lot of useful information. For example, it contains an overview of keyboard shortcuts and documents several packages and tools. You can find it in the World docking bar ▷ Help menu at the top of the screen.

About mailing-lists. There are many mailing lists, and sometimes they can be just a little bit too active. If you do not want to get flooded by mail but would still like to participate, we suggest you use https://lists.squeakfoundation.org/archives/ to browse the lists.

- Note that *squeak-dev* refers to the developers' mailing list, which can be browsed here:
  https://lists.squeakfoundation.org/archives/list/squeak-dev@lists.squeakfoundation.org/
- *Beginners* refers to a friendly mailing list for beginners where any question can be asked:
  https://lists.squeakfoundation.org/archives/list/beginners@lists.squeakfoundation.org/
  (There is so much to learn that we are all beginners in some aspect of Squeak!)

Group chats. Have a question that you need to be answered quickly? Would you like to meet with other squeakers around the world? The Slack instance at squeak.slack.com is a great place to participate in longer-term discussions. Stop by and say "Hi!"

Other sites. Several other websites support the Squeak community today in various ways. Here are some of them:

- github.com/squeak-smalltalk is the GitHub organization hosting new releases and the various Squeak websites.
- github.com/OpenSmalltalk/opensmalltalk-vm is the repository of the virtual machine running Squeak, the OpenSmalltalk-VM.
- bugs.squeak.org is the bug tracker where you can report any issues you encounter with the system. You can access debug information through World docking bar ▷ Help ▷ Send Feedback.

- source.squeak.org hosts the repositories for the base system. If you would like to contribute and want to learn about the community development process, you can learn about it directly in Squeak under World docking bar ▷ Help ▷ Contributing to Squeak. You can also learn about the community development process at https://squeak.org/development_process/.

Examples and exercises

We make use of two special conventions in this book.

We have tried to provide as many examples as possible. In particular, there are many examples that show a fragment of code that can be evaluated. We use the symbol ⟶ to indicate the result that you obtain when you select an expression and print it:

```
3 + 4  ⟶  7  "if you select 3+4 and 'print it', you will see 7"
```

The other convention that we use is to display the icon  to indicate when there is something for you to do:

 *Go ahead and read the next chapter!*

Part I

Getting started

Chapter 1

A quick tour of Squeak

In this chapter, we will give you a high-level tour of Squeak to help you get comfortable with the environment. There will be plenty of opportunities to try things out, so it would be a good idea if you have a computer handy when you read this chapter.

We will use this icon:  to mark places in the text where you should try something out in Squeak. In particular, you will fire up Squeak, learn about the different ways of interacting with the system, and discover some of the basic tools. You will also learn how to define a new method, create an object, and send it messages.

1.1 Getting started

Squeak is available as a free download from www.squeak.org. Your download will be an archive file that contents vary from platform to platform. However, there are two important components common to all versions (see Figure 1.1).

1. The *virtual machine* (VM) is the only part of the system that is different for each operating system and processor. Pre-compiled virtual machines are available for all the major computing environments. In Figure 1.1 we see the VM for the Windows is called *Squeak.exe*.

2. The current *system image* is a snapshot of a running Squeak system, frozen in time. It consists of two files: an *.image* file, which contains the state of all of the objects in the system (including classes and methods, since they are objects too), and a *.changes* file, which contains a log of



Figure 1.1: The Squeak download files (in this example, for 64-bit Windows).

all of the changes to the source code of the system. In Figure 1.1, we see that we have grabbed the *Squeak6.0* VM, image, and changes files.

Download and install Squeak on your computer. We recommend that you use the standard image provided on the Squeak web page.

Most of the introductory material in this book will work with any version, so if you already have one installed, you may as well continue to use it. However, if you notice differences between the appearance or behavior of your system and what is described here, do not be surprised.

As you work in Squeak, the *image* and *changes* files are modified, so you need to make sure that they are writable. Always keep these two files together. Never edit them directly with a text editor, as Squeak uses them to store the objects you work with and to log the changes you make to the source code. It is a good idea to keep a backup copy of the downloaded image and changes files so you can always start from a fresh image and reload your code.

The *sources* file and the *VM* can be read-only — they can be shared between different users. All of these files can be placed in the same directory, but it is also possible to put the virtual machine and sources file in a separate directory where everyone has access to them. Do whatever works best for your style of working and your operating system.

Launching. To start Squeak, do whatever your operating system expects: drag the *.image* file onto the icon of the virtual machine, or double-click the *.image* file, or double-click the virtual machine icon, or at the command line type the name of the virtual machine followed by the path to the *.image* file. (When you have multiple VMs installed on your machine, the operating system may not automatically pick the right one; in this case, it is safer to



Figure 1.2: A fresh Squeak image.

drag and drop the image onto the virtual machine, or to use the command line.)

Once Squeak is running, you should see a single large window, possibly containing some open workspace windows (see Figure 1.2). You will also notice that there is a small menu bar at the top. Beyond this one global menu, Squeak makes heavy use of context-dependent pop-up menus.

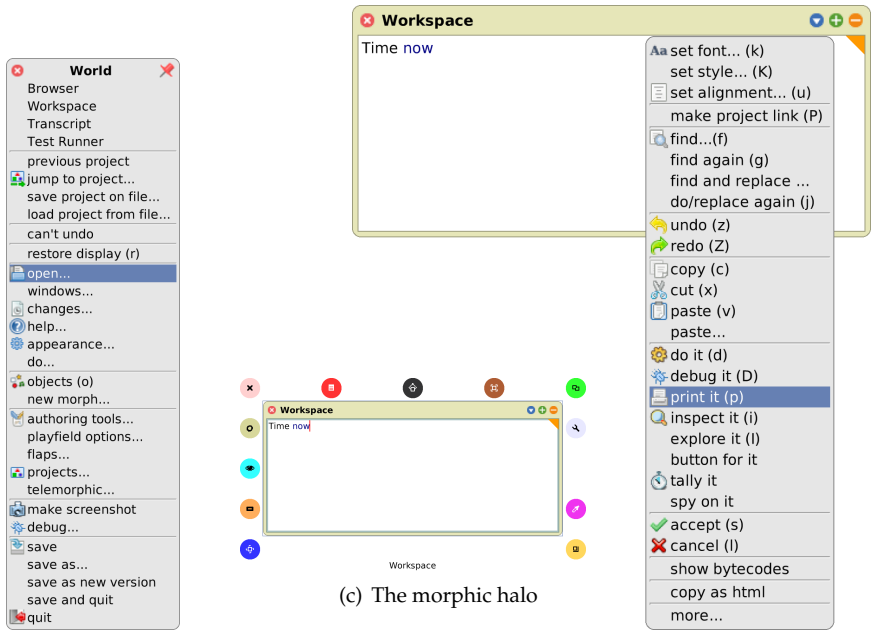
Start Squeak. You can dismiss any open workspaces by clicking on the X *in the top left corner of the workspace window. You can collapse the windows (so that they can be expanded again later) by clicking on the — in the top-right corner.*

First interaction. A good place to get started is the world menu shown in Figure 1.3 (a).

Click with the mouse on the background of the main window to show the world menu, then choose open ... ▷ workspace *to create a new workspace.*

Squeak was originally designed for a computer with a three button mouse. If your mouse has fewer than three buttons, you will have to press extra keys while clicking the mouse to simulate the extra buttons.

Squeak avoids terms like "left mouse click" because different computers, mice, keyboards, and personal configurations mean that different users will need to press different physical buttons to achieve the same effect. Instead, the mouse buttons are labeled with colors. The mouse button that you pressed to get the "World" menu is called the *red button*; it is most often



(a) The world menu

(b) The contextual menu

(c) The morphic halo

Figure 1.3: The world menu (brought up by the red mouse button), a contextual menu (yellow mouse button), and a morphic halo (blue mouse button).

used for selecting items in lists, selecting text, and selecting menu items. When you start using Squeak, it can be surprisingly helpful to actually label your mouse, as shown in Figure 1.4.[1]

The *yellow button* is the next most used button; it is used to bring up a contextual menu, that is, a menu that offers different sets of actions depending on where the mouse is pointing; see Figure 1.3 (b).

Type Time now *in the workspace. Now click the yellow button in the workspace. Select* print it.

Finally, there is the *blue button* to activate the "morphic halo", an array of handles that are used to perform operations on the on-screen objects themselves, such as rotating or resizing; see Figure 1.3 (c). If you let the mouse linger over a handle, a help balloon will explain its function.

[1] We will avoid the term "red-click" and use "click" instead since this is the default.

Click the blue button on the workspace. Grab the handle near the bottom left corner and drag it to rotate the workspace.



Figure 1.4: The author's mouse. Clicking the scroll wheel activates the blue button.

We recommend that right-handed people configure their mouse to put the red button on the left side of their mouse, the yellow button on the right, and use a clickable scroll wheel, if one is available, for the blue button. If you don't have a clickable scroll wheel, then you can get the Morphic halo by holding down the alt or option key while clicking the red button. If you use a Mac without a second mouse button, you can simulate one by holding down the ⌘ key while clicking the mouse.

You can configure your mouse to work the way you want by using the preferences of your operating system and mouse driver. Squeak has some preferences for customizing the mouse and the meta keys on your keyboard. You can find the preference browser in the open item of the World menu. In the preference browser, the general category contains an option Swap mouse buttons that switches the yellow and blue functions (see Figure 1.5). The keyboard category has options to duplicate the various command keys.

Open the preference browser and find the Swap mouse buttons *option using the search box.*

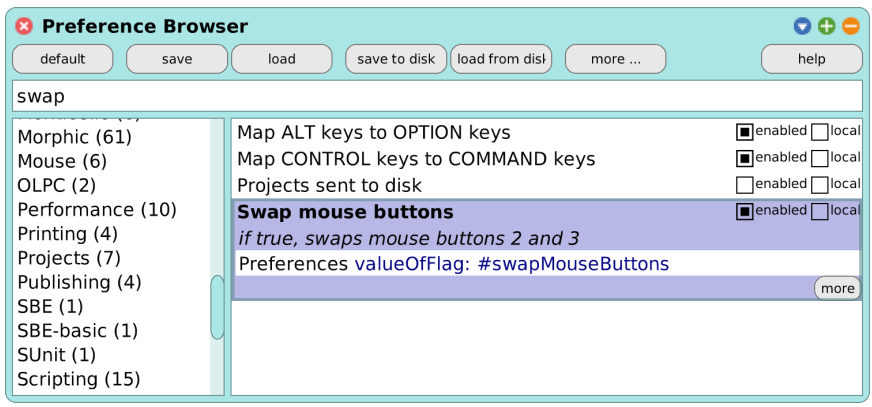


Figure 1.5: The Preference Browser.

Depending on the resolution of your display and the scaling mechanism

of your operating system, you might want to change the initial scaling Squeak chose for you. You can either change the scale factor directly in the preference wizard that opens up when you open Squeak for the first time. Later, you can change the scaling in the menu bar at the top under Extras ▷ Scale Factor. While you are there, you can also change the looks of the environment via Extras ▷ Themes & Colors.

1.2 The world menu

 *Click again on the Squeak background.*

You will see the World menu again. Most Squeak menus are not modal; you can leave them on the screen for as long as you wish by clicking the thumbtack icon in the top-right corner. Do this. Also, notice that menus appear when you click the mouse, but do not disappear when you release it; they stay visible until you make a selection, or until you click outside of the menu. You can even move the menu around by grabbing its title bar.

The world menu provides you a simple means to access many of the tools that Squeak offers.

 *Have a closer look at the* world ▷ open ... *menu.*

You will see a list of several of the core tools in Squeak, including the system browser (one of many available class browsers) and the workspace. We will encounter most of them in the coming chapters.

1.3 Saving, quitting, and restarting a Squeak session

 *Bring up the world menu. Now select* new morph ... *and navigate to* from alphabetical list ▷ A-C ▷ BouncingAtomsMorph. *You now have a morph full of bouncing atoms "in hand". Put the atoms morph down (by clicking) somewhere.*

 *Select* World ▷ save as ..., *and enter the name "SBE". Now click on the* Accept(s) *button. Now select* World ▷ save and quit.

Now if you go to the location where the original image and changes files were, you will find two new files called "SBE.image" and "SBE.changes" that represent the working state of the Squeak image at the moment before you told Squeak to save and quit. If you wish, you can move these two files

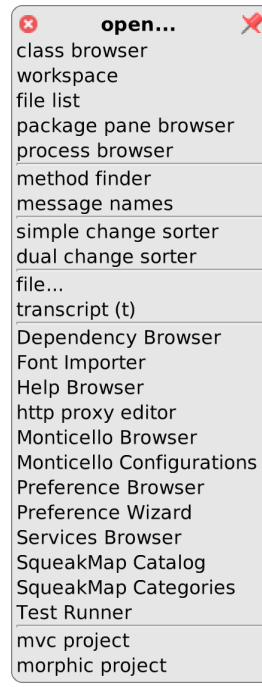


Figure 1.6: The open ... dialogue of the world menu.

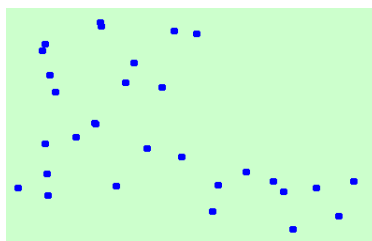


Figure 1.7: An instance of BouncingAtomsMorph.

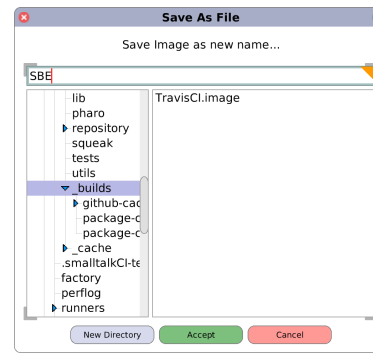


Figure 1.8: The save as ... dialogue.

anywhere that you like on your disk, but if you do so you may (depending on your operating system) need to also move, copy or link to the virtual machine and the *.source* file.

Start up Squeak from the newly created "SBE.image" file.

Now you should find yourself in precisely the state you were when you quit Squeak. The bouncing atoms morph is there again and the atoms continue to move from where they were when you left it.

When you start Squeak for the first time, the Squeak virtual machine loads the image file that you provide. This file contains a snapshot of a large number of objects, including a vast amount of pre-existing code and a large number of programming tools (all of which are objects). As you work with Squeak, you will send messages to these objects, you will create new objects, and some of these objects will die and their memory will be reclaimed (*i.e.*, garbage-collected).

When you quit Squeak, you will normally save a snapshot that contains all of your objects. If you save normally, you will overwrite your old image file with the new snapshot. Alternatively, you may save the image under a new name, as we just did.

In addition to the *.image* file, there is also a *.changes* file. This file contains a log of all the changes to the source code that you have made using the standard tools. Most of the time you do not need to worry about this file at all. As we shall see, however, the *.changes* file can be very useful for recovering from errors or replaying lost changes. More about this later!

The image that you have been working with is a descendant of the original Smalltalk-80 image created in the late 1970s. Some of these objects have been around for decades!

The image can be used as a mechanism for storing and managing software projects you are working on. However, as we shall see later on, there are also tools for managing code and sharing software developed by teams. For now, we will continue by cleaning up the image before we take a look at the tools Squeak provides.

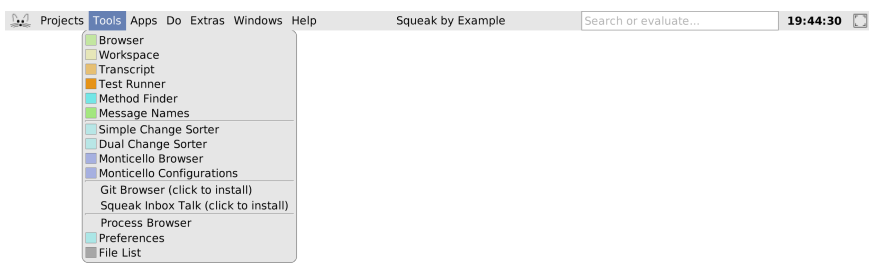


Figure 1.9: The Squeak Tools menu.

1.4 Workspaces and transcripts

 *Close all open windows. In the menu bar at the top select* Tools.

You will see entries for some of the key tools in Squeak (Figure 1.9). Open a transcript and a workspace.

 *Position and resize the transcript and workspace windows so that the workspace just overlaps the transcript.*

You can resize windows either by dragging one of the corners, or by blue-clicking the window to bring up the morphic handles, and dragging the yellow, bottom right handle.

At any time only one window is active; it is in front and has its label highlighted (e.g. the workspace window is active in Figure 1.10).

The transcript is an object that is often used for logging system messages. It is a kind of "system console". Note that the transcript is terribly slow, so if you keep it open and write to it certain operations can become 10 times slower. (You can speed up it by disabling the preference "Force transcript updates to screen" in the Preference Browser.) In addition, the transcript is not thread-safe so you may experience strange problems if multiple objects write concurrently to the transcript.

Workspaces are useful for typing snippets of Smalltalk code that you would like to experiment with. You can also use workspaces simply for typing arbitrary text that you would like to remember, such as to-do lists or instructions for anyone who will use your image. Workspaces are often used to hold documentation about a captured image, as is the case with the standard image that we downloaded earlier (see Figure 1.2).

 *Type the following text into the workspace:*

```
Transcript showln: 'hello world'.
```

Try double-clicking in the workspace at various points in the text you have just typed. Notice how an entire word, entire string, or the whole text is selected, depending on where you click.

 *Select the text you have typed and yellow-click. Select* do it (d).

Notice how the text "hello world" appears in the transcript window (Figure 1.10). Do it again. (The (d) in the menu item do it (d) tells you that the keyboard shortcut to *do it* is CMD–d. More on this in the next section!)

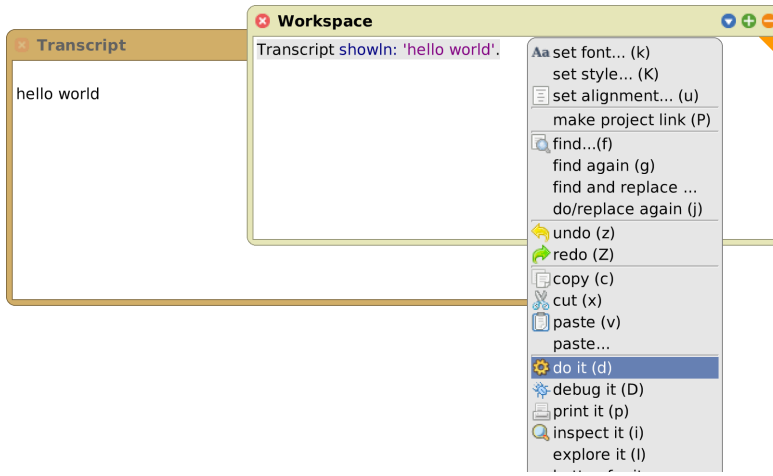


Figure 1.10: "Doing" an expression.

You have just evaluated your first Smalltalk expression! You just sent the message show: 'hello world' to the Transcript object, followed by the message cr (carriage return). The Transcript then decided what to do with this message, that is, it looked up its *methods* for handling show: and cr messages and reacted appropriately.

If you talk to Smalltalkers for a while, you will quickly notice that they generally do not use expressions like "call an operation" or "invoke a method", but instead they will say "send a message". This reflects the idea that objects are responsible for their own actions. You never *tell* an object what to do — instead, you politely *ask* it to do something by sending it a message. The object, not you, selects the appropriate method for responding to your message (more about this in Chapter 4).

1.5 Keyboard shortcuts

If you want to evaluate an expression, you do not always have to bring up the yellow-button menu. Instead, you can use keyboard shortcuts. These are the parenthesized expressions in the menu. Depending on your platform, you may have to press one of the modifier keys (control, alt, command, or meta). (We will indicate these generically as CMD–*key*.)

Evaluate the expression in the workspace again, but using the keyboard shortcut: CMD–*d*.

In addition to do it, you will have noticed print it, inspect it and explore it. Let's have a quick look at each of these.

Type the expression 3 + 4 *into the workspace. Now* do it *with the keyboard shortcut.*

Do not be surprised if you saw nothing happen! What you just did is send the message + with argument 4 to the number 3. Normally the result 7 will have been computed and returned to you, but since the workspace did not know what to do with this answer, it simply threw the answer away. If you want to see the result, you should print it instead. print it actually compiles the expression, executes it, sends the message printString to the result, and displays the resulting string.

Select 3 + 4 *and* print it *(*CMD*–p).*

This time we see the result we expect (Figure 1.11).

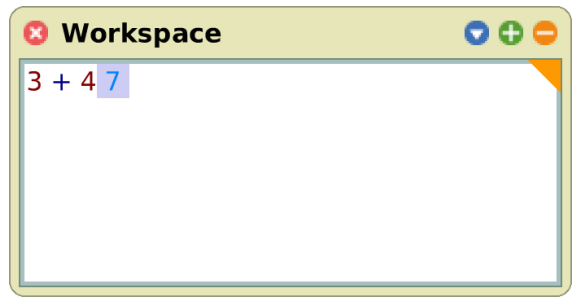


Figure 1.11: "Print it" rather than "do it".

```
3 + 4  ⟶  7
```

We use the notation ⟶ as a convention in this book to indicate that a particular Squeak expression yields a given result when you print it.

*Delete the highlighted text "*7*" (Squeak should have selected it for you, so you can just press the delete key). Select* 3 + 4 *again and this time* inspect it *(*CMD*–i).*

Now you should see a new window, called an *inspector*, with the heading SmallInteger: 7 (Figure 1.12). The inspector is an extremely useful tool that will allow you to browse and interact with any object in the system. The title tells us that 7 is an instance of the class SmallInteger. The left panel allows us to browse the instance variables of an object, the values of which are shown in the right panel. The bottom panel can be used to write expressions to send messages to the object.

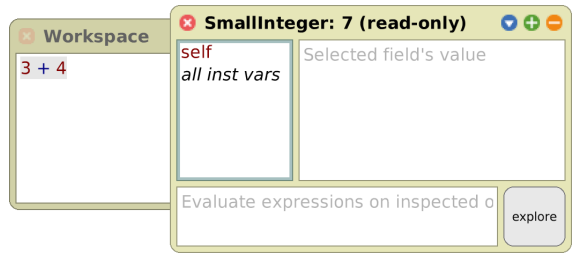


Figure 1.12: Inspecting an object.

Type **self** squared *in the bottom panel of the inspector on* 7 *and* print it.

Close the inspector. Type the expression Object *in a workspace and this time* explore it *(*CMD−*I, uppercased i).*

This time you should see a window labeled Object containing the text ▷ root: Object. Click on the triangle to open it up (Figure 1.13).

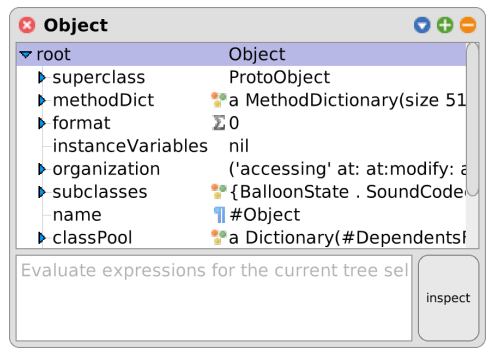


Figure 1.13: Exploring an object.

The explorer is similar to the inspector, but it offers a tree view of a complex object. In this case, the object we are looking at is the Object class. We can see directly all the information stored in this class, and we can easily navigate to all its parts, e.g. its superclass which is stored in superclass.

1.6 The system browser

The system browser is one of the key tools used for programming. As we shall see, there are several interesting browsers available for Squeak, but this is the basic one you will find in any image.

Open a browser by selecting World ▷ Browser*, or by selecting* Tools ▷ Browser *in the world menu bar.*

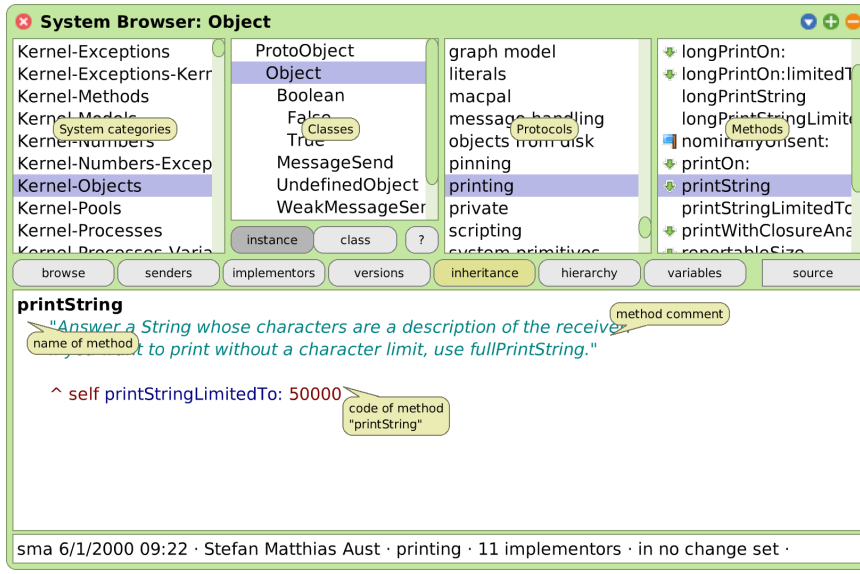


Figure 1.14: The system browser showing the printString method of class Object.

We can see a system browser in Figure 1.14. The title bar indicates that we are browsing the class Object.

When the browser first opens, all panes are empty except for the leftmost one. This first pane lists all known *system categories*, which are groups of related classes.

Click on the category Kernel-Objects.

This causes the second pane to show a list of all of the classes in the selected category.

Select the class Object.

Now the remaining two panes will be filled with text. The third pane displays the *protocols* of the currently selected class (referred to as *method categories* by the browser). These are convenient groupings of related methods. If no protocol is selected you should see all methods in the fourth pane.

Select the printing *protocol.*

You may have to scroll down to find it. Now you will see in the fourth pane only methods related to printing.

Select the printString *method.*

Now we see in the bottom pane the source code of the printString method, shared by all objects in the system (except those that override it).

1.7 Finding classes

There are several ways to find a class in Squeak. The first, as we have just seen above, is to know (or guess) what category it is in, and to navigate to it using the browser.

A second way is to send the browse message to the class, asking it to open a browser on itself. Suppose we want to browse the class Boolean.

Type Boolean browse *into a workspace and* do it.

A browser will open on the Boolean class (Figure 1.15). There is also a keyboard shortcut CMD–b (browse) that you can use in any tool where you find a class name; , select the name and type CMD–b.

Use the keyboard shortcut to browse the class Boolean.

Notice that when the Boolean class is selected but no protocol or method is selected, the bottom pane automatically shows the *class definition* (Figure 1.15). This is nothing more than an ordinary Smalltalk message that is sent to the parent class, asking it to create a subclass. Here we see that the class Object is being asked to create a subclass named Boolean with no instance variables, class variables or "pool dictionaries", and to put the class Boolean in the *Kernel-Objects* category.

If you want to see further information on the class, you can click on the ? at the bottom of the class pane. This opens the comment in a dedicated pane.

If you would like to explore Squeak's inheritance hierarchy, the *hierarchy browser* can help you. This can be useful if you are looking for an unknown subclass or superclass of a known class. The hierarchy browser is like the system browser, except that the list of classes is arranged as an indented tree mirroring the inheritance hierarchy.

Click on hierarchy *in the browser while the class* Boolean *is selected.*

This will open a hierarchy browser showing the superclasses and subclasses of Boolean.

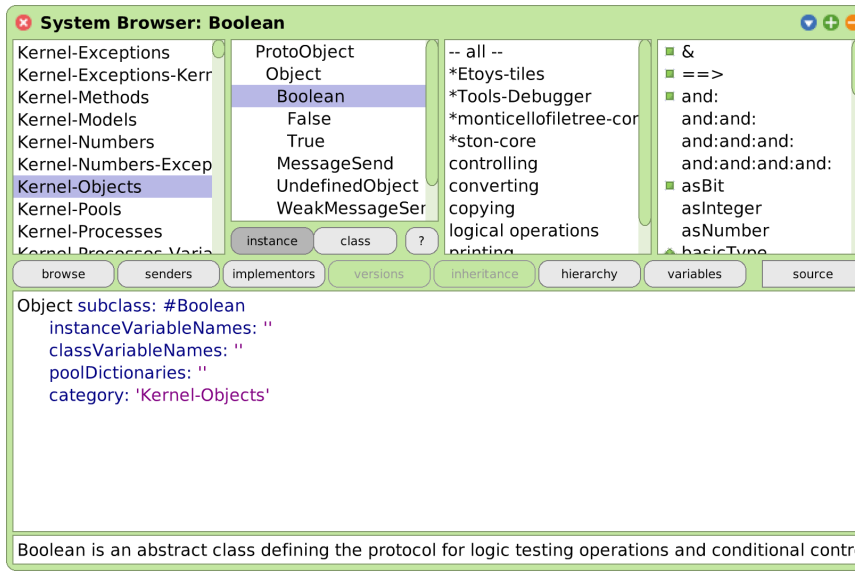


Figure 1.15: The system browser showing the definition of class Boolean.

Navigate to the immediate superclass and subclasses of Boolean.

Often, the fastest way to find a class is to search for it by name. For example, suppose that you are looking for some unknown class that represents dates and times.

Put the mouse in the system category pane of the system browser and type CMD–*f, or select* find class … (f) *from the yellow-button menu. Type "time" in the dialog box.*

You will be presented with a list of classes whose names contain "time" (see Figure 1.16). Choose one, say, Time, and the browser will show it.

1.8 Finding methods

Sometimes you can guess the name of a method, or at least part of the name of a method, more easily than the name of a class. For example, if you are interested in the current time, you might expect that there would be a method called "now", or containing "now" as a substring. But where might it be? The *method finder* can help you.

Select Tools ▷ Method Finder *in the menu bar at the top or open the world*

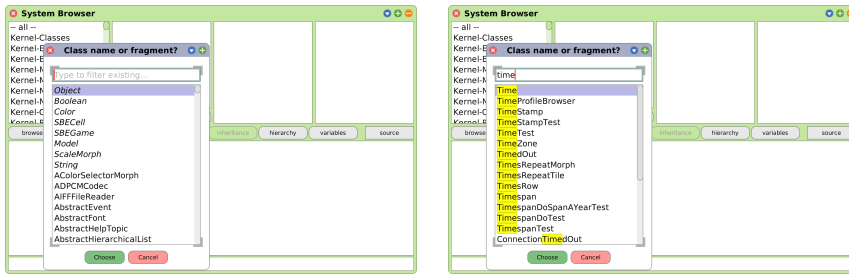


Figure 1.16: Searching for a class by name.

menu and select open... ▷ method finder. *Type "now" in the top left pane, and* accept *it (or just press the* RETURN *key).*

The method finder will display a list of all the method names that contain the substring "now". To scroll to now itself, move the cursor to the list and type "n"; this trick works in all scrolling windows. Select "now" and the right-hand pane shows you the three classes that define a method with this name, as shown in Figure 1.17. Selecting any one of them will open a browser on it.

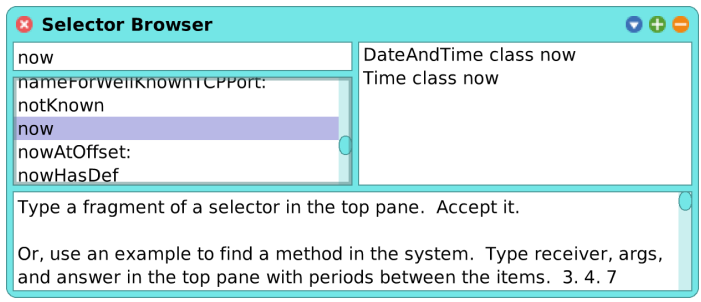


Figure 1.17: The method finder showing three classes that define a method named now.

At other times you may have a good idea that a method exists, but will have no idea what it might be called. The method finder can still help! For example, suppose that you would like to find a method that turns a string into upper case, for example, it would translate 'eureka' into 'EUREKA'.

 *Type* 'eureka' . 'EUREKA' *into the method finder and press the* RETURN *key, as shown in Figure 1.18.*

The method finder will suggest a method that does what you want.

An asterisk at the beginning of a line in the right pane of the method finder indicates that this method is the one that was actually used to obtain the requested result. So, the asterisk in front of String asUppercase lets us know that the method asUppercase defined on the class String was executed and returned the result we wanted. The methods that do not have an asterisk are just the other methods that have the same name as the ones that returned the expected result. So Character»asUppercase was not executed on our example, because 'eureka' is not a Character object.

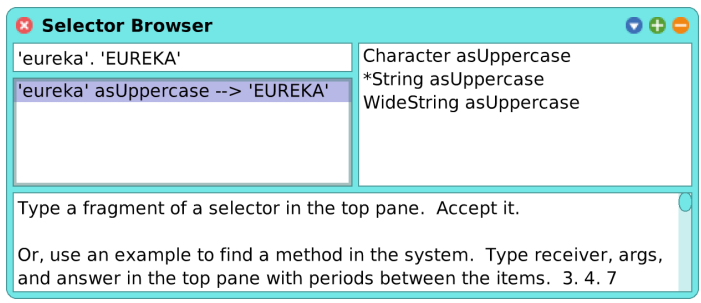


Figure 1.18: Finding a method by example.

You can also use the method finder for methods with arguments; for example, if you are looking for a method that will find the greatest common factor of two integers, you might try 25. 35. 5 as an example. You can also give the method finder multiple examples to narrow the search space; the help text in the bottom pane explains more.

There is yet another way to find both, methods, and classes. In the top right corner of the menu bar at the top you can find a pane saying "Search or evaluate…". This pane provides you a global search; you can activate it from everywhere via CMD–0. When you try previous searches for the class Time or the method now you will see that the search directly opens a browser for the class or the methods.

1.9 Defining a new method

Squeak provides many more interesting tools, such as the test runner and the debugger. In order to illustrate you how these are typically used while programming in Squeak, we will show you how you can make use of them while defining a new method.

The advent of Test Driven Development[2] has changed the way that we

[2]Kent Beck, *Test Driven Development: By Example*. Addison-Wesley, 2003, ISBN 0321146530.

write code. The idea behind Test Driven Development, also called TDD or Behavior Driven Development, is that we write a test that defines the desired behavior of our code *before* we write the code itself. Only then do we write the code that satisfies the test.

Suppose that our assignment is to write a method that "says something loudly and with emphasis". What exactly does that mean? How can we make sure that programmers who may have to maintain our method in the future have an unambiguous description of what it should do? We can start answering these questions by giving an example:

> When we send the message shout to the string "Don't panic" the result should be "DON'T PANIC!".

To make this example into something that the system can use, we turn it into a test method:

Method 1.1: *A test for a shout method.*

```
1 StringTest»testShout
2
3   self assert: 'DON"T PANIC!' equals: 'Don"t panic' shout.
```

How do we create a new method in Squeak? First, we have to decide which class the method should belong to. In this case, the shout method that we are testing will go in class String, so the corresponding test will, by convention, go in a class called StringTest.

 *Open a browser on the class* StringTest*, and select an appropriate protocol for our method, in this case* tests - converting*, as shown in Figure 1.19. The highlighted text in the bottom pane is a template that reminds you of what a Smalltalk method looks like. Delete this and enter the code from method 1.1.*

Once you have typed the text into the browser, notice that the bottom pane is outlined in red. This is a reminder that the pane contains unsaved changes. So select accept (s) from the yellow-button menu in the bottom pane, or just type CMD−s, to compile and save your method.

Because there is as yet no method called shout, the browser will ask you to confirm that this is the name that you really want — and it will suggest some other names that you might have intended (Figure 1.20). This can be quite useful if you have merely made a typing mistake, but in this case, we really *do* mean shout, since that is the method we are about to create, so we have to confirm this by selecting the first option from the menu of choices, as shown in Figure 1.20.

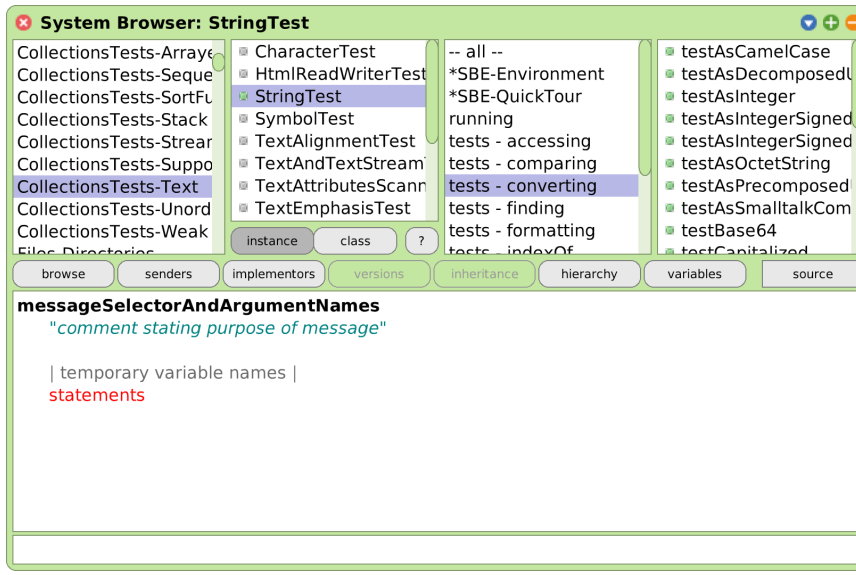


Figure 1.19: The new method template in class StringTest.

Run your newly created test: open the SUnit TestRunner, *either by dragging it from the Tools flap, or by selecting* World ▷ open... ▷ Test Runner.

The leftmost two panes are a bit like the top panes in the system browser. The left pane contains a list of system categories, but it's restricted to those categories that contain test classes.

Select CollectionsTests-Text *and the pane to the right will show all of the test classes in that category, which includes the class* StringTest. *The names of the classes are already selected, so click* Run Selected *to run all these tests.*

You should see a message like that shown in Figure 1.21, which indicates that there was an error in running the tests. The list of tests that gave rise to errors is shown in the bottom right pane; as you can see, StringTest»#testShout is the culprit. (Note that StringTest>>#testShout is the Smalltalk way of identifying the testShout method of the StringTest class.) If you click on that line of text, the erroneous test will run again, this time in such a way that you see the error happen: "MessageNotUnderstood: ByteString»shout".

The window that opens with the error message is the Smalltalk debugger (see Figure 1.22). We will look at the debugger and how to use it in Chapter 6.

The error is, of course, exactly what we expected: Running the test

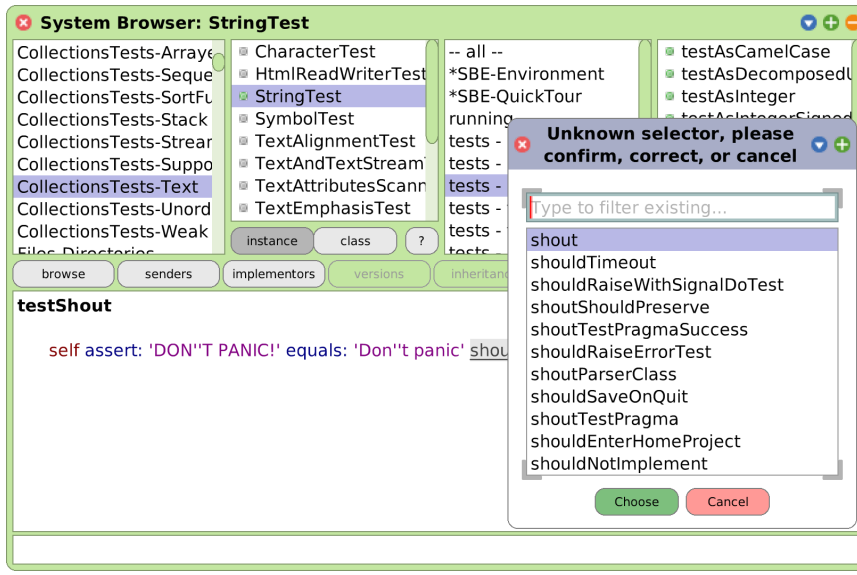


Figure 1.20: Accepting the testShout method class StringTest.

generates an error because we haven't yet written a method that tells strings how to shout. Nevertheless, it's good practice to make sure that the test fails because this confirms that we have set up the testing machinery correctly and that the new test is actually being run. Once you have seen the error, you can Abandon the running test, which will close the debugger window. Note that often with Smalltalk you can define the missing method using the Create button, edit the newly-created method in the debugger, and then Proceed with the test. Now let's define the method that will make the test succeed!

 *Select class* String *in the system browser, select the* converting *category, type the text in method 1.2 over the method creation template, and* accept *it.*

Method 1.2: *The shout method.*

```
1 shout
2
3     ^ self asUppercase , '!'
```

The comma is the string concatenation operation, so the body of this method appends an exclamation mark to an upper-case version of whatever String object the shout message was sent to. The ^ tells Squeak that the expression that follows is the answer to be returned from the method, in

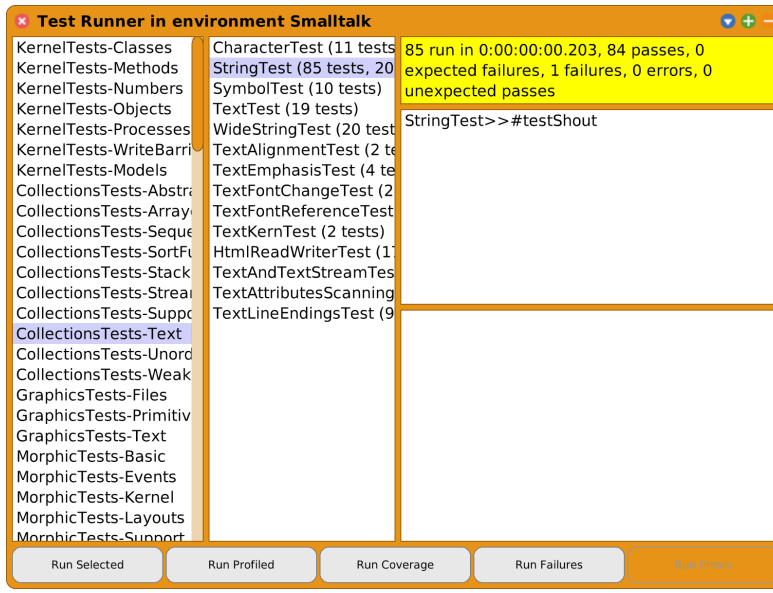


Figure 1.21: Running the String tests.

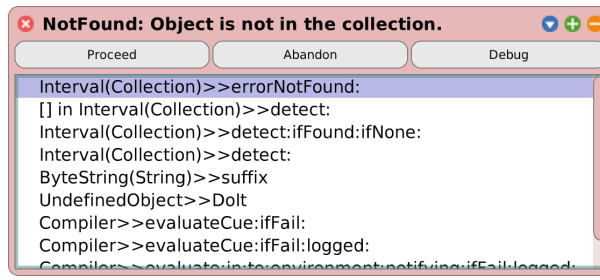


Figure 1.22: The (pre-)debugger.

this case, the new concatenated string. Does this method work? Let's run the tests and see.

Click on Run Selected *again in the test runner, and this time you should see a green bar and text indicating that all of the tests ran with no failures and no errors.*

When you get to a green bar[3], it's a good idea to save your work and take a break. So do that right now!

[3]Actually, you might not get a green bar since some images contain tests for bugs that still need to be fixed. Don't worry about this. Squeak is constantly evolving.

1.10 Chapter summary

This chapter has introduced you to the Squeak environment and shown you how to use some of the major tools, such as the system browser, the method finder, and the test runner. You have also seen a little of Squeak's syntax, even though you may not understand it all yet.

- A running Squeak system consists of a *virtual machine*, a *sources* file, and *image* and *changes* files. Only these last two change, as they record a snapshot of the running system.
- When you start a Squeak image, you will find yourself in exactly the same state — with the same running objects — that you had when you last saved that image.
- Squeak is designed to work with a three-button mouse. The buttons are known as the *red*, the *yellow* and the *blue* buttons. If you don't have a three-button mouse, you can use modifier keys to obtain the same effect.
- You use the red button on the Squeak background to bring up the *World menu* and launch various tools. You can also launch tools from the *Tools* flap at the right of the Squeak screen.
- A *workspace* is a tool for writing and evaluating snippets of code. You can also use it to store arbitrary text.
- You can use keyboard shortcuts on text in the workspace, or any other tool, to evaluate code. The most important of these are do it (CMD−d), print it (CMD−p), inspect it (CMD−i), explore it (CMD−I) and browse it (CMD−b).
- The *system browser* is the main tool for browsing Squeak code, and for developing new code.
- The *test runner* is a tool for running unit tests. It also supports Test Driven Development (TDD).

Chapter 2

Your first interactive object: the Quinto game

In this chapter, we will develop a simple game: Quinto. Along the way, we will demonstrate most of the tools that Squeak programmers use to construct and debug their programs, and show how programs are exchanged with other developers. We will see the system browser, the object inspector, the debugger, and the Monticello package browser. Development in Smalltalk is efficient: You will find that you spend far more time actually writing code and far less managing the development process. This is partly because the Smalltalk language is simple, and partly because the tools that make up the programming environment are integrated well with the language.

Nevertheless, if you find yourself not understanding all of the code, don't worry. We will explain the details of the syntax, the object model, and message passing in later chapters.

2.1 The Quinto game

To show you how to use Squeak's programming tools, we will build a simple game called *Quinto*. The game board is shown in Figure 2.1. The board consists of a rectangular grid of light yellow *cells*. When you click on one of the cells with the mouse, the four surrounding cells turn blue, but not the cell you clicked on. Click again, and the surrounding cells toggle back to light yellow. The objective of the game is to turn as many cells as possible to blue.

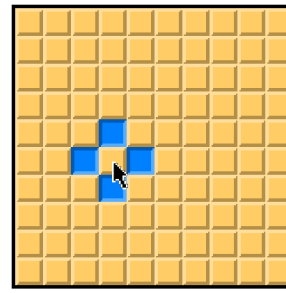


Figure 2.1: The Quinto game board. The user has just clicked the mouse as shown by the cursor.

The Quinto game shown in Figure 2.1 is made up of two kinds of objects: the game board itself, and 100 individual cell objects. The Squeak code to implement the game will contain two classes: one for the game and one for the cells. We will now show you how to define these classes using the Squeak programming tools.

2.2 Creating a new class category

We have already seen the system browser in Chapter 1, where we learned how to navigate to classes and methods, and saw how to define new methods. Now we will see how to create system categories and classes.

Open a system browser and yellow-click in the category pane. Select add item ....

Type the name of the new category (we will use *SBE-Quinto*) in the dialog box and click accept (or just press the return key); the new category is created, and positioned at the end of the category list. If you selected an existing category first, then the new category will be positioned immediately ahead of the selected one.

2.3 Defining the class SBECell

As yet there are of course no classes in the new category. However, the main editing pane displays a template to make it easy to create a new class (see Figure 2.3). This template shows us a Smalltalk expression that sends a message to a class called Object, asking it to create a subclass called NameOfSubClass. The new class has no variables and should belong to the

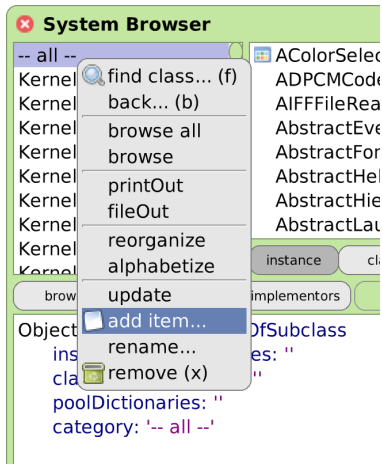


Figure 2.2: Adding a system category.

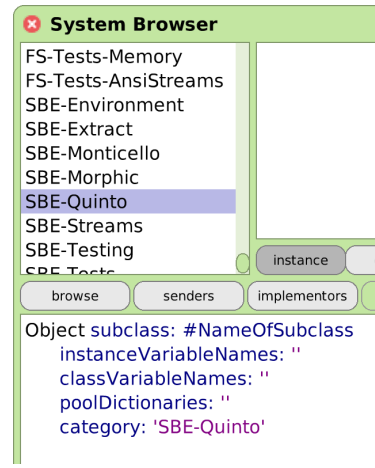


Figure 2.3: The class-creation template.

category *SBE-Quinto*. We now modify the template to create the class that we want.

Modify the class creation template as follows:

- Replace Object by SimpleSwitchMorph.
- Replace NameOfSubClass by SBECell.
- Add mouseAction to the list of instance variables.

The result should look like class 2.1.

Class 2.1: *Defining the class* SBECell.

```
SimpleSwitchMorph subclass: #SBECell
  instanceVariableNames: 'mouseAction'
  classVariableNames: ''
  poolDictionaries: ''
  category: 'SBE-Quinto'
```

This new definition consists of a Smalltalk expression that sends a message to the existing class SimpleSwitchMorph, asking it to create a subclass called SBECell. (Actually, since SBECell does not exist yet, we passed as an argument the *symbol* #SBECell which stands for the name of the class to create.) We also tell it that instances of the new class should have a

mouseAction instance variable, which we will use to define what action the cell should take if we click on it.

At this point, you still have not created anything. Note that in the upper right corner of the class template pane a red triangle appeared (Figure 2.4). This means that there are *unsaved changes*. To actually send this message, you must accept it.

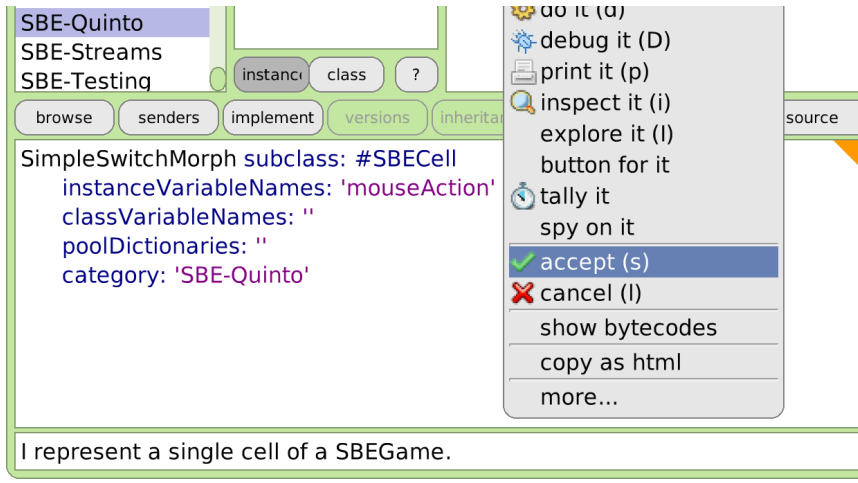


Figure 2.4: Accepting the new class definition.

Accept the new class definition.

Either yellow-click and select accept, or use the shortcut CMD–s (for "save"). The message will be sent to SimpleSwitchMorph, which will cause the new class to be compiled.

Once the class definition is accepted, the class will be created and appear in the classes pane of the browser (Figure 2.5). The editing pane now shows the class definition, and a small pane below it will remind you to write a few words describing the purpose of the class. This is called a *class comment*, and it is quite important to write one that will give other programmers a high-level overview of the purpose of this class. Smalltalkers put a very high value on the readability of their code, and detailed comments about the content of a method are unusual: The philosophy is that the code should speak for itself. (If it doesn't, you should refactor it until it does!) Be aware that this refers to the content of the method, not the purpose of the method in the larger system or assumptions of the method; these aspects should definitely be described in a method or class comment. A class comment need not contain a detailed description of the implementation

details of the class, but a few words describing its overall purpose are vital if programmers who come after you are to know whether to spend time looking at this class.

 *Type a class comment for* SBECell *and accept it; you can always improve it later.*



Figure 2.5: The newly-created class SBECell.

2.4 Adding methods to a class

Now let's add some methods to our class.

 *Select the protocol* --all-- *in the protocol pane.*

You will see a template for method creation in the editing pane. Select it, and replace it by the text of method 2.2.

Method 2.2: *Initializing instances of* SBECell.

```
1  SBECell»initialize
2
3      super initialize.
4      self
5          label: '';
6          borderWidth: 2;
7          bounds: (0 @ 0 corner: 16 @ 16);
8          offColor: Color cantaloupe;
9          onColor: Color aqua;
10         useSquareCorners.
11     self turnOff.
```

Note that the characters '' on line 3 are two separate single quotes with nothing between them, not a double quote! '' denotes the empty string.

 Accept *this method definition.*

What does the above code do? We won't go into all of the details here (that's what the rest of the book is for!), but we will give you a quick preview. Let's take it line by line.

Notice that the method is called initialize. This name has a special meaning. By convention, if a class defines a method named initialize, it will be called right after the object is created. So, when we evaluate SBECell new, the message initialize will be sent automatically to this newly created object. Initialize methods are used to set up the state of objects, typically to set their instance variables; this is exactly what we are doing here.

The first thing that this method does (line 2) is to execute the initialize method of its superclass, SimpleSwitchMorph. The idea here is that any inherited state will be properly initialized by the initialize method of the superclass. It is always a good idea to initialize inherited state by sending **super** initialize before doing anything else; we don't know exactly what SimpleSwitchMorph's initialize method will do, and we don't care, but it's a fair bet that it will set up some instance variables to hold reasonable default values, so we had better call it, or we risk starting in an unclean state.

The rest of the method sets up the state of this object. Sending **self** label: '', for example, sets the label of this object to the empty string.

The expression 0 @ 0 corner: 16 @ 16 probably needs some explanation. 0 @ 0 represents a Point object with x and y coordinates both set to 0. In fact, 0 @ 0 sends the message @ to the Number object 0 with argument 0. The effect will be that the number 0 will ask the Point class to create a new instance with coordinates $(0, 0)$. Now we send this newly created point the message corner: 16 @ 16, which causes it to create a Rectangle with corners

0 @ 0 and 16 @ 16. We use the setter method bounds, inherited from the superclass, to set the bounds of the cell to the newly created rectangle.

Note that the origin of the Squeak screen is the *top left*, and the y coordinate increases *downward*.

The rest of the method should be self-explanatory. Part of the art of writing good Smalltalk code is to pick good method names so that Smalltalk code can be read like a kind of pidgin English. You should be able to imagine the object talking to itself and saying "Self use square corners!", "Self turn off!".

2.5 Inspecting an object

You can test the effect of the code you have written by creating a new SBECell object and inspecting it.

Open a workspace. Type the expression SBECell new *and* inspect it.

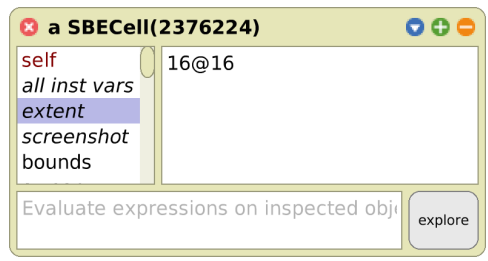


Figure 2.6: The inspector used to examine a SBECell object.

The left-hand pane of the inspector shows a list of instance variables; if you select one (try bounds), the value of the instance variable is shown in the right pane. You can also use the inspector to change the value of an instance variable.

Change the value of bounds *to* 0 @ 0 corner: 50 @ 50 *and* accept *it.*

The bottom pane of the inspector is a mini-workspace. It's useful because in this workspace the pseudo-variable self is bound to the object being inspected.

Type the text **self** openInWorld *in the bottom pane and* do it.

The cell should appear at the top left-hand corner of the screen, indeed, exactly where its bounds say that it should appear. Blue-click on the cell to

bring up the morphic halo. Move the cell with the brown (next to top-right) handle and resize it with the yellow (bottom-right) handle. Notice how the bounds reported by the inspector also change.

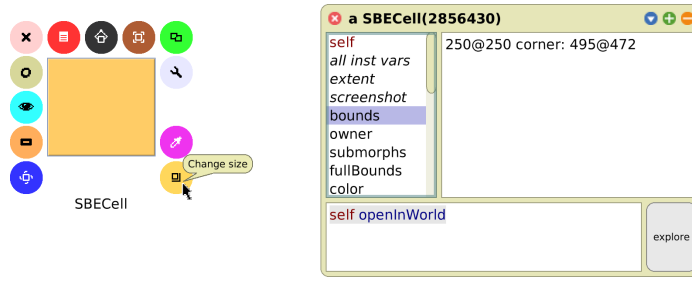


Figure 2.7: Resizing the cell.

Delete the cell by clicking on the x *in the pink handle.*

2.6 Defining the class SBEGame

Now let's create the other class that we need for the game, which we will call SBEGame.

Make the class definition template visible in the browser's main window.

Do this by clicking twice on the name of the already-selected class category, or by displaying the definition of SBECell again (by clicking the instance button.) Edit the code so that it reads as follows, and accept it.

Class 2.3: *Defining the* SBEGame *class*

```
1 BorderedMorph subclass: #SBEGame
2   instanceVariableNames: ''
3   classVariableNames: ''
4   poolDictionaries: ''
5   category: 'SBE-Quinto'
```

Here we subclass BorderedMorph; Morph is the superclass of all of the graphical shapes in Squeak, and (surprise!) a BorderedMorph is a Morph with a border. We could also insert the names of the instance variables between the quotes on the second line, but for now, let's just leave that list empty.

Now let's define an initialize method for SBEGame.

Type the following into the browser as a method for SBEGame *and try to* accept *it:*

Method 2.4: *Initializing the game.*

```
SBEGame»initialize
    | height overallBorderWidth sampleCell width |

    super initialize.
    sampleCell := SBECell new.
    width := sampleCell width.
    height := sampleCell height.
    overallBorderWidth := 2 * self borderWidth.
    self position: 5 @ 5.
    self extent: (width * self cellsPerSide) @ (height * self cellsPerSide) +
        overallBorderWidth.
    cells := Matrix new: self cellsPerSide tabulate: [:i :j | self newCellAt: i at: j].
```

Squeak will complain that it doesn't know the meaning of some of the terms. Squeak tells you that it doesn't know of a message cellsPerSide, and suggests a number of corrections, in case it was a spelling mistake.

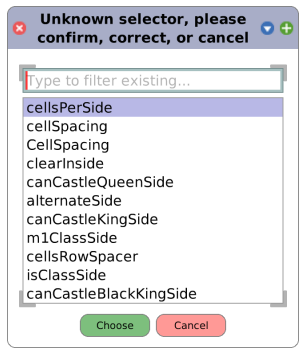


Figure 2.8: Squeak detecting an unknown selector.

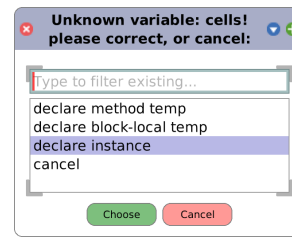


Figure 2.9: Declaring a new instance variable.

But cellsPerSide is not a mistake — it is just a method that we haven't yet defined — we will do so in a minute or two.

So just select the first item from the menu, which confirms that we really meant cellsPerSide.

Next, Squeak will complain that it doesn't know the meaning of cells. It offers you a number of ways of fixing this.

Choose declare instance *because we want* cells *to be an instance variable.*

Finally, Squeak will complain about the message newCellAt:at: sent on the last line; this is also not a mistake, so confirm that message too.

If you now look at the class definition once again (which you can do by clicking on the instance button), you will see that the browser has modified it to include the instance variable cells.

Let's look at our initialize method. We declare 4 temporary variables with the line |height overallBorderWidth sampleCell width|. They are called temporary variables because their scope and lifetime are limited to this method. Temporary variables with explanatory names are helpful in making code more readable. Smalltalk has no special syntax to distinguish constants and variables, and in fact all four of these "variables" are really constants. Lines 5–7 define these constants.

How big should our game board be? Big enough to hold some integral number of cells, and big enough to draw a border around them. How many cells is the right number? 5? 10? 100? We don't know yet, and if we did, we would probably change our minds later. So we delegate the responsibility for knowing that number to another method, which we will call cellsPerSide, and which we will write in a minute or two. It's because we are sending the cellsPerSide message before we define a method with that name that Squeak asked us to "confirm, correct, or cancel" when we accepted the method body for initialize. Don't be put off by this: It is actually good practice to write in terms of other methods that we haven't yet defined. Why? Well, it wasn't until we started writing the initialize method that we realized that we needed it, and at that point, we can give it a meaningful name, and move on, without interrupting our flow.

The lines 4–6 create a new SBECell object, and assign its width and height to the appropriate temporary variables.

Line 8 sets the extent of the new object. Without worrying too much about the details just yet, just believe us that the expression in parentheses creates a square with its origin (*i.e.*, its top-left corner) at the point $(5, 5)$ and its bottom-right corner far enough away to allow space for the right number of cells.

The last line sets the SBEGame object's instance variable cells to a newly created Matrix with the right number of rows and columns. We do this by sending the message new:tabulate: to the Matrix class (classes are objects too, so we can send them messages). We know that new:tabulate: takes two arguments because it has two colons (:) in its name. The arguments go right after the colons. If you are used to languages that put all of the arguments together inside parentheses, this may seem weird at first. Don't panic, it's only syntax! It turns out to be a very good syntax because the name of the method can be used to explain the roles of the arguments. For example, it

is pretty clear that Matrix rows: 5 columns: 2 has 5 rows and 2 columns, and not 2 rows and 5 columns.

Matrix new: **self** cellsPerSide tabulate: [:i :j | **self** newCellAt: i at: j] creates a new n×n matrix and initializes its elements. The initial value of each element will depend on its coordinates. The $(i, j)^{\text{th}}$ element will be initialized to the result of evaluating **self** newCellAt: i at: j.

2.7 Organizing methods into protocols

Before we define any more methods, let's take a quick look at the third pane at the top of the browser. In the same way that the first pane of the browser lets us categorize classes so we are not overwhelmed by a very long list of class names in the second pane, so the third pane lets us categorize methods so that we are not overwhelmed by a very long list of method names in the fourth pane. These categories of methods are also called "protocols".

If there are only a few methods in a class, the extra level of hierarchy provided by protocols is not really necessary. This is why the browser also offers us the *--all--* virtual protocol, which, you will not be surprised to learn, contains all of the methods in the class.

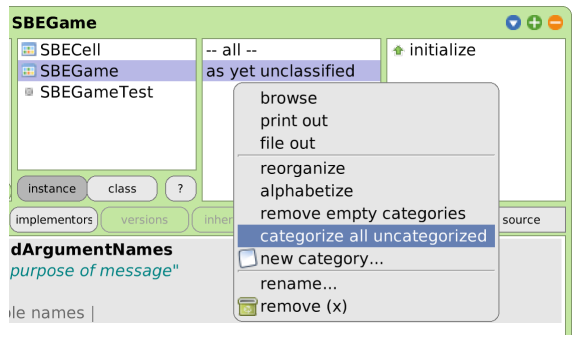


Figure 2.10: Categorize all uncategorized methods.

If you have followed along with this example, the third pane may well contain the protocol *as yet unclassified*.

Select the yellow button menu item categorize all uncategorized *to fix this, and the* initialize *method will be moved to a new protocol called* initialization.

How does Squeak know that this is the right protocol? Well, in general Squeak can't know, but in this case, there is also an initialize method in a

superclass, and Squeak assumes that our initialize method should go in the same protocol as the one that it overrides.

A typographic convention. Smalltalkers frequently use the notation ">>" to identify the class to which a method belongs, so, for example, the cellsPerSide method in class SBEGame would be referred to as SBEGame >>cellsPerSide. To indicate that this is *not* Smalltalk syntax, we will use the special symbol » instead, so this method will appear in the text as SBEGame »cellsPerSide. From now on, when we show a method in this book, we will write the name of the method in this form. Of course, when you actually type the code into the browser, you don't have to type the class name or the »; instead, you just make sure that the appropriate class is selected in the class pane.

Now let's define the other two methods that are used by the SBEGame» initialize method. Both of them can go in the *initialization* protocol.

Method 2.5: *A constant method.*

```
SBEGame»cellsPerSide
    "The number of cells along each side of the game"

    ^ 10
```

This method could hardly be simpler: It answers the constant 10. One advantage of representing constants as methods is that if the program evolves so that the constant then depends on some other features, the method can be changed to calculate this value.

Method 2.6: *An initialization helper method.*

```
SBEGame»newCellAt: i at: j
    "Create a cell for position (i,j) and add it to my on-screen
    representation at the appropriate screen position. Answer the new cell."

    | cell origin |
    cell := SBECell new.
    origin := self innerBounds origin.
    self addMorph: cell.
    cell
        position: ((i - 1) * cell width) @ ((j - 1) * cell height) + origin;
        mouseAction: [self toggleNeighboursOfCellAt: i at: j].
```

 *Add the methods* SBEGame»cellsPerSide *and* SBEGame»newCellAt:at:.

Confirm the spelling of the new selectors toggleNeighboursOfCellAt:at: and mouseAction:.

Method 2.6 answers a new SBECell, specialized to position (i, j) in the Matrix of cells. The last line defines the new cell's mouseAction to be the *block* [self toggleNeighboursOfCellAt: i at: j]. In effect, this defines the callback behavior to perform when the mouse is clicked. The corresponding method also needs to be defined.

Method 2.7: *The callback method.*

```
SBEGame»toggleNeighboursOfCellAt: i at: j

    i > 1 ifTrue: [(cells at: i - 1 at: j) toggleState].
    i < self cellsPerSide ifTrue: [(cells at: i + 1 at: j) toggleState].
    j > 1 ifTrue: [(cells at: i  at: j - 1) toggleState].
    j < self cellsPerSide ifTrue: [(cells at: i at: j + 1) toggleState].
```

Method 2.7 toggles the state of the four cells to the north, south, west and east of cell (i, j). The only complication is that the board is finite, so we have to make sure that a neighboring cell exists before we toggle its state.

Place this method in a new protocol called game logic. *To create the protocol, press the yellow mouse button in the protocol list, select* new category…, *enter "game logic", and submit the dialog.*

To move the method, you can simply click on its name and drag it to the newly-created protocol (Figure 2.11).

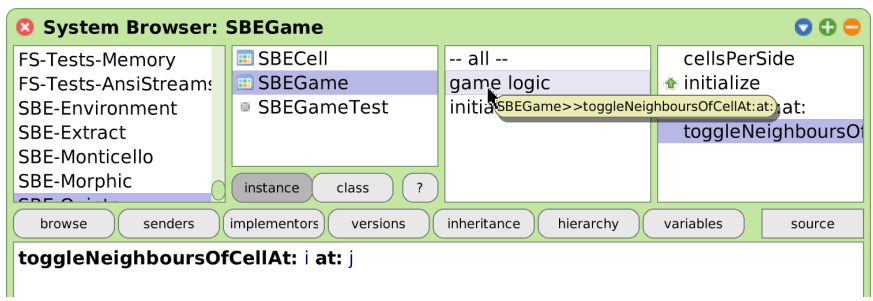


Figure 2.11: Drag a method to a protocol.

To complete the Quinto game, we need to define two more methods in class SBECell to handle mouse events.

Method 2.8: *A typical setter method.*

```
SBECell»mouseAction: aBlock

    mouseAction := aBlock
```

Method 2.8 does nothing more than set the cell's mouseAction variable to the argument, and then answers the new value. Any method that *changes* the value of an instance variable in this way is called a *setter method*; a method that *answers* the current value of an instance variable is called a *getter method*.

If you are used to getters and setters in other programming languages, you might expect these methods to be called setmouseAction and getmouseAction . The Smalltalk convention is different. A getter always has the same name as the variable it gets, and a setter is named similarly, but with a trailing ":", hence mouseAction and mouseAction:.

Collectively, setters and getters are called *accessor* methods[1], and by convention they should be placed in the *accessing* protocol. In Smalltalk, *all* instance variables are private to the object that owns them, so the only way for another object to read or write those variables in the Smalltalk language is through accessor methods like this one[2].

 *Go to the class* SBECell*, define* SBECell»mouseAction: *and put it in the* accessing *protocol.*

Finally, we need to define a method mouseUp:; this will be called automatically by the GUI framework if the mouse button is released while the mouse is over this cell on the screen.

Method 2.9: *An event handler.*

```
1 SBECell»mouseUp: anEvent
2
3   mouseAction value.
```

 *Add the method* SBECell»mouseUp: *and then* categorize all uncategorized *methods.*

This method sends the message value to the object stored in the instance variable mouseAction. Recall that in SBEGame»newCellAt: i at: j we assigned the following code fragment to mouseAction:

```
[self toggleNeighboursOfCellAt: i at: j ]
```

Sending the value message causes this code fragment to be evaluated, and consequently, the state of the cells will toggle.

[1] Edward J. Klimas, Suzanne Skublics and David A. Thomas, *Smalltalk with Style*. Prentice-Hall, 1996, ISBN 0–13–165549–3.

[2] In fact, the instance variables can be accessed in subclasses too.

2.8 Let's try our code

That's it: The Quinto game is complete!

If you have followed all of the steps, you should be able to play the game, consisting of just 2 classes and 7 methods.

 *In a workspace, type* SBEGame new openInWorld *and* do it.

The game will open, and you should be able to click on the cells and see how it works.

Well, so much for theory... When you click on a cell, a *notifier* window appears with an error message! As depicted in Figure 2.12, it says MessageNotUnderstood: SBEGame»toggleState.

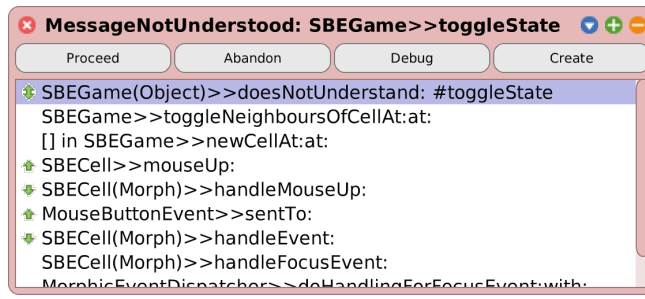


Figure 2.12: There is a bug in our game when a cell is clicked!

What happened? To find out, let's use one of Smalltalk's more powerful tools: the debugger.

 *Click on the* debug *button in the notifier window.*

The debugger will appear. In the upper part of the debugger window you can see the execution stack, showing all the active methods; selecting any one of them will show, in the middle pane, the Smalltalk code being executed in that method, with the part that triggered the error highlighted.

 *Click on the line labelled* SBEGame»toggleNeighboursOfCellAt:at: *(near the top).*

The debugger will show you the execution context within this method where the error occurred (Figure 2.13).

At the bottom of the debugger are two small inspector windows. On the left, you can inspect the object that is the receiver of the message that

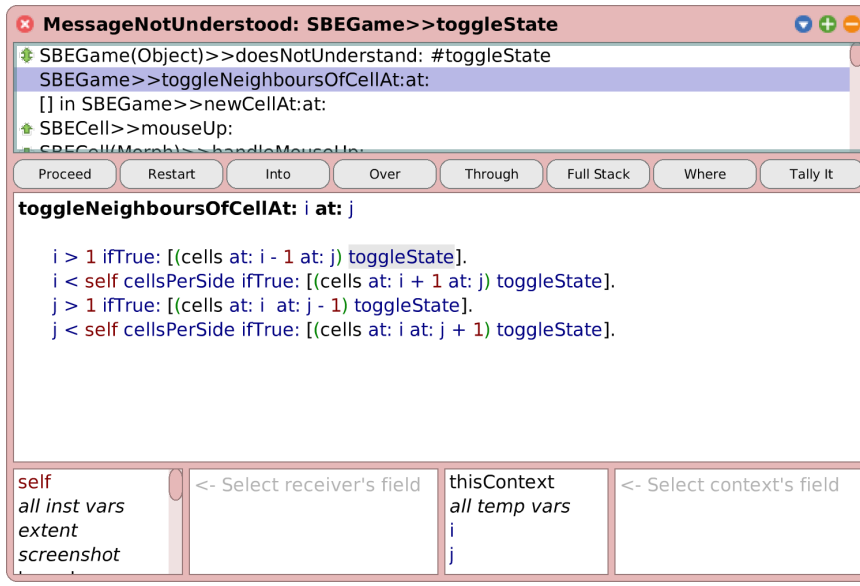


Figure 2.13: The debugger, with the method toggleNeighboursOfCell:at: selected.

caused the selected method to execute, so you can look here to see the values of the instance variables. On the right you can inspect an object that represents the currently executing method itself, so you can look here to see the values of the method's parameters and temporary variables.

Using the debugger, you can execute code step by step, inspect objects in parameters and temporary variables, evaluate code just as you can in a workspace, and, most surprisingly to those used to other debuggers, change the code while it is being debugged! Some Smalltalkers program in the debugger almost all the time, rather than in the browser. The advantage of this is that you see the method that you are writing as it will be executed, with real parameters in the actual execution context.

In this case, we can see in the first line of the top panel that the toggleState message has been sent to an instance of SBEGame, while it should clearly have been an instance of SBECell. We can also see this by selecting the cells variable in the lower-left pane in the debugger. The problem is most likely with the initialization of the cells matrix. Browsing the code of SBEGame» initialize shows that cells is filled with the return values of newCellAt:at:, but when we look at that method, we see that there is no return statement there! By default, a method returns **self**, which in the case of newCellAt:at: is indeed an instance of SBEGame.

 *Close the debugger window. Add the expression "*^ cell*" to the end of the method* SBEGame»newCellAt:at: *so that it returns* cell. *(See method 2.10.)*

Method 2.10: *Fixing the bug.*

```
SBEGame»newCellAt: i at: j
    "Create a cell for position (i,j) and add it to my on-screen
    representation at the appropriate screen position. Answer the new cell."

    | cell origin |
    cell := SBECell new.
    origin := self innerBounds origin.
    self addMorph: cell.
    cell
        position: ((i - 1) * cell width) @ ((j - 1) * cell height) + origin;
        mouseAction: [self toggleNeighboursOfCellAt: i at: j].
    ^ cell
```

Often, you can fix the code directly in the debugger window and click Proceed to continue running the application. In our case, because the bug was in the initialization of an object, rather than in the method that failed, the easiest thing to do is to close the debugger window, destroy the running instance of the game (with the halo), and create a new one.

 *Do:* SBEGame new openInWorld *again.*

Now the game should work properly.

2.9 Saving and sharing Smalltalk code

Now that you have the Quinto game working, you probably want to save it somewhere so that you can share it with your friends. Of course, you can save your whole Squeak image, and show off your first program by running it, but your friends probably have their own code in their images, and don't want to give that up to use your image. What you need is a way of getting source code out of your Squeak image so that other programmers can bring it into theirs. There is a variety of ways to do this, from ad-hoc sharing via exported files, to built-in version control systems like Monticello, or full tool support for Git.

The simplest way of doing this is by *filing out* the code. The yellow-button menu in the System Categories pane will give you the option to file out the whole of category *SBE-Quinto*. The resulting file is more or less human-readable, but is really intended for computers, not humans. You

can email this file to your friends, and they can file it into their own Squeak images using drag-and-drop.

Yellow-click on the SBE-Quinto *category and* fileOut *the contents.*

You should now find a file called "SBE-Quinto.st" in the same folder on disk where your image is saved. Have a look at this file with a text editor.

Open a fresh Squeak image, drag the exported file onto the running Squeak, and choose file in entire file. *Verify that the game now works in the new image.*

Monticello packages

Although fileouts are a convenient way of making a snapshot of the code you have written, they do not work well for long-running projects. Just as most open-source projects find it much more convenient to maintain their code in a repository using Git[3] or Mercurial[4], so Squeak programmers find it more convenient to manage their code using Monticello packages. These packages are represented as files with names ending in .mcz; they are actually zip-compressed bundles that contain the complete code of your package.

Using the Monticello package browser, you can save packages to repositories on various types of servers, including FTP and HTTP servers; you can also just write the packages to a repository in a local file system directory. A copy of your package is also always cached on your local hard-disk in the *package-cache* folder. Monticello lets you save multiple versions of your program, merge versions, go back to an old version, and browse the differences between versions. In fact, Monticello is a distributed version control system just like Git; this means it allows developers to save their work on different places, not on a single repository.

You can also send a .mcz file by email. The recipient will have to place it in her *package-cache* folder; she will then be able to use Monticello to browse and load it.

Open the Monticello browser by selecting World ▷ open... ▷ Monticello browser.

In the right-hand pane of the browser (see Figure 2.14) is a list of Monticello repositories, which will include all of the repositories from which code has been loaded into the image that you are using.

[3] www.git-scm.com
[4] www.mercurial-scm.org

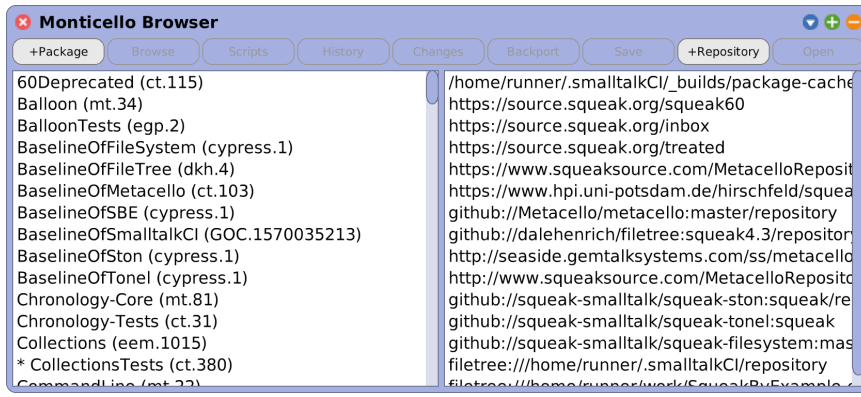


Figure 2.14: The Monticello browser.

At the top of the list in the Monticello browser is a repository in a local directory called the *package cache*, which caches copies of the packages that you have loaded or published over the network. This local cache is really handy because it lets you keep your own local history; it also allows you to work in places where you do not have internet access, or where access is slow and you do not want to save to a remote repository frequently.

Saving and loading code with Monticello

On the left-hand side of the Monticello browser is a list of packages that have a version loaded into the image; packages that have been modified since they were loaded are marked with an asterisk. (These are sometimes referred to as *dirty packages*.) If you select a package, the list of repositories is restricted to just those repositories that contain a copy of the selected package.

What is a package? For now, you can think of a package as a group of class categories and protocols that share the same prefix. Since we put all of the code for the Quinto game into the class category called *SBE-Quinto*, we can refer to it as the SBE-Quinto package.

 *Add the* SBE-Quinto *package to your Monticello browser using the* +Package *button and type* SBE-Quinto.

SqueakSource: a sharing platform for Monticello

We think that the best way to save your code and share it is to create an account for your project on a SqueakSource server. SqueakSource is similar to GitHub[5]: It is a web front-end to an HTTP Monticello server that lets you manage your projects. There is a public SqueakSource server at http://www.squeaksource.com, and a copy of the code related to our game is stored there at http://www.squeaksource.com/SqueakByExample60. You can look at this project with a web browser, but it's a lot more productive to do so from inside Squeak, using the Monticello browser, which lets you manage your packages.

 *Open a web browser to www.squeaksource.com. Create an account for yourself and then create (*i.e.*, "register") a project for the Quinto game.*

SqueakSource will show you the information that you should use when adding a repository using the Monticello browser.

Once your project has been created on SqueakSource, you have to tell your Squeak system to use it.

 *With the* SBE-Quinto *package selected, click the* +Repository *button in the Monticello browser.*

You will see a list of the different available types of repositories; to add a SqueakSource repository select HTTP. You will be presented with a dialog in which you can provide the necessary information about the server. You should copy the presented template to identify your SqueakSource project, paste it into Monticello, and supply your initials and password:

```
MCHttpRepository
    location: 'http://www.squeaksource.com/YourProject'
    user: 'yourInitials'
    password: 'yourPassword'
```

If you provide empty initials and password strings, you can still load the project, but you will not be able to update it:

```
MCHttpRepository
    location: 'http://www.squeaksource.com/YourProject'
    user: ''
    password: ''
```

Once you have accepted this template, your new repository should be listed on the right-hand side of the Monticello browser.

[5] www.github.com

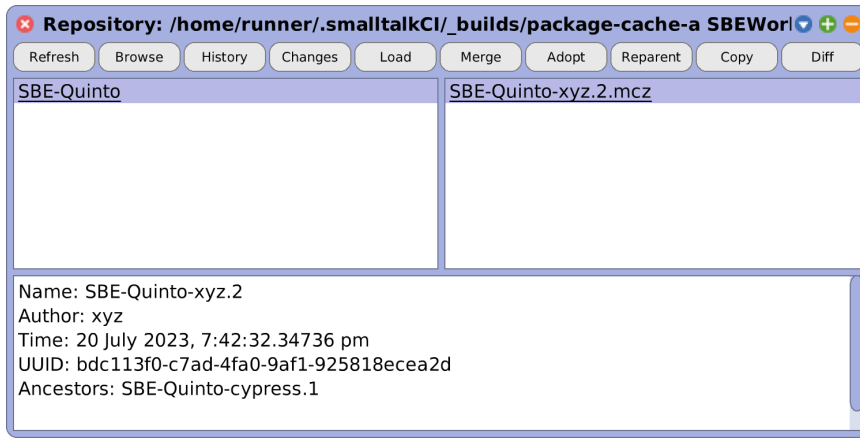


Figure 2.15: Browsing a Monticello repository.

Click on Save *to save the first version of your Quinto game on SqueakSource.*

To load a package into your image, you must first select a particular version. You can do this in the repository browser, which you can open using the Open button or the yellow-button menu. Once you have selected a version, you can load it onto your image.

Open the SBE-Quinto *repository you have just saved.*

Monticello has many more capabilities, which will be discussed in depth in Chapter 6.

2.10 Chapter summary

In this chapter, you have seen how to create categories, classes, and methods. You have seen how to use the system browser, the inspector, the debugger, and the Monticello browser.

- Categories are groups of related classes.
- A new class is created by sending a message to its superclass.
- Protocols are groups of related methods.
- A new method is created or modified by editing its definition in the browser and then *accepting* the changes.

- The inspector offers a simple, general-purpose GUI for inspecting and interacting with arbitrary objects.
- The system browser detects usage of undeclared methods and variables, and offers possible corrections.
- The initialize method is automatically executed after an object is created in Squeak. You can put any initialization code there.
- The debugger provides a high-level GUI to inspect the state of a halted program and run it step-by-step.
- You can share source code *filing out* a category.
- A better way to share code is to use Monticello to manage an external repository, for example, defined as a SqueakSource project.

Chapter 3

Syntax in a nutshell

Squeak, like most modern Smalltalk dialects, adopts a syntax very close to that of Smalltalk-80. The syntax is designed so that program text can be read aloud as though it were a kind of pidgin English:

```
(Smalltalk includes: Class) ifTrue: [Transcript show: Class superclass]
```

Squeak's syntax is minimal. Essentially there is syntax only for *sending messages* (*i.e.*, expressions). Expressions are built up from a very small number of primitive elements. There are only 6 keywords, and there is no syntax for control structures or declaring new classes. Instead, nearly everything is achieved by sending messages to objects. For instance, instead of an if-then-else control structure, Smalltalk sends messages like ifTrue: to Boolean objects. New (sub-)classes are created by sending a message to their superclass.

3.1 Syntactic elements

Expressions are composed of the following building blocks: (i) six reserved keywords, or *pseudo-variables*: **self**, **super**, **nil**, **true**, **false**, and **thisContext**, (ii) constant expressions for *literal objects* including numbers, characters, strings, symbols, and arrays, (iii) variable declarations, (iv) assignments, (v) block closures, and (vi) messages.

We can see examples of the various syntactic elements in Table 3.1.

Local variables. startPoint is a variable name, or identifier. By convention, identifiers are composed of words in "camelCase" (*i.e.*, each word except the first starting with an upper case letter). The first letter of an

Syntax	What it represents
startPoint	a local variable name
Transcript	a shared variable name
self	pseudo-variable
1	decimal integer
2r101	binary integer
1.5	floating point number
2.4e7	exponential notation
1.79s2	scaled decimal
$a	the character 'a'
'Hello'	the string "Hello"
#Hello	the symbol #Hello
{1 . 2 . 1 + 2}	an array (brace construct)
#(1 2 3)	an array literal
#[255 33 200 1]	a byte array literal
"a comment"	a comment
|x y|	declaration of temporary variables x and y
x := 1	assign 1 to x
[x + y]	a block that evaluates to x+y
<primitive: 1>	primitive pragma to invoke virtual machine
3 factorial	unary message
3 + 4	binary messages
2 raisedTo: 6 modulo: 10	keyword message
^ true	return the value true
Transcript show: 'hello'. Transcript cr	expression separator (.)
Transcript show: 'hello'; cr	message cascade (;)

Table 3.1: Squeak Syntax in a nutshell.

instance variable, method or block argument, or temporary variable must be lower case. This indicates to the reader that the variable has a private scope.

Shared variables. Identifiers that start with upper case letters are global variables, class variables, pool dictionaries, or class names. Transcript is a global variable, an instance of the class TranscriptStream.

The receiver. **self** is a keyword that refers to the object in which the current method is executing. We call it "receiver" because this object will normally have received the message that caused the method to execute. self is called a "pseudo-variable" since we cannot assign to it.

Integers. In addition to ordinary decimal integers like 42, Squeak also provides a radix notation. 2r101 is 101 in radix 2 (*i.e.*, binary), which is equal to decimal 5. 16rFf is FF in radix 16 (*i.e.*, hexadecimal), which is equal to decimal 255.

Floating point numbers can be specified with their base-ten exponent: 2.4e7 is 2.4×10^7. Floats also support radix notation, but the exponent is always in base ten: 2r0.1e2 is 0.5×10^2, which is equal to 2.

Scaled decimals are fixed-point numbers with a fixed number of digits after the decimal point. They support unbounded precision, but are slower than floating point numbers. For example, 1.79s2 is equal to 179/100. You can omit the scale, in which case it is taken from the number of digits specified: 1.99s is equivalent to 1.99s2 and 1s2 is equivalent to 1.00s2.

Characters. A dollar sign introduces a literal character: $a is the literal for 'a'. Instances of non-printing characters can be obtained by sending appropriately named messages to the Character class, such as Character space and Character tab. Nevertheless, you can also write any non-printing character verbatim after the dollar sign, but this is often hard to read:

```
$ = Character space    ⟶    true
$
 = Character cr        ⟶    true
```

Strings. Single quotes are used to define a literal string. If you want a string with a quote inside, just double the quote, as in 'G''day'.

Symbols are like strings in that they contain a sequence of characters. However, unlike a string, a literal symbol is guaranteed to be globally unique. There is only one Symbol object #Hello but there may be multiple String objects with the value 'Hello'. Amongst others, symbols are used for referencing messages by their selector.

Run-time arrays. Curly braces { } create an array dynamically at run-time. Elements are expressions separated by periods. So {1 . 2 . 1 + 2} defines an array with elements 1, 2, and the result of evaluating $1 + 2$. (The curly-brace notation is peculiar to the Squeak dialect of Smalltalk! In other Smalltalks you must build up dynamic arrays explicitly.)

Array literals are defined by #( ), surrounding space-separated literals. Everything within the parentheses must be a compile-time constant. These arrays are created at compile-time of the method. For example,

#(27 #(**true false**) abc) is a literal of three elements: the integer 27, the literal array containing the two booleans, and the symbol #abc. Whenever a method including such an array is executed the same instance is used. That is why these arrays are often used to represent constant values and avoid repeated array instantiations.

Byte arrays are defined by #[ ], surrounding integers between 0 and 255. This creates an instance of ByteArray which can be used to work with binary data. Just as for #( ), byte arrays that are defined using #[ ] are static literals.

Comments are enclosed in double quotes. *"hello"* is a comment, not a string, and is ignored by the Squeak compiler. Comments may span multiple lines.

Temporary variable declaration. Vertical bars | | enclose the declaration of one or more temporary variables in a method (and also in a block).

Assignment. := assigns an object to a variable. Sometimes you will see ← used instead. Unfortunately, since this is not an ASCII character, it will appear as an underscore unless you are using a special font. So, x := 1 is the same as x ← 1 or x _ 1. You should use := since the other representations have been deprecated since Squeak 3.9.

Blocks. Square brackets [ ] define a block, also known as a block closure or a lexical closure, which is a first-class object representing a function. As we shall see, blocks may take arguments and can have temporary variables.

Pragmas. Angle brackets < > right after a method header denote a pragma which is a possibility to add static annotations to a method. The kind of pragma you will probably see most frequently in Squeak are *primitive* pragmas which are indicated by the pragma selector #primitive: and its siblings #primitive:error:, #primitive:module: etc. Primitives denote an invocation of a virtual machine primitive. For instance, <primitive: 1> is the VM primitive for SmallInteger»+. Any code following the primitive is executed only if the primitive fails. pragma

Unary messages consist of a single word (such as factorial) sent to a receiver (like the number 3).

Binary messages are operators (like +) sent to a receiver and taking a single argument. In 3 + 4, the receiver is 3 and the argument is 4.

Keyword messages consist of multiple keywords (like raisedTo:modulo:), each ending with a colon and taking a single argument. In the expression 2 raisedTo: 6 modulo: 10, the *message selector* raisedTo:modulo: takes

the two arguments 6 and 10, one following each colon. We send the message to the receiver 2.

Method return. ^ is used to *return* a value from a method.

Sequences of statements. A period or full-stop (.) is the *statement separator*. Putting a period between two expressions turns them into independent statements.

Cascades. Semicolons can be used to send a *cascade* of messages to a single receiver. In Transcript show: 'hello'; cr we first send the keyword message show: 'hello' to the receiver Transcript, and then we send the unary message cr to the same receiver.

The classes Number, Character, String, and Boolean are described in more detail in Chapter 9.

3.2 Pseudo-variables

In Smalltalk, there are six reserved keywords, or *pseudo-variables*: **true**, **false**, **nil**, **self**, **super**, and **thisContext**. They are called pseudo-variables because they are predefined and cannot be assigned to. **true**, **false**, and **nil** are constants while the values of **self**, **super**, and **thisContext** vary dynamically as code is executed.

true and **false** are the unique instances of the Boolean classes True and False. See Chapter 9 for more details.

nil is the undefined object. Instance variables, class variables, and temporary variables are initialized to **nil**. It is the unique instance of the class UndefinedObject.

self always refers to the receiver of the currently executing method.

super also refers to the receiver of the current method, but when you send a message to super, the method-lookup changes so that it starts from the superclass of the class containing the method that uses **super**. For further details see Chapter 5.

thisContext is a pseudo-variable that represents the top frame of the run-time stack. In other words, it represents the currently executing Context . **thisContext** is normally not of interest to most programmers, but it is essential for implementing development tools like the debugger and it is also used to implement exception handling and continuations.

3.3 Message sends

There are three kinds of messages in Squeak.

1. *Unary* messages take no argument. `1 factorial` sends the message `factorial` to the object `1`.

2. *Binary* messages take exactly one argument. `1 + 2` sends the message `+` with argument `2` to the object `1`.

3. *Keyword* messages take an arbitrary number of arguments. `2 raisedTo: 6 modulo: 10` sends the message consisting of the message selector `raisedTo:modulo:` and the arguments `6` and `10` to the object `2`.

Unary message selectors consist of alphanumeric characters, and start with a lower case letter.

Binary message selectors consist of one or more special characters. The following set are the most commonly used characters for binary message selectors:

```
+ - / \ * ~ < > = @ % | & ? ,
```

Keyword message selectors consist of a series of alphanumeric keywords, where each keyword starts with a lower-case letter and ends with a colon.

Unary messages have the highest precedence, then binary messages, and finally keyword messages, so:

```
2 raisedTo: 1 + 3 factorial  ⟶  128
```

(First we send `factorial` to `3`, then we send `+ 6` to `1`, and finally we send `raisedTo: 7` to `2`.) Recall that we use the notation *expression* ⟶ *result* to show the result of evaluating an expression.

Precedence aside, evaluation is strictly from left to right, so

```
1 + 2 * 3  ⟶  9
```

not `7`. Parentheses must be used to alter the order of evaluation:

```
1 + (2 * 3)  ⟶  7
```

Message sends may be composed using periods and semi-colons. A period-separated sequence of expressions causes each expression in the series to be evaluated as a *statement*, one after the other.

```
Transcript cr.
Transcript show: 'hello world'.
Transcript cr.
```

This will send cr to the Transcript object, then send it show: 'hello world', and finally send it another cr.

When a series of messages is being sent to the *same* receiver, then this can be expressed more succinctly as a *cascade*. The receiver is specified just once, and the sequence of messages is separated by semi-colons:

```
Transcript
   cr;
   show: 'hello world';
   cr.
```

This has precisely the same effect as the previous example.

3.4 Method syntax

Whereas expressions may be evaluated anywhere in Squeak (for example, in a workspace, a debugger, or a browser), methods are normally defined in a browser or debugger window. (Methods can also be filed in from an external medium, but this is not the usual way to program in Squeak.)

Programs are developed one method at a time, in the context of a given class. (A class is defined by sending a message to a class, asking it to create a subclass, so there is no special syntax required for defining classes.)

Here is the method lineCount in the class String. (The usual convention is to refer to methods as ClassName»methodName, so we call this method String»lineCount.)

Method 3.1: *Line count.*

```
1  String»lineCount
2     "Answer the number of lines represented by the receiver,
3     where every cr adds one line."
4
5     | cr count |
6     cr := Character cr.
7     count := 1 min: self size.
8     self do: [:eachChar |
9        eachChar = cr ifTrue: [count := count + 1]].
10    ^ count
```

Syntactically, a method consists of:

1. the method pattern, containing the name (*i.e.*, lineCount) and any arguments (none in this example);
2. comments (these may occur anywhere, but the convention is to put one at the top that explains what the method does);
3. declarations of temporary variables (*i.e.*, cr and count); and
4. any number of expressions separated by dots; here there are four.

The evaluation of any expression preceded by a ^ (typed as ^) will cause the method to exit at that point, returning the value of that expression. A method that terminates without explicitly returning some expression will implicitly return **self**.

Arguments and temporary variables should always start with lower case letters. Names starting with upper-case letters are assumed to be global variables. Class names, like Character, for example, are simply global variables referring to the object representing that class.

However, you are discouraged to define your own global variables since they introduce *global state* which is often considered a code smell because it is hard to keep the overview of all global variables. In most cases, best practice is to define a class variable and implement accessor methods on its class-side instead.

3.5 Block syntax

Blocks provide a mechanism to defer the evaluation of expressions. A block is essentially an anonymous function. A block is evaluated by sending it the message value. The block answers the value of the last expression in its body unless there is an explicit return (with ^), in which case it does not answer any value.

```
[1 + 2] value    ⟶    3
```

Blocks may take parameters, each of which is declared with a leading colon. A vertical bar separates the parameter declaration(s) from the body of the block. To evaluate a block with one parameter, you must send it the message value: with one argument. A two-parameter block must be sent value:value:, and so on, up to 4 arguments.

```
[:x | 1 + x] value: 2    ⟶    3
[:x :y | x + y] value: 1 value: 2    ⟶    3
```

If you have a block with more than four parameters, you must use valueWithArguments: and pass the arguments in an array. (A block with a large number of parameters is often a sign of a design problem.)

Blocks may also declare temporary variables, which are surrounded by vertical bars, just like temporary variable declarations in a method. Locals are declared after any arguments:

```
[:x :y | | z | z := x + y. z] value: 1 value: 2    ⟶    3
```

Blocks are actually lexical *closures* since they can refer to variables of the surrounding environment. The following block refers to the variable x of its enclosing environment:

```
| x |
x := 1.
[:y | x + y] value: 2    ⟶    3
```

Blocks are instances of the class BlockClosure. This means that they are objects, so they can be assigned to variables and passed as arguments just like any other object.

Returning inside blocks. Returning inside blocks acts as an escaping mechanism. Return expressions inside a nested block expression will terminate the lexically enclosing method. In the following example (see method 3.2), when the expression ^ foundBlock value: element is executed, the method detect:ifFound:ifNone: escapes the current iteration and returns it.

Method 3.2: *The method* detect:ifFound:ifNone: *illustrating how returning inside blocks can be used.*

```
1 Collection»detect: aBlock ifFound: foundBlock ifNone: exceptionBlock
2   "foundBlock takes one argument, the found object."
3
4   self do: [:element |
5     (aBlock value: element) ifTrue: [^ foundBlock value: element]].
6   ^ exceptionBlock value
```

3.6 Conditionals and loops in a nutshell

Smalltalk offers no special syntax for control constructs. Instead, these are typically expressed by sending messages to booleans, numbers, and collections, with blocks as arguments.

Conditionals are expressed by sending one of the messages ifTrue:, ifFalse: or ifTrue:ifFalse: to the result of a boolean expression. See Chapter 9 for more about booleans.

```
17 * 13 > 220
  ifTrue: ['bigger']
  ifFalse: ['not bigger']    ⟶    'bigger'.
```

Loops are typically expressed by sending messages to blocks, integers, or collections. Since the exit condition for a loop may be repeatedly evaluated, it should be a block rather than a boolean value. Here is an example of a very procedural loop:

```
n := 1.
[n < 1000] whileTrue: [n := n * 2].
n    ⟶    1024
```

whileFalse: reverses the exit condition.

```
n := 1.
[n > 1000] whileFalse: [n := n * 2].
n    ⟶    1024
```

timesRepeat: offers a simple way to implement a fixed iteration:

```
n := 1.
10 timesRepeat: [n := n * 2].
n    ⟶    1024
```

We can also send the message to:do: to a number which then acts as the initial value of a loop counter. The two arguments are the upper bound, and a block that takes the current value of the loop counter as its argument:

```
result := String new.
1 to: 10 do: [:n | result := result , n , ' '].
result    ⟶    '1 2 3 4 5 6 7 8 9 10 '
```

Higher-order iterators. Collections comprise a large number of different classes, many of which support the same protocol. The most important messages for iterating over collections include do:, collect:, select:, reject:, detect:, and inject:into:. These messages define high-level iterators that allow one to write very compact code.

An Interval is a collection that lets one iterate over a sequence of numbers from the starting point to the end. 1 to: 10 represents the interval from 1 to 10. Since it is a collection, we can send the message do: to it. The argument is a block that is evaluated for each element of the collection.

```
result := String new.
(1 to: 10) do: [:n | result := result , n , ' '].
result    ⟶    '1 2 3 4 5 6 7 8 9 10 '
```

collect: builds a new collection of the same size, transforming each element.

```
(1 to: 10) collect: [:each | each * each]    ⟶    #(1 4 9 16 25 36 49 64 81 100)
```

select: and reject: build new collections, each containing a subset of the elements satisfying (or not) the boolean block condition. detect: returns the first element satisfying the condition. Don't forget that strings are also collections, so you can iterate over all the characters.

```
'hello there' select: [:char | char isVowel]    ⟶    'eoee'
'hello there' reject: [:char | char isVowel]    ⟶    'hll thr'
'hello there' detect: [:char | char isVowel]    ⟶    $e
```

Finally, you should be aware that collections also support a functional-style generic *fold* message called inject:into:. This lets you generate a cumulative result using an expression that starts with a seed value and injects each element of the collection. Sums and products are typical examples.

```
(1 to: 10) inject: 0 into: [:sum :each | sum + each]    ⟶    55
```

This is equivalent to 0+1+2+3+4+5+6+7+8+9+10.

More about collections can be found in Chapter 10.

3.7 Primitives and pragmas

In Smalltalk everything is an object, and everything happens by sending messages. Nevertheless, at certain points, we hit rock bottom. Certain objects can only get work done by invoking virtual machine primitives.

For example, the following are all implemented as primitives: memory allocation (new, new:), bit manipulation (bitAnd:, bitOr:, bitShift:), pointer and integer arithmetic (+, -, <, >, *, / , =, ==...), and array access (at:, at:put:).

Primitives are invoked with the syntax <primitive: aNumber>. A method that invokes such a primitive may also include Smalltalk code, which will be evaluated *only* if the primitive fails.

Here we see the code for SmallInteger»+. If the primitive fails, the expression **super** + aNumber will be evaluated and returned.

Method 3.3: *A primitive method.*

```
+ aNumber
  "Primitive. Add the receiver to the argument and answer with the result
  if it is a SmallInteger. Fail if the argument or the result is not a
  SmallInteger  Essential  No Lookup. See Object documentation whatIsAPrimitive."

  <primitive: 1>
  ^ super + aNumber
```

The angle bracket syntax is also used for method annotations called pragmas. A pragma basically looks like a message, it consists of a selector symbol, filled up with argument values if the selector has keyword arguments. Pragma selectors can be unary (`<generated>`) or keyword (`generatedBy: 'compiler' level: 42`) but not binary selectors. Argument values must be constant literals such as symbols, numbers, or arrays, but you must not evaluate any code in a pragma value. Pragmas don't have any effect on the method by themselves, but other objects can read and handle those pragmas to perform arbitrary actions such as querying methods or modifying the way a method is compiled when you accept it.

3.8 Chapter summary

- Squeak has (only) six reserved identifiers also called *pseudo-variables*: **true**, **false**, **nil**, **self**, **super**, and **thisContext**.
- There are five kinds of literal objects: numbers (`5`, `2.5`, `1.9e15`, `2r111`), characters (`$a`), strings (`'hello'`), symbols (`#hello`), and arrays (`#('hello' #hello)`)
- Strings are delimited by single quotes, comments by double quotes. To get a quote inside a string, double it.
- Unlike strings, symbols are guaranteed to be globally unique.
- Use `#( ... )` to define a literal array. Use `{ ... }` to define an array dynamically. Note that `#(1 + 2) size` ⟶ `3`, but `{1 + 2} size` ⟶ `1`
- There are three kinds of messages: *unary* (*e.g.*, `1 asString`, `Array new`), *binary* (*e.g.*, `3 + 4`, `'hi' , ' there'`), and *keyword* (*e.g.*, `'hi' at: 2 put: $o`)
- A *cascaded* message send is a sequence of messages sent to the same target, separated by semi-colons: `OrderedCollection new add: #calvin; add: #hobbes; size` ⟶ `2`

- Temporary variables are declared with vertical bars. Use := for assignment. `| x | x := 1`
- Expressions consist of message sends, cascades and assignments, possibly grouped with parentheses. *Statements* are expressions separated by periods.
- Block closures are expressions enclosed in square brackets. Blocks may take arguments and can contain temporary variables. The expressions in the block are not evaluated until you send the block a `value...` message with the correct number of arguments.
  `[:x | x + 2] value: 4` ⟶ `6.`
- There is no dedicated syntax for control constructs, just messages that conditionally evaluate blocks.
  `(Smalltalk includes: Class) ifTrue: [Transcript show: Class superclass]`

Chapter 4

Understanding message syntax

Although Smalltalk's message syntax is very simple, it is unconventional and it can take some time to get used to it. This chapter offers some guidance to help you get accustomed to this special syntax for sending messages. If you already feel comfortable with the syntax, you may choose to skip this chapter or come back to it later.

4.1 Identifying messages

In Smalltalk, except for the syntactic elements listed in Chapter 3 (:= ^ . ; # () {} [: |]), everything is a message send. Even operations such as + are message sends and you can implement methods for these operations in your classes. At the same time, the arity of a method is set. "-" is always a binary message; there is no way to have a unary "-" with different overloading.

In Smalltalk the order in which messages are sent is determined by the kind of message. There are just three kinds of messages: *unary*, *binary*, and *keyword* messages. Unary messages are always sent first, then binary messages and finally keyword ones. As in most languages, parentheses can be used to change the order of evaluation. These rules make Smalltalk code as easy to read as possible. We discuss the rules for evaluating messages in detail in Section 4.2.

As most computation in Smalltalk is done by message passing, correctly identifying messages and their evaluation order is crucial. The following terminology will help us:

- A message is composed of the message *selector* and the optional message arguments.
- A message is sent to a *receiver*.
- The combination of a message and its receiver is called a *message send* as shown in Figure 4.1.

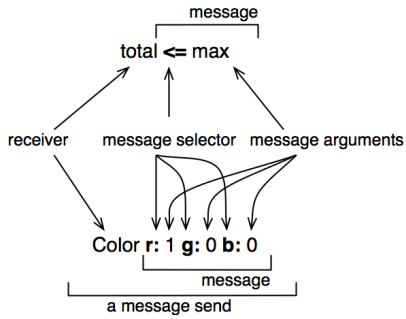


Figure 4.1: Two messages composed of a receiver, a method selector, and a set of arguments.

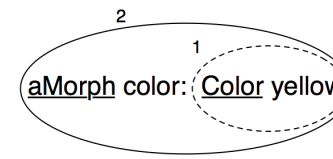


Figure 4.2: aMorph color: Color yellow is composed of two message sends: Color yellow and aMorph color: Color yellow.

> A message is always sent to a receiver, which can be a single literal, a block or a variable or the result of evaluating another message.

To help you identify the receiver of a message send, we will underline it for you in the following examples. We will also surround each message send with an ellipse and number message sends starting from the first one that will be sent to help you see the order in which messages are sent.

Figure 4.2 represents two message sends, Color yellow and aMorph color: Color yellow, hence there are two ellipses. The message send Color yellow is executed first so its ellipse is numbered 1. There are two receivers: aMorph which receives the message color: ... and Color which receives the message yellow. Both receivers are underlined.

A receiver can be the first element of a message, such as 100 in the message send 100 + 200 or Color in the message send Color yellow. However, a receiver can also be the result of other messages. For example in the message Pen new go: 100, the receiver of the message go: 100 is the object

Message send	Message type	Result
`Color yellow`	unary	Creates a color.
`aPen go: 100`	keyword	The receiving pen moves forward 100 pixels.
`100 + 20`	binary	The number 100 receives the message + with the number 20.
`Pen new go: 100`	unary and keyword	A pen is created and moved 100 pixels.
`aPen go: 100 + 20`	keyword and binary	The receiving pen moves forward 120 pixels.

Table 4.1: Examples of message sends and their types.

returned by the message send `Pen new`. In all the cases, a message is sent to an object called the *receiver* which may be the result of another message send.

4.2 Three kinds of messages

Smalltalk defines a few simple rules to determine the order in which the messages are sent. These rules are based on the distinction between 3 different kinds of messages:

- *Unary messages* are messages that are sent to an object without arguments. For example in `3 factorial`, `factorial` is a unary message.
- *Binary messages* are messages consisting of operators (often arithmetic). They are binary because they always involve only two objects: the receiver and the argument object. For example in `10 + 20`, `+` is a binary message sent to the receiver `10` with argument `20`.
- *Keyword messages* are messages consisting of one or more keywords, each ending with a colon (:) and taking an argument. For example in `anArray at: 1 put: 10`, the keyword `at:` takes the argument `1` and the keyword `put:` takes the argument `10`.

Table 4.1 shows several examples of simple message sends and composed ones. `Color yellow` and `100 + 20` are simple: A message is sent to an object, while the message send `aPen go: 100 + 20` is composed of two messages: `+ 20` is sent to `100` and `go:` is sent to `aPen` with the argument being

the result of the first message. A receiver can be an expression (such as an assignment, a message send, or a literal) which returns an object. In Pen new go: 100, the message go: 100 is sent to the object that results from the execution of the message send Pen new. We will now take a closer look at the three kinds of messages.

Unary messages

Unary messages are messages that do not require any argument. They follow the syntactic template: receiver messageName. The selector is simply made up of a succession of characters not containing : (*e.g.*, factorial, open, class).

```
89 sin          ⟶   0.8600694058124533
3 sqrt          ⟶   1.7320508075688772
Float pi        ⟶   3.141592653589793
'blop' size     ⟶   4
true not        ⟶   false
Object class    ⟶   Object class  "The class of Object is Object class (!)"
```

> Unary messages are messages that do not require any argument.
> They follow the syntactic template: receiver **selector**

Binary messages

Binary messages are messages that require exactly one argument *and* whose selector consists of a sequence of one or more characters from the set: +, -, *, /, &, =, >, |, <, ∼, and @. Note that -- is not allowed for parsing reasons.

```
100 @ 100          ⟶   100@100  "creates a Point object"
3 + 4              ⟶   7
10 - 1             ⟶   9
4 <= 3             ⟶   false
(4 / 3) * 3 = 4    ⟶   true  "equality is a binary message; Fractions are exact"
(3 / 4) == (3 / 4) ⟶   false  "two equal Fractions are not the same object"
```

> Binary messages are messages that require exactly one argument *and* whose selector is composed of a sequence of characters from: +, -, *, /, &, =, >, |, <, ∼, and @. -- is not possible.
> They follow the syntactic template: receiver **selector** argument

Keyword messages

Keyword messages are messages that require one or more arguments and whose selector consists of one or more keywords each ending in :. Keyword messages follow the syntactic template: receiver **selectorWordOne:** argumentOne **wordTwo:** argumentTwo

Each keyword takes an argument. Hence r:g:b: is a method with three arguments, playFileNamed: and at: are both methods with one argument, and at:put: is a method with two arguments. To create an instance of the class Color one can use the method r:g:b: as in Color r: 1 g: 0 b: 0, which creates the color red. Note that the colons are part of the selector.

> In Java-style syntax, the Smalltalk method invocation Color r: 1 g: 0 b: 0 would be written Color.rgb(1,0,0).

```
1 to: 10                    ⟶   (1 to: 10)  "creates an interval"
Color r: 1 g: 0 b: 0        ⟶   Color red  "creates a new color"
12 between: 8 and: 15       ⟶   true

nums := Array newFrom: (1 to: 5).
nums at: 1 put: 6.
nums    ⟶    #(6 2 3 4 5)
```

> Keyword based messages are messages that require one or more arguments. Their selector consists of one or more keywords each ending in a colon (:). They follow the syntactic template:
> receiver **selectorWordOne:** argumentOne **wordTwo:** argumentTwo

4.3 Message composition

The three kinds of messages each have different precedence, which allows them to be composed in an elegant way.

1. Unary messages are always sent first, then binary messages and finally keyword messages.
2. Messages in parentheses are sent prior to any kind of message.
3. Messages of the same kind are evaluated from left to right.

These rules lead to a very natural reading order. Now, if you want to be sure that your messages are sent in the order that you want you can always put more parentheses as shown in Figure 4.3. In this figure, the message yellow is a unary message and the message color: a keyword message, therefore the message send Color yellow is sent first. However as message sends in parentheses are sent first, putting (unnecessary) parentheses around Color yellow just emphasizes that it will be sent first. The rest of the section illustrates each of these points.

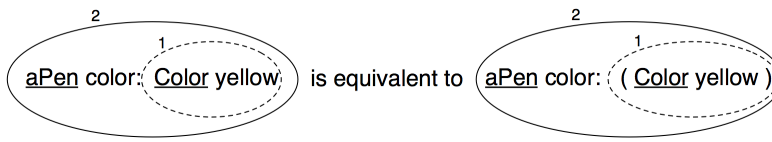


Figure 4.3: Unary messages are sent first so Color yellow is sent first. This returns a Color object which is passed as an argument of the message aPen color:.

Unary > Binary > Keywords

Unary messages are sent first, then binary messages, and finally, keyword messages. We also say that unary messages have a higher priority over other kinds of messages.

> **Rule One.** Unary messages are sent first, then binary messages, and finally keyword-based messages.
> Unary > Binary > Keyword

As the following examples show, Smalltalk's syntax rules generally ensure that message sends can be read in a natural way:

```
1000 factorial / 999 factorial    ⟶    1000
2 raisedTo: 1 + 3 factorial       ⟶    128
```

Unfortunately, the rules are a bit too simplistic for arithmetic message sends, so you need to introduce parentheses whenever you want to impose a priority over binary operators:

```
1 + 2 * 3      ⟶    9
1 + (2 * 3)    ⟶    7
```

The following example, which is a bit more complex (!), offers a nice illustration that even complicated Smalltalk expressions can be read naturally:

```
[:aClass | aClass methodDict keys sorted select: [:aMethod | (aClass>>aMethod)
      isAbstract]] value: Boolean    ⟶    #(#& #==> #and: #asBit #ifFalse:
      #ifFalse:ifTrue: #ifTrue: #ifTrue:ifFalse: #not #or: #|)
```

Here we want to know which methods of the Boolean class are abstract. We ask some argument class, aClass, for the keys of its method dictionary, sort them, and select those methods of that class that are abstract. Then we bind the argument aClass to the concrete value Boolean. We need parentheses only to send the binary message >>, which selects a method from a class, before sending the unary message isAbstract to that method The result shows us which methods must be implemented by Boolean's concrete subclasses True and False.

Example. In the message aPen color: Color yellow, there is one *unary* message yellow sent to the class Color and a *keyword* message color: sent to aPen. Unary messages are sent first so the message send Color yellow is sent (1). This returns a Color object, which is used as an argument in the message send aPen color: aColor (2) as shown in example 4.1. Figure 4.3 shows graphically how messages are sent.

Example 4.1: *Decomposing the evaluation of* aPen color: Color yellow.

```
    aPen color: Color yellow.
(1)             Color yellow      "unary message is sent first"
                ⟶    aColor
(2) aPen color: aColor            "keyword message is sent next"
```

Example. Example 4.2 shows an example of a combination of a keyword and binary message send and how it is executed. In the message send aPen go: 100 + 20, there is a *binary* message + 20 and a *keyword* message go:. Binary messages are sent prior to keyword messages, so 100 + 20 is sent first (1): The message + 20 is sent to the object 100 and returns the number 120. Then the message send aPen go: 120 is sent with 120 as argument (2).

Example 4.2: *Decomposing* aPen go: 100 + 20.

```
    aPen go: 100 + 20.
(1)          100 + 20       "binary message first"
             ⟶    120
(2) aPen go: 120            "then keyword message"
```

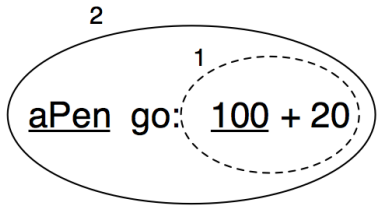


Figure 4.4: Binary messages are sent before keyword messages.

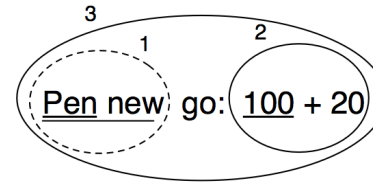


Figure 4.5: Decomposing Pen new go: 100 + 20.

Example. As an exercise, we let you decompose the evaluation of the message send Pen new go: 100 + 20, which is composed of one unary, one keyword, and one binary message (see Figure 4.5).

Parentheses first

> **Rule Two.** Parenthesised messages are sent prior to other messages.
>
> (Msg) > Unary > Binary > Keyword

```
1.5 tan rounded asString = (((1.5 tan) rounded) asString)    ⟶    true
            "parentheses not needed for the sends above"
3 + 4 factorial    ⟶    27    "not 5040"
(3 + 4) factorial    ⟶    5040
```

In the following example, we need the parentheses to force sending lowMajorScaleOn: before play.

```
(FMSound lowMajorScaleOn: FMSound clarinet) play
"(1) send the message clarinet to the FMSound class to create a clarinet sound.
 (2) send this sound to FMSound as an argument to the lowMajorScaleOn: keyword
      message.
 (3) play the resulting sound."
```

Example. The message (65 @ 325 extent: 134 @ 100) center returns the center of a rectangle whose top-left point is $(65, 325)$ and whose size is 134×100. Example 4.3 shows how the message is decomposed and sent. First, the message between parentheses is sent: It contains two binary messages 65 @ 325 and 134 @ 100 that are sent first and return points, and a keyword message extent: which is then sent and returns a rectangle. Finally, the unary

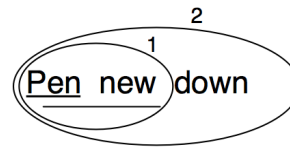


Figure 4.6: Decomposing Pen new down.

message center is sent to the rectangle and a point is returned. Evaluating the message without parentheses would lead to an error because the object 100 does not understand the message center.

Example 4.3: *Example of Parentheses.*

```
    (65 @ 325 extent: 134 @ 100) center.
(1) 65 @ 325                                  "binary"
      ⟶   aPoint
(2)                  134 @ 100                "binary"
                     ⟶   anotherPoint
(3) aPoint extent: anotherPoint               "keyword"
      ⟶   aRectangle
(4) aRectangle center                         "unary"
      ⟶   132 @ 375
```

From left to right

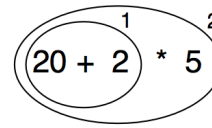
Now we know how messages of different kinds or priorities are handled. The final question to be addressed is how messages with the same priority are sent: The answer is *from the left to the right*. Note that you already saw this behavior in example 4.3 where the two point creation messages (@) were sent first.

> **Rule Three.** When the messages are of the same kind, the order of evaluation is from left to right.

Example. In the message sends Pen new down all messages are unary messages, so the leftmost one, Pen new, is sent first. This returns a newly created pen to which the second message down is sent, as shown in Figure 4.6.

The absence of arithmetic precedence

The message composition rules are simple, but, as they apply to all message sends, might not always match your expectations. In particular, arithmetic



message sends, such as + and * are also subject to the same set of rules. Thus, there is no special precedence between these message sends. Here, we see the common situations where extra parentheses are needed:

```
3 + 4 * 5          ⟶   35   "(not 23)  Binary messages sent from left to right"
3 + (4 * 5)        ⟶   23
1 + 1 / 3          ⟶   (2/3)   "not 4/3"
1 + (1 / 3)        ⟶   (4/3)
1 / 3 + 2 / 3      ⟶   (7/9)   "not 1"
(1 / 3) + (2 / 3)  ⟶   1
```

Example. In the message sends 20 + 2 * 5, there are only binary messages + and *. However, in Smalltalk there is no specific priority for the operations + and *. They are just binary messages, hence * does not have priority over +. So, the leftmost message + is sent first (1) and then the * is sent to the result as shown in example 4.4.

Example 4.4: *Decomposing* 20 + 2 * 5.

```
"As there is no priority among binary messages, the leftmost message + is evaluated
      first even if by the rules of arithmetic the * should be sent first."

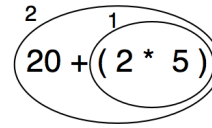
    20 + 2 * 5.
(1) 20 + 2    ⟶   22
(2) 22    * 5 ⟶   110
```

As shown in example 4.4, the result of this message send is not 30 but 110. Even though the result may be unexpected, it follows directly from the rules for the ordering of message sends. This is somehow the price to pay for the simplicity of the Smalltalk model. To get the correct result, we should use parentheses. When messages are enclosed in parentheses, they are evaluated first. Hence the message send 20 + (2 * 5) returns the result as shown in example 4.5.

Example 4.5: *Decomposing* 20 + (2 * 5).

```
"The messages surrounded by parentheses are evaluated first. Therefore * is sent
      prior to +, which produces the correct behavior."

  20 + (2 * 5).
```



Implicit precedence	Explicitly parenthesized equivalent
aPen color: Color yellow	aPen color: (Color yellow)
aPen go: 100 + 20	aPen go: (100 + 20)
aPen penSize: aPen penSize + 2	aPen penSize: ((aPen penSize) + 2)
2 factorial + 4	(2 factorial) + 4

Figure 4.7: Message sends and their fully parenthesized equivalents.

```
(1)      (2 * 5)   ⟶   10
(2) 20 + 10        ⟶   30
```

> In Smalltalk, arithmetic operators such as + and * do not have different priorities. + and * are just binary messages, therefore * does not have priority over +. Use parentheses to obtain the desired result.

Note that the first rule stating that unary messages are sent prior to binary and keyword messages avoids the need to put explicit parentheses around them. Table 4.7 shows message sends written following the rules and equivalent message sends if the rules would not exist. Both message sends result in the same effect or return the same value.

4.4 Hints for identifying keyword messages

Often beginners face the challenge of when they need to add parentheses. Let's see how keywords messages are recognized by the compiler.

Parentheses or not?

The characters [] and () delimit distinct areas. Whatever is in these areas is evaluated before the result is used in any other evaluation, as you have seen in examples of arithmetic messages. Further, parentheses are also necessary to keep the parts of nested keyword messages apart.

In the following example, there are two distinct keyword messages: includes: and ifTrue:. Without the parenthesis we would actually send the unknown message includes:ifTrue: to the collection. By adding parenthesis we make it clear where one keyword message ends and where another one starts.

```
ord := OrderedCollection new.
(ord includes: $a) ifTrue: [...]
```

The same applies to the following example, in which we use the result of the keyword message send rotateBy:magnify:smoothing: as an argument in the keyword message send at:put:.

```
aDict
  at: (rotatingForm
        rotateBy: angle
        magnify: 2
        smoothing: 1)
  put: 3.
```

Hints. If you have problems with these precedence rules, you may start simply by putting parentheses whenever you want to distinguish two messages having the same precedence. To keep your code short and clear you should however try to use them less often when you become more experienced.

When messages have different precedence, you do not have to use parentheses. For example, the following piece of code does not require parentheses because the message send x isNil is unary, hence it is always sent prior to the keyword message ifTrue:.

```
x isNil ifTrue: [...]
```

When to use [ ] or ( )

Another challenge you might face is understanding the difference between square brackets and parentheses. The basic principle is that you should use [ ] when you do not know how many times, potentially zero, an expression should be evaluated. [*expression*] will create a block closure (*i.e.*, an object) from *expression* which may be evaluated any number of times (possibly zero), depending on the context. The expression can be any code.

Hence the conditional branches of ifTrue: or ifTrue:ifFalse: require blocks, as depending on the condition, the code after the ifTrue: or after the ifFalse:

might be executed. Following the same principle, both the receiver and the argument of a whileTrue: message require the use of square brackets since we do not know how many times either the receiver or the argument should be evaluated.

Parentheses, on the other hand, only affect the order of sending messages. So in (*expression*), the *expression* will *always* be evaluated exactly once.

```
"both the receiver and the argument must be blocks"
[x isReady] whileTrue: [y doSomething].
"the argument is evaluated more than once, so must be a block"
4 timesRepeat: [Beeper beep].
"the receiver is evaluated once, so it is not a block"
(x isReady) ifTrue: [y doSomething].
```

4.5 Expression sequences

Expressions (*i.e.*, messages sends, assignments...) separated by periods are evaluated in sequence. Note that there is no period between a variable definition and the following expression. The value of a sequence is the value of the last expression. The values returned by all the expressions except the last one are ignored. Note that the period is a separator and not a terminator. Therefore a final period is optional.

```
| box |
box := 20 @ 30 corner: 60 @ 90.
box containsPoint: 40 @ 50    ⟶    true
```

4.6 Cascaded messages

Smalltalk offers a way to send multiple messages to the same receiver using a semicolon (;). This is called the *cascade* in Smalltalk jargon.

Expression msg1; msg2

```
Transcript show: 'Squeak is '.
Transcript show: 'fun '.
Transcript cr.
```

is equivalent to:

```
Transcript
  show: 'Squeak is';
  show: 'fun ';
  cr.
```

Note that the object receiving the cascaded messages can itself be the result of a message send. In fact, the receiver of all the cascaded messages is the receiver of the first message involved in a cascade. In the following example, the first cascaded message is setX:setY since it is followed by a cascade. The receiver of the cascaded message setX:setY: is the newly created point resulting from the evaluation of Point new, and *not* Point. The subsequent message isZero is sent to that same receiver.

```
Point new setX: 25 setY: 35; isZero  ⟶  false
```

4.7 Chapter summary

- A message is always sent to an object named the *receiver* which may be the result of other message sends.
- Unary messages are messages that do not require any argument. They are of the form of receiver **selector**.
- Binary messages are messages that involve two objects, the receiver and another object *and* whose selector is composed of one or more characters from the following list: +, -, *, /, |, &, =, >, <, ~, and @. They are of the form: receiver **selector** argument
- Keyword messages are messages that involve more than one object and that contain at least one colon character (:).
  They are of the form: receiver **selectorWordOne:** argumentOne **wordTwo:** argumentTwo
- **Rule One.** Unary messages are sent first, then binary messages, and finally keyword messages.
- **Rule Two.** Messages in parentheses are sent before any others.
- **Rule Three.** When the messages are of the same kind, the order of evaluation is from left to right.
- In Smalltalk, traditional arithmetic operators such as + and * have the same priority. + and * are just binary messages, therefore * does not have priority over +. You must use parentheses to obtain a different result.

Part II

Developing in Squeak

Chapter 5

The Smalltalk object model

Smalltalk's programming model is simple and uniform: Everything is an object, and objects communicate only by sending each other messages. However, this simplicity and uniformity can be a source of difficulty for programmers used to other languages. In this chapter, we present the core concepts of the Smalltalk object model; in particular we discuss the consequences of representing classes as objects.

5.1 The rules of the model

The Smalltalk object model is based on a set of simple rules that are applied *uniformly*. The rules are as follows:

Rule 1. Everything is an object.

Rule 2. Every object is an instance of a class.

Rule 3. Every class has a superclass.

Rule 4. Everything happens by message sends.

Rule 5. Method lookup follows the inheritance chain.

Let us look at each of these rules in some detail.

5.2 Everything is an object

The mantra "everything is an object" is highly contagious. After only a short while working with Smalltalk, you will start to be surprised at how this rule simplifies everything you do. Integers, for example, are true objects, so you can send messages to them, just as you do to any other object.

```
3 + 4          ⟶   7    "send '+ 4' to 3, yielding 7"
20 factorial   ⟶   2432902008176640000   "send factorial, yielding a big number"
```

The representation of a number as large as 20 factorial is different from the representation of a number as small as 7, but because they are both objects, none of the code — not even the implementation of factorial — needs to know about this.

Perhaps the most fundamental consequence of this rule is the following:

> Classes are objects too.

Furthermore, classes are not second-class objects: they are really first-class objects that you can send messages to, inspect, and so on. This means that Squeak is a truly reflective system, which gives a great deal of expressive power to developers.

Deep in the implementation of Smalltalk, there are three different kinds of objects. There are (1) ordinary objects with instance variables that are passed by references, there are (2) *immediate objects* such as SmallInteger s that are passed by value, and there are (3) indexable objects like arrays that hold a contiguous portion of memory. The beauty of Smalltalk is that you normally do not need to care about the differences between these three kinds of objects.

5.3 Every object is an instance of a class

Every object has a class; you can find out which by sending it the message class.

```
1 class              ⟶   SmallInteger
20 factorial class   ⟶   LargePositiveInteger
'hello' class        ⟶   ByteString
#(1 2 3) class       ⟶   Array
(4 @ 5) class        ⟶   Point
```

```
Object new class  ⟶  Object
```

A class defines the *structure* of its instances via instance variables, and the *behavior* of its instances via methods. Each method has a name, called its *selector*, which is unique within the class.

Since *classes are objects*, and *every object is an instance of a class*, it follows that classes must also be instances of classes. A class whose instances are classes is called a *metaclass*. Whenever you create a class, the system automatically creates a metaclass for you. The metaclass defines the structure and behavior of the class that is its instance. Most of the time you will not need to think about metaclasses, and may happily ignore them. (We will have a closer look at metaclasses in Chapter 13.)

Instance variables

Instance variables in Smalltalk are private to the *instance* itself. This is in contrast to Java and C++, which allow instance variables (also known as "fields" or "member variables") to be accessed by any other instance that happens to be of the same class. We say that the *encapsulation boundary* of objects in Java and C++ is the class, whereas in Smalltalk it is the instance.

In Smalltalk, two instances of the same class cannot access each other's instance variables unless the class defines "accessor methods". There is no language syntax that provides direct access to the instance variables of any other object. (Actually, a mechanism called reflection does provide a way to ask another object for the values of its instance variables; metaprogramming is intended for writing tools like the object inspector, whose sole purse is to look inside other objects.)

Instance variables can be accessed by name in any of the instance methods of the class that defines them, and also in the methods defined in its subclasses. This means that Smalltalk instance variables are similar to *protected* variables in C++ and Java. However, we prefer to say that they are private because it is considered bad style in Smalltalk to access an instance variable directly from a subclass.

Example

Method `Point»dist:` (see Method 5.1) computes the distance between the receiver and another point. The instance variables `x` and `y` of the receiver are accessed directly by the method body. However, the instance variables of the other point must be accessed by sending it the messages `x` and `y`.

Method 5.1: *The distance between two points.*

```
Point»dist: aPoint
    "Answer the distance between aPoint and the receiver."

    | dx dy |
    dx := aPoint x - x.
    dy :=  aPoint y - y.
    ^ ((dx * dx) + (dy * dy)) sqrt
```

```
1 @ 1 dist: 4 @ 5    ⟶    5
```

The key reason to prefer instance-based encapsulation to class-based encapsulation is that it enables different implementations of the same abstraction to coexist. For example, method Point»dist:, need not know or care whether the argument aPoint is an instance of the same class as the receiver. The argument object might be represented in polar coordinates, or as a record in a database, or on another computer in a distributed system; as long as it can respond to the messages x and y, the code in Method 5.1 will still work.

Methods

All methods are public. Methods are grouped into categories that indicate their intent. Some common category names have been established by convention, for example, *accessing* for all accessor methods, and *initialization* for establishing a consistent initial state for the object. The protocol *private* is sometimes used to group methods that should not be seen from outside. Nothing, however, prevents you from sending a message that is implemented by such a "private" method.

Methods can access all instance variables of the object. Some Smalltalk developers prefer to access instance variables only through their accessors. This practice has some value, but it also clutters the interface of your classes, and worse, exposes some private state to the world.

The instance side and the class side

Since classes are objects, they can have their own instance variables and their own methods. We call these *class instance variables* and *class methods*, but they are really no different from ordinary instance variables and methods: Class instance variables are just instance variables defined by a metaclass, and class methods are just methods defined by a metaclass.

A class and its metaclass are two separate classes, even though the former is an instance of the latter. However, this is largely irrelevant to you as a programmer: You are concerned with defining the behavior of your objects and the classes that create them.

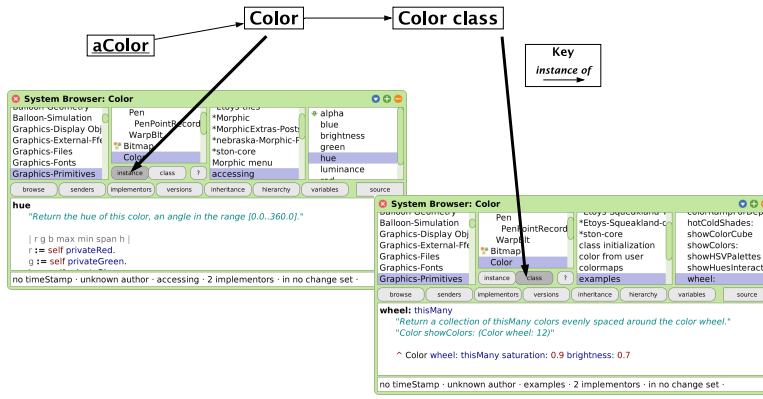


Figure 5.1: Browsing a class and its metaclass.

For this reason, the browser helps you to browse both class and metaclass as if they were a single thing with two "sides": the "instance side" and the "class side", as shown in Figure 5.1. Clicking on the instance button browses the class Color, *i.e.*, you browse the methods that are executed when messages are sent to an instance of Color, like the color blue. Pressing the class button browses the class Color class, *i.e.*, you see the methods that will be executed when messages are sent to the class Color itself. For example, Color blue sends the message blue to the class Color. You will therefore find the method blue defined on the class side of Color, not on the instance side.

```
aColor := Color blue.          "Class side method blue"
aColor      ⟶   Color blue
aColor red  ⟶   0.0     "Instance side accessor method red"
aColor blue ⟶   1.0     "Instance side accessor method blue"
```

You define a class by filling in the template proposed on the instance side. When you accept this template, the system creates not just the class that you defined, but also the corresponding metaclass. You can browse the metaclass by clicking on the class button. The only part of the metaclass creation template that makes sense for you to edit directly is the list of instance variable names.

Once a class has been created, clicking the instance button lets you edit and browse the methods that will be possessed by instances of that class

(and of its subclasses). For example, we can see in Figure 5.1 that the method hue is defined on instances of the class Color. In contrast, the class button lets you browse and edit the metaclass (in this case Color class).

Class methods

Class methods can be quite useful; browse Color class for some good examples. You will see that there are two kinds of method defined on a class: those that create instances of the class, like Color class»blue and those that perform a utility function, like Color class»showColorCube. This is typical, although you will occasionally find class methods used in other ways.

It is convenient to place utility methods on the class side because they can be executed without having to create any additional objects first. Indeed, many of them will contain a comment designed to make it easy to execute them.

Browse method Color class»showColorCube*, double-click just inside the quotes on the comment "Color showColorCube" and type* CMD−*d.*

You will see the effect of executing this method directly within the world. (Select World ▷ restore display (r) to undo the effects.)

For those familiar with Java and C++, class methods may seem similar to static methods. However, the uniformity of Smalltalk means that they are somewhat different: Whereas Java static methods are really just statically-resolved procedures, Smalltalk class methods are dynamically-dispatched methods. This means that inheritance, overriding, and super-sends work for class methods in Smalltalk, whereas they do not work for static methods in Java.

Class instance variables

With ordinary instance variables, all the instances of a class have the same set of variable names, and the instances of its subclasses inherit those names. However, each instance has its own private set of values. The story is exactly the same with class instance variables: each class has its own private class instance variables. A subclass will inherit those class instance variables, *but it has its own private copies of those variables*. Just as objects do not share instance variables, neither do classes and their subclasses share class instance variables.

You could use a class instance variable called count to keep track of how many instances you create of a given class. However, any subclass

would have its own count variable, so subclass instances would be counted separately.

Example: Class instance variables are not shared with subclasses. Suppose we define classes Dog and Hyena, where Hyena inherits the class instance variable count from Dog.

Class 5.2: *Dogs and Hyenas.*

```
Object subclass: #Dog
    instanceVariableNames: ''
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-CIV'

Dog class
    instanceVariableNames: 'count'

Dog subclass: #Hyena
    instanceVariableNames: ''
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-CIV'
```

Now suppose we define class methods for Dog to initialize its count to 0, and to increment it when new instances are created:

Methods 5.3: *Keeping count of new dogs.*

```
Dog class»initialize

    super initialize.
    count := 0.


Dog class»new

    count := count +1.
    ^ super new


Dog class»count

    ^ count
```

Now when we create a new Dog its count is incremented, and so is that of every Hyena, but they are counted separately:

```
Dog initialize.
Hyena initialize.
Dog count      ⟶  0
Hyena count    ⟶  0
Dog new.
Dog count      ⟶  1
Dog new.
Dog count      ⟶  2
Hyena new.
Hyena count    ⟶  1
```

Note also that class instance variables are private to a class in exactly the same way that instance variables are private to the instance. Since classes and their instances are different objects, this has the following immediate consequences:

> A class does not have access to the instance variables of its own instances.

> An instance of a class does not have access to the class instance variables of its class.

For this reason, instance initialization methods must always be defined on the instance side —the class side has no access to instance variables, so it cannot initialize them! All that the class can do is to send initialization messages, possibly using accessors, to newly created instances.

Similarly, instances can only access class instance variables indirectly, by sending accessor messages to their class.

Java has nothing equivalent to class instance variables. Java and C++ static variables are more like Smalltalk class variables, which we will discuss in Section 5.7: all of the subclasses and all of their instances share the same static variable.

Example: Defining a Singleton. The Singleton pattern[1] provides a typical example of the use of class instance variables and class methods. Imagine that we would like to implement a class SBEWebServer and use the Singleton pattern to ensure that it has only one instance.

[1] Sherman R. Alpert, Kyle Brown and Bobby Woolf, *The Design Patterns Smalltalk Companion*. Addison Wesley, 1998, ISBN 0–201–18462–1.

Clicking on the instance button in the browser, we define the class SBEWebServer as follows (class 5.4).

Class 5.4: *A singleton class.*

```
Object subclass: #SBEWebServer
    instanceVariableNames: 'sessions'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'Web'
```

Then, we click on the class button to switch to the class side and add the instance variable uniqueInstance there.

Class 5.5: *The class side of the singleton class.*

```
SBEWebServer class
    instanceVariableNames: 'uniqueInstance'
```

The consequence of this is that the class SBEWebServer now has another instance variable, in addition to the variables that it inherits, such as superclass and methodDict.

We can now define a class method named uniqueInstance as shown in method 5.6. This method first checks whether uniqueInstance has been initialized. If it has not, the method creates an instance and assigns it to the class instance variable uniqueInstance. We use a **super** send, as we have forbidden the use of new for our web server class. Finally the value of uniqueInstance is returned. Since uniqueInstance is a class instance variable, this method can directly access it.

Method 5.6: *uniqueInstance and new (on the class side).*

```
SBEWebServer class»uniqueInstance

    uniqueInstance ifNil: [uniqueInstance := super new].
    ^ uniqueInstance


SBEWebServer class»new

    ^ self error: 'Singleton: use #uniqueInstance'
```

The first time that SBEWebServer uniqueInstance is executed, an instance of the class SBEWebServer will be created and assigned to the uniqueInstance variable. The next time, the previously created instance will be returned instead of creating a new one.

Note that the instance creation code inside the conditional in method 5.6 is written as **self** new and not as SBEWebServer new. What is the difference?

Since the uniqueInstance method is defined in SBEWebServer class, you might think that they were the same. And indeed, until someone creates a subclass of SBEWebServer, they are the same. But suppose that SBEReliableWebServer is a subclass of SBEWebServer and inherits the uniqueInstance method. We would clearly expect SBEReliableWebServer uniqueInstance to answer a SBEReliableWebServer. Using self ensures that this will happen since it will be bound to the respective class. Note also that SBEWebServer and SBEReliableWebServer will each have their own class instance variable called uniqueInstance. These two variables will of course have different values.

5.4 Every class has a superclass

Each class in Smalltalk inherits its behavior and the description of its structure from a single *superclass*. This means that Smalltalk has single inheritance.

```
SmallInteger superclass   ⟶  Integer
Integer superclass        ⟶  Number
Number superclass         ⟶  Magnitude
Magnitude superclass      ⟶  Object
Object superclass         ⟶  ProtoObject
ProtoObject superclass    ⟶  nil
```

Traditionally, the root of the Smalltalk inheritance hierarchy is the class Object (since everything is an object). In Squeak, the root is actually a class called ProtoObject, but you will normally not pay any attention to this class. ProtoObject encapsulates the minimal set of messages that all objects *must* have. However, most classes inherit from Object, which defines many additional messages that almost all objects ought to understand and respond to. Unless you have a very good reason to do otherwise, when creating application classes, you should normally subclass Object, or one of its subclasses.[2]

 *A new class is normally created by sending a message to an existing class, most often the message* subclass:instanceVariableNames: .... *There are a few other methods to create classes. Have a look at the protocol* Kernel-Classes ▷ Class ▷ subclass creation *to see what they are.*

Although Squeak does not provide multiple inheritance, it incorporates a mechanism called *trait*s for sharing behavior across unrelated classes.

[2]Actually, the only use case for directly subclassing ProtoObject is when you write a *transparent decorator* that implements *message forwarding* by overriding #doesNotUnderstand:. In Squeak, see MessageCatcher for a simple example.

Traits are collections of methods that can be reused by multiple classes that are not related by inheritance. Using traits allows one to share code between different classes without duplicating code.

Abstract methods and abstract classes

An abstract class is a class that exists to be subclassed, rather than to be instantiated. An abstract class is usually incomplete, in the sense that it does not define all of the methods that it uses. The "missing" methods — those that the other methods assume, but which are not themselves defined — are called abstract methods.

Smalltalk has no dedicated syntax to specify that a method or a class is abstract. By convention, the body of an abstract method consists of the expression **self** subclassResponsibility. This is known as a "marker method", and indicates that subclasses have the responsibility to define a concrete version of the method. **self** subclassResponsibility methods should always be overridden, and thus should never be executed. If you forget to override one, and it is executed, an exception will be raised.

A class is considered abstract if one of its methods is abstract. Nothing actually prevents you from creating an instance of an abstract class; everything will work until an abstract method is invoked.

Example: the class Magnitude

Magnitude is an abstract class that helps us to define objects that can be compared to each other. Subclasses of Magnitude should implement the methods <, = and hash. Using such messages Magnitude defines other methods such as >, >=, <=, max:, min: between:and: and others for comparing objects. Such methods are inherited by subclasses. The method < is abstract and defined as shown in method 5.7.

Method 5.7: Magnitude»<.

```
1 Magnitude»< aMagnitude
2     "Answer whether the receiver is less than the argument."
3
4     ^ self subclassResponsibility
```

By contrast, the method >= is concrete; it is defined in terms of <:

Method 5.8: Magnitude»>=.

```
1 Magnitude»>= aMagnitude
2     "Answer whether the receiver is greater than or equal to the argument."
```

```
3
4     ^ (self < aMagnitude) not
```

The same is true of the other comparison methods.

Character is a subclass of Magnitude; it overrides the subclassResponsibility method for < with its own version of < (see method 5.9). Character also defines methods = and hash; it inherits from Magnitude the methods >=, <=, ~= and others.

Method 5.9: Character»<.

```
1 Character»< aCharacter
2     "Answer true if the receiver's value < aCharacter's value."
3
4     ^ self asciiValue < aCharacter asciiValue
```

Traits

A *trait* is a collection of methods that can be included in the behavior of a class without the need for inheritance. This makes it easy for classes to have a unique superclass, yet still share useful methods with otherwise unrelated classes.

To define a new trait, simply replace the subclass creation template by a message to the class Trait.

Class 5.10: *Defining a new trait.*

```
1 Trait named: #TAuthor
2     uses: { }
3     category: 'SBE-Quinto'
```

Here, we define the trait TAuthor in the category *SBE-Quinto*. This trait does not *use* any other existing traits. In general, we can specify a *trait composition expression* of other traits to use as part of the uses: keyword argument. Here we simply provide an empty array.

Traits may contain methods, but no instance variables. Suppose we would like to be able to add an author method to various classes, independent of where they occur in the hierarchy. We might do this as follows:

Method 5.11: *An author method.*

```
1 TAuthor»author
2     "Returns author initials"
3
4     ^ 'taa'    "the anonymous author"
```

Now we can use this trait in a class that already has its own superclass, for instance the SBEGame class that we defined in Chapter 2. We simply modify the class creation template for SBEGame to include a uses: keyword argument that specifies that TAuthor should be used.

Class 5.12: *Using a trait.*

```
BorderedMorph subclass: #SBEGame
    uses: TAuthor
    instanceVariableNames: 'cells'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-Quinto'
```

If we now instantiate SBEGame, it will respond to the author message as expected.

```
SBEGame new author    ⟶    'taa'
```

Trait composition expressions may combine multiple traits using the + operator. In case of conflicts (*i.e.*, if multiple traits define methods with the same name), these conflicts can be resolved by explicitly removing these methods (with -), or by redefining these methods in the class or trait that you are defining. It is also possible to *alias* methods (with @), providing a new name for them.

5.5 Everything happens by sending messages

This rule captures the essence of programming in Smalltalk.

In procedural programming, the choice of which piece of code to execute when a procedure is called is made by the caller. The caller chooses the procedure or function to execute *statically*, by name.

In object-oriented programming, we do *not* "call methods": we "send messages." The choice of terminology is significant. Each object has its own responsibilities. We do not *tell* an object what to do by applying some procedure to it. Instead, we politely *ask* an object to do something for us by sending it a message. The message is *not* a piece of code: it is nothing but a name and a list of arguments. The receiver then decides how to respond by selecting its own *method* for doing what was asked. Since different objects may have different methods for responding to the same message, the method must be chosen *dynamically*, when the message is received.

```
3 + 4          ⟶    7        "send message + with argument 4 to integer  3"
(1 @ 2) + 4    ⟶    5@6      "send message + with argument 4 to point (1@2)"
```

As a consequence, we can send the *same message* to different objects, each of which may have *its own method* for responding to the message. We do not tell the SmallInteger 3 or the Point 1@2 how to respond to the message + 4. Each has its own method for +, and responds to + 4 accordingly.

One of the consequences of Smalltalk's model of message sending is that it encourages a style in which objects tend to have very small methods and delegate tasks to other objects, rather than implementing huge, procedural methods that assume too much responsibility. Joseph Pelrine expresses this principle succinctly as follows:

> Don not do anything that you can push off onto someone else.

Many object-oriented languages provide both static and dynamic operations for objects; in Smalltalk there are only dynamic message sends. Instead of providing static class operations, for instance, classes are objects and we simply send messages to classes.

Nearly everything in Smalltalk happens by message sends. At some point, action must take place:

- *Variable declarations* are not message sends. In fact, variable declarations are not even executable. Declaring a variable just causes space to be allocated for an object reference.
- *Assignments* are not message sends. An assignment to a variable causes that variable name to be freshly bound in the scope of its definition.
- *Returns* are not message sends. A return simply causes the computed result to be returned to the sender.
- *Primitives* are not message sends. They are implemented in the virtual machine.

Other than these few exceptions, pretty much everything else does truly happen by sending messages. In particular, since there are no "public fields" in Smalltalk, the only way to update an instance variable of another object is to send it a message asking that it update its own field. Of course, providing setter and getter methods for all the instance variables of an object is no good object-oriented style. Joseph Pelrine also states this very nicely:

> Do not let anyone else play with your data.

5.6 Method lookup follows the inheritance chain

What exactly happens when an object receives a message?

The process is quite simple: The class of the receiver looks up the method to use to handle the message. If this class does not have a method, it asks its superclass, and so on, up the inheritance chain. When the method is found, the arguments are bound to the parameters of the method, and the virtual machine executes it.

It is essentially as simple as this. Nevertheless, there are a few questions that need some care to answer:

- *What happens when a method does not explicitly return a value?*
- *What happens when a class reimplements a superclass method?*
- *What is the difference between* **self** *and* **super** *sends?*
- *What happens when no method is found?*

The rules for method lookup that we present here are conceptual: Virtual machine implementors use all kinds of tricks and optimizations to speed-up method lookup. That is their job, but you should never be able to detect that they are doing something different from the rules mentioned above.

First let us look at the basic lookup strategy, and then consider these further questions.

Method lookup

Suppose we create an instance of EllipseMorph.

```
anEllipse := EllipseMorph new.
```

If we now send this object the message defaultColor, we get the result Color yellow:

```
anEllipse defaultColor  ⟶  Color yellow
```

The class EllipseMorph implements defaultColor, so the appropriate method is found immediately.

Method 5.13: *A locally implemented method.*

```
1 EllipseMorph»defaultColor
2     "answer the default color/fill style for the receiver"
```

```
3
4    ^ Color yellow
```

In contrast, if we send the message openInWorld to anEllipse, the method is not immediately found, since the class EllipseMorph does not implement openInWorld. The search therefore continues in the superclass, BorderedMorph, and so on, until an openInWorld method is found in the class Morph (see Figure 5.2).

Method 5.14: *An inherited method.*

```
1  Morph»openInWorld
2     "Add this morph to the world."
3
4     self openInWorld: self currentWorld.
```

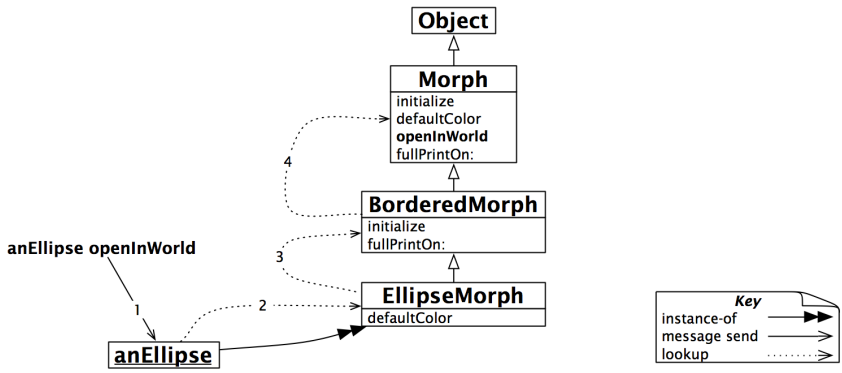


Figure 5.2: Method lookup follows the inheritance hierarchy.

Returning self

Notice that EllipseMorph»defaultColor (method 5.13) explicitly returns Color yellow whereas Morph»openInWorld (method 5.14) does not appear to return anything.

Actually, a method *always* answers a message with a value — which is, of course, an object. The answer may be defined by the ^ construct in the method, but if execution reaches the end of the method without executing a ^, the method still answers a value: It answers the object that received the message. We usually say that the method "answers self", because in Smalltalk the pseudo-variable self represents the receiver of the message, rather like this in Java.

This suggests that method 5.14 is equivalent to method 5.15:

Method 5.15: *Explicitly returning self.*

```
Morph»openInWorld
    "Add this morph to the world."

    self openInWorld: self currentWorld.
    ^ self      "Don't do this unless you mean it"
```

Why is writing ^ **self** explicitly not a good thing to do? Well, when you return something explicitly, you are communicating that you are returning something of interest to the sender. When you explicitly return self, you are saying that you expect the sender to use the returned value. This is not the case here, so it is best not to explicitly return self.

This is a common idiom in Smalltalk, which Kent Beck refers to as "Interesting return value"[3]:

> Return a value only when you intend for the sender to use the value.

Overriding and extension

If we look again at the EllipseMorph class hierarchy in Figure 5.2, we see that the classes Morph and EllipseMorph both implement defaultColor. In fact, if we open a new morph (Morph new openInWorld) we see that we get a blue morph, whereas an ellipse will be yellow by default.

We say that EllipseMorph *overrides* the defaultColor method that it inherits from Morph. The inherited method no longer exists from the point of view of anEllipse.

Sometimes, we do not want to override inherited methods, but rather *extend* them with some new functionality, that is, we would like to be able to invoke the overridden method *in addition to* the new functionality we are defining in the subclass. In Smalltalk, as in many object-oriented languages that support single inheritance, this can be done with the help of super sends.

The most important application of this mechanism is in the initialize method. Whenever a new instance of a class is initialized, it is critical to also initialize any inherited instance variables. However, the knowledge of how to do this is already captured in the initialize methods of each of

[3] Kent Beck, *Smalltalk Best Practice Patterns*. Prentice-Hall, 1997.

the superclass in the inheritance chain. The subclass has no business even trying to initialize inherited instance variables!

It is therefore good practice whenever implementing an initialize method to send **super** initialize before performing any further initialization:

Method 5.16: *Super initialize.*

```
1 BorderedMorph»initialize
2     "Initialize the state of the receiver."
3
4     super initialize.
5     self initializeBorder.
```

> An initialize method should always start by sending **super** initialize.

Self sends and super sends

We need super sends to compose inherited behavior that would otherwise be overridden. The usual way to compose methods, whether inherited or not, however, is using self sends.

How do self sends differ from super sends? Like self, super represents the receiver of the message. The only thing that changes is the method lookup. Instead of lookup starting in the class of the receiver, it starts in the superclass of the class of the method where the super send occurs.

Note that super is *not* the superclass! It is a common and natural mistake to think so. It is also a mistake to think that lookup starts in the superclass of the receiver. We shall see with the following example precisely how this works.

Consider the message initString, which we can send to any morph:

```
anEllipse initString  ⟶  '(EllipseMorph newBounds: (0@0 corner: 50@40) color:
    Color yellow) setBorderWidth: 1 borderColor: Color black'
```

The return value is a string that can be evaluated to recreate the morph.

How exactly is this result obtained through a combination of self and super sends? First, anEllipse initString will cause the method initString to be found in the class Morph, as shown in Figure 5.3.

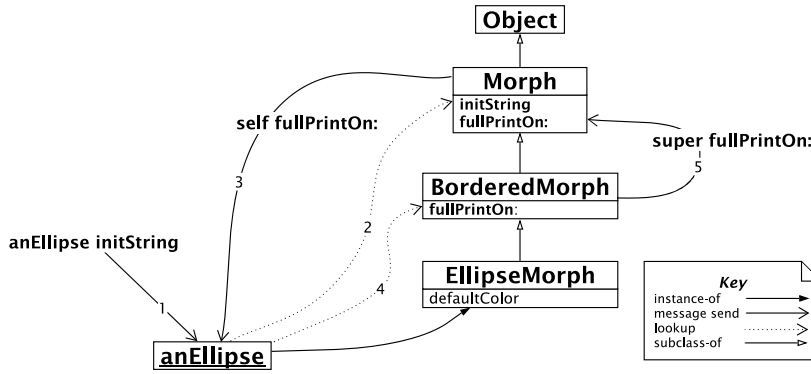


Figure 5.3: self and super sends.

Method 5.17: *A* self *send.*

```
1 Morph»initString
2
3   ^ String streamContents: [:stream | self fullPrintOn: stream]
```

The method Morph»initString performs a self send of fullPrintOn:. This causes a second lookup to take place, starting in the class EllipseMorph, and finding fullPrintOn: in BorderedMorph (see Figure 5.3 once again). What is critical to notice is that the self send causes the method lookup to start again in the class of the receiver, namely the class of anEllipse.

> A self send triggers a *dynamic* method lookup starting in the class of the receiver.

Method 5.18: *Combining* super *and* self *sends.*

```
1 BorderedMorph»fullPrintOn: aStream
2
3   aStream nextPutAll: '('.
4   super fullPrintOn: aStream.
5   aStream
6     nextPutAll: ') setBorderWidth: ';
7     print: self borderWidth;
8     nextPutAll: ' borderColor: ' , (self colorString: self borderColor).
```

At this point, BorderedMorph»fullPrintOn: does a super send to extend the fullPrintOn: behavior it inherits from its superclass. Because this is a super send, the lookup now starts in the superclass of the class where the super send occurs, namely in Morph. We then immediately find and evaluate Morph»fullPrintOn:.

Note that the super lookup did not start in the superclass of the receiver. This would have caused the lookup to start from BorderedMorph, resulting in an infinite loop!

> A super send triggers a *static* method lookup starting in the superclass of the class of the method performing the super send.

If you think carefully about super sends and Figure 5.3, you will realize that super bindings are static: all that matters is the class in which the text of the super send is found. By contrast, the meaning of self is dynamic: It always represents the receiver of the currently executing message. This means that *all* messages sent to self are looked up by starting in the receiver's class.

Message not understood

What happens if the method we are looking for is not found?

Suppose we send the message foo to our ellipse. First, the normal method lookup would go through the inheritance chain all the way up to Object (or rather ProtoObject) looking for this method. When this method is not found, the virtual machine will notify the object of the failure by causing it to send **self** doesNotUnderstand: #foo. (See Figure 5.4.)

Now, this is an ordinary, dynamic message send, so the lookup starts again from the class EllipseMorph, but this time searching for the method doesNotUnderstand:. As it turns out, Object implements doesNotUnderstand:. This method will create a new MessageNotUnderstood object which is capable of starting a Debugger in the current execution context.

Why do we take this convoluted path to handle such an obvious error? Well, this offers developers an easy way to intercept such errors and take alternative action. One could easily override the method doesNotUnderstand: in any subclass of Object and provide a different way of handling the error.

In fact, this can be an easy way to implement automatic delegation of messages from one object to another. A Delegator object could simply

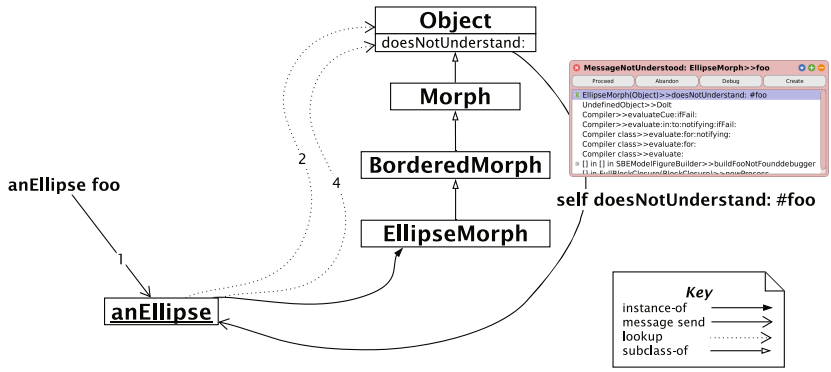


Figure 5.4: Message foo is not understood.

delegate all messages it does not understand to another object whose responsibility it is to handle them, or raise an error itself!

5.7 Shared variables

Next, we will look at an aspect of Smalltalk that is not so easily covered by our five rules: shared variables.

Smalltalk provides three kinds of shared variables: (1) *globally* shared variables; (2) variables shared between instances and classes (*class variables*), and (3) variables shared amongst a group of classes (*pool variables*). The names of all of these shared variables start with a capital letter, to warn us that they are indeed shared between multiple objects.

Global variables

In Squeak, all global variables are stored in a namespace called Smalltalk which is an instance of the class SmalltalkImage. Global variables are accessible from anywhere. Every class is named by a global variable; in addition, a few globals are used to name special or commonly useful objects.

However, what may sound so handy and convenient at the first glance has turned out as bad practice during the evolution of Squeak. While we state in rule 4 that "everything happens by message sends", the concept of globals is a violation of this rule. Accessing a global allows you to exchange

information between two objects — *without* sending any message. As a consequence, when using globals, we lose an important advantage of the Smalltalk object model, which is the encapsulation of every object's state. Furthermore, global variables introduce *global state* as mentioned earlier which impedes the modularity of Smalltalk objects and hinders us to use a Smalltalk object directly without taking care of its environment.

For these reasons, current practice is to strictly limit the use of global variables; it is usually better to use class instance variables or class variables and to provide class methods to access them. Still, there are a number of globals defined in Squeak, but for most of them, there is a preferred alternative too, which is a message that usually can be sent to a particular class.

For example, the variable Transcript names an instance of TranscriptStream, a stream that writes to a scrolling window. The following code displays some information in a new of the Transcript.

```
Transcript showln: 'Squeak is fun and powerful'
```

Before you do it, open a transcript from the world menu.

HINT *Writing to the Transcript is slow, especially when the transcript window is open. So, if you experience some sluggishness and are writing to the Transcript, think about collapsing it.*

Transcript outputs are mostly used for debugging and experimenting purposes where code quality is not the most important thing. If you want to use a transcript in a real application, you should use Project current transcript instead, which makes sure that the transcript you are using actually matches your current Squeak environment.

Another useful global variable is Smalltalk which is the single instance of SmalltalkImage that, inter alia, provides access to all globals — including Smalltalk itself. Smalltalk globals answers an Environment instance which defines all of the globals and provides a Dictionary-like interface. The keys for these declarations are the symbols that name the global objects in Smalltalk code. So, for example,

```
Smalltalk globals at: #Boolean    ⟶    Boolean
```

Or even shorter, using a convenience method defined on SmalltalkImage:

```
Smalltalk at: #Boolean    ⟶    Boolean
```

Since Smalltalk is itself a global variable,

```
Smalltalk at: #Smalltalk    ⟶    Smalltalk
```

and

```
(Smalltalk at: #Smalltalk) == Smalltalk    ⟶    true
```

There is also a dictionary that contains all the undeclared variables, which can be retrieved via Smalltalk globals undeclared. If you write a method that references an undeclared variable, the browser will normally prompt you to declare it, for example, as a global or as an instance variable of the class. However, if you later delete the declaration, the code will then reference an undeclared variable. Inspecting the undeclared dictionary can sometimes help explain strange behavior!

In case you really need to define a new global, the common way to do so is just to do it on an assignment to a capitalized but undeclared identifier. The parser will then offer to declare the global for you. If you want to define a global programmatically, just execute Smalltalk at: #AGlobalName put: **nil**. To remove it, execute Smalltalk removeKey: #AGlobalName.

Class variables

Sometimes, we need to share some data amongst all the instances of a class and the class itself. This is possible using *class variables*. The term class variable indicates that the lifetime of the variable is the same as that of the class. However, what the term does not convey is that these variables are shared amongst all the instances of a class as well as the class itself, as shown in Figure 5.5. Indeed, a better name would have been *shared variables* since this expresses more clearly their role, and also warns of the danger of using them, particularly if they are modified.

In Figure 5.5 we see that rgb and cachedDepth are instance variables of Color, hence only accessible to instances of Color. We also see that superclass, subclass, methodDict and so on are class instance variables, *i.e.*, instance variables only accessible to the Color class.

But we can also see something new: ColorNames and CachedColormaps are *class variables* defined for Color. The capitalization of these variables gives us a hint that they are shared. In fact, not only may all instances of Color access these shared variables, but also the Color class itself, *and any of its subclasses*. Both instance methods and class methods can access these shared variables.

A class variable is declared in the class definition template. For example, the class Color defines a large number of class variables to speed up color creation; its definition is shown below (class 5.19).

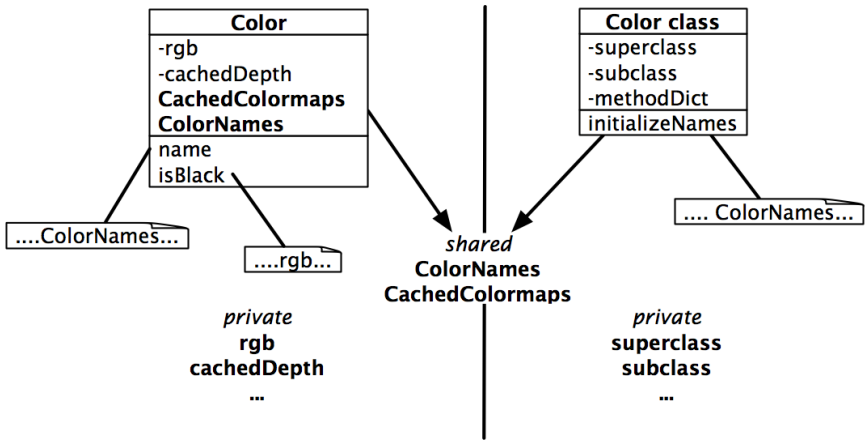


Figure 5.5: Instance and class methods accessing different variables.

Class 5.19: *Color and its class variables.*

```
1 Object subclass: #Color
2       instanceVariableNames: 'rgb cachedDepth cachedBitPattern'
3       classVariableNames: 'Black Blue BlueShift Brown CachedColormaps ColorChart
         ColorNames ComponentMask ComponentMax Cyan DarkGray Gray
        GrayToIndexMap Green GreenShift HalfComponentMask HighLightBitmaps
        IndexedColors LightBlue LightBrown LightCyan LightGray LightGreen
        LightMagenta LightOrange LightRed LightYellow Magenta MaskingMap Orange
        PaleBlue PaleBuff PaleGreen PaleMagenta PaleOrange PalePeach PaleRed
        PaleTan PaleYellow PureBlue PureCyan PureGreen PureMagenta PureRed
        PureYellow RandomStream Red RedShift TranslucentPatterns Transparent
        VeryDarkGray VeryLightGray VeryPaleRed VeryVeryDarkGray
        VeryVeryLightGray White Yellow'
4       poolDictionaries: ''
5       category: 'Graphics-Primitives'
```

The class variable ColorNames is an array containing the name of frequently-used colors. This array is shared by all the instances of Color and its subclass TranslucentColor. It is accessible from all the instance and class methods.

ColorNames is initialized once in Color class»initializeNames, but it is accessed from instances of Color. The method Color»name uses the variable to find the name of a color. Since most colors do not have names, it was thought inappropriate to add an instance variable name to every color.

Class initialization

The presence of class variables raises the question: How do we initialize them? One solution is lazy initialization. This can be done by introducing an accessor method which, when executed, initializes the variable if it has not yet been initialized. This implies that we must use the accessor all the time and never use the class variable directly. This furthermore imposes the cost of the accessor send and the initialization test. It also arguably defeats the point of using a class variable, since in fact it is no longer shared.

Method 5.20: *Color class»colorNames.*

```
1 Color class»colorNames
2
3   ColorNames ifNil: [self initializeNames].
4   ^ ColorNames
```

Another solution is to override the class method initialize.

Method 5.21: *Color class»initialize.*

```
1 Color class»initialize
2
3   ...
4   self initializeNames
```

If you adopt this solution, you need to remember to invoke the initialize method after you define it, *e.g.*, by evaluating Color initialize. Although class side initialize methods are executed automatically when code is loaded into memory, they are *not* executed automatically when they are first typed into the browser and compiled, or when they are edited and re-compiled.

Pool variables

Pool variables are variables that are shared between several classes that may not be related by inheritance. Pool variables were originally stored in pool dictionaries; now they should be defined as class variables of dedicated classes (subclasses of SharedPool). Our advice is to avoid them; you will need them only in rare and specific circumstances. Our goal here is therefore to explain pool variables just enough so that you can understand them when you are reading code.

A class that accesses a pool variable must mention the pool in its class definition. For example, the class Text indicates that it is using the pool dictionary TextConstants, which contains all the text constants such as Tab and CrLf. This dictionary has a key #Tab that is bound to the value Character tab, *i.e.*, the tabulator character.

Class 5.22: *Pool dictionaries in the* Text *class.*

```
ArrayedCollection subclass: #Text
    instanceVariableNames: 'string runs'
    classVariableNames: ''
    poolDictionaries: 'TextConstants'
    category: 'Collections-Text'
```

This allows methods of the class Text to access the keys of the dictionary in the method body *directly*, *i.e.*, by using variable syntax rather than an explicit dictionary lookup. For example, we can write the following method.

Method 5.23: *Text»testCR.*

```
Text»testCR

    ^ CR == Character cr
```

Once again, we recommend that you avoid the use of pool variables and pool dictionaries.

5.8 Chapter summary

The object model of Squeak is both simple and uniform. Everything is an object, and pretty much everything happens by message sends.

- Everything is an object. Primitive entities like integers are objects, but also classes are first-class objects.
- Every object is an instance of a class. Classes define the structure of their instances via *private* instance variables and the behavior of their instances via *public* methods. Each class is the unique instance of its metaclass. Class variables are private variables shared by the class and all the instances of the class. Classes cannot directly access instance variables of their instances, and instances cannot access instance variables of their class. Accessors must be defined if this is needed.
- Every class has a superclass. The root of the single inheritance hierarchy is ProtoObject. Classes you define, however, should normally inherit from Object or its subclasses. There is no syntax for defining abstract classes. An abstract class is simply a class with an abstract

method — one whose implementation consists of the expression **self** subclassResponsibility. Although Squeak supports only single inheritance, it is easy to share implementations of methods by packaging them as *traits*.

- Everything happens by message sends. We do not "call methods", we "send messages". The receiver then chooses its own method for responding to the message.
- Method lookup follows the inheritance chain; self sends are dynamic and start the method lookup again in the class of the receiver, whereas super sends are static, and start in the superclass of class in which the super send is written.
- There are three kinds of shared variables. Global variables are accessible everywhere in the system. Class variables are shared between a class, its subclasses and its instances. Pool variables are shared between a selected set of classes. You should avoid shared variables as much as possible.

Chapter 6

The Squeak programming environment

The goal of this chapter is to show you how to develop programs in the Squeak programming environment. You have already seen how to define methods and classes using the system browser; this chapter will show you more of the features of the system browser, and introduce you to some of the other browsers.

Sometimes your program does not work as you expect. Squeak has an excellent debugger, but like most powerful tools, it can be confusing on first use. We will walk you through a debugging session and demonstrate some of the features of the debugger. One of the unique features of Smalltalk is that while you are programming, you are living in a world of live objects, not in a world of static program text. This makes it possible to get very rapid feedback while programming, which makes you more productive. There are two tools that let you look at, and indeed change, live objects: the *inspector* and the *explorer*.

6.1 Overview

Smalltalk and modern graphical interfaces were developed together. Even before the first public release of Smalltalk in 1983, Smalltalk had a self-hosting graphical development environment, and all Smalltalk development was taking place in it. Let's start by looking at the main tools in Squeak, all of which can be opened through the *Tools* menu in the world docking bar at the top of the screen. Some of the tools can also be opened

directly from the world menu.

- The Browser is a central development tool. You will use it to create, define, and organize your classes and methods. Using it you can also navigate through all the library classes: Unlike other environments where the source code is stored in separate files, in Smalltalk, all classes and methods are contained in the image.
- The Workspace is a window into which you can type input. It can be used for any purpose, but is most often used for typing Smalltalk expressions and executing them as do its. The use of the workspace was also illustrated in Section 1.4.
- The Transcript is a window on the Transcript output stream, which is useful for writing log messages and has already been described in Section 1.4.
- The Test Runner lets you run and debug SUnit tests, and is described in Chapter 8.
- The Message Names tool is used to browse the methods with a particular selector or with a selector containing a substring.
- The Method Finder tool will let you find methods, but according to what they *do* as well as what they are called.
- The Process Browser provides a view on all of the processes (threads) executing in Smalltalk.

The Debugger has an obvious role but you will discover that it has a more central place compared to debuggers for other programming languages, because in Smalltalk you can *program* in the debugger. The debugger is not launched from a menu or from the *Tools* flap; it is normally entered by running a failing test, by typing CMD–. to interrupt a running process, or by inserting a **self** halt expression in code.

6.2 The system browser

There are actually several browsers in Squeak: the standard system browser, the package browser, and the Monticello snapshot browser. We will take a look at the standard system browser first since the others are variations on it. Figure 6.1 shows the browser as it appears when you first open it from the world menu.

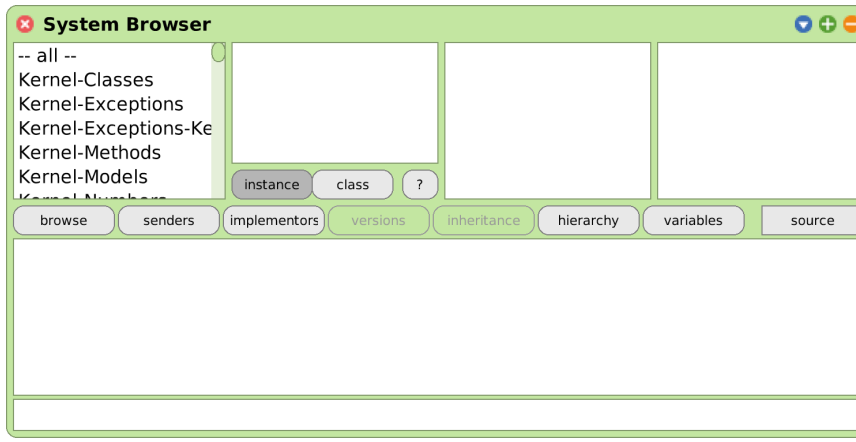


Figure 6.1: The system browser.

The four small panes at the top of the browser represent a hierarchic view of the methods in the system, much in the same way as the Mac OS X *Finder* in column mode provide a view of the files on the disk (or the original NeXTstep *File Viewer*). The leftmost pane lists *categories* of classes; select one (say *Kernel-Objects*) and the pane immediately to the right will then show all of the classes in that category.

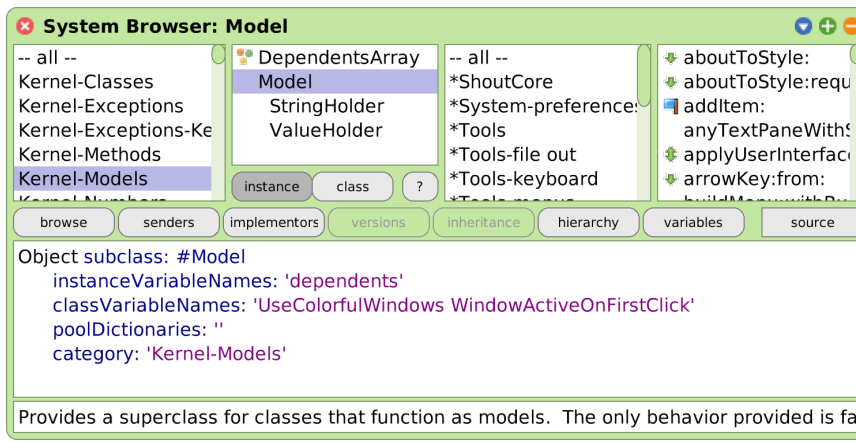


Figure 6.2: System Browser with the class Model selected.

Similarly, if you select one of the classes in the second pane, say, Model

(see Figure 6.2), the third pane will show all of the *protocols* defined for that class, as well as a virtual protocol *--all--*, which is selected by default (the browser sometimes refers to protocols as "message categories"). Protocols help organizing methods; they make it easier to find and think about the behavior of a class by breaking it up into smaller, conceptually coherent pieces. The fourth pane shows the names of all of the methods defined in the selected protocol. If you then select a method name, the source code of the corresponding method appears in the large pane at the bottom of the browser, where you can view it, edit it, and save the edited version. If you select the class `Model`, the protocol *dependents*, and the method `myDependents`, the browser should look like Figure 6.3.

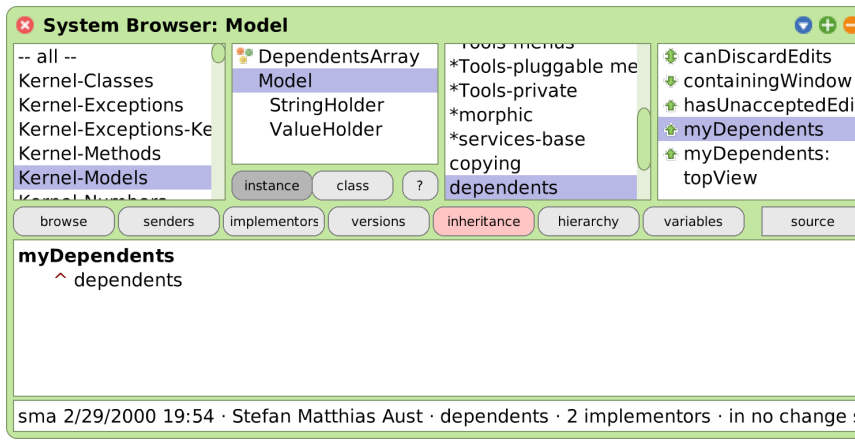


Figure 6.3: System Browser showing the `myDependents` method in class `Model`.

Creating categories and protocols

Unlike directories in the Mac OS X *Finder*, the four top panes of the browser are not quite equal. Whereas classes and methods are part of the Smalltalk language, system categories and protocols are not: They are a convenience introduced by the programming tools to limit the amount of information that needs to be shown in each pane. For example, if there were no categories, the browser would have to show a list of all of the methods in the selected class; for many classes, this list would be too large to navigate conveniently.

Because of this, the way that you create a new class category or protocol is different from the way that you create a new class or a new method. To create a new class category, select `new category` from the yellow button

menu in the class category pane; to create a new protocol, select new category from the yellow button menu in the protocol pane. Enter the name of the new thing in the dialog, and you are done: there is nothing more to a class category or a protocol than its name and its contents.

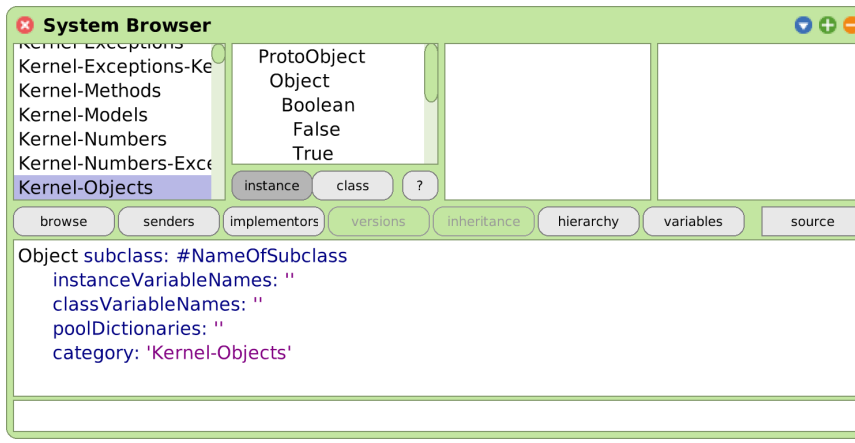


Figure 6.4: System Browser showing the class-creation template.

Finally, you can influence what kind of information you want to see about a class through the three buttons below the second pane. By default, the instance view is selected that shows you the ordinary methods provided by the class. The class button changes the view to show the methods understood by the class itself (for more details see Chapter 5 and Chapter 13). The ? button changes the view to show the class comment of the class in the bottom pane, where you can edit and save it as you would with a method.

Creating classes and methods

In contrast to creating categories or protocols, to create a new class or a new method, you will actually have to write some Smalltalk code. If you deselect the currently selected category (most left pane) and then reselect it again, the main browser pane will display a class creation template (Figure 6.4).

You create a new class by editing this template: replace Object by the name of the existing class of which you wish to create a subclass, replace NameOfSubclass by the name that you would like to give to your new subclass, and fill in the instance variable names if you want some. The category for the new class is, by default, the currently selected category, but you can change this too if you like. If you already have the browser focussed on the

class that you wish to subclass, you can get the same template with slightly different initialization by using the yellow button menu in the class pane, and selecting more ... ▷ subclass template. You can also just edit the definition of an existing class, changing the class name to something new. In all cases, when you accept the new definition, the new class (the one whose name follows the #) is created (as is the corresponding metaclass). Creating a class also creates a global variable that references the class, which is why you can refer to all of the existing classes by using their names.

The process of creating a new method is similar. First, select the class in which you want the method to live, and then select a protocol. The browser will display a method-creation template, as shown in Figure 6.5, which you can fill-in or edit.

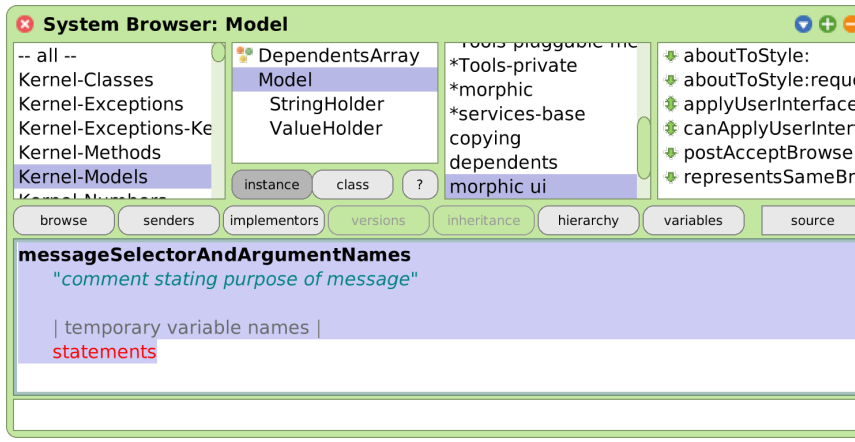


Figure 6.5: System Browser showing the method-creation template.

The button bar

The system browser provides several tools for exploring and analyzing code. Those tools are most simply accessed from the horizontal button bar in the middle of the browser window. The buttons are labeled browse, senders, implementors, ... Figure 6.5 shows the complete set.

Browsing code

The browse button opens a new system browser on the class or method that is currently selected. It's often useful to have multiple browsers open at the

same time. When you are writing code you will almost certainly need at least two: one for the method that you are typing, and another to browse around the system to see what to type. You can also open a browser on a class named by any selected text using the CMD–b keyboard shortcut.

 *Try this: in a workspace window, type the name of a class (for instance* ScaleMorph*), select it, and then press* CMD–*b. This trick is often useful; it works in any text window.*

Senders and implementors of a message

The senders button will give you a list of all methods that may use the selected method. With the browser open on ScaleMorph, click on the checkExtent: method in the method pane near the top right corner of the browser; the body of checkExtent: displays in the bottom part of the browser. If you now press the senders button, a menu will appear with checkExtent: as the topmost item, and below it, all the messages that checkExtent: sends (see Figure 6.6). Selecting an item in this menu will open a browser with the list of all methods in the image that send the selected message. You can also quickly access the senders of messages by typing CMD–n (for se**n**ders).

The implementors button works similarly, but instead of listing the senders of a message, it lists all of the classes that implement a method with the same selector. To see this, select drawOn: in the message pane and then bring up the "implementors of drawOn:" browser, either using the implementors button, or the yellow button menu on the method pane, or just by typing CMD–m (for i**m**ple**m**entors) in the method pane with drawOn: selected. You should get a method list window showing a scrolling list of over a hundred classes that implement a drawOn: method. It shouldn't be all that surprising that so many classes implement this method: drawOn: is the message that is understood by every object that is capable of drawing itself on the screen. While viewing any one of these methods, try browsing the senders of the drawOn: message: We found 72 methods that send this message (the number can vary depending on your image version). You can also bring up an implementors browser at any time by selecting a message (including the arguments if it is a keyword message) and typing CMD–m.

The senders button lists *all* methods that send the chosen message: not all of these sends will result in the execution of the method you are looking at. Indeed, much of the power of object-oriented programming comes from the fact that message sends are *polymorphic*, that is, they can work equally well on objects of any class. Sometimes it is easy to figure out which method will be executed as the result of a message send, and sometimes it is impossible; the senders and implementors tools don't try.

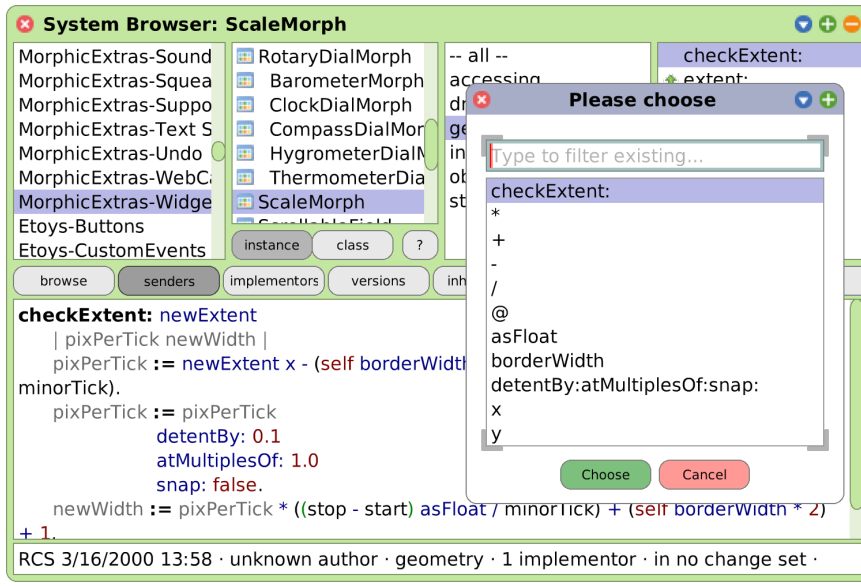


Figure 6.6: A Class Browser opened on the ScaleMorph class. Note the horizontal bar of buttons at the center of the browser; here we are using the senders button.

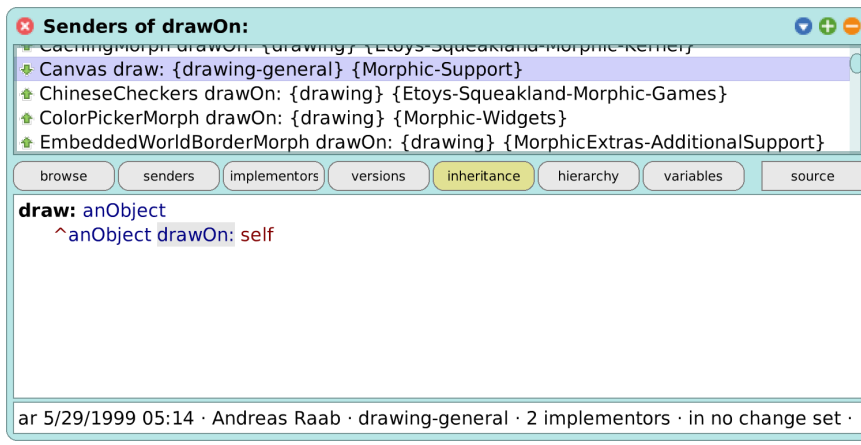


Figure 6.7: The Senders Browser showing that the Canvas»draw: method sends the drawOn: message to its argument.

As an example, we can look at the method Canvas»draw:, shown in Figure 6.7. You can see that this method sends drawOn: to whatever object is passed to it as an argument, which could potentially be an instance of any class at all. In general, there is no simple way for the browser to know which message sends might cause which methods to be executed[1]. For this reason, the "senders" browser shows exactly what its name suggests: all of the senders of the message with the chosen selector. The senders button is nevertheless extremely useful when you need to understand how you can *use* a method: it lets you navigate quickly through example uses. Since all of the methods with the same selector ought to be used in the same way, all of the uses of a given message ought to be similar.

Another way to determine implementors of a message is to browse the hierarchy. If you look at the send of drawOn: in AtomMorph»drawOn:, you will see that it is a super send. So we know that the method that will be executed will be in AtomMorph's superclass. What class is that? Click the hierarchy button and you will see that it is EllipseMorph.

Versions of a method

When you save a new version of a method, the old one is not lost. Squeak keeps all of the old versions, and allows you to compare different versions and to go back ("revert") to an old version.

The versions button gives access to the successive modifications made to the selected method. In Figure 6.8 we can see the versions of the mouseUp: method that one of the authors created while writing the Quinto game described in Chapter 2.

The top pane displays one line for each version of the method, listing the initials of the programmer who wrote it, the date and time at which it was saved, the names of the class and the method, and the protocol in which it was defined. The current (active) version is at the top of the list; whichever version is selected is displayed in the bottom pane. If the showDiffs view is selected from the right most button, as it is in Figure 6.8, the display also shows the differences between the selected version and the one immediately older. The prettyDiffs view is useful if there have been changes to the layout: It pretty-prints both versions before differencing so that the differences that are displayed exclude formatting changes. Through the leftmost button you can revert the method to the selected version. The yellow button menu in the list of versions provides you further actions, such as comparing any version to the current one.

[1] Dataflow analysis can generally help figure out the class of the receiver of some messages, but it is not applied in the Smalltalk tools

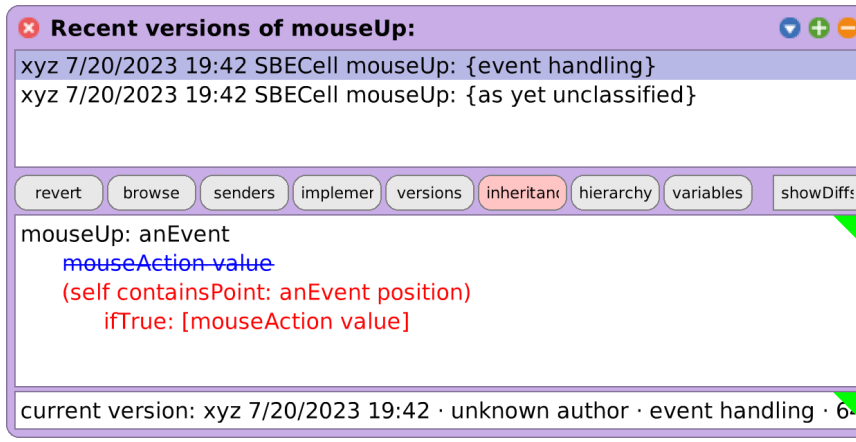


Figure 6.8: The versions browser showing several versions of the SBECell»mouseUp: method.

The existence of the versions browser means that you never have to worry about preserving code that you think might no longer be needed: Just delete it. If you find that you *do* need it, you can always revert to the old version, or copy the needed code fragment out of the old version and paste it into another method. Get into the habit of using versions; "commenting out" code that is no longer needed is a bad practice because it makes the current code harder to read. Smalltalkers rate code readability extremely highly.

HINT *What if you delete a method entirely, and then decide that you want it back? You can find the deletion in a change set, where you can ask to see versions with the yellow button menu. The change set browser is described in Section 7.2. If you do not have a change set for the method, you can also browse your changes log using the Recover changes or the Recover method versions dialog from the* Extras *menu in the main docking bar, see Section 7.4.*

Method overridings

The inheritance button opens a specialized browser that displays all the methods overridden by the displayed method. To see how it works, display the ScaleMorph»defaultColor method and click inheritance. This method definition overrides RectangleMorph»defaultColor, which itself overrides Morph»defaultColor, as shown in Figure 6.9. The color of the inheritance button

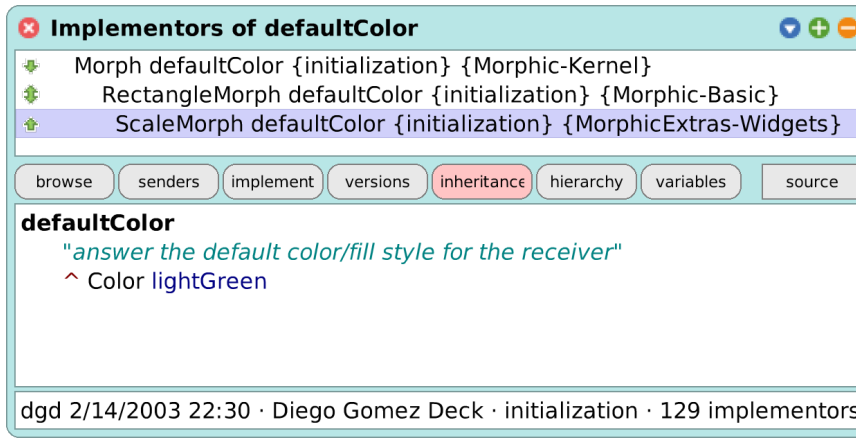


Figure 6.9: ScaleMorph»defaultColor and the methods that it overrides, in inheritance order. The inheritance button is pink because the displayed method is overridden in a subclass.

depends on how the overriding occurs. The colors are explained in a help balloon:

pink: the displayed method overrides another method but does not call super;

green: the displayed method overrides another method and uses it via super;

tan: the displayed method is itself overridden in a subclass;

mauve: the displayed method overrides another method, and it itself overridden;

pink tan: the displayed method overrides, is overridden, and makes a **super**-send.

The hierarchy browser

The hierarchy button opens a hierarchy browser on the current class; this browser can also be opened by using the browse hierarchy menu item in the yellow button menu of the class pane. The hierarchy browser is similar to the system browser, but instead of displaying the system categories and the classes in each category, it shows a single list of classes indented to represent inheritance. The category of the selected class is displayed in the title of the browser window. The hierarchy browser is designed to make it easy to navigate through the inheritance hierarchy but does not show all of the classes in the system: only the superclasses and subclasses of the

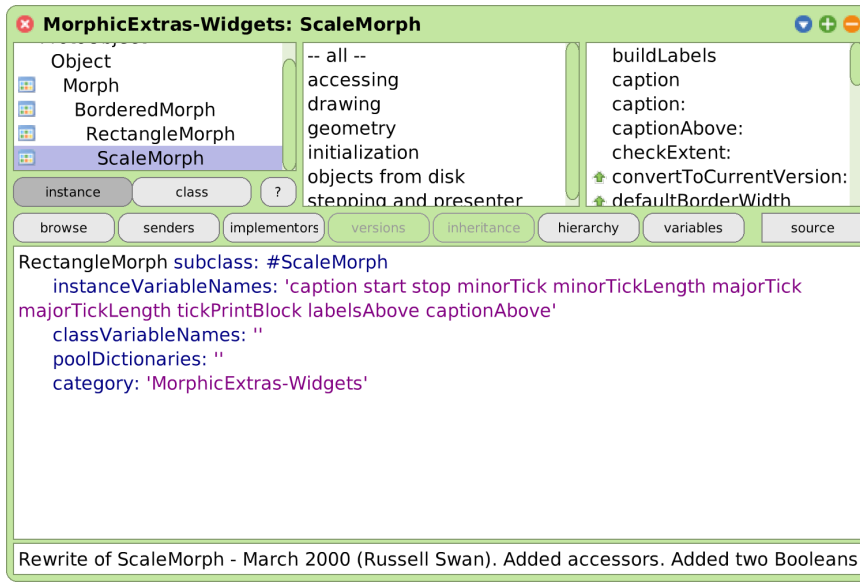


Figure 6.10: A hierarchy browser open on ScaleMorph.

initial class are shown. In Figure 6.10, the hierarchy browser reveals that the direct superclass of ScaleMorph is RectangleMorph.

Finding variable or class references

The variables button helps you find where an instance variable or a class variable is used; the same information is accessible from the yellow button menu item references in the class pane. The menu also includes assignments, which shows the subset of the instance variable references that assign to the variable. Once you click on the button or select the menu item, you will be presented with a dialog that invites you to choose a variable from all of the variables defined in the current class, and all of the variables that it inherits. The list is in inheritance order; it can often be useful to bring up this list just to remind yourself of the name of an instance variable. If you click outside the list, it will go away and no variable browser will be created.

Another interesting action in the yellow button menu of the class pane is class refs (N), which displays a list of all of the methods that directly reference the current class.

Source

The source button brings up the "what to show" menu, which allows you to choose what the browser shows in the source pane. Options include the source code, documentation, prettyPrinted source code, show diffs, byteCodes, and source code decompiled from the byte codes. The label on the button changes if you select one of the other modes. There are other options too; if you let the mouse linger over the names, a help balloon will appear. Try some of them.

Note that selecting prettyPrint in the "what to show" menu is *not* the same as prettyPrinting a method before you save it. The menu controls only what the browser displays, and does not affect the code stored in the system. You can verify this by opening two browsers, and selecting prettyPrint in one and source in the other. Focussing two browsers on the same method and selecting byteCodes in one and decompile in another is a good way to learn about the Squeak virtual machine's byte-coded instruction set.

The browser menus

Many additional functions are available from the browser's yellow button menu. Since yellow button menus are context-sensitive, each pane in the browser has its own menu. Even if the labels on the menu items are the same, their *meaning* depends on the context. For example, the category pane, the class pane, the protocol pane, and the messages pane all have a file out menu item. However, they do different things: In the category pane, it files out the whole category, in the class pane it files out the whole class, in the protocol pane it files out the whole protocol, and in the method pane it files out just the displayed method. Although this may seem obvious, it can be a source of confusion for beginners.

Possibly the most useful menu item for navigating is find class... (f) in the category pane. Although the categories are useful for the code that we are actively developing, most of us do not know the categorization of the whole system, and it is much faster to type CMD−f followed by a few subsequent characters from the name of a class than to guess which category it might be in. The find dialog also shows the classes that you have browsed recently, even if you have forgotten its name.

In the class pane, find method is a similar menu item for browsing a particular method. However, unless the list of methods is very long, it is often quicker to browse the *--all--* protocol (which is the default), place the mouse in the method pane, and type the first letter of the name of the method that you are looking for. This will filter the elements in the list so that the sought-for method name is visible.

Try both ways of navigating to OrderedCollection»removeAt:

There are many other options available in the menus. It pays to spend a few minutes working with the browser and seeing what is there.

Compare the result of the following three menu items in the class pane: Browse Protocol, Browse Hierarchy, *and* Show Hierarchy.

In general, the common navigation functions, such as searching for a class, browsing senders, or browsing implementors have keyboard shortcuts, which you can find in parentheses after the corresponding menu items. Further, the feature that you can filter the list of methods by typing while having the list selected is a general feature of all lists in Squeak. Even in menus, such as the world menu, you can simply start typing to filter entries.

Other class browsers

Next to the standard browser there is also another class browser: the *package pane browser*. This browser can be opened from the world menu: World ▷ open... ▷ package pane browser

It's basically the same as the class browser, but it knows about the naming convention for system categories. You will have noticed that the names of categories have two parts. For example, the ScaleMorph class belongs to the *Morphic-Widgets* category. The package browser assumes that the part before the hyphen, Morphic, is the name of a "package", and adds a fifth pane that allows you to browse only those categories in a particular package. However, if you select no package at all, then all the categories are available, just as with the ordinary four-pane browser. You can also open a package pane browser from a regular browser by yellow-clicking any system category on the left and choosing browse package.

Note that the meaning of the term package as used in the package pane browser is different from the concept of Monticello packages. To browse all classes of a Monticello package, you can open a Snapshot Browser from the Monticello Browser by selecting a package and then pressing Browse.

Browsing programmatically

The class SystemNavigation provides a number of utility methods that are useful for navigating around the system. Many of the functions offered by the classic browser are implemented by SystemNavigation.

Open a workspace and do it *the following code to browse the senders of* checkExtent*:*

```
self systemNavigation browseAllCallsOn: #checkExtent:.
```

To restrict the search for senders to the methods of a specific class:

```
self systemNavigation browseAllCallsOn: #drawOn: from: ScaleMorph.
```

Because the development tools are objects, they are completely accessible from programs and you can develop your own tools or adapt the existing tools to your needs.

The programmatic equivalent to the implementors button is:

```
self systemNavigation browseAllImplementorsOf: #checkExtent:.
```

You can also find methods or classes based on custom criteria using SystemNavigation»browseAllSelect: or ClassListBrowser class»browseClassesSatisfying:title: . To learn more about what is available, explore the class SystemNavigation with the browser. Further navigation examples can be found in the FAQ (Appendix A).

Besides SystemNavigation, there are also several subclasses of Categorizer that manage the categories in which classes or methods are organized. In particular, a ClassOrganizer records the organization of methods in a class, and analogously, a SystemOrganizer records the organization of classes within system categories. For example, you can retrieve all classes that are in the system category Collections-Strings by sending classesIn: to the default SystemOrganizer instance:

```
SystemOrganizer default classesIn: 'Collections-Strings'  ⟶  an
    OrderedCollection(ByteString ByteSymbol Character String Symbol WideString
    WideSymbol)
```

Summary

As you have seen, there are many ways to navigate around Smalltalk code. You may find this confusing at first, in which case you can always fall back to the traditional system browser. However, we usually find that once beginners gain more experience with Squeak, the availability of different browsers becomes one of its most valued features, because they provide many ways to help you to understand and organize your code. The problem of understanding code is one of the greatest challenges of large-scale software development.

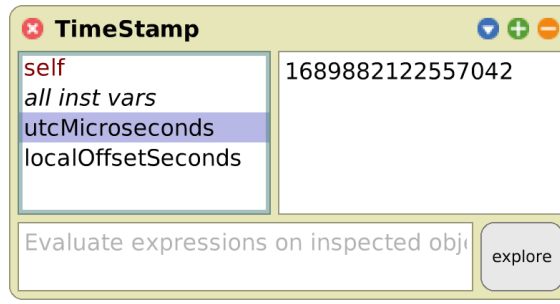


Figure 6.11: Inspecting TimeStamp now.

6.3 The inspector and the explorer

One of the things that make Smalltalk so different from many other programming environments is that it provides you with a window onto a world of live objects, not a world of static code. Any of those objects can be examined by the programmer, and even changed — although some care is necessary when changing the basic objects that support the system. By all means experiment, but save your image first!

The inspector

As an illustration of what you can do with an inspector, type TimeStamp now *in a workspace, and then choose* inspect it *from the yellow button menu. (It's not necessary to select the text before using the menu; if no text is selected, the menu operations work on the whole of the current line. You can also type* CMD–*i for* inspect it.*)*

A window like that shown in Figure 6.11 will appear. This is an inspector, and can be thought of as a window onto the internals of a particular object — in this case, the particular instance of TimeStamp that was created when you evaluated the expression TimeStamp now. The title bar of the window shows the *class* of the object that is being inspected. If you select self at the top of the left pane, the right pane will show the print string of the object. If you select all inst vars in the left pane, the right pane will show a list of the instance variables in the object, and the print string for each one. The remaining items in the left pane represent the instance variables; this makes it easy to examine them one at a time, and also to change them.

The horizontal pane at the bottom of the inspector is a small workspace

window. In this window, the pseudo-variable **self** is bound to the very object that you are inspecting. So, if you inspect it on

```
self - TimeStamp today
```

in the workspace pane, the result will be a Duration object that represents the time interval between midnight today and the instant at which you evaluated TimeStamp now and created the TimeStamp object that you are inspecting. You can also try evaluating TimeStamp now - **self**; this will tell you how long you have spent reading this section of this book!

In addition to **self**, all the instance variables of the object are in scope in the workspace pane, so you can use them in expressions or even assign values to them. For example, if you evaluate localOffsetSeconds := localOffsetSeconds - 1 in the workspace pane, you will see that the value of the localOffsetSeconds instance variable will indeed change, and the value of TimeStamp now - **self** will increase by one second.

You can change instance variables directly by selecting them, replacing the old value in the right-hand pane by a Squeak expression, and accepting. Squeak will evaluate the expression and assign the result to the instance variable.

There are many special variants of the inspector for different kinds of collections, methods, morphs, and a few other classes that make it easier to examine the contents of these special objects. Squeak will automatically choose a fitting variant for the object you want to inspect. For instance, if you inspect an EllipseMorph by doing EllipseMorph new openInWorld; inspect, you will see a window like that shown in Figure 6.12 that has special fields for accessing the geometry, the screenshot, and the properties of the ellipse.

If you do a yellow-button click on any field in an inspector, you will see a menu of operations that can be performed on that field. For instance, you can inspect the value of the field, browse its class, or search its implementation for all references or assignments to the selected variable.

You can also add your own fields to the inspector by using the add field... menu item. This concept is similar to what is known as *watch expressions* in some other IDEs. For instance, in the inspector of the EllipseMorph, choose add field..., type **self** position, and accept. As you drag the ellipse around in the world, you will see the position of the ellipse change in the inspector.

The object explorer

The *object explorer* is conceptually similar to the inspector, but presents its information in a different way. To see the difference, we'll *explore* the same object that we were just inspecting.

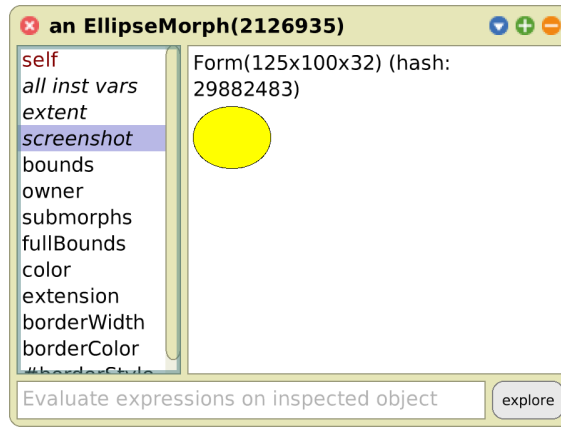


Figure 6.12: Inspecting an EllipseMorph.

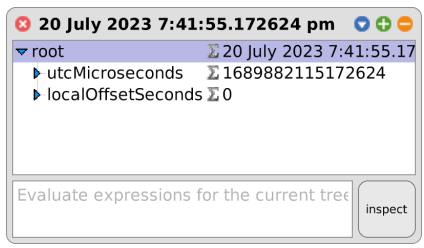


Figure 6.13: Exploring TimeStamp now.

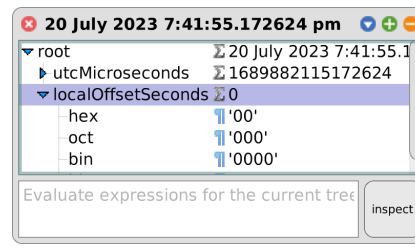


Figure 6.14: Exploring the instance variables.

Select self *in the inspector's left-hand pane, and choose* explore (I) *from the yellow button menu.*

The explorer window looks like Figure 6.13. If you click on the small triangle next to root, the view will change to Figure 6.14, which shows the instance variables of the object that you are exploring. Click on the triangle next to localOffsetSeconds, and you will see *its* instance variables. (Actually, in this example, hexadecimal, octal, etc. are no instance variables of the Integer object, but rather different representations.) The explorer is really useful when you need to explore a complex hierarchic structure — hence the name.

The workspace pane of the object explorer works slightly differently to that of the inspector. **self** is not bound to the root object, but rather to the

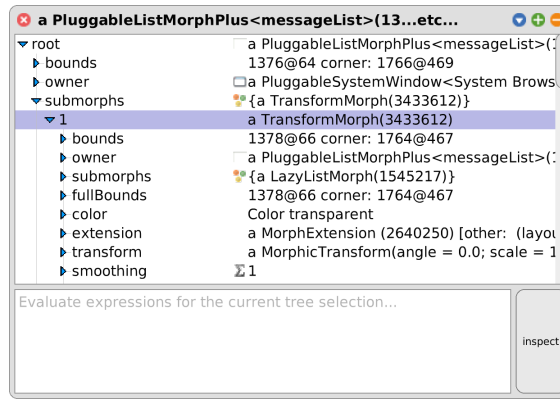


Figure 6.15: Exploring a PluggableListMorph.

object that is currently selected; the instance variables of the selected object are also in scope.

To see the value of the explorer, let's use it to explore a deeply-nested structure of objects.

Open a browser, and blue-click twice on the method pane to bring-up the Morphic halo on the PluggableListMorph *that is used to represent the list of messages. Click on the* debug *handle and select* explore morph *from the menu that appears. This will open an Explorer on the* PluggableListMorph *object that represents the method list on the screen. Open the root object (by clicking in its triangle), open its* submorphs, *and continue exploring the structure of the objects that underlie this Morph, as shown in Figure 6.15.*

The instance browser

Sometimes, you might want to explore what a specific object can do. In the class browser, you can look through the methods to learn what they should do, but you cannot execute them and see what they actually do. In the inspector or explorer, you can see the state of an object and execute methods, but you cannot directly see the code corresponding to the methods. In case you want both, reading code and evaluating code for a single object, you can use the *instance browser* tool. The instance browser allows you to read the code of all methods that an object can execute, and evaluate any code snippets in the context of this object.

You can open an instance browser from the explorer.

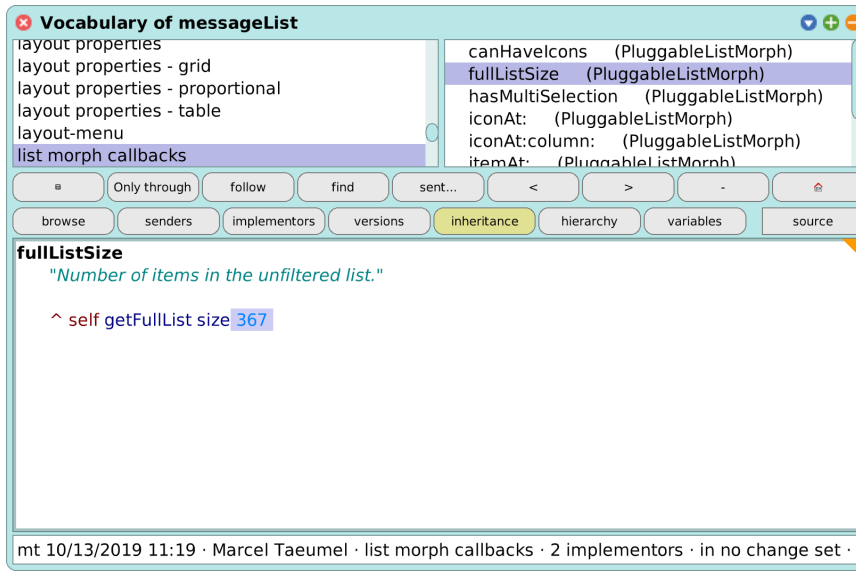


Figure 6.16: Exploring the method fullListSize for a PluggableListMorph.

In the explorer above, select the root entry for the PluggableListMorph, *open the yellow button menu, and choose* browse protocol (p).

In the new browser (Figure 6.16), you can see a list of methods on the top right, which contains all methods the object can execute, gathered from its class and all its superclasses. The methods are categorized according to their protocols on the top left. At the bottom, you see a code pane, just like in an ordinary browser. However, this window is more interesting as it still knows about the PluggableListMorph for which we opened it. Thus, we can evaluate code in the code pane.

In the browser, open the method fullListSize *by selecting it in the list at the top right. Then, navigate to the last code line and* print it *(*CMD*–p). You should see the result of evaluating the expression right within the code editor.*

6.4 The debugger

The debugger is arguably the most powerful tool in the Squeak tool suite. It is used not just for debugging, but also for writing new code. To demonstrate the debugger, let's start by writing a bug!

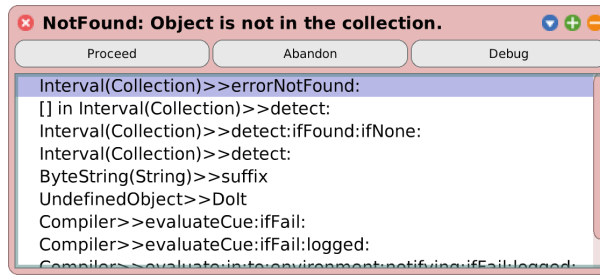


Figure 6.17: A predebugger notifies us of a bug.

Using the browser, add the following method to the class String*:*

Method 6.1: *A buggy method.*

```
1 suffix
2     "assumes that I'm a file name, and answers my suffix, the part after the last dot"
3
4     | dot dotPosition |
5     dot := FileDirectory dot.
6     dotPosition := (self size to: 1 by: -1) detect: [:i | (self at: i) = dot].
7     ^ self copyFrom: dotPosition to: self size
```

Of course, we are sure that such a trivial method will work, so instead of writing an SUnit test, we just type 'readme.txt' suffix into a workspace and print it (p). What a surprise! Instead of getting the expected answer 'txt', a *predebugger* pops up, as shown in Figure 6.17.

The predebugger has a title bar that tells us what error occurred, and shows us a *stack trace* of the messages that led up to the error. Starting from the bottom of the trace, UndefinedObject»DoIt represents the code that was compiled and run when we selected 'readme.txt' suffix in the workspace and asked Squeak to print it. This code, of course, sent the message suffix to a ByteString object ('readme.txt'). This caused the inherited suffix method in class String to execute; all this information is encoded in the next line of the stack trace, ByteString(String)»suffix. Working up the stack, we can see that suffix sent detect:... and eventually detect:ifFound:ifNone: sent errorNotFound.

To find out *why* the dot was not found, we need the debugger itself, so click on Debug.

You can also open the debugger by clicking on any of the lines on the stack trace. If you do this, the debugger will open already focussed on the corresponding method.

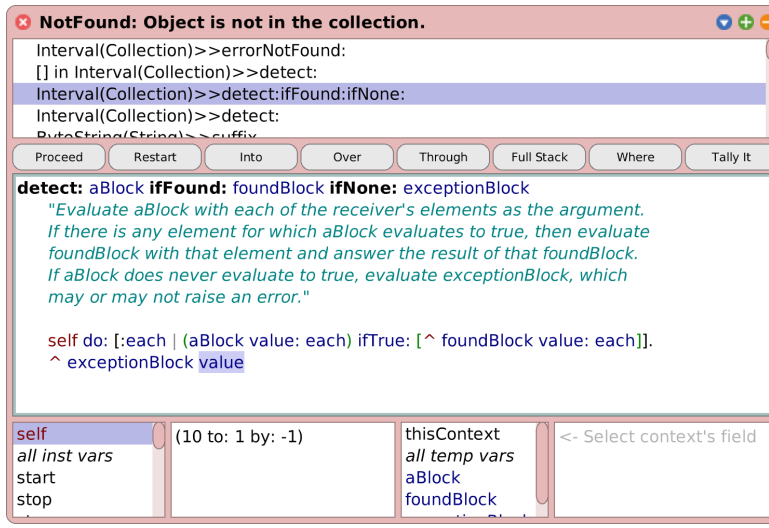


Figure 6.18: The debugger.

The debugger is shown in Figure 6.18; it looks intimidating at first, but it is quite easy to use. The title bar and the top pane are very similar to those that we saw in the predebugger. However, the debugger combines the stack trace with a method browser, so when you select a line in the stack trace, the corresponding method is shown in the pane below. It's important to realize that the execution that caused the error is still in your image, but in a suspended state. Each line of the stack trace represents a frame on the execution stack that contains all of the information necessary to continue the execution. This includes all of the objects involved in the computation, with their instance variables, and all of the temporary variables of the executing methods.

In Figure 6.18 we have selected the detect:ifFound:ifNone: method in the top pane. The method body is displayed in the center pane; the blue highlight in the source code around the message value shows that the current method has sent the message value and is waiting for an answer.

The four panes at the bottom of the debugger are really two mini-inspectors (without workspace panes). The inspector on the left shows the current object, that is, the object named self in the center pane. As you select different stack frames, the identity of self may change, and so will the contents of the self-inspector. If you click on self in the bottom-left pane, you will see that self is the interval (10 to: 1 by: -1), which is what we expect. The workspace panes are not needed in the debugger's mini-inspectors

because all of the variables are also in scope in their value panes and in the method pane; you should feel free to type or select expressions in this pane and evaluate them. You can always cancel (l) your changes using the menu or CMD–*l*.

The inspector on the right shows the temporary variables of the current context. In Figure 6.18, value was sent to the parameter exceptionBlock.

 *To see the current value of this parameter, click on* exceptionBlock *in the context inspector. This will tell you that* exceptionBlock *is* [**self** errorNotFound: ...]. *So, it is not surprising that we see the corresponding error message.*

Incidentally, if you want to open a full inspector or explorer on one of the variables shown in the mini-inspectors, select the name of the variable and choose inspect (i) or explore (I) from the yellow button menu. This can be useful if you want to watch how a variable changes while you execute other code.

Looking back at the method window, we see that we expected the penultimate line of the suffix method to find dot in the string 'readme.txt', and that execution should never have reached the final line. Squeak does not let us run an execution backward, but it does let us start a method again, which works very well in code such as this that does not mutate objects, but instead creates new ones.

 *Click* Restart, *and you will see that the locus of execution returns to the first statement of the current method. The blue highlight shows that the next message to be sent will be* do: *(see Figure 6.19).*

The Into and Over buttons give us two different ways to step through the execution. If you click Over, Squeak executes the current message send (in this case the do:) in one step, unless there is an error. So Over will take us to the next message send in the current method, which is value: This is exactly where we started, and not much help. What we need to do is to find out why the do: is not finding the character that we are looking for.

 Click Over, *and then click* Restart *to get back to the situation shown in Figure 6.19.*

 *Click* Into; *Squeak will go into the method corresponding to the highlighted message send, in this case,* Interval»do:.

However, it turns out that this is not much help either: We can be fairly confident that Interval»do: is not broken. The bug is much more likely to be in *what* we asked Squeak to do. Through is the appropriate button to use in this case: We want to ignore the details of the do: itself and focus on the execution of the argument block.

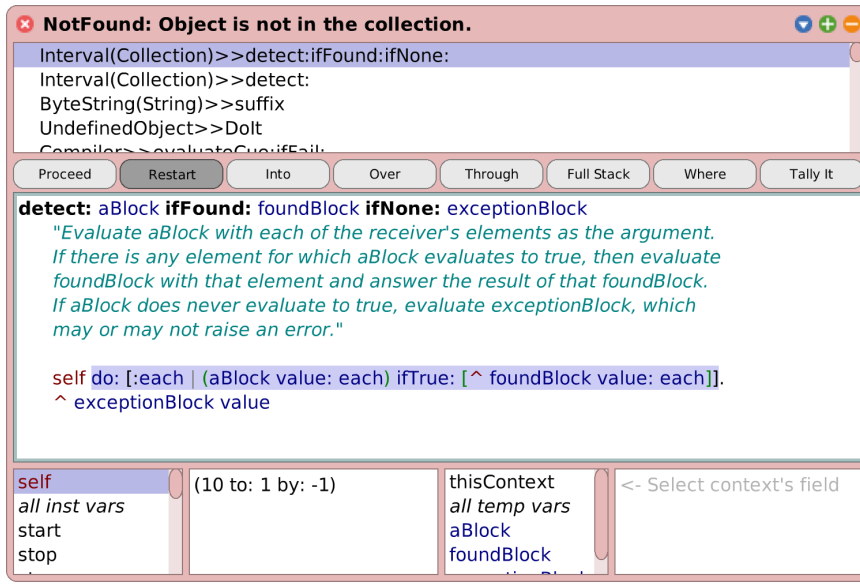


Figure 6.19: The debugger after restarting the detect:ifFound:ifNone: method.

Click on Through *a few times. Select* each *in the context window as you do so. You should see* each *count down from* 10 *as the* do: *method executes.*

When each is 7 we expect the ifTrue: block to be executed because the dot is at position 7, but the ifTrue: block is not executed. To see what is going wrong, go Into the execution of value: as illustrated in Figure 6.20.

After clicking Into, we find ourselves in the position shown in Figure 6.21. It looks at first that we have gone *back* to the suffix method, but this is because we are now executing the block that suffix provided as argument to detect:. If you select i in the context inspector, you can see its current value, which should be 7 if you have been following along. You can then select the corresponding element of self from the self-inspector. In Figure 6.21 you can see that element 7 of the string is character 46, which is indeed a dot. If you select dot in the context inspector, you will see that its value is '.'. And now you see why they are not equal: the seventh character of 'readme.txt' is a Character, while dot is a String.

Now that we see the bug, the fix is obvious: we have to convert dot to a character before starting to search for it.

Change the code right in the debugger so that the assignment reads dot :=

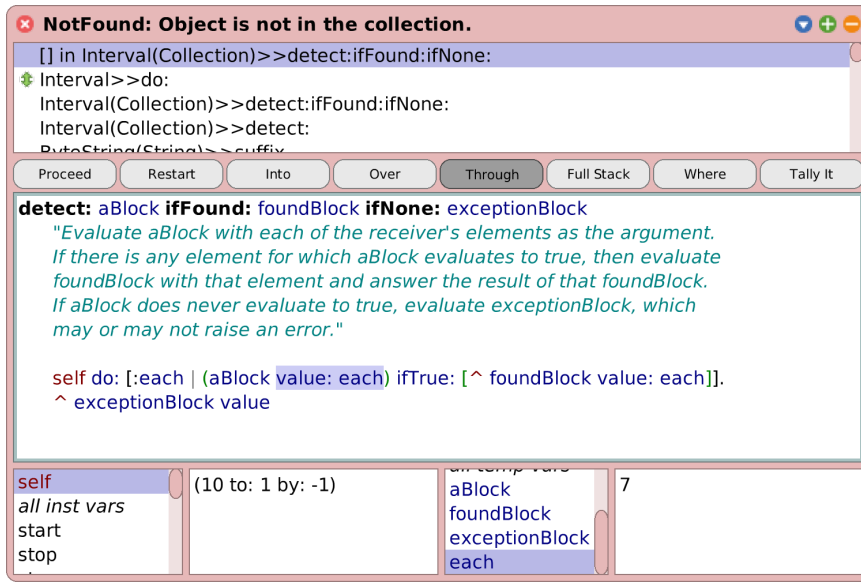


Figure 6.20: The debugger after stepping Through the do: method several times.

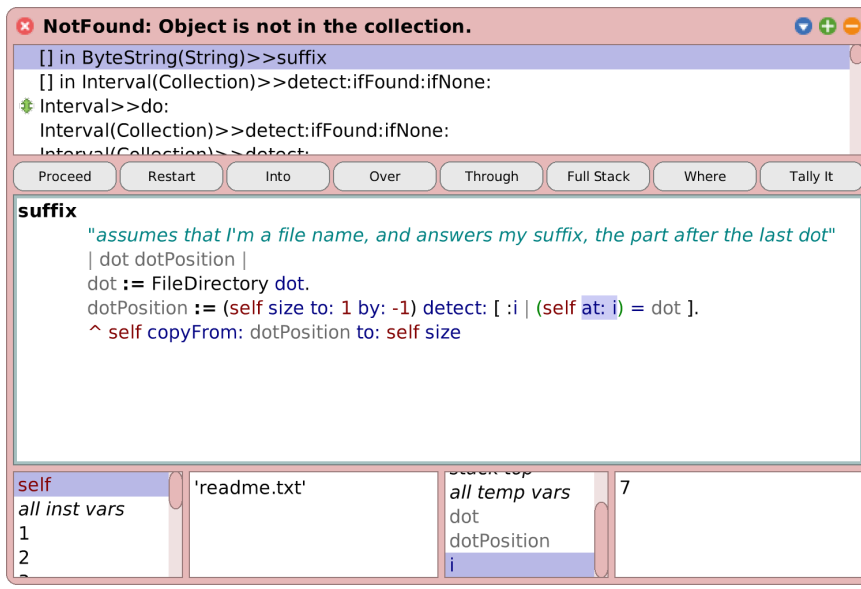


Figure 6.21: The debugger showing why 'readme.txt' at: 7 is not equal to dot.

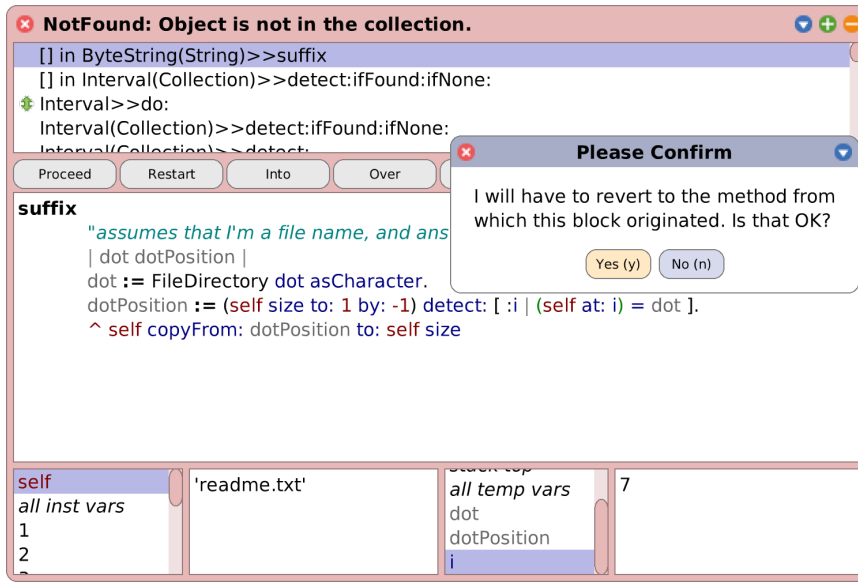


Figure 6.22: Changing the suffix method in the debugger: asking for confirmation of the exit from an inner block.

FileDirectory dot asCharacter *and* accept *the change.*

Because we are executing code inside a block that is inside a detect:, several stack frames will have to be abandoned in order to make this change. Squeak asks us if this is what we want (see Figure 6.22), and, assuming that we click yes, will save (and compile) the new method.

 *Click* Restart *and then* Proceed*; the debugger window will vanish, and the evaluation of the expression* 'readme.txt' suffix *will complete, and print the answer* '.txt'

This was a first tour through the basic features of the debugger. As a result of the reflective nature of the Squeak system, the debugger is a very powerful tool with numerous advanced features. You can discover some of them via the yellow button menu in the mini-inspectors, the stack trace, and the method browser. For example, you might want to try return entered value in the yellow button menu of the stack trace, which directly returns from the context with the entered value. You might also want to try the following two features in the yellow button menu of the code editor: run to selection, which runs the process up to the selected expression, and run until, which asks you for an expression and continues to run the process until the

expression evaluates to true.

From the debugger to tests and back again

Is the answer correct? Unfortunately, we can't say for sure. Should the suffix be .txt or txt? The method comment in suffix is not very precise. A good way to avoid this kind of problem is to write an SUnit test that defines the answer. As you will see in the following example, we can also use the tests to advance our debugging sessions quickly.

We start by adding the following test method 6.2 to StringTest.

Method 6.2: *A simple test for the* suffix *method*

```
testSuffixFound

    self assert: 'txt' equals: 'readme.txt' suffix.
```

The effort required to do that was little more than to run the same code in the workspace, but by creating a test, we save the code as executable documentation, and make it easy for others to run. Moreover, if you now run that test suite with SUnit, you can very quickly get back to debugging the error. SUnit opens the debugger on the failing assertion, but you need only go back down the stack two frames, Restart the test, and go Into the suffix method, and you can correct the error, as we are doing in Figure 6.23. It is then only a second of work to click on the Run Failures button in the SUnit Test Runner and confirm that the test now passes.

Here is a better test:

Method 6.3: *A better test for the* suffix *method.*

```
testSuffixFound

    self
        assert: 'txt' equals: 'readme.txt' suffix;
        assert: 'txt' equals: 'read.me.txt' suffix.
```

Why is this test better? Because it tells the reader what the method should do if there is more than one dot in the target String.

That's all there is to our initial test, but it's not all there is to the suffix method. The initial bug should have made you realize that if there is no dot in the target string, the suffix method will raise an error. This isn't the behavior that we want, so let's add a second test to specify what should happen in this case.

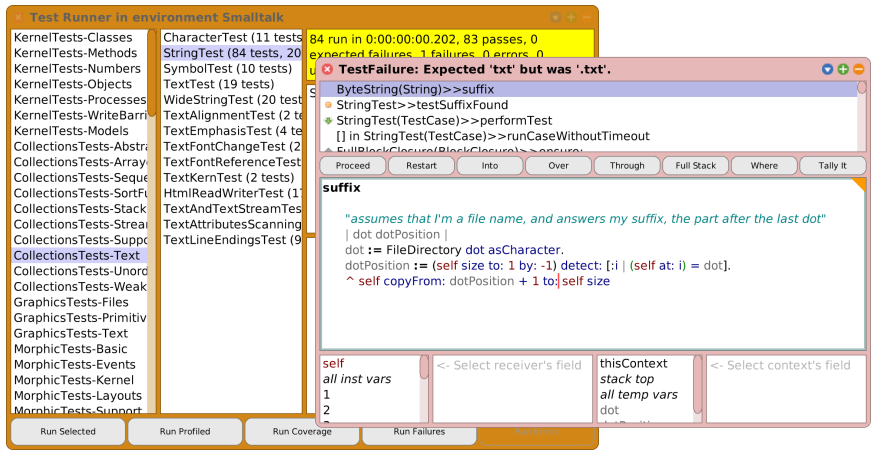


Figure 6.23: Changing the suffix method in the debugger: fixing the off-by-one error after an SUnit assertion failure.

Method 6.4: *A second test for the* suffix *method: the target has no suffix.*

```
1 testSuffixNotFound
2
3   self assert: '' equals: 'readme' suffix.
```

Add method 6.4 to the test suite in class StringTest*, and watch the test raise an error. Enter the debugger by selecting the erroneous test in SUnit, and edit the code so that the test passes. The easiest and clearest way to do this is to replace the* detect: *message by* detect:ifNone:*, where the second argument is a block that simply returns the string size.*

We will learn more about SUnit in Chapter 8.

Ways into the debugger

There are a few other ways to get into the debugger in addition to catching errors and assertion failures. First of all, you can interrupt processes and open them in the debugger interactively. If you execute code that goes into an infinite loop, you can interrupt it and open a debugger on the computation by typing CMD–. (that's a full stop or a period, depending on where you learned English). You can also open a debugger on almost all running processes (see Section 6.5). To do so, open the process browser and in the yellow button menu of a process, you can select to debug it.

When you open a debugger on a computation interactively, you suspend the computation wherever it is at that moment. Sometimes, you want to specify exactly at what point in the code you want to open a debugger. Therefore, you edit the suspect code and insert **self** halt at the point at which you want to open a debugger. So, for example, we might edit the suffix method to read as follows:

Method 6.5: *Inserting a* halt *into the* suffix *method.*

```
1 suffix
2     "assumes that I'm a file name, and answers my suffix, the part after the last dot"
3
4     | dot dotPosition |
5     dot := FileDirectory dot asCharacter.
6     dotPosition := (self size to: 1 by: -1) detect: [:i | (self at: i) = dot].
7     self halt.
8     ^ self copyFrom: dotPosition to: self size
```

When we run this method, the execution of the **self** halt will bring up the pre-debugger, from where we can proceed, or go into the debugger and look at variables, step the computation, and edit the code.

Advanced ways into the debugger Sometimes, a simple halt can be problematic or insufficient. For these cases, there are some special ways to interrupt and inspect the execution.

For instance, when putting a halt in code that restarts in another process whenever it is stopped, you would get an infinite number of debuggers. In such a situation, we might only want to interrupt the execution once, let the execution restart in another process, and leave that new process alone. For this use case, you can use haltOnce, which only interrupts the execution the first time it is executed. You can use it in source code just like halt, but before you start your execution, you first have to do 0 setHaltOnce, which activates the haltOnce (actually the receiver of setHaltOnce does not matter, send it to whoever you want). The corresponding message inspectOnce opens an inspector for whatever object the message was sent to and only does so once, again to save you from vast numbers of inspectors.

Another problem you might encounter while debugging is that the system encounters your halt repeatedly, but you have to proceed many of the debuggers to find the point in time you are interested in. For example, when you have a halt in a loop working on the characters of a long string, you might only be interested in the loop iteration in which a separator is encountered, which unfortunately is the last third of the string. To only interrupt the execution whenever a condition is met, for example, the separator character was read from the string, you can use haltIf:. The

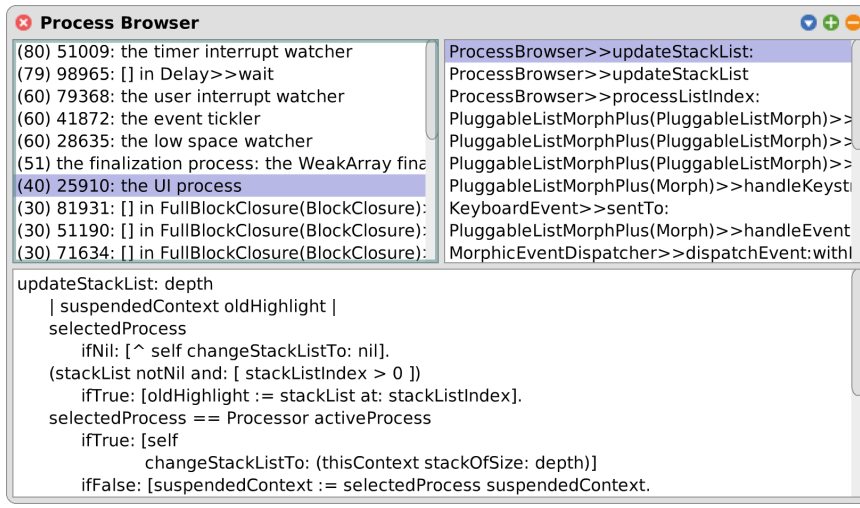


Figure 6.24: The process browser.

method accepts a block that tests for the condition. Alternatively, you can pass a selector and the debugger will stop as soon as a method appears on the stack that matches this selector.

You can also interactively set for which methods the execution should be interrupted. You can do so in the yellow button menu on any list of methods via the option break on entry. When you activate it, a debugger will open every time an execution enters the method.

6.5 The process browser

Smalltalk is a multi-threaded system: there are many lightweight processes (also known as threads) running concurrently in your image. The Squeak virtual machine implements concurrency by time-slicing.

The process browser is a cousin of the debugger that lets you look at the various processes running inside Squeak. Figure 6.24 shows a screenshot. The top-left pane lists all of the processes in Squeak, in priority order, from the timer interrupt watcher at priority 80 to the idle process at priority 10. Of course, on a uniprocessor, when you look which program is running, the only process that can be running is the UI process, as it is currently displaying the information to you; all of the other will be waiting for some kind of event. By default, the display of processes is static; it can be updated

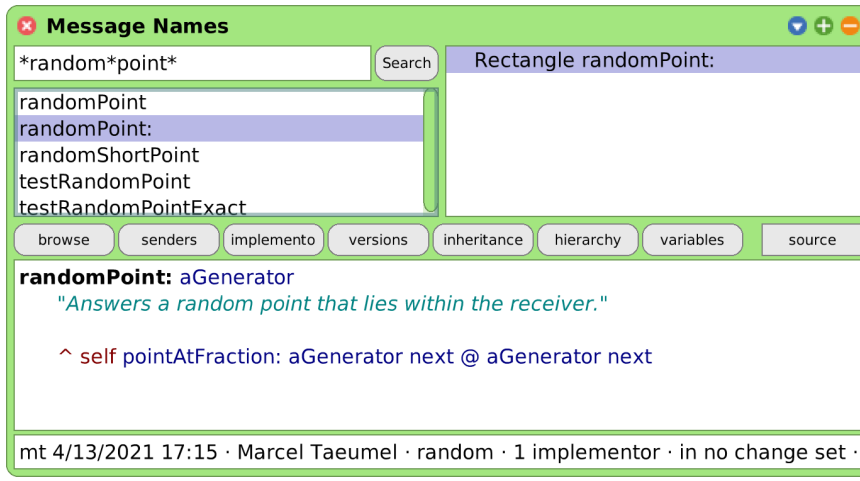


Figure 6.25: The method names browser showing all methods containing the substring random, followed by anything else, followed by the substring point in their selectors.

using the yellow button menu, which also provides an option to turn on auto-update (a)

If you select a process in the top-left pane, its stack trace is displayed in the top-right pane, just as with the debugger. If you select a stack frame, the corresponding method is displayed in the bottom pane. The process browser is not equipped with mini-inspectors for self and thisContext, but yellow button menu items on the stack frames provide equivalent functionality.

6.6 Finding methods

There are three tools in Squeak to help you find messages. You can open the first two, *method finder* and *message names*, from World docking bar ▷ Tools. The third one is the global search in the right corner of the world docking bar. They differ in both interface and functionality.

The *method finder* was described at some length in Section 1.8; you can use it to find methods by name or by functionality. However, to look at the body of a method, the method finder opens a new browser. This can quickly become overwhelming.

The *message names* browser has more limited search functionality: You type a fragment of a message selector in the search box, and the browser lists all methods that contain that fragment in their names, as shown in Figure 6.25. Instead of the fragment, you can also specify a search term with wildcards (* for zero or more characters, and # for exactly one character). However, it is a full-fledged browser: If you select one of the names in the left pane, all of the methods with that name are listed in the right pane and can be browsed in the bottom pane. As with the class browser, the message names browser has a button bar that can be used to open other browsers on the selected method or its class.

The *search bar* in the top right corner allows you to quickly access any method in the system. You can either directly enter the full selector of the method, which will open a class browser, or enter a search term including wildcards, which will open a message names browser. You can also use the search bar to search for classes also using wildcards. As this search bar allows you to quickly jump to methods, it is also accessible via the keyboard shortcut CMD−0.

6.7 Other interesting tools

Beyond the common tools introduced in this chapter, Squeak also provides a number of other interesting and rich tools. We briefly introduce some of these tools which you might want to use when you are more proficient in using Squeak:

Dependency browser Whenever you use a class in a method, the method then depends on the presence of this class. Running the method will result in an exception if the class is not in the environment, as Squeak does not know who should receive the message. This can occur, for example, whenever you use a class that is not part of the base system but stems from a project you installed. Using the dependency browser you can see the dependencies of your package. You can open the dependency browser through world main docking bar ▷ Apps ▷ Dependency Browser or from the context menu of a system category in a browser (browse package ▷ dependencies).

Lexicon Objects understand all messages for which their class or their superclasses have methods. The system browser introduced earlier can only show you the methods of one class. If you want to know all the methods an instance of a class understands you can open the lexicon. You can get the lexicon from the system browser by opening the yellow button menu for the class and selecting browse protocol (p).

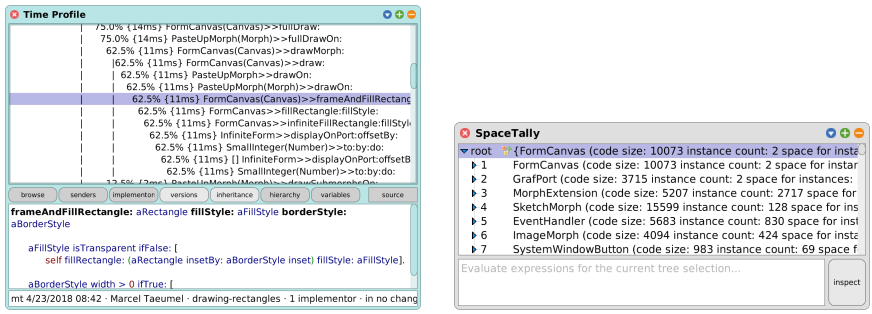


Figure 6.26: A time profiler (left) and a space tally (right).

Profiling tools Sometimes you might write code that is too slow. For that, Squeak provides a number of profiling tools to measure execution time and help you find bottlenecks.

First, you can send the message timeToRun to a block to measure the time in milliseconds it takes to execute it:

```
[10000 factorial] timeToRun    ⟶    27
```

Alternatively, you can select a piece of code in a workspace, debugger, autc., open the yellow button menu, and select tally it. If a single execution of the code is too fast to measure (confidently), you can also *benchmark* it by sending bench or benchFor: to a block. This will run the block repeatedly and compute a few statistics about the execution time.

```
[10 factorial] bench    ⟶    '14,500,000 per second. 68.8 nanoseconds per run. 0 %
      GC time.'
```

If you have discovered that your code is too slow, you might wonder which parts of your code are responsible for the slowness. For that, Squeak provides the *message tally* tool. You can start it to record the execution times for your code by selecting the code and choosing spy on it from the context menu. Alternatively, you can let it observe all processes in the image from the menu item world main docking bar ▷ Apps ▷ Message Tally. Note that the message tally is a sampling profiler, so for detailed information, you will have to let your code run for some seconds (*e.g.*, by running it 1000 times if it only takes a little time).

An extension of the message tally is the *time profile browser* which allows you to browse the results of a message tally conveniently (see Figure 6.7). You can invoke it by sending timeProfile to a block. To get rid of the long stack of the existing process, a practical way to invoke it is:

```
[[10000 factorial] timeProfile] fork
```

Time is not the only resource that is unfortunately limited. You can also use the *space tally* to find out which objects consume the most memory:

```
SpaceTally new spaceForInstance: Object new depth: 0.    ⟶  16
SpaceTally new spaceForInstance: Morph new depth: 0.     ⟶  56
SpaceTally new spaceForInstance: Morph new depth: 1.     ⟶  144
SpaceTally new spaceForInstance: Morph new depth: 2.     ⟶  208
```

The depth parameter specifies how many levels of the object graph should be traversed. As you can see, even a plain Object instance consumes 16 bytes. Like the message tally, you can also ask a space tally to analyze all objects in the system by providing a list of classes and exploring the result (see Figure 6.7):

```
SpaceTally new spaceTally: {TextMorph. Point}.
SpaceTally new depth: 2; spaceTally: (PackageInfo named: #Morphic) classes.
SpaceTally new systemWideSpaceTally.
```

6.8 Chapter summary

In order to develop effectively with Squeak, it is important to invest some effort into learning the tools available in the environment.

- The standard *system browser* is your main interface for browsing existing class categories, classes, method protocols and methods, and for defining new ones. The system browser offers several useful buttons to directly jump to senders or implementors of a message, versions of a method, and so on.
- There exist several different class browsers (such as the System Browser and the Snapshot Browser from Monticello), and several specialized browsers (such as the hierarchy browser) which provide different views of classes and methods.
- From any of the tools, you can highlight the name of a class or a method and immediately jump to a browser by using the keyboard shortcut CMD–b.
- You can also browse the Smalltalk system programmatically by sending messages to **self** systemNavigation.

- The *inspector* (CMD–i) and the *explorer* (CMD–I) are two tools that are useful for exploring and interacting with live objects in your image. You can even inspect tools by blue-clicking to bring up their morphic halo and selecting the debug handle  .

- The *debugger* is a tool that not only lets you inspect the run-time stack of your program when an error is raised, but it also enables you to interact with all of the objects of your application, including the source code. In many cases, you can modify your source code from the debugger and continue executing. The debugger is especially effective as a tool to support test-first development in tandem with SUnit (Chapter 8).

- The *process browser* lets you monitor, query and interact with the processes that currently exist in your image.

- The *method finder*, the *message names browser*, and the *search bar* are three tools for locating methods and/or classes. The first is more useful when you are not sure of the name, but you know the expected behavior. The second offers a more advanced browsing interface when you know at least a fragment of the name. The third allows you to quickly navigate to methods and classes.

Chapter 7

Storing and sharing source code

The consequence of programming in a world of live objects rather than with files and a text editor is that you have to do something explicit to export your program from your Smalltalk image. The old way of doing this, also supported by all Smalltalk dialects, is by creating a *fileout* or a *change set*, which are essentially encoded text files that can be imported into another system. A newer way of doing this in Squeak is to upload your code to a versioned repository on a server using a tool called Monticello. Both have their strengths and weaknesses: Monticello makes it easier to track and distribute changes, especially when working in a team, while change sets do not require you to set up a server repository and a package organization.

In this chapter, we will look at the tools that Squeak provides for sharing and storing source code:

- The Monticello Browser is the starting point for loading code from, and saving code in, Monticello packages.
- Using the Change Sorter, you can organize your source code modifications, review the code, and write it to a file.
- The File List Browser allows you to browse the local file system on your machine, including an option to load source code from a file into your image.
- Through the Change List Browser, you can recover any code changes that you have made to your image, even if you have not saved your image.

- Git Tools make it possible to manage your source code using the Git version control system from within Squeak.

7.1 Packages and Monticello

We gave you a quick overview of Monticello, Squeak's packaging tool, in Section 2.9. However, Monticello has many more features than were discussed there. Monticello manages *Packages*. Thus, before telling you more about Monticello, we will first explain exactly what a package is.

Packages: declarative categorization of Squeak code

The package system is a simple, lightweight way of organizing Smalltalk source code. It leverages the long-used naming convention of Squeak (Section 6.2), but adds to it in an important way.

Let's look at this in an example. Suppose that you are developing a framework to facilitate the use of relational databases from Squeak. You have decided to call your framework SqueakLink, and have created a series of system categories to contain all of the classes that you have written, *e.g.*:

- the category 'SqueakLink-Connections' contains classes OracleConnection, MySQLConnection, PostgresConnection,
- the category 'SqueakLink-Model' contains DBTable, DBRow, DBQuery.

We will manage the code for these classes in a package named SqueakLink .

Extension methods

However, not all of your code will reside in these classes. For example, you may also have a series of methods to convert objects into an SQL-friendly format:

```
Object»asSQL
String»asSQL
Date»asSQL
```

These methods belong in the same package as the classes in the categories SqueakLink-Connections and SqueakLink-Model. But clearly the whole of class Object does not belong in your package! So you need a way of putting

certain *methods* in a package, even though the rest of the class is in another package.

This is where the concept of *extension methods* comes in. You can mark these converter methods as extension methods by placing them in a protocol (of Object, String, Date, and so on) named **SqueakLink-Model-converting* (note the initial asterisk). As a result, the three methods are now being managed as part of the SqueakLink package. In general, any method in any protocol that starts with an asterisk, followed by a package name, followed by a hyphen, is an extension method to that package.

Override methods

Sometimes, you may also want to alter a method from another package that already exists and store the changed method in your package. For instance, this can be required if you need to extend an existing feature or fix a bug in the standard Squeak image. Let's say that you want to add a menu item to an inspector's context menu for opening its SQL version. Therefore, you have edited Inspector>>addObjectItemsTo: and now would like to check it into your package repository.

While you could place it in a simple extension protocol as above, this would introduce some problems: First, the method would be displayed as missing in the original package, so the package would unintentionally become dirty. Second, if you unloaded your own package again, the original method would be lost.

A better way to do this is to mark the method as an overriding method by placing it in a protocol named **SqueakLink-menu-override*. In general, an override protocol is an extension protocol that ends with the suffix *-override*. Usually, it's a good idea to repeat the name of the original protocol in the override protocol name.

However, beware that every change to an existing package might break other packages that depend on it. Thus, use override methods as seldom as possible. Often, there are better ways to extend the functionality of a package, such as subclassing or using explicit hooks[1].

Surveying the contents of a package

The combination of the *SqueakLink-...* categories and the **SqueakLink-...* protocols form the package named SqueakLink. To be precise, the rules for what

[1]Actually, there is a better way to extend the menu in our example: Place a new (non-override) extension method named squeakLinkMenu: in the Inspector class and add the pragma <fieldListMenu> to it.

goes in a package are as follows.

A package named Foo contains:

1. all class definitions of classes in the category *Foo*, or in categories with names starting with *Foo-* (or *foo-*, *fOO*, autc., as Monticello ignores the case of the letters),
2. all method definitions in any class in a protocol named **Foo* or whose name starts with **Foo-*, and
3. all methods in classes in the category *Foo* or in a category whose name starts with *Foo-*, *except* for those methods in protocols whose names start with ***.

A consequence of these rules is that each class definition and each method belongs to exactly one package. The *exception* in the last rule takes care of excluding methods that belong to other packages.

The class PackageInfo implements these rules, and one way to get a feel for them is to experiment with this class.

 *Try this in your image, which should contain the classes* PackageInfo *and* Morph.

The Morphic code uses these package naming conventions, with Morphic as the package name. In a workspace, create a model of this package with

```
morphic := PackageInfo named: 'Morphic'.
```

It is now possible to introspect on this package. For example, morphic classes will return the long list of classes from the core of Morphic as well as a number of specific Morph classes. morphic coreMethods will return a list of MethodReferences for all of the methods in those classes. morphic extensionMethods is perhaps one of the most interesting queries: It will return a list of all methods contained in the Morphic package but not contained within a Morphic class. This includes, for example, Object»asDraggableMorph: and Text»embeddedMorphs.

 *Evaluate* (PackageInfo named: 'Collections') externalSubclasses*; this expression will answer a list of all subclasses of* Collection *that are* not *in the* Collections *package.*

You can send fileOut to an instance of PackageInfo to get a change set of the entire package. For more sophisticated versioning of packages, we use Monticello.

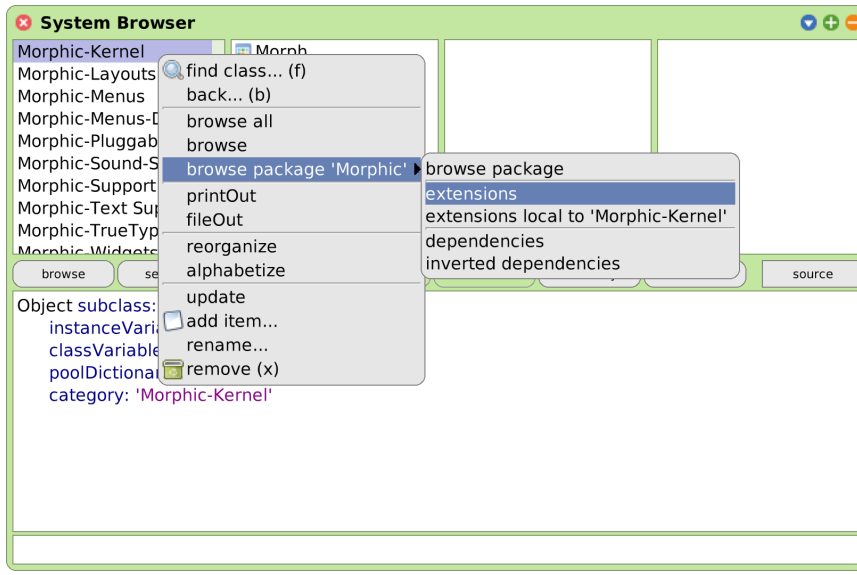


Figure 7.1: Browsing all extension methods of the package Morphic.

If you do not feel like playing with the PackageInfo class manually, you can also use the package pane browser introduced in Section 6.2. You can also browse all extension methods of a package by opening a yellow-button menu on any package or system category in the browser and selecting browse package extensions (see Figure 7.1).

Basic Monticello

Monticello is named after the mountaintop home of Thomas Jefferson, third president of the United States and author of the Statute of Virginia for Religious Freedom. The name means "little mountain" in Italian and so it is always pronounced with an Italian "c" ([mˌɔntɪʃˈɛloʊ], which sounds like *Mont-y'-che-llo*).

When you open the Monticello browser (*e.g.*, in World docking bar ▷ Apps ▷Monticello Browser), you will see two list panes and a row of buttons, as shown in Figure 7.2. The left-hand pane lists all of the packages that have been loaded into the image that you are running; the particular version of the package is shown in parentheses after the name.

The right-hand pane lists all of the source-code repositories that Monticello knows about, usually because it has loaded code from them. If you

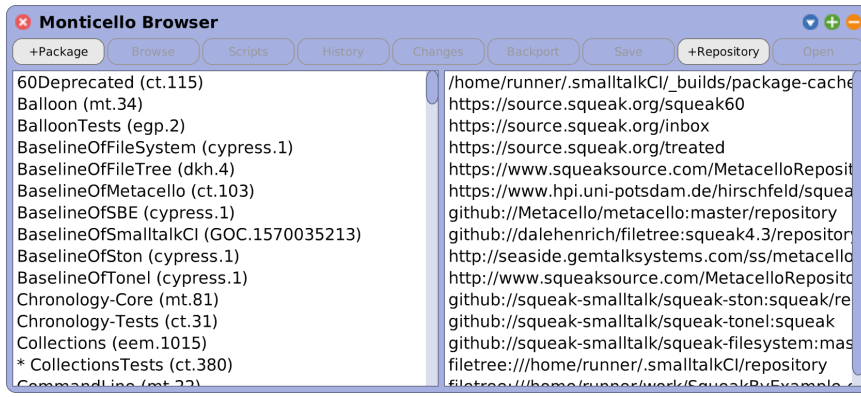


Figure 7.2: The Monticello browser.

select a package in the left pane, the right pane is filtered to show only those repositories that contain versions of the selected package.

One of the repositories is a directory named *package-cache*, which is a subdirectory of the directory in which your image is running. When you load code from or write code to a remote repository, a copy is also saved in the package cache. This can be useful if the network is not available and you need to access a package. Also, if you are given a Monticello (.mcz) file directly, for example as an email attachment, the most convenient way to access it is to place it in the package-cache directory.

To add a new repository to the list, click the +Repository, and choose the kind of repository from the pop-up menu. Let's add an HTTP repository.

Open Monticello, click on +Repository*, and select* HTTP*. Edit the dialog to read:*

```
MCHttpRepository
    location: 'http://www.squeaksource.com/SqueakByExample60'
    user: ''
    password: ''
```

Then click on Open to open a repository browser on this repository. You should see something like Figure 7.3. On the left is a list of all of the packages in the repository; if you select one, then the pane on the right will show all of the versions of the selected package in this repository.

If you select one of the versions, you can Browse it (without loading it into your image), Load it, or look at the Changes that will be made to

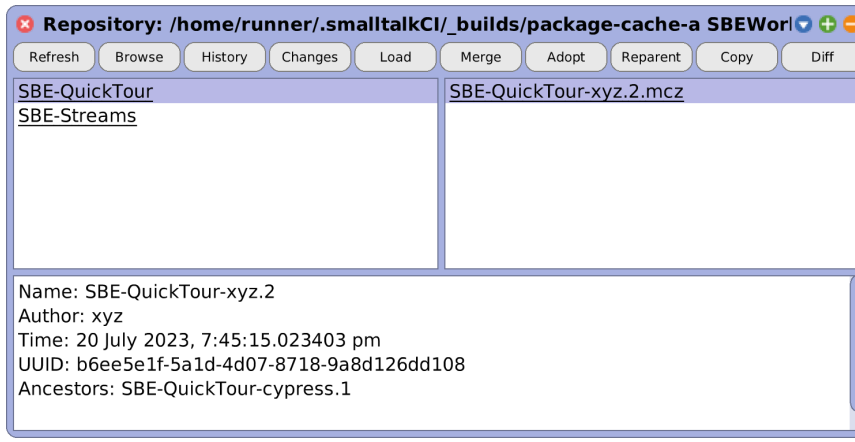


Figure 7.3: A repository browser.

your image by loading the selected version. If you have local changes to the package you want to load, you should use Merge, as Load will fully load the selected version and thereby delete your own changes. You can also make a Copy of a version of a package, which you can then write to another repository.

As you can see, the names of versions contain the name of the package, the initials of the author of the version, and a version number. The version name is also the name of the file in the repository. Never change these names; the correct functionality of Monticello depends on them! Monticello version files are just zip archives, and if you are curious you can unpack them with a zip tool, but the best way to look at their contents is using Monticello itself.

To create a package with Monticello, you have to do two things: write some code, and tell Monticello about it.

Create a category called SBE-Monticello, *and put a couple of classes in it, as shown in Figure 7.4. Also, create a method in an existing class, and put it in the same package as your classes, using the rules from page 144 — see Figure 7.5.*

To tell Monticello about your package, click on +Package, and type the name of the package, in this case "SBE". Monticello will add SBE to its list of packages; the package entry will be marked as *dirty* with an asterisk to show that the changes in the image have not yet been written to any repository.

Initially, the only repository associated with this package will be your

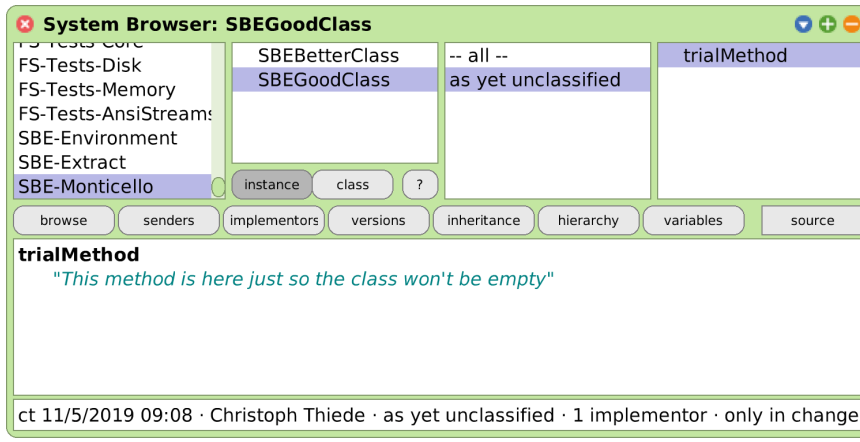


Figure 7.4: Two classes in the "SBE" package.

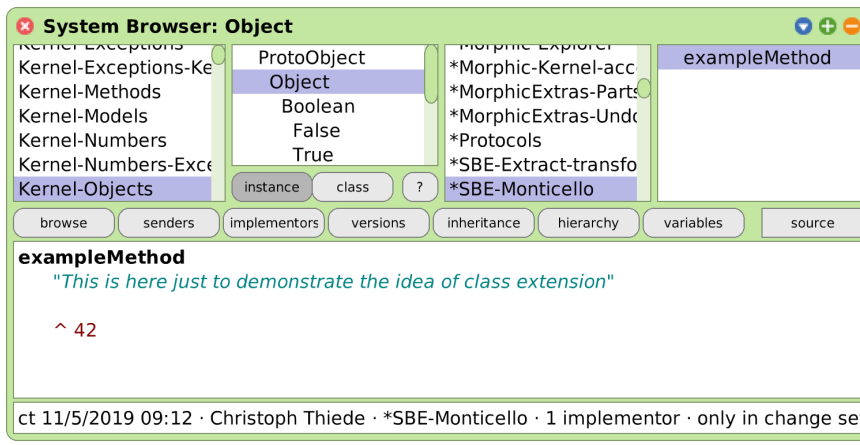


Figure 7.5: An extension method that will also be in the "SBE" package.

package cache, as shown in Figure 7.6. That's OK: You can still save the code, which will cause it to be written to the package cache. Just click Save and you will be invited to provide a log message for the version of the package that you are about to save, as shown in Figure 7.7; when you accept the message, Monticello will save your package. To indicate this, the asterisk decorating the name in Monticello's package pane will be removed, and the version number added.

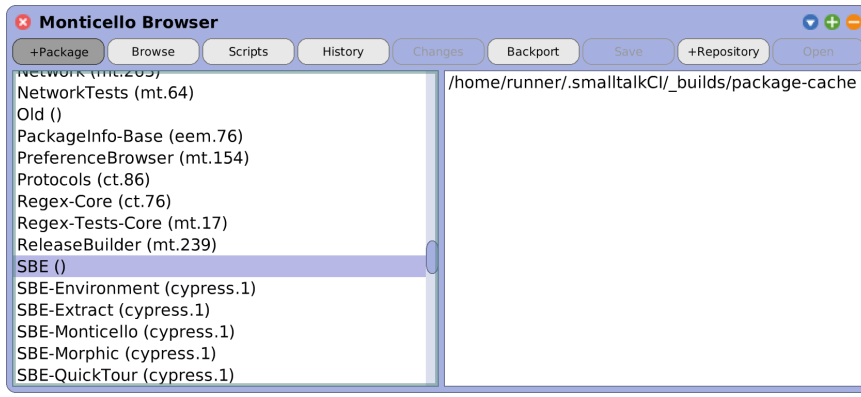


Figure 7.6: The as-yet-unsaved SBE package in Monticello.

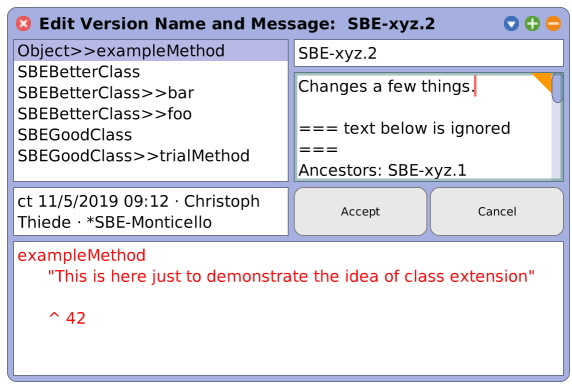


Figure 7.7: Providing a log message for a new version of a package.

If you then make a change to the package — say by adding a method to one of the classes — the asterisk will re-appear, showing that you have unsaved changes. If you open a repository browser on the package cache, you can select the saved version, and use Changes and the other buttons. You can of course save the new version to the repository too; once you Refresh the repository view, it should look like Figure 7.8.

To save the new package to a repository other than the package cache, you need to first make sure that Monticello knows about the repository, adding it if necessary. Then you can use the Copy in the package-cache repository browser, and select the repository to which the package should

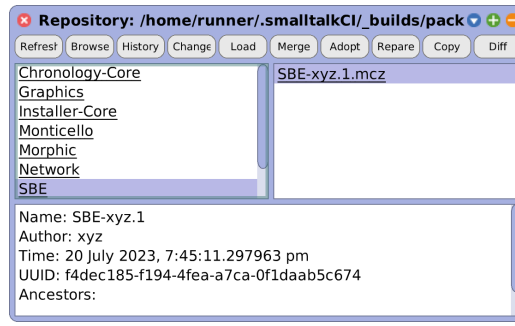


Figure 7.8: Two versions of our package are now in the package cache.

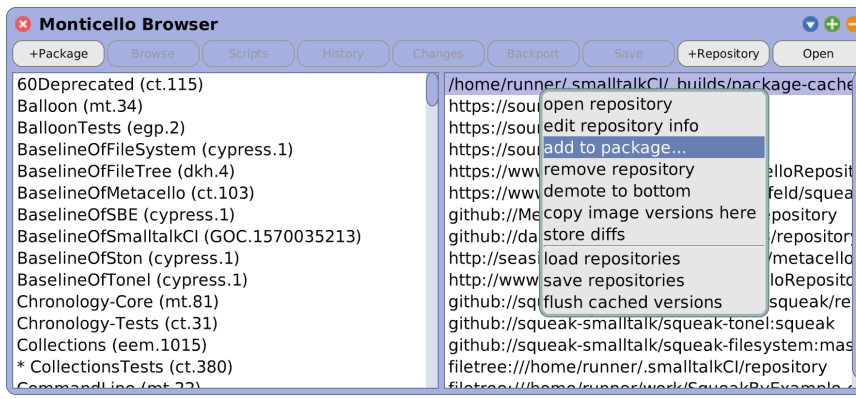


Figure 7.9: Adding a repository to the set of repositories associated with a package.

be copied. You can also associate the desired repository with the package by using the yellow button menu item add to package... on the repository, as shown in Figure 7.9. Once the package knows about a repository, you can save a new version by selecting the repository and the package in the Monticello Browser, and clicking Save. Of course, you must have permission to write to a repository. The SqueakByExample repository on *SqueakSource* is world-readable but not world-writable, so if you try and save there, you will see an error message. However, you can create your own repository on *SqueakSource* by using the web interface at http://www.squeaksource.com, and use this to save your work. This is especially useful as a mechanism to share your code with friends, or if you use multiple computers.

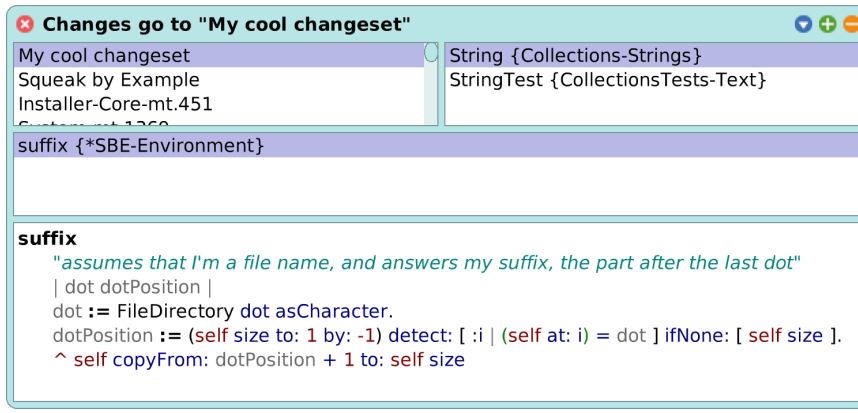


Figure 7.10: The change set browser.

If you do try and save to a repository where you don't have write permission, a version will nevertheless be written to the package-cache. So you can recover by editing the repository information (yellow button menu in the Monticello Browser) or choosing a different repository, and then using Copy from the package-cache browser.

7.2 Change sets and the change sorter

Whenever you are working in Squeak, any changes that you make to methods and classes are recorded in a change set. This includes creating new classes, re-naming classes, changing categories, adding methods to existing classes — just about everything of significance. However, arbitrary *doits* are not included, so if, for example, you create a new global variable by assigning to it in a workspace, the variable creation will not make it into a change set.

At any time, many change sets exist, but only one of them — ChangeSet current — is collecting the changes that are being made to the image. You can see which change set is current and can examine all of the change sets using the change set browser, available from World ▷ open... ▷ simple change sorter or World docking bar ▷ Tools.

Figure 7.10 shows this browser. The title bar shows which change set is current, and this change set is selected when the browser opens.

Other change sets can be selected in the top-left pane; the yellow button menu allows you to make a different change set current (make changes go

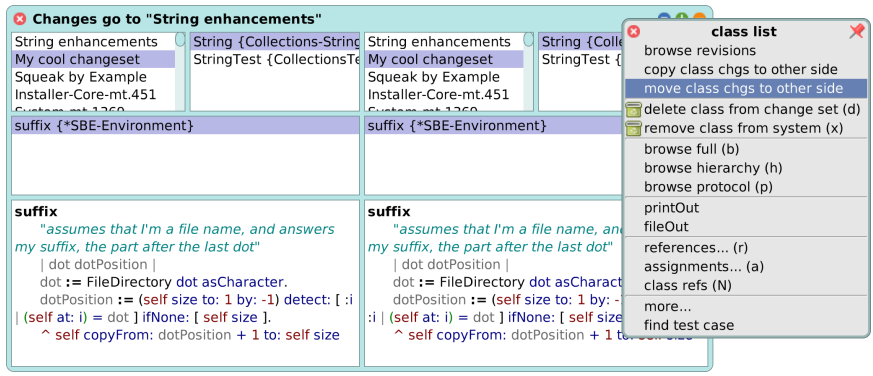


Figure 7.11: The dual change sorter.

to me (m)), or to create a new change set. The top-right pane lists all of the classes affected by the selected change set (with their categories). Selecting one of the classes displays the names of those of its methods that are also in the change set (*not* all of the methods in the class) in the central pane, and selecting a method name displays the method definition in the bottom pane. If you do not select a method, the bottom pane also shows you whether the creation of the class itself is part of the change set.

The change set browser also lets you delete classes and methods from the change set using the yellow button menu on the corresponding items. However, for more elaborate editing of change sets, you should use a second tool, the *Dual Change Sorter*, available under World docking bar ▷ Tools or by selecting World ▷ open... ▷ dual change sorter, which is shown in Figure 7.11.

The dual change sorter is essentially two change set browsers side by side; each side can focus on a different change set, class, or method. This layout supports the change sorter's main feature: the ability to move or copy changes from one change set to another, as shown by the yellow button menu in Figure 7.11. It is also possible to copy individual methods from one side to the other.

You may be wondering why you should care about the composition of a change set. The answer is that change sets provide a simple mechanism for exporting code from Squeak to the file system, from where it can be imported into another Squeak image, or into another non-Squeak Smalltalk. Change set export is known as "filing-out", and can be accomplished using the yellow button menu on any change set, class, or method in either browser. Repeated file-outs create new versions of the file, but change

sets are not a versioning tool like Monticello: They do not keep track of dependencies.

Change sets vs. Monticello

Before the advent of Monticello, change sets were the main means for exchanging code between Squeak users. They have the advantage of simplicity (the file-out is just a text file, although we *don't* recommend that you try to edit it with a text editor) and a degree of portability. It's also relatively easy to create a change set that makes changes to many different, unrelated parts of the system — something that Monticello is not yet equipped to do.

Compared to Monticello packages, the main drawback of change sets is that they do not support the notion of dependencies. A filed-out change set is a set of *actions* that change any image into which it is loaded. To successfully load a change set requires that the image be in an appropriate state. For example, the change set might contain an action to add a method to a class; this can only be accomplished if the class is already defined in the image. Similarly, the change set might rename or re-categorize a class, which obviously will only work if the class is present in the image; methods may use instance variables that were declared when they were filed out, but which do not exist in the image into which they are imported. The problem is that change sets do not explicitly describe the conditions under which they can be filed in: The file-in process just hopes for the best, usually resulting in a cryptic error message and a stack trace when things go wrong. Even if the file-in works, one change set might silently undo a change made by another change set.

In contrast, Monticello packages represent code in a declarative fashion: They describe the state of the image should be in after they have been loaded. This permits Monticello to warn you about conflicts (when two packages require contradictory final states) and to offer to load a series of packages in their dependency order.

In spite of these shortcomings, change sets still have their uses; in particular, you may find change sets on the Internet that you want to look at and perhaps use. So, having filed-out a change set using the change sorter, we will now tell you how to file one in. The simplest way to do this is to drag a change set file from your host system's file browser, drop it on the Squeak image, and then choose install into new changeset (see Figure 7.12). Alternatively, you can file in a change set without leaving your Squeak image by using another tool, the file list browser.

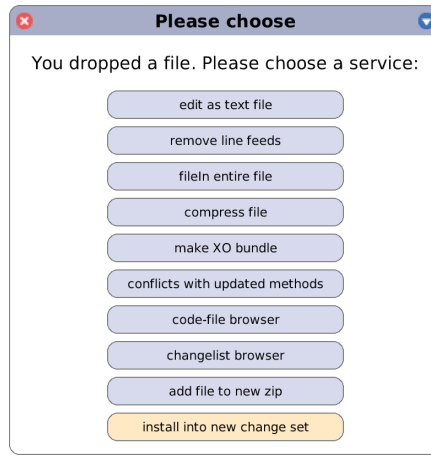


Figure 7.12: Dropping a change set file into the image to file it in.

7.3 The file list browser

The file list browser is in fact a general-purpose tool for browsing the file system (and also FTP servers) from Squeak. You can open it from the World ▷ open... ▷ file list menu, or World docking bar ▷ Tools. What you see of course depends on the contents of your local file system, but a typical view is shown in Figure 7.13.

When you first open a file list browser, it will be focused on the current directory, that is, the one from which you started Squeak. The title bar shows the path to this directory. The larger pane on the left-hand side can be used to navigate the file system in the conventional way. When a directory is selected, the files that it contains (but not the directories) are displayed on the right. This list of files can be filtered by entering a Unix-style pattern in the small box at the top-left of the window. Initially, this pattern is the current folder followed by a *, which matches all file names in the folder, but you can type a different string there and accept it, changing the pattern. (Note that a * is implicitly prepended and appended to the pattern that you type.) The sort order of the files can be changed using the name, date and size buttons. The rest of the buttons depends on the name of the file selected in the browser. In Figure 7.13, the file name has the suffix .cs, so the browser assumes that it is a change set, and provides buttons to install it (which *files it in* to a new change set whose name is derived from the name of the file), to browse the changes in the file, to examine the code

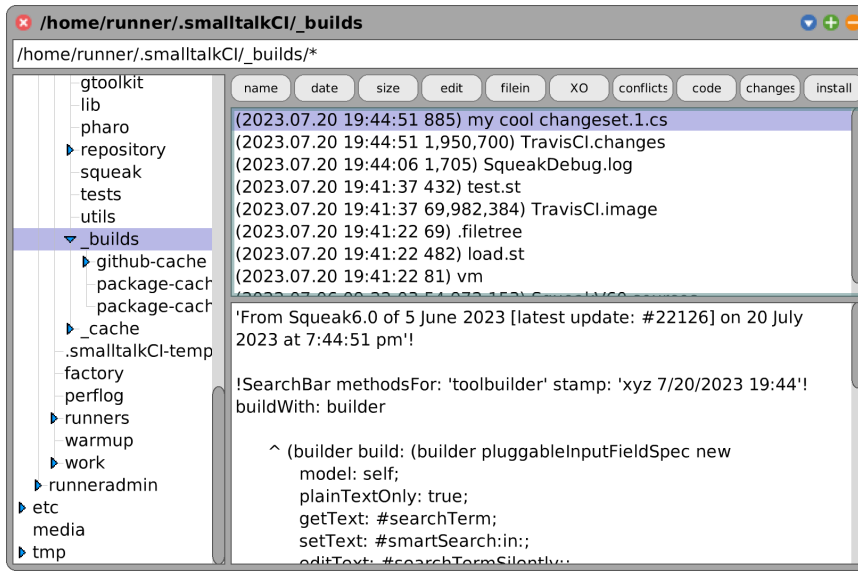


Figure 7.13: A file list browser.

in the file, and to filein the code into the *current* change set.

Because the choice of buttons to display depends on the file's *name*, and not on its contents, sometimes the button that you want won't be on the screen. However, the full set of options is always available from the yellow button more... menu, so you can easily work around this problem.

The code button is perhaps the most useful for working with change sets; it opens a browser on the contents of the change set file; an example is shown in Figure 7.14. The file contents browser is similar to the system browser except that it does not show categories, just classes, protocols and methods. For each class, the browser will tell you whether the class already exists in the system and whether it is defined in the file. It will show the methods in each class, and (as shown in Figure 7.14) will show you the differences between the current version and the version in the file. Yellow-button menu items in each of the top four panes will also let you file in the whole of the change set, or the corresponding class, protocol or method.

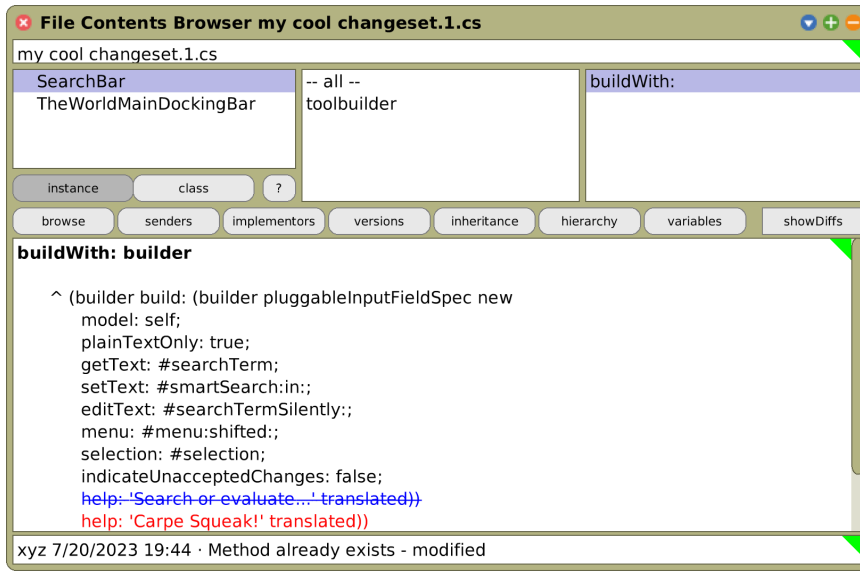


Figure 7.14: A file contents browser.

7.4 In Smalltalk, you can't lose code

It is quite possible to crash Squeak: as a system designed to be open to experimenting, Squeak lets you change anything, including things that are vital to make Squeak work!



To maliciously crash Squeak, try Object removeFromSystem.

The good news is that you need never lose any work, even if you crash and go back to the last saved version of your image, which might be hours old. This is because all of the code that you executed or saved is stored in the *.changes* file. All of it! This includes one-liners that you evaluate in a workspace, as well as code that you add to a class while programming.

So here are the instructions on how to get your code back. There is no need to read this until you need it. However, when you do need it, you'll find it here waiting for you.

In the worst case, you can use a text editor on the *.changes* file, but since it is many megabytes in size, this can be slow and is not recommended. Squeak offers you better ways.

How to get your code back

To start recovering, restart Squeak from the most recent snapshot. Then select World Docking Bar ▷ Extras ▷ Recover Changes. This will ask you how far back in history you wish to browse. The list includes snapshots that get older the further down they are in the list. The topmost snapshot is the most recent one. Selecting it will give you all changes that have been recorded since this snapshot. Normally, it's sufficient to browse changes as far back as the last snapshot.

Once you have a *recent changes* browser, showing, say, changes back as far as your last snapshot, you will have a list of everything that you have done in Squeak during that time. You can delete items from this list using the yellow button menu. When you are satisfied, you can file-in what is left also using the yellow button menu, thus incorporating the changes into your new image.

One useful thing to do in the *recent changes* browser is to remove doIts. Usually, you won't want to file in (and thus re-execute) doIts. However, there is an exception. Creating a class shows up as a doIt. *Before you can file in the methods for a class, the class must exist.* So, if you have created any new classes, *first* file-in the class creation doIts, then remove doIts and file in the methods.

7.5 Git tools

Git is a commonly used version control system. For sharing you projects via Git on platforms such as GitHub, GitLab, or Bitbucket, you can use the Squeak Git tools. The Git tools are not part of the standard environment and you have to install them. You can do so by clicking world docking bar ▷ Tools ▷ Git Browser. The Git Browser allows you to manage a local Git repository directly from within the image using parts of the gitless[2] semantic.

7.6 Chapter summary

In this chapter, you have met some tools that are useful for managing, versioning, and sharing your code.

- *Monticello* is a tool for exporting, importing, versioning and sharing packages of classes and methods. A Monticello package consists of

[2] https://gitless.com/

a system category, subcategories, and related methods protocols in other categories.

- *Change sets* are automatically generated logs of all changes to the source code of your image. They have largely been superseded by Monticello as a means to store and exchange versions of your source code, but are still useful, especially for recovering from catastrophic failures, however rare these may be.
- The *file list browser* is a tool for browsing the file system. It also allows you to `filein` source code from the file system.
- In case your image crashes before you could save it or backup your source code with Monticello, you can always recover your most recent changes using a *change list browser*. You can then select the changes you want to replay and file them into the most recent copy of your image.

Chapter 8

SUnit

8.1 Introduction

SUnit is a minimal yet powerful framework that supports the creation and deployment of tests. As might be guessed from its name, the design of SUnit focussed on *Unit Tests*, but in fact it can be used for integration tests and functional tests as well. SUnit was originally developed by Kent Beck and subsequently extended by Joseph Pelrine and others to incorporate the notion of a resource, which we will describe in Section 8.6.

The interest in testing and Test Driven Development is not limited to Squeak or Smalltalk. Automated testing has become a hallmark of the agile software development movement, and any software developer concerned with improving software quality would do well to adopt it. Indeed, developers in many languages have come to appreciate the power of unit testing, and versions of xUnit now exist for many languages, including Java, Python, and .NET.

Neither testing, nor the building of test suites, is new: Everybody knows that tests are a good way to catch errors. eXtreme Programming, by making testing a core practice and by emphasizing *automated* tests, has helped to make testing productive and fun, rather than a chore that programmers dislike. The Smalltalk community has a long tradition of testing because of the incremental style of development supported by its programming environment. In traditional Smalltalk development, the programmer would write tests in a workspace as soon as a method was finished. Sometimes a test would be incorporated as a comment at the head of the method that it exercised, or tests that needed some setup would be included as example methods in the class. The problem with these practices is that tests in a

workspace are not available to other programmers who modify the code; comments and example methods are better in this respect, but there is still no easy way to keep track of them and to run them automatically. Tests that are not run do not help you to find bugs! Moreover, an example method does not inform the reader of the expected result: You can run the example and see the — perhaps surprising — result, but you will not know if the observed behavior is correct.

SUnit is valuable because it allows us to write self-checking tests: The test itself defines what the correct result should be. It also helps us to organize tests into groups, to describe the context in which the tests must run, and to run a group of tests automatically. In less than two minutes you can write tests using SUnit, so instead of writing small code snippets in a workspace, we encourage you to use SUnit and get all the advantages of stored and automatically executable tests.

In this chapter we start by discussing why we test, and what makes a good test. We then present a series of small examples showing how to use SUnit. Finally, we look at the implementation of SUnit, so that you can understand how Smalltalk uses the power of reflection for its tools.

8.2 Why testing is important

Unfortunately, many developers believe that tests require to much of their time. After all, *they* do not write bugs — only *other* programmers do that. Most of us have said, at some time or other: "I would write tests if I had more time." If you never write a bug, and if your code will never be changed in the future, then indeed you do not need tests. However, this most likely also means that your application is trivial, or that it is not used by you or anyone else. Think of tests as an investment for the future: Having a suite of tests will be quite useful now, but it will be *extremely* useful when your application, or the environment in which it executes, changes in the future.

Tests play several roles. First, they provide documentation of the functionality that they cover, and, moreover, the documentation is active: Watching the tests pass tells you that the documentation is up-to-date. Second, tests help developers to confirm that some changes that they have just made to a package have not broken anything else in the system — and to find the parts that break when that confidence turns out to be misplaced. Finally, writing tests at the same time as — or even before — programming forces you to think about the functionality that you want to design, *and how it should appear to the client*, rather than about how to implement it. By writing the tests first — before the code — you are compelled to state the

context in which your functionality will run, the way it will interact with the client code, and the expected results. Your code will improve: Try it.

We cannot test all aspects of any realistic application. Covering a complete application is simply impossible and should not be the goal of testing. Even with a good test suite some bugs will still creep into the application, where they can lay dormant waiting for an opportunity to damage your system. If you find that this has happened, take advantage of it! As soon as you uncover the bug, write a test that exposes it, run the test, and watch it fail. Now you can start to fix the bug: The test will tell you when you are done.

8.3 What makes a good test?

Writing good tests is a skill that can be learned most easily by practicing. Let us look at the properties that tests should have to get a maximum benefit.

1. Tests should be repeatable. You should be able to run a test as often as you want, and always get the same answer.
2. Tests should run without human intervention. You should even be able to run them during the night.
3. Tests should tell a story. Each test should cover one aspect of a piece of code. A test should act as a scenario that you or someone else can read to understand a piece of functionality.
4. Tests should have a change frequency lower than that of the functionality they cover: You do not want to have to change all your tests every time you modify your application. One way to achieve this is to write tests based on the public interfaces of the class that you are testing. It is OK to write a test for a private "helper" method if you feel that the method is complicated enough to need the test, but you should be aware that such a test may have to be changed, or thrown away entirely when you think of a better implementation.

A consequence of property (3) is that the number of tests should be somewhat proportional to the number of functions to be tested: Changing one aspect of the system should not break all the tests but only a limited number. This is important because having 100 tests fail should send a much stronger message than having 10 tests fail. However, it is not always possible to achieve this ideal: In particular, if a change breaks the initialization of an object, or the setup of a test, it is likely to cause all of the tests to fail.

eXtreme Programming advocates writing tests before writing code. This may seem to go against our deep instincts as software developers. All we can say is: Go ahead and try it. We have found that writing the tests before the code helps us to know what we want to code, helps us know when we are done, and helps us conceptualize the functionality of a class and to design its interface. Moreover, test-first development gives us the courage to go fast, because we are not afraid that we will forget something important.

8.4 SUnit by example

Before going into the details of SUnit, we will show a step by step example testing the class Set. Try entering the code as we go along.

Step 1: create the test class

 *First you should create a new subclass of* TestCase *called* ExampleSetTest. *Add two instance variables so that your new class looks like this:*

Class 8.1: *An Example Set Test class.*

```
1 TestCase subclass: #ExampleSetTest
2   instanceVariableNames: 'full empty'
3   classInstanceVariableNames: ''
4   poolDictionaries: ''
5   category: 'MyTest'
```

We will use the class ExampleSetTest to group all the tests related to the class Set. It defines the context in which the tests will run. Here the context is described by the two instance variables full and empty that we will use to represent a full and an empty set.

The name of the class is not critical, but by convention, it should end in Test. If you define a class called Pattern and call the corresponding test class PatternTest, the two classes will be alphabetized together in the browser (assuming that they are in the same category). It *is* critical that your class be a subclass of TestCase.

Step 2: initialize the test context

The method setUp defines the context in which the tests will run, a bit like an initialize method. setUp is invoked before the execution of each test method defined in the test class.

 *Define the* setUp *method as follows, to initialize the* empty *variable to refer to an empty set and the* full *variable to refer to a set containing two elements. Never forget to call* **super** setUp *as the very first operation in a* setUp *method!*

Method 8.2: *Setting up a fixture.*

```
ExampleSetTest»setUp

    super setUp.

    empty := Set new.
    full := Set with: 5 with: 6.
```

In testing jargon the context is called the *fixture* for the test. You can also clean up the fixture of the test again in the tearDown method.

Step 3: write some test methods

Let's create some tests by defining some methods in the class ExampleSetTest. Each method represents one test; the name of the method should start with the string 'test' so that SUnit will collect them into test suites. Test methods take no arguments.

 *Define the following test methods.*

The first test, named testIncludes, tests the includes: method of Set. The test says that sending the message includes: 5 to a set containing 5 should return **true**. Clearly, this test relies on the fact that the setUp method has already run.

Method 8.3: *Testing set membership.*

```
ExampleSetTest»testIncludes

    self
        assert: (full includes: 5);
        assert: (full includes: 6).
```

The second test, named testOccurrences, verifies that the number of occurrences of 5 in full set is equal to one, even if we add another element 5 to the set. Here we make use of the specific assertion assert:equals: that asserts whether the two arguments are equal in value.

Method 8.4: *Testing occurrences.*

```
ExampleSetTest»testOccurrences

    self
        assert: 0 equals: (empty occurrencesOf: 0);
        assert: 1 equals: (full occurrencesOf: 5).
    full add: 5.
    self assert: 1 equals: (full occurrencesOf: 5).
```

Finally, we test that the set no longer contains the element 5 after we have removed it.

Method 8.5: *Testing removal.*

```
ExampleSetTest»testRemove

    full remove: 5.
    self
        assert: (full includes: 6);
        deny: (full includes: 5).
```

Note the use of the method deny: to assert something that should not be true. aTest deny: anExpression is equivalent to aTest assert: anExpression not, but is much more readable.

Step 4: run the tests

The easiest way to execute the tests is with the SUnit *Test Runner*, which you can open from the World ▷ Test Runner menu, or from World docking bar ▷ Tools. The *TestRunner*, shown in Figure 8.1, is designed to make it easy to execute groups of tests. The left-most pane lists all of the system categories that contain test classes (*i.e.*, subclasses of TestCase); when some of these categories are selected, the test classes that they contain appear in the pane to the right. Abstract classes are italicized, and the test class hierarchy is shown by indentation, so subclasses of ClassTestCase are indented more than subclasses of TestCase.

 *Open a Test Runner, select the category* MyTest*, and click the* Run Selected *button.*

You can also run your test by executing a print it on the following code: ExampleSetTest run: #testRemove. Finally, you can also run a test directly from the method pane in the browser by opening the yellow button menu on the test method and selecting run test.

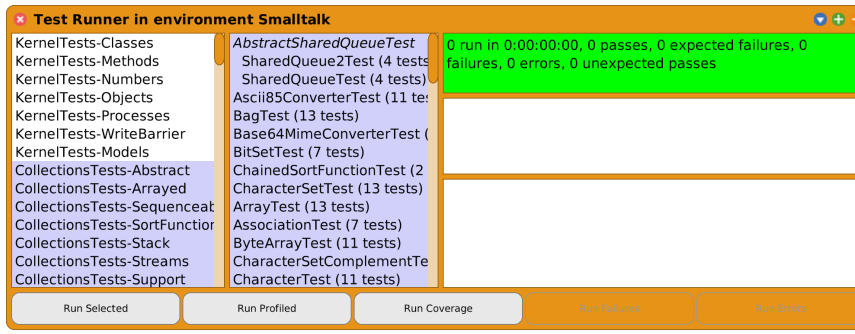


Figure 8.1: The Squeak SUnit test runner.

Introduce a bug in ExampleSetTest»testRemove *and run the tests again. For example, change* 5 *to* 4.

The tests that did not pass (if any) are listed in the right-hand panes of the *Test Runner*; if you want to debug one, to see why it failed, just click on the name. Alternatively, you can also directly start debugging the test from the browser by opening the yellow button menu on the test method and selecting debug test. If you prefer working with scripts, you can execute the following expression:

```
ExampleSetTest debug: #testRemove
```

Step 5: interpret the results

The method assert: , which is defined in the class TestCase, expects a boolean argument, usually the value of a tested expression. When the argument is true, the test passes; when the argument is false, the test fails.

There are actually three possible outcomes of a test and corresponding visualizations in the test runner. The outcome that we hope for is that all of the assertions in the test are true, in which case the test passes. When all of the tests pass, the bar at the top of the test runner turns green. However, there are also two kinds of things that can go wrong when you run a test. Most obviously, one of the assertions can be false, causing the test to *fail*. This will turn the bar at the top of the test runner yellow and the failed tests are listed at the middle pane on the right. However, it is also possible that some kind of error occurs during the execution of the test, such as a *message not understood* error or an *index out of bounds* error. If an error occurs, the assertions in the test method may not have been executed at all, so we

can't say that the test has failed; nevertheless, something is clearly wrong! Erroneous tests cause the bar at the top of the test runner to turn red and the erroneous tests are listed in the bottom pane on the right.

 *Modify your tests to provoke both errors and failures.*

8.5 The SUnit cook book

This section will give you more details on how to use SUnit. If you have used another testing framework such as JUnit[1], much of this will be familiar, since all these frameworks have their roots in SUnit. Normally you will use SUnit's GUI to run tests, but there are situations where you may not want to use it.

Other assertions

In addition to assert: and deny:, several other methods can be used to make assertions.

First, assert:description: and deny:description: take a second argument which is a message string that can be used to describe the reason for the failure if it is not obvious from the test itself. These methods are described in Section 8.7. Next, SUnit provides two additional methods, should:raise: and shouldnt:raise: for testing exception propagation. For example, you would use (**self** should: aBlock raise: anException) to test that a particular exception is raised during the execution of aBlock. Method 8.6 illustrates the use of should:raise:.

 *Try running this test.*

Note that the first argument of the should: and shouldnt: methods is a *block* that *contains* the expression to be evaluated.

Method 8.6: *Testing error raising.*

```
1 ExampleSetTest»testIllegal
2
3   self
4     should: [empty at: 5] raise: Error;
5     should: [empty at: 5 put: #zork] raise: Error.
```

SUnit is portable: it can be used from all dialects of Smalltalk. To make SUnit portable, its developers factored-out the dialect-dependent aspects.

[1] junit.org

The class method TestResult class»error answers the system's error class in a dialect-independent fashion. You can take advantage of this: If you want to write tests that will work in any dialect of Smalltalk, instead of method 8.6 you would write:

Method 8.7: *Portable error handling.*

```
ExampleSetTest»testIllegal

    self
        should: [empty at: 5] raise: TestResult error;
        should: [empty at: 5 put: #zork] raise: TestResult error.
```

 *Give it a try.*

Running a single test

Normally, you will run your tests using the Test Runner. If you don't want to launch the Test Runner from the menus, you can execute TestRunner open as a print it.

You can run a single test as follows:

```
ExampleSetTest run: #testRemove    ⟶    1 run in 0:00:00:00, 1 passes, 0
    expected failures, 0 failures, 0 errors, 0 unexpected passes
```

Running all the tests in a test class

Any subclass of TestCase responds to the message suite, which will build a test suite that contains all the methods in the class whose names start with the string "test". To run the tests in the suite, send it the message run. For example:

```
ExampleSetTest suite run    ⟶    5 run in 0:00:00:01, 5 passes, 0 expected failures,
    0 failures, 0 errors, 0 unexpected passes
```

Must I subclass TestCase?

In JUnit you can build a TestSuite from an arbitrary class containing test* methods. In Smalltalk you can do the same but you will then have to create a suite by hand and your class will have to implement all the essential TestCase methods like assert:. We recommend that you not try to do this. The framework is there, use it.

8.6 The SUnit framework

The core of SUnit consists of four main classes: TestCase, TestSuite, TestResult , and TestResource, as shown in Figure 8.2. The notion of a *test resource* represents a resource that is expensive to set-up but which can be used by a whole series of tests. A TestResource specifies a setUp method that is executed just once before a suite of tests; this is in distinction to the TestCase»setUp method, which is executed before each test.

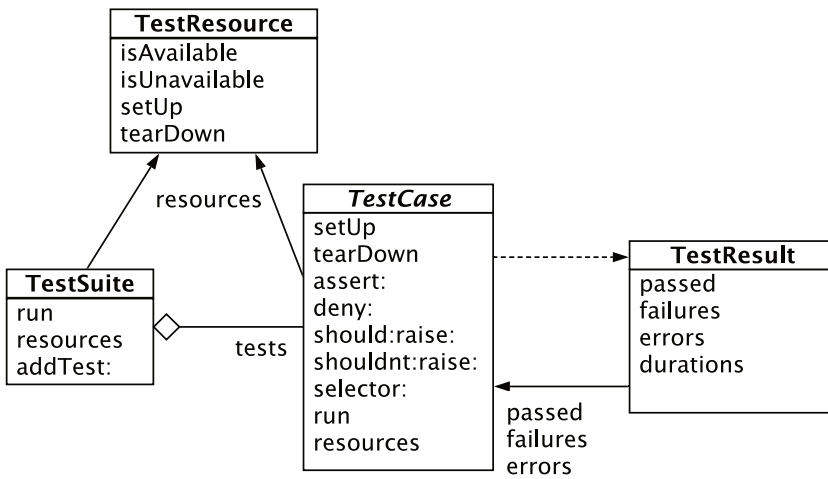


Figure 8.2: The four classes representing the core of SUnit.

TestCase

TestCase is an abstract class that is designed to be subclassed; each of its subclasses represents a group of tests that share a common context (that is, a test suite). Each test is run by creating a new instance of a subclass of TestCase, running setUp, running the test method itself, and then running tearDown.

The context is specified by instance variables of the subclass and by the specialization of the method setUp, which initializes those instance variables. Subclasses of TestCase can also override method tearDown, which is invoked after the execution of each test, and can be used to release any objects allocated during setUp.

TestSuite

Instances of the class TestSuite contain a collection of test cases. An instance of TestSuite contains tests, and other test suites. That is, a test suite contains sub-instances of TestCase and TestSuite. Both individual TestCases and TestSuites understand the same protocol, so they can be treated in the same way; for example, both can be run. This is in fact an application of the composite pattern in which TestSuite is the composite and the TestCases are the leaves — see *Design Patterns* for more information on this pattern[2].

TestResult

The class TestResult represents the results of a TestSuite execution. It records the number of tests passed, the number of tests failed, and the number of errors raised.

TestResource

One of the important features of a suite of tests is that they should be independent of each other: The failure of one test should not cause an avalanche of failures of other tests that depend upon it, nor should the order in which the tests are run matter. Performing setUp before each test and tearDown afterwards helps to reinforce this independence.

However, there are occasions where setting up the necessary context is just too time-consuming for it to be practical to do once before the execution of each test. Moreover, if it is known that the test cases do not disrupt the resources used by the tests, then it is wasteful to set them up afresh for each test; it is sufficient to set them up once for each suite of tests. Suppose, for example, that a suite of tests need to query a database, or do some analysis on some compiled code. In such cases, it may make sense to set up the database and open a connection to it, or to compile some source code, before any of the tests start to run.

Where should we cache these resources, so that they can be shared by a suite of tests? The instance variables of a particular TestCase subinstance won't do, because such an instance persists only for the duration of a single test. A global variable would work, but using too many global variables pollutes the namespace, and the binding between the global and the tests that depend on it will not be explicit. A better solution is to put the necessary resources in a singleton object of some class. The

[2] Erich Gamma et al., *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, Mass.: Addison Wesley Professional, 1995, ISBN 978–0201633610.

class TestResource exists to be subclassed by such resource classes. Each subclass of TestResource understands the message current, which will answer a singleton instance of that subclass. Methods setUp and tearDown should be overridden in the subclass to ensure that the resource is initialized and finalized.

One thing remains: Somehow, SUnit has to be told which resources are associated with which test suite. A resource is associated with a particular subclass of TestCase by overriding the *class* method resources. By default, the resources of a TestSuite are the union of the resources of the TestCases that it contains.

Here is an example. We define our own subclass of TestResource called MyTestResource and we associate it with MyTestCase by specializing the class method resources to return an array of the test classes that it will use.

Class 8.8: *An example of a TestResource subclass.*

```
1 TestResource subclass: #MyTestResource
2    instanceVariableNames: ''
3
4 MyTestCase class»resources
5    "associate the resource with this class of test cases"
6
7    ^ {MyTestResource}
```

In your tests, you can now access the test resource through MyTestCase current. This message will return the test resource instance that is initialized for the current run of the test suite.

8.7 Advanced features of SUnit

In addition to TestResource, the current version of SUnit contains assertion description strings and logging support.

Assertion description strings

The TestCase assertion protocol includes a number of methods that allow the programmer to supply a description of the assertion. The description is a String; if the test case fails, this string will be displayed by the test runner. Of course, this string can be constructed dynamically.

```
| e |
e := 42.
self
```

```
assert: e = 23
description: 'expected 23, got ', e.
```

The relevant methods in TestCase are:

```
#assert:description:
#deny:description:
#should:description:
#shouldnt:description:
```

Logging support

The description strings described above may also be logged to a Stream such as the Transcript, or a file stream. You can choose whether to log by overriding TestCase»isLogging in your test class; you must also choose where to log by overriding TestCase»failureLog to answer an appropriate stream.

8.8 The implementation of SUnit

The implementation of SUnit makes an interesting case study of a Smalltalk framework. Let's look at some key aspects of the implementation by following the execution of a test.

Running one test

To execute one test, we evaluate the expression (aTestClass selector: aSymbol) run.

The method TestCase»run creates an instance of TestResult that will accumulate the results of the tests. Then the TestCase instance sends itself the message run:. (See Figure 8.3.)

Method 8.9: *Running a test case.*

```
1 TestCase»run
2
3     | result |
4     result := TestResult new.
5     self run: result.
6     ^ result
```

The method TestCase»run: sends the message runCase: to the test result:

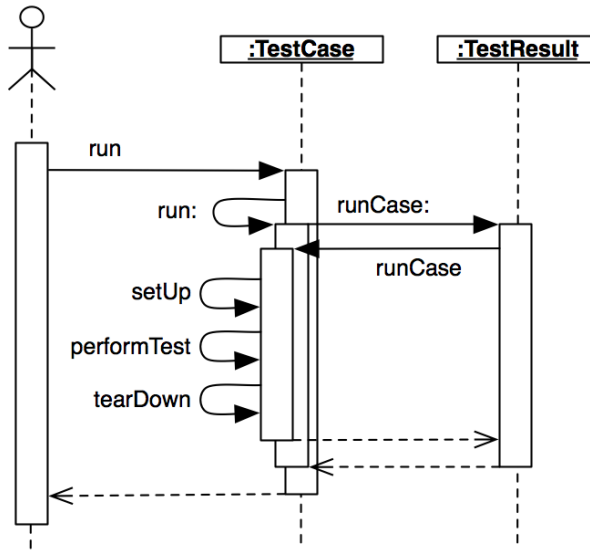


Figure 8.3: Running one test.

Method 8.10: *Passing the test case to the test result.*

```
TestCase»run: aResult

    aResult runCase: self.
```

The method TestResult»runCase: sends the message runCase to an individual test, to execute the test. TestResult»runCase deals with any exceptions that may be raised during the execution of a test, runs a TestCase by sending it the message runCase, and counts the errors, failures, and passes.

Method 8.11: *Catching test case errors and failures.*

```
TestResult»runCase: aTestCase

    | testCasePassed timeToRun |
    testCasePassed := true.

    [timeToRun := [aTestCase runCase] timeToRunWithoutGC]
        on: self class failure
        do: [:signal |
                failures add: aTestCase.
                testCasePassed := false.
                signal return: false]
        on: self class error
```

```
13      do: [:signal |
14          errors add: aTestCase.
15          testCasePassed := false.
16          signal return: false].
17
18  testCasePassed ifTrue: [passed add: aTestCase].
19  self durations at: aTestCase put: timeToRun.
```

The method TestCase»runCase sends the messages setUp and tearDown as shown below.

Method 8.12: *Test case template method.*

```
1  TestCase»runCase
2    "Run this TestCase. Time out if the test takes too long."
3
4    [self timeout: [self setUp] after: self timeoutForSetUp.
5    self timeout: [self performTest] after: self timeoutForTest]
6        ensure: [self tearDown].
```

Running a TestSuite

To run more than one test, we send the message run to a TestSuite that contains the relevant tests. TestCase class provides some functionality to build a test suite from its methods. The expression MyTestCase buildSuiteFromSelectors returns a suite containing all the tests defined in the MyTestCase class. The core of this process is:

Method 8.13: *Auto-building the test suite.*

```
1  TestCase class»testSelectors
2
3    ^ (self selectors asArray select: [:each |
4      (each beginsWith: 'test') and: [each numArgs isZero]]) sort
```

The method TestSuite»run creates an instance of TestResult, verifies that all the resources are available, and then sends itself the message run:, which runs all the tests in the suite. All the resources are then released.

Method 8.14: *Running a test suite.*

```
1  TestSuite»run
2
3    | result |
4    result := self resultClass new.
5    self resources do: [:resource |
6      res isAvailable ifFalse: [^ resource signalInitializationError]].
```

```
    [self run: result] ensure: [self resources do: [:each | each reset]].
    ^ result
```

Method 8.15: *Passing the test suite to the test result.*

```
TestSuite»run: aResult

    self tests do: [:each |
        self sunitChanged: each.
        each run: aResult].
```

The class TestResource and its subclasses keep track of their currently created instances (one per class) that can be accessed and created using the class method current. This instance is cleared when the tests have finished running and the resources are reset.

The resource availability check makes it possible for the resource to be re-created if needed, as shown in the class method TestResource class» isAvailable. During the TestResource instance creation, it is initialized and the method setUp is invoked.

Method 8.16: *Test resource availability.*

```
TestResource class»isAvailable

    ^ self current notNil
```

Method 8.17: *Test resource creation.*

```
TestResource class»current

    current ifNil: [current := self new].
    ^ current
```

Method 8.18: *Test resource initialization.*

```
TestResource»initialize

    self setUp.
```

8.9 Some advice on testing

While the mechanics of testing are easy, writing good tests is not. Here is some advice on how to design tests.

Feathers' Rules for Unit tests. Michael Feathers, an agile process consultant and author, writes:[3]

> *A test is not a unit test if:*
>
> - *it talks to the database,*
> - *it communicates across the network,*
> - *it touches the file system,*
> - *it can't run at the same time as any of your other unit tests, or*
> - *you have to do special things to your environment (such as editing config files) to run it.*
>
> *Tests that do these things aren't bad. Often they are worth writing, and they can be written in a unit test harness. However, it is important to be able to separate them from true unit tests so that we can keep a set of tests that we can run fast whenever we make our changes.*

Never get yourself into a situation where you don't want to run your unit test suite because it takes too long.

Unit Tests *vs.* Acceptance Tests. Unit tests capture one piece of functionality, and as such make it easier to identify bugs in that functionality. As far as possible try to have unit tests for each method that could possibly fail, and group them per class. However, for certain deeply recursive or complex setup situations, it is easier to write tests that represent a scenario in the larger application; these are called *acceptance tests* or functional tests. Tests that break Feathers' rules may make good acceptance tests. Group acceptance tests according to the functionality that they test. For example, if you are writing a compiler, you might write acceptance tests that make assertions about the code generated for each possible source language statement. Such tests might exercise many classes, and might take a long time to run because they touch the file system. You can write them using SUnit, but you won't want to run them each time you make a small change, so they should be separated from the true unit tests.

One Test per Property. For every test in the system, you should be able to identify some property for which the test increases your confidence. Obviously there should be no important property that you are not testing. This rule states the less obvious fact that there should be no test that does not add value to the system by increasing your confidence that a useful property holds. For example, several tests

[3] See https://web.archive.org/web/20060212020903/http://www.artima.com/weblogs/viewpost.jsp?thread=126923.

of the same property do no good. In fact they do harm: They make it harder to infer the behavior of the class by reading the tests, and because one bug in the code might then break many tests at the same time. Have a property in mind when you write a test.

8.10 Chapter summary

This chapter explained why tests are an important investment in the future of your code. We explained in a step by step fashion how to define a few tests for the class Set. Then we gave an overview of the core of the SUnit framework by presenting the classes TestCase, TestResult, TestSuite, and TestResources. Finally, we looked deep inside SUnit by following the execution of a test and a test suite.

- To maximize their potential, unit tests should be fast, repeatable, independent of any direct human interaction, and cover a single unit of functionality.
- Tests for a class called MyClass belong in a class classed MyClassTest, which should be introduced as a subclass of TestCase.
- Initialize your test data in a setUp method.
- Each test method should start with the word test.
- Use the TestCase methods assert:, deny: and others to make assertions.
- Run tests using the SUnit test runner tool (in the tool bar).

Chapter 9

Basic Classes

Most of the power of Smalltalk is not in the language but the class libraries. To program effectively with Smalltalk, you need to learn how the class libraries support the language and environment. The class libraries are entirely written in Smalltalk and we can easily extend them since a package may add new functionality to a class even if it does not define this class.

Our goal here is not to present in tedious detail the whole of the Squeak class library, but rather to point out the key classes and methods that you will need to use or override to program effectively. In this chapter we cover the basic classes that you will need for nearly every application: Object, Number and its subclasses, Character, String, Symbol, Boolean, and Exception. In the following chapter Chapter 10, we will discuss the equally important Collection subclasses.

9.1 Object

Object is the root of the inheritance hierarchy. Actually, in Squeak the true root of the hierarchy is ProtoObject, which is used to define minimal entities that masquerade as objects, but we can ignore this point for the time being.

Object can be found in the *Kernel-Objects* category. Astonishingly, there are almost 500 methods to be found here (including extensions). In other words, every class that you define will automatically provide these methods, whether you know what they do or not.

The class comment for the Object states:

> Object *is the root class for almost all of the other classes in the class hierarchy. The exceptions are* ProtoObject *(the superclass of* Object*) and its subclasses.*
>
> *Class* Object *provides default behavior common to all normal objects, such as access, copying, comparison, error handling, message sending, and reflection. Also, utility messages that all objects should respond to are defined here.*
>
> Object *has no instance variables, nor should any be added. This is due to several classes of objects that inherit from* Object *that have special implementations (*SmallInteger *and* UndefinedObject *for example) or the VM knows about and depends on the structure and layout of certain standard classes.*

If we begin to browse the protocols on the instance side of Object we start to see some of the key behavior it provides.

Printing

Every object in Smalltalk can return a printed form of itself. You can select any expression in a workspace and select the print it menu item: This executes the expression and asks the returned object to print itself. In fact, this sends the message printString to the returned object. The method printString, which is a template method, at its core sends the message printOn: to its receiver. The message printOn: is a hook that can be specialized.

Object»printOn: is very likely one of the methods that you will most frequently override. This method takes as its argument a Stream (for more on streams see Chapter 11) on which a String representation of the object will be written. The default implementation simply writes the class name preceded by "a" or "an". Object»printString returns the String that is written.

For example, the class Browser does not redefine the method printOn: and sending the message printString to a Browser instance executes the method defined in Object.

```
Browser new printString  ⟶  'a Browser'
```

In contrast, the class PackageInfo shows an example of printOn: specialization. In addition to the usual name of the class, it prints the name of the package as shown by the code below which prints an instance of the class.

Method 9.1: *printOn: redefinition.*

```
PackageInfo»printOn: aStream
   super printOn: aStream.
   aStream
      nextPut: $(;
      nextPutAll: self packageName;
      nextPut: $)
```

```
Object packageInfo printString  ⟶  'a PackageInfo(Kernel)'
```

Note that the message printOn: is not the same as storeOn:. The message storeOn: puts an expression that can be used to recreate the receiver on its argument stream . This expression is evaluated when the stream is read using the message readFrom:. printOn: just returns a textual version of the receiver that may or may not be a self-evaluating expression.

A word about representation and self-evaluation. In functional programming, expressions return values when executed. In Smalltalk, messages (expressions) return objects (values). Some objects have the nice properties that their value is themselves. For example, the value of the object **true** is itself *i.e.*, the object **true**. We call such objects *self-evaluating objects*. You can see a *printed* version of an object value when you print the object in a workspace. Here are some examples of such self-evaluating expressions.

```
true      ⟶  true
3 @ 4     ⟶  3@4
$a        ⟶  $a
#(1 2 3)  ⟶  #(1 2 3)
```

Note that some objects, such as arrays, are self-evaluating or not depending on the objects they contain. For example, an array of booleans is self-evaluating whereas an array of persons is not. The following example shows that an array is self-evaluating only if its elements are:

```
{10 @ 10 . 100 @ 100}          ⟶  {10@10 . 100@100}
{Browser new . 100 @ 100}      ⟶  {a Browser . 100@100}
```

Remember that literal arrays can only contain literals. Hence the following array does not contain two points but rather six literal elements.

```
#(10 @ 10 100 @ 100)  ⟶  #(10 #@ 10 100 #@ 100)
```

Lots of printOn: method specializations implement self-evaluating behavior. The implementations of Point»printOn: and Interval»printOn: are self-evaluating.

Method 9.2: Point *print string definition that enables self-evaluation.*

```
Point»printOn: aStream
    "The receiver prints on aStream in terms of infix notation."

    x printOn: aStream.
    aStream nextPut: $@.
    ...
    y printOn: aStream.
```

Method 9.3: Interval *print string definition that enables self-evaluation.*

```
Interval»printOn: aStream

    aStream nextPut: $(;
        print: start;
        nextPutAll: ' to: ';
        print: stop.
    step ~= 1 ifTrue: [aStream nextPutAll: ' by: '; print: step].
    aStream nextPut: $).
```

```
1 to: 10    ⟶    (1 to: 10)    "intervals are self-evaluating"
```

Identity and equality

In Smalltalk, the message = tests object *equality* (*i.e.,* whether two objects represent the same value) whereas == tests object *identity* (*i.e.,* whether two expressions represent the same object).

The default implementation of object equality is to test for object identity:

Method 9.4: *Object equality.*

```
Object»= anObject
    "Answer whether the receiver and the argument represent the same object.
    If = is redefined in any subclass, consider also redefining the message hash."

    ^ self == anObject
```

This is a method that you will frequently want to override. Consider the case of Complex numbers:

```
(1 + 2 i) = (1 + 2 i)     ⟶    true     "same value"
(1 + 2 i) == (1 + 2 i)    ⟶    false    "but different objects"
```

This works because Complex overrides = as follows:

Method 9.5: *Equality for complex numbers.*

```
Complex»= anObject

    anObject isNumber ifFalse: [^ false].
    anObject isComplex
        ifTrue: [^ real = anObject real and: [imaginary = anObject imaginary]]
        ifFalse: [^ anObject adaptToComplex: self andSend: #=]
```

The default implementation of Object»~= simply negates Object»=, and should not normally need to be changed.

```
(1 + 2 i) ~= (1 + 4 i)    ⟶    true
```

> If you override =, you should override hash.

If instances of your class are ever used as keys in a Dictionary, then you should make sure that instances that are considered to be equal have the same hash value. This is because the implementation of Dictionary will only use the hash to determine whether it includes an object as a key:

Method 9.6: *Hash must be reimplemented for complex numbers.*

```
Complex»hash
    "Hash is reimplemented because = is implemented."

    ^ real hash bitXor: imaginary hash
```

Although you should override = and hash together, you should *never* override ==. (The semantics of object identity is the same for all classes.) == is a primitive method of ProtoObject.

Class membership

Several methods allow you to query the class of an object.

class. You can ask any object about its class using the message class.

```
1 class    ⟶    SmallInteger
```

Conversely, you can ask if an object is an instance of a specific class:

```
1 isMemberOf: SmallInteger    ⟶    true    "must be precisely this class"
1 isMemberOf: Integer         ⟶    false
1 isMemberOf: Number          ⟶    false
1 isMemberOf: Object          ⟶    false
```

Since Smalltalk is written in itself, you can really navigate through its structure using the right combination of superclass and class messages (see Chapter 13).

isKindOf:. Object»isKindOf: answers whether the receiver's class is either the same as, or a subclass of the argument class.

```
1 isKindOf: SmallInteger   ⟶   true
1 isKindOf: Integer        ⟶   true
1 isKindOf: Number         ⟶   true
1 isKindOf: Object         ⟶   true
1 isKindOf: String         ⟶   false

1 / 3 isKindOf: Number     ⟶   true
1 / 3 isKindOf: Integer    ⟶   false
```

1 / 3 which is a Fraction is a kind of Number, since the class Number is a superclass of the class Fraction, but 1 / 3 is not a Integer.

respondsTo:. Object»respondsTo: answers whether the receiver understands the message selector given as an argument.

```
1 respondsTo: #,   ⟶   false
```

Normally, it is a *bad idea* to query an object for its class, or to ask it which messages it understands. Instead of making decisions based on the class of object, you should simply send a message to the object and let it decide (*i.e.*, based on its class) how it should behave.

Copying

There are several ways to copy objects in Squeak, depending on what you want to copy. For example, when you copy a collection, you might want to or might not want to copy all its elements. If you are not sure what the right way to copy an object is, the first message to send is copy, as it takes care of copying the right instance variables for you.

To understand the other messages for copying, we will discuss different ways at looking at copying one after another. In general, copying objects introduces some subtle issues. Since instance variables are accessed by reference, a *shallow copy* of an object would share its references to instance variables with the original object:

```
array1 := { { 'harry' } }.
array1   ⟶   #(#('harry'))
```

```
array2 := array1 shallowCopy.
array2    ⟶    #(#('harry'))
(array1 at: 1) at: 1 put: 'sally'.
array1    ⟶    #(#('sally'))
array2    ⟶    #(#('sally'))    "the subarray is shared"
```

Object»shallowCopy creates a shallow copy of an object, which means that the content of the object is not copied. Since a2 is only a shallow copy of a1, the two arrays share a reference to the nested Array that they contain.

For some classes even a shallow copy does not make sense if instances are unique, for example for the classes Boolean, Character, SmallInteger, Symbol and UndefinedObject. These classes override Object»shallowCopy and instances simply return themselves.

Object»copyTwoLevel can be used when a simple shallow copy does not suffice. It copies the object and its contents, but not deeper.

```
array1 := { { 'harry' } }.
array2 := array1 copyTwoLevel.
(array1 at: 1) at: 1 put: 'sally'.
array1    ⟶    #(#('sally'))
array2    ⟶    #(#('harry'))    "fully independent state"
```

Object»deepCopy makes an arbitrarily deep copy of an object.

```
array1 := { { { 'harry' } } }.
array2 := array1 deepCopy.
(array1 at: 1) at: 1 put: 'sally'.
array1    ⟶    #(#('sally'))
array2    ⟶    #(#(#('harry')))
```

However, deepCopy does not keep track of previously copied objects. This means that it does not preserve the identity of re-occurring objects. When the object to be copied contains another object twice in its contents, there will be two new objects.

```
object := {'harry'}.
array := {object . object}.
array first == array second    ⟶    true
copiedArray := array deepCopy.
copiedArray first == copiedArray second    ⟶    false
```

Also, not keeping track of previously copied objects means that deepCopy will not terminate when applied to a mutually recursive structure.

```
array1 := {'harry'}.
array2 := {array1}.
```

```
array1 at: 1 put: array2.
array1 deepCopy    ⟶    ... does not terminate!
```

Another way to copy an object is provided by Object»veryDeepCopy: It creates arbitrarily deep copies of objects but preserves object identities and cycles.

```
array1 := {'harry'}.
array2 := {array1}.
array1 at: 1 put: array2.
copiedArray := array1 veryDeepCopy.
copiedArray first first = copiedArray    ⟶    true
```

Whether any of these four copy methods is the right one depends on the meaning of the object and its instance variables. As this can not be decided in general for all objects and all use cases, Object»copy delegates this decision to specific subclasses. Object»copy first creates a shallow copy, and then asks this shallow copy to copy any instance variables that should be copied by sending postCopy to it.

Method 9.7: *Copying objects as a template method.*

```
1 Object»copy
2     "Answer another instance just like the receiver. Subclasses typically override
        postCopy; they typically do not override shallowCopy."
3
4     ^ self shallowCopy postCopy
```

You should override postCopy to copy any instance variables that should not be shared. postCopy should always do a **super** postCopy. For example, a Form object has instance variables for width, height, bit depth, an offset, and the actual bits. As the first four instance variables are all described by numbers, there is no need to copy them as you can see in postCopy.

Method 9.8: *Copying objects as a template method.*

```
1 Form»postCopy
2
3     super postCopy.
4     bits := bits copy.
```

Debugging

The most important method here is halt. In order to set a breakpoint in a method, simply insert the message send **self** halt at some point in the body of the method. When this message is sent, execution will be interrupted

and a debugger will open to this point in your program. (See Chapter 6 for more details about the debugger.)

The next most important message is assert:, which takes a block as its argument. If the block returns **true**, execution continues. Otherwise, an AssertionFailure exception will be raised, which is a subclass of Error. If this exception is not otherwise caught, the debugger will open to this point in the execution. assert: is especially useful to support *design by contract*. The most typical usage is to check non-trivial pre-conditions to public methods of objects. Stack»pop could easily have been implemented as follows:

Method 9.9: *Checking a pre-condition.*

```
1 Stack»pop
2     "Return the first element and remove it from the stack."
3
4     self assert: [self isEmpty not].
5     ^ self linkedList removeFirst element
```

Do not confuse Object»assert: with TestCase»assert:, which occurs in the SUnit testing framework and raises a TestFailure (see Chapter 8). While the latter also has variants such as assert:equals: or deny:, the former has no variants. Although both are useful for debugging, they each serve a very different intent.

Error handling

This protocol contains several methods useful for signaling run-time errors.

Sending self deprecated: *anExplanationString* signals that the current method should no longer be used, if deprecation warnings have been turned on in the "Debug"category of the preference browser. The String argument should offer an alternative.

```
Object new exploreAndYourself ⟶ "Object>>#exploreAndYourself has been
    deprecated. Use #explore because it does not return the tool window anymore.
    Only calls via ToolSet do so."
```

doesNotUnderstand: is sent whenever a message lookup fails. The default implementation, *i.e.*, Object»doesNotUnderstand: will trigger the debugger at this point. It may be useful to override doesNotUnderstand: to provide some other behavior.

Object»error and Object»error: are generic methods that can be used to raise exceptions. (Generally, it is better to raise your own custom exceptions, so you can distinguish errors arising from your code from those coming from kernel classes.)

Abstract methods in Smalltalk are implemented by convention with the body self subclassResponsibility. Should an abstract class be instantiated by accident, then calls to abstract methods will result in Object»subclassResponsibility being evaluated.

Method 9.10: *Signaling that a method is abstract.*

```
Object»subclassResponsibility
    "This message sets up a framework for the behavior of the class' subclasses.
    Announce that the subclass should have implemented this message."

    ^ SubclassResponsibility
        signal: ('My {1} subclass should have overridden {2}'
            format: {self className. thisContext sender selector})
```

Magnitude, Number, and Boolean are classical examples of abstract classes that we shall see shortly in this chapter.

```
Number new + 1    ⟶    Error: My subclass should have overridden #+
```

self shouldNotImplement is sent by convention to signal that an inherited method is not appropriate for this subclass. This is generally a sign that something is not quite right with the design of the class hierarchy. Due to the limitations of single inheritance, however, sometimes it is very hard to avoid such workarounds. A typical example is Collection»remove: which is inherited by Dictionary but flagged as not implemented. (A Dictionary provides removeKey: instead.)

Testing

The methods in *testing* are not related to unit tests! A testing method lets you ask a question about the state of the receiver and returns a Boolean.

Numerous testing methods are provided by Object. We have already seen isComplex. Others include isArray, isBoolean, isBlock, isCollection, and so on. Generally, such methods are to be avoided since querying an object for its class is a form of violation of encapsulation. Instead of testing an object for its class, one should simply send a request and let the object decide how to handle it.

Nevertheless, some of these testing methods are undeniably useful. The most useful are probably ProtoObject»isNil and Object»notNil (though the Null Object[1] design pattern can obviate the need for even these methods).

[1] Bobby Woolf, Null Object. In Robert Martin, Dirk Riehle and Frank Buschmann, editors, Pattern Languages of Program Design 3. Addison Wesley, 1998.

Initialize-release

A final key method that occurs not in Object but in ProtoObject is initialize.

Method 9.11: initialize *as an empty hook method.*

```
ProtoObject»initialize
    "Subclasses should redefine this method to perform initializations on instance
        creation"
```

The reason this is important is that in Squeak, the default new method defined for every class in the system will send initialize to newly created instances.

Method 9.12: new *as a class-side template method.*

```
Behavior»new
    "Answer a new initialized instance of the receiver (which is a class) with no
        indexable variables. Fail if the class is indexable."

    ^ self basicNew initialize
```

This means that simply by overriding the initialize hook method, new instances of your class will automatically be initialized. The initialize method should normally perform a **super** initialize to establish the class invariant for any inherited instance variables. (Note that this is *not* the standard behavior of other Smalltalks.)

9.2 Numbers

Remarkably, numbers in Smalltalk are not primitive data values but true objects. Of course, numbers are implemented efficiently in the virtual machine, but the Number hierarchy is as perfectly accessible and extensible as any other portion of the Smalltalk class hierarchy.

Numbers are found in the *Kernel-Numbers* category. The abstract root of this hierarchy is Magnitude, which represents all kinds of classes supporting comparison operators. Number adds various arithmetic and other operators as mostly abstract methods. Float and Fraction represent, respectively, floating-point numbers and fractional values. Integer is also abstract, thus distinguishing between subclasses SmallInteger, LargePositiveInteger and LargeNegativeInteger. For the most part, users do not need to be aware of the difference between the three Integer classes, as values are automatically converted as needed.

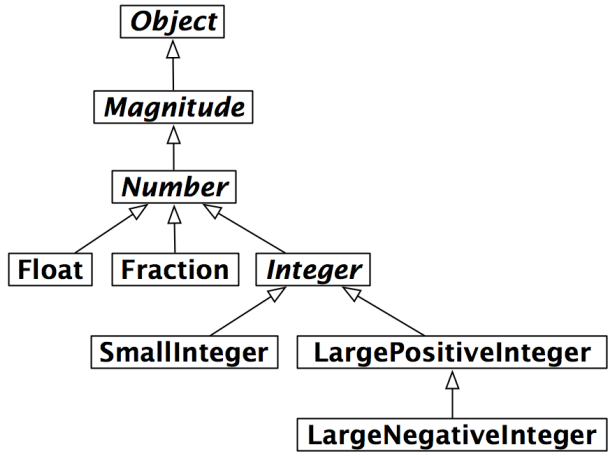


Figure 9.1: The number hierarchy (excerpt).

Magnitude

Magnitude is the parent not only of the Number classes, but also of other classes supporting comparison operations, such as Character, Duration, and Timespan. (Complex numbers are not comparable, and so do not inherit from Number nor from Magnitude.)

The methods Magnitude»< and Magnitude»= are abstract. The remaining operators are defined generically. For example:

Method 9.13: *Abstract comparison methods.*

```
1  Magnitude» < aMagnitude
2      "Answer whether the receiver is less than the argument."
3
4      ^ self subclassResponsibility
5
6
7  Magnitude» > aMagnitude
8      "Answer whether the receiver is greater than the argument."
9
10     ^ aMagnitude < self
```

Number

Similarly, Number defines +, -, *, and / to be abstract, but all other arithmetic operators are generically defined.

All Number objects support various *converting* operators, such as asFloat and asInteger. There are also numerous *shortcut constructor methods*, such as i, which converts a Number to an instance of Complex with a zero real component, and others which generate Durations, such as hour, day and week.

Numbers directly support common *math functions* such as sin, log, raiseTo:, squared, sqrt, and so on.

Number»printOn: is implemented in terms of the abstract method Number» printOn:base:. (The default base is 10.)

Testing methods include even, odd, positive, and negative. Unsurprisingly, Number overrides isNumber. More interesting, isInfinite is defined to return **false**.

Truncation methods include floor, ceiling, integerPart, fractionPart, and so on. *Rounding* methods include rounded, roundTo:, roundUpTo:, and so on.

```
1 + 2.5          ⟶   3.5          "Addition of two numbers"
10 - 8.5         ⟶   1.5          "Subtraction of two numbers"
3.4 * 5          ⟶   17.0         "Multiplication of two numbers"
8 / 2            ⟶   4            "Division of two numbers"
2 ** 3           ⟶   8            "Exponentiation of a number"
12 = 11          ⟶   false        "Equality between two numbers"
12 ~= 11         ⟶   true         "Test if two numbers are different"
12 > 9           ⟶   true         "Greater than"
12 >= 10         ⟶   true         "Greater or equal  than"
12 < 10          ⟶   false        "Smaller than"
100 @ 10         ⟶   100@10       "Point creation"

2.718 truncated          ⟶   2      "Truncate to integer"
2.718 rounded            ⟶   3      "Round to integer"
2.718 roundTo: 0.01      ⟶   2.72   "Round to a given precision"
```

The following example works very well in Smalltalk:

```
1000 factorial / 999 factorial   ⟶   1000
```

Note that 1000 factorial is really calculated which in many other languages can be quite difficult to compute. This is an excellent example of automatic coercion and exact handling of a number.

 *Try to display the result of* 1000 factorial*. It takes more time to display it than to calculate it!*

Float

Float implements the abstract Number methods for floating point numbers.

More interestingly, Float class (*i.e.*, the class-side of Float) provides methods to return the following *constants*: e, infinity, nan, and pi.

```
Float pi                    ⟶  3.141592653589793
Float infinity              ⟶  Infinity
Float infinity isInfinite   ⟶  true
```

Fraction

Fractions are represented by instance variables for the numerator and denominator, which should be Integers. Fractions are normally created by Integer division (rather than using the constructor method Fraction»numerator:denominator:):

```
6 / 8              ⟶  (3/4)
(6 / 8) class      ⟶  Fraction
```

Multiplying a Fraction by an Integer or another Fraction may yield an Integer:

```
6 / 8 * 4    ⟶  3
```

Integer

Integer is the abstract parent of three concrete integer implementations. In addition to providing concrete implementations of many abstract Number methods, it also adds a few methods specific to integers, such as factorial, atRandom, isPrime, gcd:, and many others.

SmallInteger is special in that its instances are represented compactly — instead of being stored as a reference, a SmallInteger is represented directly using the bits that would otherwise be used to hold a reference. The first bit of an object reference indicates whether the object is a SmallInteger or not.

The class methods minVal and maxVal tell us the range of a SmallInteger, which depends on the bit-width of your processor.

When a SmallInteger goes out of this range, it is automatically converted to a LargePositiveInteger or a LargeNegativeInteger, as needed:

```
(SmallInteger maxVal + 1) class    ⟶  LargePositiveInteger
(SmallInteger minVal - 1) class    ⟶  LargeNegativeInteger
```

Large integers are similarly converted back to small integers when appropriate.

As in most programming languages, integers can be useful for specifying iterative behavior. There is a dedicated method timesRepeat: for evaluating a block repeatedly. We have already seen a similar example in Chapter 3:

```
n := 2.
3 timesRepeat: [n := n * n].
n    ⟶    256
```

ScaledDecimal

ScaledDecimal represents a decimal number with a fixed number of digits after the decimal point. Other than floats, they do not suffer from rounding errors and can represent numbers of arbitrary size; however, they also take up more space and time to calculate.

You can create a literal scaled decimal by putting the letter s at the end of a literal number, followed by an integer that specifies the number of digits after the decimal point. If you omit the scale after the s, the number of visible digits will be used:

```
2.5      ⟶    2.5       "a Float"
2.5s1    ⟶    2.5s1     "a ScaledDecimal"
2.5s2    ⟶    2.50s2    "same value, but with a higher precision"
2.5s0    ⟶    2s0       "same value, but with a lower precision"
2.5s     ⟶    2.5s1     "same value, but with automatic precision"
2s2      ⟶    2.00s2    "a ScaledDecimal without a decimal point"
```

You can also create a ScaledDecimal from another number by sending it the message asScaledDecimal:, indicating the number of digits after the decimal point:

```
(1.342 asScaledDecimal: 2)    ⟶    1.34s2
```

9.3 Characters

Character is defined in the *Collections-Strings* category as a subclass of Magnitude . Printable characters are represented in Squeak as $⟨*char*⟩. For example:

```
$a < $b    ⟶    true
```

Non-printing characters can be generated by various class methods. Character class»value: takes the Unicode (or ASCII) integer value as argument

and returns the corresponding character. The protocol *accessing untypeable characters* contains a number of convenience constructor methods such as backspace, cr, escape, euro, space, tab, and so on.

```
Character space = (Character value: Character space asciiValue)    ⟶    true
```

The printOn: method is clever enough to know which of the three ways to generate characters offers the most appropriate representation:

```
Character value: 1    ⟶    Character home
Character value: 32   ⟶    Character space
Character value: 97   ⟶    $a
```

Various convenient *testing* methods are built in: isAlphaNumeric, isCharacter , isDigit, isLowercase, isVowel, and so on.

To convert a Character to the string containing just that character, send asString. In this case asString and printString yield different results:

```
$a asString     ⟶    'a'
$a              ⟶    $a
$a printString  ⟶    '$a'
```

Every Character is an immediate object, meaning that they are managed by the VM via special pointers. As a consequence, characters are unique, immutable instances.

9.4 Strings

The String class is also defined in the category *Collections-Strings*. A String is an indexed Collection that holds only Characters.

In fact, String is abstract, and Squeak Strings are usually instances of the concrete class ByteString, or WideString if they contain any special character that cannot be represented by a single byte. The distinction between the two does not concern you most of the time, as the two classes convert their instances between each other automatically.

```
'hello world' class    ⟶    ByteString
(String value: 16r1F388) class    ⟶    WideString
```

The other important subclass of String is Symbol. The key difference is that there is only ever a single instance of Symbol with a given value. (This is sometimes called "the unique instance property"). In contrast, two separately constructed Strings that happen to contain the same sequence of characters will often be different objects.

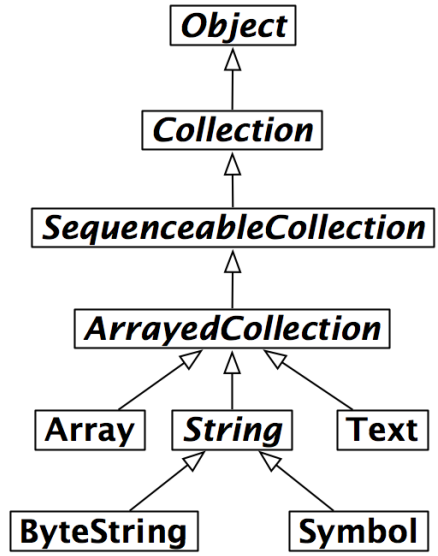


Figure 9.2: The String Hierarchy.

```
'hel','lo' == 'hello'    ⟶    false
```

```
('hel','lo') asSymbol == #hello    ⟶    true
```

Another important difference is that a String is mutable, whereas a Symbol is immutable. Still, strings that are part of a method need to be copied before altering them because they are constants:

```
'hello' copy at: 2 put: $u; yourself    ⟶    'hullo'
```

```
#hello at: 2 put: $u    ⟶    error!
```

It is easy to forget that since strings are collections, but they understand the same messages that other collections do:

```
#hello indexOf: $o    ⟶    5
```

Although String does not inherit from Magnitude, it does support the usual *comparing* methods, <, =, and so on. In addition, String»match: is useful for some basic glob-style pattern-matching:

```
'*or*' match: 'zorro'    ⟶    true
```

Should you need more advanced support for regular expressions, you can also use the Regex extension methods defined on String from the RegexCore package. For example:

```
'zorro' matchesRegex: 'z[or]{4}'    ⟶    true
'foo, bar, baz' copyWithRegex: '\w+' matchesTranslatedUsing: [:match | match
      capitalized]    ⟶    'Foo, Bar, Baz'
```

Strings support rather a large number of conversion methods. Many of these are shortcut constructor methods for other classes, such as asDate , asFileName, and so on. There are also a number of useful methods for converting a string to another string, such as capitalized and asLowercase.

For more on strings and collections, see Chapter 10.

A note on protocols. As you have seen, many classes such as String or Integer provide you with dozens of useful selectors, and we only introduced a few of them here. It is hard to memorize the full protocol of all basic classes, and many Squeakers frequently look up the relevant selectors. When you are trying to do something with some object while writing a script or application, it is often a good idea to keep a browser browser open (see Section 6.2) on its class and consult it from time to time as you are searching for a specific functionality.

 *Open a browser on the class* String, *focus the message list, and type* numb *to discover messages that you can send your string to convert it into a number.*

Also remember that you can use the method finder (see Section 6.6) for finding messages based on their output rather than their name, which works especially well for strings, collections, and numbers.

9.5 Booleans

The class Boolean offers a fascinating insight into how much of the Smalltalk language has been pushed into the class library. Boolean is the abstract superclass of the Singleton classes True and False.

Most of the behavior of Booleans can be understood by considering the method ifTrue:ifFalse:, which takes two Blocks as arguments.

```
4 factorial > 20 ifTrue: ['bigger'] ifFalse: ['smaller']    ⟶    'bigger'
```

The method is abstract in Boolean. The implementations in its concrete subclasses are both trivial:

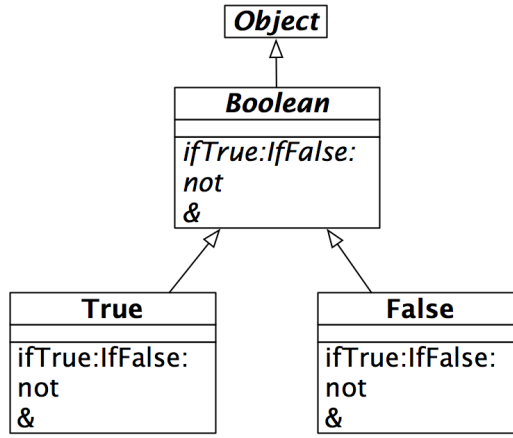


Figure 9.3: The Boolean Hierarchy.

Method 9.14: *Implementations of* ifTrue:ifFalse:.

```
True»ifTrue: trueAlternativeBlock ifFalse: falseAlternativeBlock

    ^ trueAlternativeBlock value


False»ifTrue: trueAlternativeBlock ifFalse: falseAlternativeBlock

    ^ falseAlternativeBlock value
```

In fact, this is the essence of OOP: When a message is sent to an object, the object itself determines which method will be used to respond. In this case an instance of True simply evaluates the *true* alternative, while an instance of False evaluates the *false* alternative. All the abstract Boolean methods are implemented in this way for True and False. For example:

Method 9.15: *Implementing negation.*

```
True»not
    "Negation--answer false since the receiver is true."

    ^ false
```

Booleans offer several useful convenience methods, such as ifTrue:, ifFalse:, and ifFalse:ifTrue. You also have a choice between eager and lazy conjunctions and disjunctions.

```
(1 > 2) & (3 < 4)          ⟶   false   "must evaluate both sides"
(1 > 2) and: [3 < 4]       ⟶   false   "only evaluate receiver"
```

```
(1 > 2) and: [(1 / 0) > 0]    ⟶    false    "argument block is never evaluated, so no exception"
```

In the first example, both Boolean subexpressions are evaluated, since & takes a Boolean argument. In the second and third examples, only the first is evaluated, since and: expects a Block as its argument. The Block is evaluated only if the first argument is **true**. While you have the choice, by default you should always use the lazy conjunction (and:) and disjunction (or:).

 *Try to imagine how* and: *as well as* or: *are implemented. Check the implementations in* Boolean, True, *and* False.

9.6 Exceptions

Exception might not be the first class you will need when starting to explore Squeak. However, Exceptions provide a fundamental way to influence the control flow in your application, thus we already introduce them with the other basic classes.

Similar to ifTrue: and ifFalse:, exceptions are not part of the language, but implemented as ordinary objects in the class library. Thus, we can work with them as we would work with any other object.

The common superclass of all exceptions is the abstract class Exception. There are numerous subclasses of Exception representing specific exceptions, such as ZeroDivide, MessageNotUnderstood, or ProgressNotification. In particular, subclasses of Error represent erroneous situations, and subclasses of Notification indicate that something interesting occurred.

To signal (or raise) an exception, you can use the message signal. If you want to pass a message along to help users figure out what happened, you can use signal:. When an exception is not handled by anyone, it is sent the message defaultAction which is implemented differently by different types of exceptions. The default action for Errors is to open a debugger, while the default action for Notifications is to do nothing but proceed with the execution of the program:

```
Error signal    ⟶    "Will open a debugger"
Error signal: 'We wanted this to happen'    ⟶    "Will open a debugger with a message"
Notification signal: 'We wanted this to happen'    ⟶    "Nothing happens!"
```

As we do not want to present users with debuggers every time something unexpected happens, we can also *handle* exceptions directly within our code. The methods for handling exceptions are associated with blocks,

so you can find them in the exceptions protocol of the class BlockClosure. To catch an exception, you can use the method BlockClosure»on:do:[2], which takes an exception class as the first argument and a block as the second:

```
[(-2 to: 2) collect: [:i | 100 / i]]
  on: ZeroDivide
  do: [:ex | "do nothing"].
```

The code above does not return a collection, as the block was interrupted by the ZeroDivide exception before it was able to return the result of the #collect: send. Instead of doing nothing, you can also return another value from the handler block:

```
result := (-2 to: 2) collect: [:i |
   [100 / i]
      on: ZeroDivide
      do: [:ex | Float nan]].
result    ⟶    {-50 . -100 . NaN . 100 . 50}
```

There are also ways to handle multiple exceptions at once. You can combine many exceptions into an ExceptionSet using the method Exception», and pass that ExceptionSet to on:do:. Another option is to use BlockClosure»on:do:on:do:, BlockClosure»on:do:on:do:on:do:, etc. for attaching multiple different handlers to a block.

Sometimes you might not be interested in handling an exception. Instead, you might only want to *ensure* that some code is always executed after your block, regardless of whether an exception occurs, the block returns early, or the block finishes normally. To achieve this, use the method BlockClosure»ensure:[3] which will always execute the block passed to it.

```
[connectedSocket sendData: 'my network message']
  ensure: [connectedSocket close].

[(1 to: 10) pairsDo: [:number1 :number2 | | sum |
   sum := number1 + number2.
   sum isPrime ifTrue: [^ sum]]]
      ensure: [Transcript showln: 'My execution is ensured.'].
```

Note that under normal circumstances, you should never place a return statement (^) in an ensure block. If you do so, the execution will not reach further ensure: blocks but will continue as if no exception had occurred. In particular, this can create a zombie process when you abandon a program from a debugger.

[2]This is similar to try...catch in other languages.
[3]This is similar to finally in other languages.

So far, we only have *returned* from an exception. However, exception handlers have a couple of other options for dealing with exceptions by sending messages to them:

- pass passes the exception to the next handler on the stack[4] (or to the defaultAction, if there is no further exception handler). Unlike sending signal again to the exception, pass will maintain the existing signaler stack, which is usually desired behavior (as you can debug and resume from the original exception). Using pass, you can design your exception handlers to only handle some exceptions based on their state and pass all others to the next handler:

```
[self downloadData]
   on: ConnectionRefused do: [:ex |
      self cache at: ex host ifAbsent: [ex pass]].
```

  This example tries to download some data from a server, but if the connection is refused, it will try to fallback to an offline cache. If the data is in that cache, it will return it. If the cache does not contain the data either, it will pass the exception to the next handler, which will open a debugger.

- resume continues the execution of the method that signaled the exception[5]. This is similar to pressing Proceed in the debugger for an exception. You can also pass a value along with resume: which will be returned from the original signal: send. This is a powerful mechanism that allows your code to communicate with other methods on the stack that it does not know about.

  For instance, most dialogs in Squeak signal a ProvideAnswerNotification before they show up to the user, which allows the code that eventually triggered the dialog to automatically answer the question:

```
Morph new exportAsPNG.    ⟶    "a file dialog opens for entering a file
   name"

[Morph new exportAsPNG]
   on: ProvideAnswerNotification
   do: [:ex | ex resume: 'myMorph.png'].    ⟶    "no dialog opens, the file is
    saved as 'myMorph.png'"
```

[4] This is similar to a plain throw in other languages.

[5] Most other languages do not support resumable exceptions. However, there are some similar concepts such as On Error Resume Next in Visual Basic, begin...rescue...retry in Ruby, or restart-case in Common Lisp.

Note that specifically for handling ProvideAnswerNotifications, there are the more convenient methods BlockClosure»valueSupplyingAnswers: and BlockClosure»valueSuppressingMessages:.

- retry restarts the execution of the block that signaled the exception. However, you should use this with caution as the exact method that will signal an exception is often unknown, and restarting the wrong method can lead to unexpected behavior and infinite loops.

- return returns from the handler, and return: passes along a value. This is the implicit default behavior of every handler block. The advantage of using return explicitly is that you can leave the handler block immediately, similar to a ^ return statement in a method. For instance, the following would be equivalent to the first example above:

```
[(-2 to:2) collect: [:i | 100 / i]]
  on: ZeroDivide
  do: [:ex | ex return: nil].
```

As exceptions are instances of classes like any other objects, it is a good practice to define your own exceptions by subclassing Error or Notification. This allows you to handle them with ease and to add instance variables and accessors, which can be useful for passing additional information to the exception handler. There are other subclasses of Exception as well, but most of the time you will not have to deal with them. In particular, you should never instantiate, subclass, or handle the abstract base class Exception itself.

9.7 Chapter summary

- If you override = then you should override hash as well.
- Override postCopy to correctly implement copying for your objects.
- Send **self** halt to set a breakpoint.
- Return **self** subclassResponsibility to make a method abstract.
- To give an object a String representation you should override printOn:.
- Override the hook method initialize to properly initialize instances.
- Number methods automatically convert between Floats, Fractions and Integers.

- Fractions truly represent rational numbers rather than floats.
- Characters are unique instances.
- Strings are mutable; Symbols are not. However, take care that string literals must not be mutated!
- Symbols are unique; Strings are not.
- Strings and Symbols are Collections and therefore support the usual Collection methods.
- Signal errors using Object»error: or Exception»signal: and handle them using BlockClosure»on:do:.
- You can return, resume, pass, or retry exceptions.

Chapter 10

Collections

10.1 Introduction

To make good use of the collection classes, Smalltalk programmers need at least an overview of the wide variety of collections and their commonalities and differences. Programming with collections rather than individual elements is an important way to raise the level of abstraction of a program. The Lisp function map, which applies an argument function to every element of a list and returns a new list containing the results is an early example of this style. Smalltalk-80 adopted collection-based programming as a central tenet.

Why is this a good idea? Suppose you have a data structure containing a collection of student records and wish to perform some action on all of the students that meet a particular criterion. Programmers used to an imperative language might reach for a loop. But Smalltalk programmers will write:

```
students select: [:each | each gpa < threshold].
```

which evaluates to a new collection containing precisely those elements of students for which the bracketed function returns **true**[1]. The Smalltalk code has the simplicity and elegance of a domain-specific query language.

The message select: is understood by *all* collections in Smalltalk. There is no need to find out if the student data structure is an array or a linked list: select: is understood by both. Note that this is quite different from using a loop, where one must know whether students is an array or a linked list.

[1]The expression in brackets can be thought of as a λ-expression defining an anonymous function $\lambda x.x$ gpa $<$ threshold.

Collection
- **Bag**
  - **IdentityBag**
- Bitset
- *CharacterSet ...*
- *HashedCollection*
  - **Dictionary**
    - **IdentityDictionary**
      - WeakIdentityDictionary
    - **PluggableDictionary**
      - OrderedDictionary
    - **WeakKeyDictionary**
      - **WeakIdentityKeyDictionary ...**
    - **WeakValueDictionary**
  - Set
    - **IdentitySet**
    - KeyedSet
      - KeyedIdentitySet
    - **PluggableSet**
    - **WeakSet**
- **Heap**
- Matrix
- *SequenceableCollection*
  - *ArrayedCollection ...*
    - **Array**
      - **WeakArray**
    - **String**
      - ByteString
      - Symbol
        - ByteSymbol
        - WideSymbol
      - WideString
    - **Text**
  - **Interval ...**
  - **LinkedList**
  - **OrderedCollection ...**
    - **SortedCollection**
    - WeakOrderedCollection
  - RunArray

Figure 10.1: The collection classes in Squeak. Indentation indicates subclassing. *Italicized* classes are abstract. **Bold** classes are described in the "Blue Book". If the class name is followed by ... then some or all subclasses have been omitted.

Protocol	**Methods**
accessing	size, capacity, at: *anIndex*, at: *anIndex* put: *anElement*
testing	isEmpty, includes: *anElement*, contains: *aBlock*, occurrencesOf: *anElement*
adding	add: *anElement*, addAll: *aCollection*
removing	remove: *anElement*, remove: *anElement* ifAbsent: *aBlock*, removeAll: *aCollection*
enumerating	do: *aBlock*, collect: *aBlock*, select: *aBlock*, reject: *aBlock*, detect: *aBlock*, detect: *aBlock* ifNone: *aNoneBlock*, inject: *aValue* into: *aBinaryBlock*
converting	asBag, asSet, asOrderedCollection, asSortedCollection, asArray, asSortedCollection: *aBlock*
creation	with: *anElement*, with:with:, with:with:with:, with:with:with:with:, withAll: *aCollection*
copying	., copyWith:, copyWithout:

Figure 10.2: Standard collection protocols.

Smalltalk provides a number of different kinds of collections, that all are represented as subclasses of Collection. In Squeak, the abstract class Collection has 101 subclasses, but many of these (like Bitmap, or CompiledMethod) are special-purpose classes crafted for use in other parts of the system or in applications, and hence not categorized as "Collections" by the system organization. For the purposes of this chapter, we use the term "Collection Hierarchy" to mean Collection and its 65 subclasses that are *also* in the system categories labeled *Collections-**. In this chapter, we focus mainly on the subset of collection classes shown in Figure 10.1[2].

In Smalltalk, when one speaks of a collection without being more specific about the kind of collection, one means an object that supports well-defined protocols for testing membership and enumerating the elements. *All* collections understand the *testing* messages includes:, isEmpty, and occurrencesOf:. *All* collections understand the *enumeration* messages do:, select: (which is like filter in Lisp, JavaScript, and others), reject: (which is the opposite of select:), collect: (which is like map in Lisp etc.), detect:ifNone:, inject:into: (which performs a left fold), and many more. It is the ubiquity of this protocol, as well as its variety, that makes it so powerful.

Figure 10.2 summarizes the standard protocols supported by most of the classes in the collection hierarchy. These methods are defined, redefined, optimized, or occasionally even forbidden by subclasses of Collection.

Beyond this basic uniformity, the different kinds of collections support different protocols or provide different behavior for the same requests. Let us briefly survey some of the key differences:

- **Sequenceable:** Instances of all subclasses of SequenceableCollection start from a first element and proceed in a well-defined order to a last element. Examples are LinkedList and OrderedCollection. In contrast, Set, Bag, and Dictionary are not sequenceable.
- **Sortable:** A SortedCollection maintains its elements in sort order.
- **Indexable:** Most sequenceable collections are also indexable, that is, elements can be retrieved with at:. Array is the familiar indexable data structure with a fixed size; anArray at: n retrieves the nth element of anArray, and anArray at: n put: v changes the nth element to v. LinkedLists are sequenceable but not indexable, that is, they understand first and last but not at:.

[2]The group of classes that appears in the "Blue Book" Adele Goldberg and David Robson, *Smalltalk 80: the Language and its Implementation*. Reading, Mass.: Addison Wesley, May 1983 ⟨URL: https://dl.acm.org/doi/pdf/10.5555/273⟩, ISBN 0–201–13688–0 contains 17 subclasses of Collection, and had already been redesigned several times before the Smalltalk-80 system was released. This group of classes is often considered to be a paradigmatic example of object-oriented design.

- **Keyed:** Instances of Dictionary and its subclasses are accessed by keys instead of indices.

- **Mutable:** Most collections are mutable, but Intervals and Symbols are not. An Interval is an immutable collection representing a range of Integers. For example, `5 to: 16 by: 2` is an interval that contains the elements 5, 7, 9, 11, 13, and 15. It is indexable with at:, but cannot be changed with at:put:.

- **Growable:** (Sub-)instances of Interval and ArrayedCollection are always of a fixed size. Other kinds of collections (sorted collections, ordered collections, and linked lists) can grow after creation.

  The class OrderedCollection is more flexible than Array; the size of an OrderedCollection grows on demand, and it has methods for addFirst: and addLast: as well as at: and at:put:.

- **Accepts duplicates:** A Set will filter out duplicates, but a Bag will not. Dictionary, Set, and Bag use the = method provided by the elements; the Identity variants of these classes use the == method, which tests whether the arguments are the same object, and the Pluggable variants use an arbitrary equivalence relation supplied by the creator of the collection.

- **Heterogeneous:** Most collections will hold any kind of element. A String, CharacterArray, or Symbol, however, only holds Characters. An Array will hold any mix of objects, but this is not true for RawBitsArrays: a ByteArray only holds single bytes (*i.e.*, SmallIntegers between 0 and 255), a FloatArray only holds Floats, a ColorArray only holds Colors etc. A LinkedList is constrained to hold elements that conform to the *Link ▷ accessing* protocol.

10.2 Implementations of collections

These categorizations by functionality are not our only concern; we must also consider how the collection classes are implemented. As shown in Figure 10.3, five main implementation techniques are employed.

1. ArrayedCollections store their elements in the (indexable) instance variables of the collection object itself; as a consequence, arrays must be of a fixed size, but can be created with a single memory allocation message.

Arrayed	Ordered	Hashed	Linked	Interval
Array String Symbol	OrderedCollection SortedCollection Heap Text	Set IdentitySet PluggableSet Bag IdentityBag Dictionary IdentityDictionary PluggableDictionary	LinkedList	Interval

Figure 10.3: Some collection classes categorized by implementation technique.

2. OrderedCollections and SortedCollections store their elements in an array that is referenced by one of the instance variables of the collection. Consequently, the internal array can be replaced with a larger one if the collection grows beyond its storage capacity.

3. The various kinds of Set and Dictionary also reference a subsidiary array for storage but use the array as a hash table. Bags use a subsidiary Dictionary, with the elements of the bag as keys and the number of occurrences as values.

4. LinkedLists use a standard singly-linked representation.

5. Intervals are represented by three integers that record the two endpoints and the step size.

In addition to these classes, there are also "weak" variants of Array, Set, and of the various kinds of dictionary. These collections hold onto their elements weakly, *i.e.*, in a way that does not prevent the elements from being garbage collected. The Squeak virtual machine is aware of these classes and handles them specially.

Readers interested in learning more about the Smalltalk collections are referred to LaLonde and Pugh's excellent book[3].

10.3 Examples of key classes

We present now the most common or important collection classes using simple code examples. The main protocols of collections are: at:, at:put: — to access an element, add:, remove: — to add or remove an element, size,

[3]Wilf LaLonde and John Pugh, *Inside Smalltalk: Volume 1*. Prentice Hall, 1990, ISBN 0–13–468414–1.

isEmpty, include: — to get some information about the collection, do:, collect:, and select: — to iterate over the collection. Each collection may implement or not such protocols, and when they do, they interpret them to fit with their semantics. We suggest you browse the classes themselves to identify specific and more advanced protocols.

We will focus on the most common collection classes: OrderedCollection, Set, SortedCollection, Dictionary, Interval, and Array.

Common creation protocol. There are several ways to create instances of collections. The most generic ones use the methods new: and with:. new: anInteger creates a collection of size anInteger whose elements will all be **nil**. with: anObject creates a collection and adds anObject to the created collection. Different collections will realize this behavior differently.

You can create collections with initial elements using the methods with:, with:with:, etc. for up to six elements.

```
Array with: 1    ⟶    #(1)
Array with: 1 with: 2    ⟶    #(1 2)
Array with: 1 with: 2 with: 3    ⟶    #(1 2 3)
Array with: 1 with: 2 with: 3 with: 4    ⟶    #(1 2 3 4)
Array with: 1 with: 2 with: 3 with: 4 with: 5    ⟶    #(1 2 3 4 5)
Array with: 1 with: 2 with: 3 with: 4 with: 5 with: 6    ⟶    #(1 2 3 4 5 6)
```

You can also use addAll: to add all elements of one kind of collection to another kind:

```
(1 to: 5) asOrderedCollection addAll: '678'; yourself    ⟶    an OrderedCollection(1
    2 3 4 5 $6 $7 $8)
```

You can also create many collections with withAll: or newFrom:

```
Array withAll: #(7 3 1 3)                    ⟶    #(7 3 1 3)
OrderedCollection withAll: #(7 3 1 3)        ⟶    an OrderedCollection(7 3 1 3)
SortedCollection withAll: #(7 3 1 3)         ⟶    a SortedCollection(1 3 3 7)
Set withAll: #(7 3 1 3)                      ⟶    a Set(7 1 3)
Bag withAll: #(7 3 1 3)                      ⟶    a Bag(7 1 3 3)
```

```
Array newFrom: #(7 3 1 3)                                ⟶    #(7 3 1 3)
OrderedCollection newFrom: #(7 3 1 3)                    ⟶    an OrderedCollection(7
    3 1 3)
SortedCollection newFrom: #(7 3 1 3)                     ⟶    a SortedCollection(1 3 3
    7)
Set newFrom: #(7 3 1 3)                                  ⟶    a Set(7 1 3)
Bag newFrom: #(7 3 1 3)                                  ⟶    a Bag(7 1 3 3)
Dictionary newFrom: {1 -> 7 . 2 -> 3 . 3 -> 1 . 4 -> 3}  ⟶    a Dictionary(1->7 2->3
    3->1 4->3 )
```

Note that these two methods are not identical. In particular, Dictionary class»withAll: interprets its argument as a collection of values, whereas Dictionary class»newFrom: expects a collection of associations.

Array

An Array is a fixed-sized collection of elements accessed by integer indices. Contrary to the C convention, the first element of a Smalltalk array is at position 1 and not 0. The main protocol to access array elements is the method at: and at:put:. at: anInteger returns the element at index anInteger. at: anInteger put: anObject puts anObject at index anInteger. Arrays are fixed-size collections, therefore we cannot add or remove elements at the end of an array. The following code creates an array of size 5, puts values in the first 3 locations, and returns the first element.

```
anArray := Array new: 5.
anArray at: 1 put: 4.    ⟶    4
anArray at: 2 put: 3 / 2.
anArray at: 3 put: 'ssss'.
anArray at: 1    ⟶    4
```

Beware that at:put: returns the argument and not the receiver, so you often need to build a cascade yourself!

There are several ways to create instances of the class Array. We can use new:, with:, and the constructs #( ) and { }.

Creation with new:. new: anInteger creates an array of size anInteger. Array new: 5 creates an array of size 5.

Creation with with:. with: methods allows one to specify the value of the elements. The following code creates an array of three elements consisting of the number 4, the fraction 3/2, and the string 'lulu'.

```
Array with: 4 with: 3 / 2 with: 'lulu'    ⟶    {4 . (3/2) . 'lulu'}
```

Literal creation with #(). #() creates an *array literal* with static (or "literal") elements that have to be known when the expression is compiled, and not when it is executed. The following code creates an array of size 2 where the first element is the (literal) number 1 and the second the (literal) string 'here'.

```
#(1 'here') size    ⟶    2
```

Now, if you evaluate #(1 + 2), you do not get an array with a single element 3 but instead you get the array #(1 #+ 2), *i.e.*, with three elements: 1, the symbol #+, and the number 2.

```
#(1 + 2)    ⟶    #(1 #+ 2)
```

This occurs because the construct #() causes the compiler to interpret literally the expressions contained in the array. The expression is scanned and the resulting elements are fed to a new array. Array literals can contain numbers, **nil**, **true**, **false**, characters, strings, symbols, and nested array literals.

Analogously, *byte arrays* that are created using #[ ] are literal as well.

Dynamic creation with { }. Finally, you can create an array dynamically using the brace construct {}. {a . b} is equivalent to Array with: a with: b. This means in particular that the expressions enclosed by { and } are executed.

```
{1 + 2}    ⟶    #(3)
{(1 / 2) asFloat} at: 1    ⟶    0.5
{10 atRandom . 1 / 3} at: 2    ⟶    (1/3)
```

Element Access. Elements of all sequenceable collections can be accessed with at: and at:put:.

```
anArray := #(1 2 3 4 5 6) copy.
anArray at: 3    ⟶    3
anArray at: 3 put: 33.
anArray at: 3    ⟶    33
```

Be careful with code that modifies array literals! The compiler allocates space just once for them. Unless you copy the array, it will be a *read-only object* and signal a ModificationForbidden error every time you try to alter it. Dynamically created arrays do not have this constraint.

OrderedCollection

OrderedCollection is one of the collections that can grow, and to which elements can be added sequentially. It offers a variety of methods such as add:, addFirst:, addLast:, and addAll:.

```
ordCol := OrderedCollection new.
ordCol add: 'Seaside'.    ⟶    'Seaside'
ordCol addFirst: 'Monticello'.    ⟶    'Monticello'
```

```
ordCol add: 'SqueakSource'; yourself    ⟶    an OrderedCollection('Monticello' '
    Seaside' 'SqueakSource')
```

Similarly to Collection»at:put:, all add methods return the argument and not the receiver.

Removing Elements. The method remove: anObject removes the first occurrence of an object from the collection and returns it. If the collection does not include such an object, it raises an error.

```
ordCol add: 'Monticello'.
ordCol remove: 'Monticello'.    ⟶    'Monticello'
ordCol    ⟶    an OrderedCollection('Seaside' 'SqueakSource' 'Monticello')
```

There is a variant of remove: named remove:ifAbsent: that allows one to specify as second argument a block that is executed in case the element to be removed is not in the collection.

```
ordCol remove: 'zork' ifAbsent: [33].    ⟶    33
```

Conversion. It is possible to get an OrderedCollection from an Array (or any other collection) by sending the message asOrderedCollection:

```
#(1 2 3) asOrderedCollection    ⟶    an OrderedCollection(1 2 3)
'hello' asOrderedCollection    ⟶    an OrderedCollection($h $e $l $l $o)
```

Interval

The class Interval represents ranges of numbers. For example, the interval of numbers from 1 to 100 is defined as follows:

```
Interval from: 1 to: 100    ⟶    (1 to: 100)
```

The printString of this interval reveals that the class Number provides us with a convenience method called to: to generate intervals:

```
(Interval from: 1 to: 100) = (1 to: 100)    ⟶    true
```

We can use Interval class»from:to:by: or Number»to:by: to specify the step between two numbers as follow:

```
(Interval from: 1 to: 100 by: 0.5) size    ⟶    199
(1 to: 100 by: 0.5) at: 198    ⟶    99.5
(1 / 2 to: 54 / 7 by: 1 / 3) last    ⟶    (15/2)
```

Dictionary

Dictionaries are important collections whose elements are accessed using keys. Among the most commonly used messages of dictionary you will find at:, at:put:, at:ifAbsent:, keys, and values.

```
colors := Dictionary new.
colors at: #yellow put: Color yellow.
colors at: #blue put: Color blue.
colors at: #red put: Color red.    ⟶    Color red
colors at: #yellow    ⟶    Color yellow
colors keys           ⟶    #(#red #yellow #blue)
colors values         ⟶    {Color red . Color yellow . Color blue}
```

Dictionaries compare keys by equality. Two keys are considered to be the same if they return true when compared using =. A common and difficult to spot bug is to use as key an object whose = method has been redefined but not its hash method. Both methods are used in the implementation of Dictionary and when comparing objects.

Even though Dictionary is a subclass of Collection, we would normally not want to use a Dictionary where a Collection is expected. In its implementation, however, a Dictionary can be seen as consisting of an unordered collection of associations (key-value pairs) created using the message ->. We can create a Dictionary from a collection of associations, or we may convert a dictionary to an array of associations.

```
colors := Dictionary newFrom: {#blue -> Color blue . #red -> Color red . #yellow ->
      Color yellow}.
colors removeKey: #blue.
colors associations    ⟶    {#red->Color red . #yellow->Color yellow}
```

IdentityDictionary. While a dictionary uses the result of the messages = and hash to determine if two keys are the same, the class IdentityDictionary uses the identity (message ==) of the key instead of its values, *i.e.*, it considers two keys to be equal *only* if they are the same object.

If your dictionary only uses Symbols as keys, you can also instead use an IdentityDictionary since every symbol is guaranteed to be globally unique. If, on the other hand, your keys are Strings, it is better to use a plain Dictionary, or you may get into trouble:

```
a := 'foobar'.
b := a copy.
trouble := IdentityDictionary new.
trouble
```

```
    at: a put: 'first value';
    at: b put: 'second value'.
trouble at: a              ⟶    'first value'
trouble at: b              ⟶    'second value'
trouble at: 'foobar'       ⟶    'first value'
```

a and b both point to a string 'foobar'. However, due to the copying in the first line, a and b are different objects. So, although we have used 'foobar' twice as a key to add values to the dictionary, we did not overwrite the 'first value' with 'second value', but actually added two associations to the dictionary. A plain Dictionary would give the same value for any key equal to 'foobar'.

The last line of the example is the result of the way literals are represented. The string 'foobar' occurs two times in the source code, in the first line and in the last line of the example. The compiler, however, only allocates the literal once, so a and the literal string 'foobar' are actually the same object. As this requires intricate knowledge about the implementation of the language, you should not depend on this behavior.

Set

The class Set is a collection which behaves like a mathematical set, *i.e.*, as a collection with no duplicate elements and without any order. In a Set elements are added using the message add: and they cannot be accessed using the message at:. Objects put in a set should implement the methods hash and =.

```
s := Set new.
s add: 4 / 2; add: 4; add: 2.
s size    ⟶    2
```

You can also create sets using Set class»newFrom: or the conversion message Collection»asSet:

```
(Set newFrom: #(1 2 3 1 4)) = #(1 2 3 4 3 2 1) asSet    ⟶    true
```

asSet offers us a convenient way to eliminate duplicates from a collection:

```
{Color black . Color white . (Color red + Color blue + Color green)} asSet size
        ⟶    2
```

The result of the example is 2, as the color white was included twice in the collection. Once as the result of Color white, and once as the result of the combination of red, blue, and green. However, if you are working with a

sequenceable collection and want to preserve its type or order, you can use withoutDuplicates instead.

A Bag is much like a Set except that it does allow duplicates:

```
{Color black . Color white . (Color red + Color blue + Color green)} asBag size
        ⟶  3
```

The set operations *union*, *intersection*, and *membership test* are implemented by the Collection messages union:, intersection:, and includes:. The receiver may be first converted to a Set, so these operations work for all kinds of collections!

```
(1 to: 6) union: (4 to: 10)     ⟶  a Set(1 2 3 4 5 6 7 8 9 10)
'hello' intersection: 'there'   ⟶  'he'
#Smalltalk includes: $k         ⟶  true
```

As we explain below, elements of a set are accessed using iterators (see Section 10.4).

Analogous to Dictionary and IdentityDictionary, there is also an IdentitySet class, which uses equality of identity instead of equality of value to determine whether an element is in the set or not.

SortedCollection

In contrast to an OrderedCollection, a SortedCollection maintains its elements in sort order. You can create a SortedCollection by creating a new instance and adding elements to it:

```
SortedCollection new add: 5; add: 2; add: 50; add: -10; yourself.   ⟶  a
    SortedCollection(-10 2 5 50)
```

While SortedCollections are quite useful if the collection should always be in sorted order, they should not be used to simply sort a collection once. To achieve the latter, you can use the messages sorted and sorted: which are introduced in Section 10.6.

```
'hello' sorted   ⟶  #($e $h $l $l $o)
```

If you really want a SortedCollection, you can more easily create one by sending the conversion message asSortedCollection to an existing collection:

```
#(5 2 50 -10) asSortedCollection   ⟶  a SortedCollection(-10 2 5 50)
'hello' asSortedCollection         ⟶  a SortedCollection($e $h $l $l $o)
```

How do you get a String back from this result? asString unfortunately returns the printString representation, which is not what we want:

```
'hello' asSortedCollection asString    ⟶    'a SortedCollection($e $h $l $l $o)'
```

The correct answer is to either use String class»newFrom:, String class»withAll:, or Object»as::

```
'hello' asSortedCollection as: String              ⟶    'ehllo'
String newFrom: ('hello' asSortedCollection)       ⟶    'ehllo'
String withAll: ('hello' asSortedCollection)       ⟶    'ehllo'
```

It is possible to have different kinds of elements in a SortedCollection as long as they are all comparable. For example, we can mix different kinds of numbers such as integers, floats, and fractions:

```
{5 . 2 / -3 . 5.21} asSortedCollection    ⟶    a SortedCollection((-2/3) 5 5.21)
```

Imagine that you want to sort objects that do not define the method <= or that you would like to have a different sorting criterion. You can do this by supplying a so-called sort function to the sorted collection. For example, the class Color is not a Magnitude and it does not implement the method <=, but we can specify that the colors should be sorted according to their luminance (a measure of brightness).

```
col := { Color red . Color yellow . Color white . Color black }.
col asSortedCollection: #luminance ascending    ⟶    a SortedCollection(Color
    black Color red Color yellow Color white)
```

We will explain sort functions in more detail in Section 10.6.

String

A Smalltalk String represents a collection of Characters. It is sequenceable, indexable, mutable, and homogeneous, containing only Character instances. Like Arrays, Strings have a dedicated syntax, and are normally created by directly specifying a String literal within single quotes, but the usual collection creation methods will work as well.

```
'Hello'                                ⟶    'Hello'
String with: $A                        ⟶    'A'
String with: $h with: $i with: $!      ⟶    'hi!'
String newFrom: #($h $e $l $l $o)      ⟶    'hello'
```

In actual fact, String is abstract. When we instantiate a String we actually get either an 8-bit ByteString or a 32-bit WideString. To keep things simple, we usually ignore the difference and just talk about instances of String.

Two instances of String can be concatenated with a comma.

```
s := 'no' , ' ' , 'worries'.
s    ⟶    'no worries'
```

Since a string is a mutable collection (unless it is a method literal) we can also change it using the method at:put:.

```
s at: 4 put: $h; at: 5 put: $u.
s    ⟶    'no hurries'
```

Note that the comma method is defined by Collection, so it will work for any kind of collection!

```
(1 to: 3) , '45'    ⟶    #(1 2 3 $4 $5)
```

We can also modify an existing string in place using replaceAll:with: or replaceFrom:to:with: as shown below. Note that the number of characters and the interval should have the same size.

```
s replaceAll: $n with: $N.
s    ⟶    'No hurries'
s replaceFrom: 4 to: 5 with: 'wo'.
s    ⟶    'No worries'
```

In contrast to the methods described above, the method copyReplaceAll: creates a new string. (Curiously, here the arguments are substrings rather than individual characters, and their sizes do not have to match.)

```
s copyReplaceAll: 'rries' with: 'mbats'    ⟶    'No wombats'
```

If you take a quick look at the implementation of these methods, you can see that they are defined not only for Strings but for any kind of SequenceableCollection, so the following also works:

```
(1 to: 6) copyReplaceAll: (3 to: 5) with: {'three' . 'etc.'}    ⟶    #(1 2 'three' 'etc.' 6)
```

String matching. It is possible to ask whether a pattern matches a string by sending the match: message. The pattern can specify * to match an arbitrary series of characters and # to match a single character. Note that match: is sent to the pattern and not the string to be matched.

```
'Linux *' match: 'Linux mag'                ⟶    true
'GNU/Linux #ag' match: 'GNU/Linux tag'      ⟶    true
```

Another useful method is findString:.

```
'GNU/Linux mag' findString: 'Linux'                                    ⟶  5
'GNU/Linux mag' findString: 'linux' startingAt: 1 caseSensitive: false  ⟶  5
```

More advanced pattern matching facilities based on regular expressions are provided by the Regex package.

Some tests on strings. The following examples illustrate the use of isEmpty, includes:, and anySatisfy: which are further messages defined not only on Strings but more generally on collections.

```
'Hello' isEmpty  ⟶  false
'Hello' includes: $a  ⟶  false
'JOE' anySatisfy: [:c | c isLowercase]  ⟶  false
'Joe' anySatisfy: [:c | c isLowercase]  ⟶  true
```

String templating. There are three messages that are useful to manage string templating: format:, expandMacros, and expandMacrosWith:.

```
'{1} is {2}' format: {'Squeak' . 'cool'}  ⟶  'Squeak is cool'
```

The messages of the expandMacros family offer variable substitution, using <n> for carriage return, <t> for tabulation, <1s>, <2s>, <3s> for arguments (<1p>, <2p>, surrounds the string with single quotes), and <1?value1:value2> for conditional.

```
'look-<t>-here' expandMacros                                          ⟶  'look- -here'
'<1s> is <2s>' expandMacrosWith: 'Squeak' with: 'cool'                ⟶  'Squeak is cool'
'<2s> is <1s>' expandMacrosWith: 'Squeak' with: 'cool'                ⟶  'cool is Squeak'
'<1p> or <1s>' expandMacrosWith: 'Squeak' with: 'cool'                ⟶  '"Squeak" or
    Squeak'
'<1?Quentin:Thibaut> plays' expandMacrosWith: true                    ⟶  'Quentin plays'
'<1?Quentin:Thibaut> plays' expandMacrosWith: false                   ⟶  'Thibaut plays'
```

Some other utility methods. The class String offers numerous other utilities including the messages asLowercase, asUppercase, and capitalized.

```
'XYZ' asLowercase  ⟶  'xyz'
'xyz' asUppercase  ⟶  'XYZ'
'hilaire' capitalized  ⟶  'Hilaire'
'1.54' asNumber  ⟶  1.54
'this sentence is without a doubt far too long' contractTo: 20  ⟶  'this sent...too
    long'
```

Note that there is generally a difference between asking an object its string representation by sending the message printString and converting it to a string by sending the message asString. Here is an example of the difference.

```
#ASymbol printString  ⟶  '#ASymbol'
#ASymbol asString     ⟶  'ASymbol'
```

A symbol is similar to a string but is guaranteed to be globally unique. For this reason, symbols are preferred to strings as keys for dictionaries, in particular for instances of IdentityDictionary. See also Chapter 9 for more about String and Symbol.

10.4 Collection iterators

In Smalltalk, loops and conditionals are simply messages sent to collections or other objects such as integers or blocks (see also Chapter 3). In addition to low-level messages such as to:do: which evaluates a block with an argument ranging from an initial to a final number, the Smalltalk collection hierarchy offers various high-level iterators. Using such iterators will make your code more robust and compact.

Iterating (do:)

The method do: is the basic collection iterator. It applies its argument (a block taking a single argument) to each element of the receiver. The following example prints all the strings contained in the receiver to the transcript.

```
#('bob' 'joe' 'toto') do: [:each | Transcript showln: each].
```

Variants. There are a lot of variants of do:, such as do:without:, withIndexDo:, and reverseDo:: For the indexed collections (Array, OrderedCollection, and SortedCollection), the method withIndexDo: also gives access to the current index. This method is related to to:do: which is defined in class Number.

```
#('bob' 'joe' 'toto') withIndexDo: [:each :i | each = 'joe' ifTrue: [^ i]]  ⟶  2
```

For ordered collections, reverseDo: walks the collection in the reverse order.

The following code shows an interesting message: do:separatedBy: which executes the second block only in between two elements.

```
res := ''.
#('bob' 'joe' 'toto') do: [:e | res := res , e] separatedBy: [res := res , '.'].
res    ⟶    'bob.joe.toto'
```

Note that this code is not especially efficient since it creates intermediate strings and it would be better to use a write stream to buffer the result (see Chapter 11):

```
String streamContents: [:stream | #('bob' 'joe' 'toto') joinOn: stream separatedBy: '.']
        ⟶    'bob.joe.toto'
```

This particular example can be expressed even more concisely as follows:

```
#('bob' 'joe' 'toto') joinSeparatedBy: '.'    ⟶    'bob.joe.toto'
```

Dictionaries. When the message do: is sent to a dictionary, the elements taken into account are the values, not the associations. The proper methods to use are keysDo:, valuesDo:, and associationsDo:, which iterate respectively on keys, values or associations.

```
colors := Dictionary newFrom: {#yellow -> Color yellow . #blue -> Color blue . #red ->
      Color red}.
colors keysDo: [:key | Transcript showln: key].                "displays keys"
colors valuesDo: [:value | Transcript showln: value].          "displays values"
colors associationsDo: [:value | Transcript showln: value].    "displays associations"
```

Collecting results (collect:)

If you want to process the elements of a collection and produce a new collection as a result, rather than using do:, you are probably better off using collect:, or one of the other iterator methods. Most of these can be found in the *enumerating* protocol of Collection and its subclasses.

Imagine that we want a collection containing the doubles of the elements in another collection. Using the method do: we must write the following:

```
double := OrderedCollection new.
#(1 2 3 4 5 6) do: [:e | double add: e * 2].
double    ⟶    an OrderedCollection(2 4 6 8 10 12)
```

The method collect: executes its argument block for each element and returns a new collection containing the results. Using collect: instead, the code is much simpler:

```
#(1 2 3 4 5 6) collect: [:e | e * 2]    ⟶    #(2 4 6 8 10 12)
```

The advantages of collect: over do: are even more dramatic in the following example, where we take a collection of integers and generate as a result a collection of absolute values of these integers:

```
aCol := #(2 -3 4 -35 4 -11).
result := aCol species new: aCol size.
1 to: aCol size do: [:each | result at: each put: (aCol at: each) abs].
result    ⟶    #(2 3 4 35 4 11)
```

Contrast the above with the much simpler following expression:

```
#(2 -3 4 -35 4 -11) collect: [:each | each abs]    ⟶    #(2 3 4 35 4 11)
```

A further advantage of the second solution is that it will also work for sets and bags.

Generally, you should only use do: when you only want to send messages to each of the elements of a collection.

Collection types. Note that sending the message collect: returns the same kind of collection as the receiver. For this reason, the following code fails. (A String cannot hold integer values.)

```
'abc' collect: [:each | each asciiValue].    "error!"
```

Instead we must first convert the string to an Array or an OrderedCollection:

```
'abc' asArray collect: [:each | each asciiValue]    ⟶    #(97 98 99)
```

We can even save one redundant copy operation of the string by using collect:as: instead:

```
'abc' collect: [:each | each asciiValue] as: Array    ⟶    #(97 98 99)
```

Actually collect: is not guaranteed to return a collection of exactly the same class as the receiver, but only the same *"species"*. In the case of an Interval, the species is actually Array!

```
(1 to: 5) collect: [:each | each * 2]    ⟶    #(2 4 6 8 10)
```

Shortcuts. If you look up the implementation of collect:, do:, autc. in any collection class, you will find that they evaluate the argument block by sending the message value: to it. That means that you actually can pass

any other object that understands the message value: to collect:, do:, etc. In particular, this includes the classes Symbol and MessageSend. Thus, you could also write the above expression as follows:

```
#(2 -3 4 -35 4 -11) collect: #abs    ⟶    #(2 3 4 35 4 11)
```

While using a selector symbol instead of a block is a popular shortcut, it is not always considered good style. As opposed to blocks, symbols are slower to evaluate, and you cannot directly step into them with the debugger.

Selecting and rejecting elements

select: returns the elements of the receiver that satisfy a particular condition:

```
(2 to: 20) select: [:each | each isPrime]    ⟶    #(2 3 5 7 11 13 17 19)
```

reject: does the opposite:

```
(2 to: 20) reject: [:each | each isPrime]    ⟶    #(4 6 8 9 10 12 14 15 16 18 20)
```

Identifying an element with detect:

The method detect: returns the first element of the receiver that matches the block argument.

```
'through' detect: [:each | each isVowel]    ⟶    $o
```

If detect: can not find an element, it will signal a NotFound error. The method detect:ifNone: is a variant of the method detect:. Its second block is evaluated when there is no element matching the block.

```
Smalltalk allClasses detect: [:each | '*java*' match: each asString] ifNone: [nil]
        ⟶    nil
```

Correspondingly, there is also the variant detect:ifFound: that let's you handle the case when an element is found or otherwise returns **nil**:

```
'Smalltalk' detect: [:each | each isVowel] ifFound: [:char | char asString capitalized]
        ⟶    'A'
```

Accumulating results

Functional programming languages often provide a higher-order function called *fold* or *reduce* to accumulate a result by applying some binary operator iteratively over all elements of a collection. In Squeak this is done by Collection»inject:into: or Collection»fold: (also available as Collection»reduce:).

The first argument of inject:into: is an initial value, and the second argument is a two-argument block which is applied to the result this far, and each element in turn.

A trivial application of inject:into: is to produce the sum of a collection of numbers. For example, we could write this expression to sum the first 100 integers:

```
(1 to: 100) inject: 0 into: [:sum :each | sum + each]    ⟶    5050
```

Because Symbol also understands Symbol»value:value:, we can also write this:

```
(1 to: 100) inject: 0 into: #+    ⟶    5050
```

Another example is the following one-argument block which computes factorials:

```
factorial := [:n | (1 to: n) inject: 1 into: [:product :each | product * each]].
factorial value: 10    ⟶    3628800
```

fold: is a simpler version of inject:into: that uses the first element of the collection as the initial value. However, it signals an error if the collection is empty, whereas inject:into: lets you specify a default value in that case.

```
#(1 2 3 4) fold: [:sum :each | sum + each]    ⟶    10
#() fold: [:sum :each | sum + each]    ⟶    Error: This collection is empty.
```

Other messages

count:. The message count: returns the number of elements satisfying a condition. The condition is represented as a boolean block.

```
Smalltalk allClasses count: [:each | 'Collection*' match: each asString]    ⟶    4
```

includes:. The message includes: checks whether the argument is contained in the collection.

```
colors := {Color white . Color yellow. Color red . Color blue . Color orange}.
colors includes: Color blue.    ⟶    true
```

anySatisfy:. The message anySatisfy: answers true if at least one element of the collection satisfies the condition represented by the argument. Analogously, there are also **allSatisfy:** and **noneSatisfy:**.

```
colors anySatisfy: [:color | color red > 0.5]   ⟶   true
colors allSatisfy: [:color | color red > 0.5]   ⟶   false
colors noneSatisfy: [:color | color red > 1]    ⟶   true
```

gather:. The message gather: can be used to turn nested collections into a flat collection while at the same time converting the values. Some languages refer to this collection operation as flat map.

```
colors gather: [:color | color names]   ⟶   #(#white #snow #yellow #lemon #red
    #maraschino #blue #blueberry #orange)
```

10.5 Some hints for using collections

A common mistake with add:. The following error is one of the most frequent Smalltalk mistakes.

```
collection := OrderedCollection new add: 1; add: 2.
collection   ⟶   2
```

Here the variable collection does not hold the newly created collection but rather the last number added. This is because the method add: returns the element added and not the receiver.

The following code yields the expected result:

```
collection := OrderedCollection new.
collection add: 1; add: 2.
collection   ⟶   an OrderedCollection(1 2)
```

You can also use the message yourself to return the receiver of a cascade of messages:

```
collection := OrderedCollection new add: 1; add: 2; yourself   ⟶   an
    OrderedCollection(1 2)
```

Removing an element of the collection you are iterating on. Another mistake you may make is to remove an element from a collection you are currently iterating over.

```
range := (2 to: 20) asOrderedCollection.
range do: [:aNumber | aNumber isPrime ifFalse: [range remove: aNumber]].
range    ⟶    an OrderedCollection(2 3 5 7 9 11 13 15 17 19)
```

This result is clearly incorrect since 9 and 15 should have been filtered out!

The solution is to copy the collection before going over it.

```
range := (2 to: 20) asOrderedCollection.
range copy do: [:aNumber | aNumber isPrime ifFalse: [range remove: aNumber]].
range    ⟶    an OrderedCollection(2 3 5 7 11 13 17 19)
```

Redefining both = and hash. A difficult error to spot is when you redefine = but not hash. The symptoms are that you will lose elements that you put in sets or other strange behavior. One solution proposed by Kent Beck is to use bitXor: to redefine hash. Suppose that we want two books to be considered equal if their titles and authors are the same. Then we would redefine not only = but also hash as follows:

Method 10.1: *Redefining* = *and* hash.

```
1  Book»= aBook
2
3    self class = aBook class ifFalse: [^ false].
4    ^ title = aBook title and: [authors = aBook authors]
5
6
7  Book»hash
8
9    ^ title hash bitXor: authors hash
```

Another nasty problem arises if you use a mutable object, *i.e.*, an object that can change its hash value over time, as an element of a Set or as a key to a Dictionary. Don't do this unless you love debugging! (If you need to do it anyway, you can send the message rehash to the collection to force it to recompute the hash values of all its elements.)

10.6 Sorting collections

Basically, there are two selectors for sorting a collection: sort and sorted. So when should you use which of them? Actually, sort is only defined on certain specializations of Collection, such as ArrayedCollection or OrderedCollection. This message performs an in-place-sort on the collection, *i.e.*, after sending

sort to a collection instance, the order of its elements will have been updated. Conversely, sorted: performs an out-of-place sort that will not modify the original collection but will answer a sorted copy of the receiver. While an in-place-sort might reach higher performance in some situations, an out-of-place sort is the preferable option in most situations because a) it does not restrict the collection you use to be an array or an ordered collection, b) it is applicable to read-only literal arrays and strings as well, and c) it helps you avoid unintended side-effects that eventually can make your code harder to debug and understand.

The following examples illustrates the difference between in-place and out-of-place sorting:

```
array := {3. 5. 1}.

"Out-of-place sort"
array sorted.    ⟶    #(1 3 5)
array.    ⟶    #(3 5 1)

"In-place sort"
array sort.
array.    ⟶    #(1 3 5)
```

Sort methods use the message <= to establish sort order, so by default, they can sort magnitudes such as numbers, characters, timestamps, etc. (see Chapter 9 on Magnitude), but also other objects such as strings or points. However, what can you do if you want to sort objects that do not implement <=, or if you would like to customize the sort order? For this purpose, Collection also provides sorted: in addition to sorted (note the colon), and analogously, arrays etc. also implement sort:. These messages take a two-argument sort block or a *sort function* that, speaking formally, defines a partial order on the collection to be sorted.[4] Here is a simple example of how to use a sort block:

```
#('Foo' 'Bar' 'baz') sorted: [:a :b | a caseInsensitiveLessOrEqual: b]    ⟶    #('Bar' '
    baz' 'Foo')
```

In many cases, you will need to sort a collection by a particular property or transformation of each element. This is often realized by implementing the property function twice, *e.g.* (beware, negative example!):

```
(1 to: 10) sorted: [:a :b | (a raisedTo: 2 modulo: 3) <= (b raisedTo: 2 modulo: 3)]
        ⟶    #(3 6 9 1 2 4 5 7 8 10)
```

However, the sort function protocol provides a much more elegant solution to this problem by defining a dedicated PropertySortFunction class. It

[4] Further reading: Wikipedia – Partially ordered set

can be instantiated from any unary block by sending the message ascending or descending to it:

```
(1 to: 10) sorted: [:ea | ea raisedTo: 2 modulo: 3] ascending    ⟶    #(3 6 9 1 2 4 5 7
    8 10)
```

ascending and descending are defined on Symbol as well, so we could also write the previous example in the following way:

```
#('Foo' 'Bar' 'baz') sorted: #asUppercase descending    ⟶    #('Foo' 'baz' 'Bar')
```

Sort functions provide a bunch of other useful functionality. For example, you can chain them by using ,, reverse the order of elements by using reversed, or make sure that your code does not founder at **nil** values in a collection by using undefinedFirst or undefinedLast. Let's lump them all together to demonstrate the overwhelming power of sort functions:

```
{1. 100. nil. 1000. 10} sorted: [:ea | ea ifNotNil: [ea asWords first]] ascending
    undefinedLast , SortFunction default reversed.    ⟶    #(1000 100 1 10 nil)
```

For further information on sort functions, you can browse the system category *Collections-SortFunctions* and its extension methods on BlockClosure and Symbol.

10.7 Chapter summary

The Smalltalk collection hierarchy provides a common vocabulary for uniformly manipulating a variety of different kinds of collections.

- A key distinction is between SequenceableCollections, which maintain their elements in a given order, Dictionary and its subclasses, which maintain key-to-value associations, and Sets and Bags, which are unordered.
- You can convert most collections to another kind of collection by sending them the messages asArray, asOrderedCollection etc.
- To sort a collection, send it the message sorted or sorted:, to sort it in-place, send sort or sort:, and to create a collection that always keeps its element sorted send asSortedCollection or asSortedCollection:.
- Literal Arrays are created with the special syntax #(...). Dynamic Arrays are created with the syntax {...}.

- A Dictionary compares keys by equality. It is most useful when keys are instances of String. An IdentityDictionary instead uses object identity to compare keys. It is more suitable when Symbols are used as keys, or when mapping object references to values.
- Strings also understand the usual collection messages. In addition, a String supports a simple form of pattern-matching. For more advanced applications, look instead at the Regex package.
- The basic iteration message is do:. It is useful for imperative code, such as modifying each element of a collection, or sending each element a message.
- Instead of using do:, it is more common to use collect:, select:, reject:, includes:, inject:into:, and other higher-level messages to process collections in a uniform way.
- Never remove an element from a collection you are iterating over. If you must modify it, iterate over a copy instead.
- If you override =, remember to override hash as well!

Chapter 11

Streams

Streams are used to iterate over sequences of elements such as sequenced collections, files, and network streams. With Streams you can represent different iterations on the same collection, speed up the creation of large collections, and even iterate over potentially infinite collections without running into an infinite loop. A Stream instance represents an iteration for reading, writing, or both. Reading or writing is always relative to the current position in the stream.

11.1 Two sequences of elements

A good model to understand a stream is the following: A stream can be represented as two sequences of elements: a past element sequence and a future element sequence. The stream is positioned between the two sequences. Understanding this model is important since all stream operations in Smalltalk rely on it. For this reason, most of the Stream classes are subclasses of PositionableStream. Figure 11.1 presents a stream which contains five characters. This stream is in its original position, *i.e.*, there is no element in the past. You can go back to this position using the message reset.

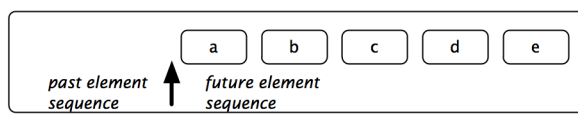


Figure 11.1: A stream positioned at its beginning.

You can now read the next element from the stream using the message next. Reading an element conceptually means removing the first element of the future element sequence, returning it, and putting it after the last element in the past element sequence. After having read one element using the message next, the state of your stream is that shown in Figure 11.2.

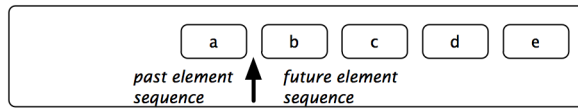


Figure 11.2: The same stream after the execution of the method next. The character a is "in the past" whereas b, c, d and e are "in the future".

You can write an element to a stream using the message nextPut: anElement . Writing an element means replacing the first element of the future sequence with the new one and moving it to the past. Figure 11.3 shows the state of the same stream after having written an x.

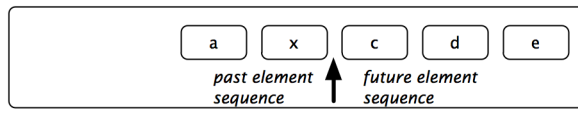


Figure 11.3: The same stream after having written an x.

11.2 Streams vs. collections

The collection protocol supports the storage, removal, and enumeration of the elements of a collection, but does not allow these operations to be intermingled. For example, if the elements of an OrderedCollection are processed by a do: method, it is not possible to add or remove elements from inside the do: block. Nor does the collection protocol offer ways to iterate over two collections at the same time, choosing which collection goes forward and which does not. Procedures like these require that a traversal index or position reference is maintained outside of the collection itself: This is exactly the role of ReadStream, WriteStream, and ReadWriteStream.

These three classes are defined to *stream over* some collection. For example, the following snippet creates a stream on an interval, then it reads two elements.

```
r := ReadStream on: (1 to: 1000).
r next      ⟶    1
r next      ⟶    2
r atEnd     ⟶    false
```

WriteStreams can write data to the collection:

```
w := WriteStream on: (String new: 5).
w nextPut: $a; nextPut: $b.
w contents     ⟶     'ab'
```

It is also possible to create ReadWriteStreams that support both the reading and writing protocols.

Streams are not only meant for collections, they can be used for files or sockets too. The following example creates a file named test.txt, writes two strings to it, separated by a carriage return, and closes the file. After that, the file contents are read again into the image.

```
StandardFileStream
  fileNamed: 'someTestFile.txt'
  do: [:stream | stream
       nextPutAll: '123';
       space;
       nextPutAll: 'abcd'].

StandardFileStream readOnlyFileNamed: 'someTestFile.txt' do: [:stream | stream
       contents]     ⟶     '123 abcd'
```

The following sections present the protocols in more depth.

11.3 Streaming over collections

Streams are really useful when dealing with collections of elements. They can be used for reading and writing elements in collections.

Reading collections

This section presents features used for reading collections. Using a stream to read a collection essentially provides you a pointer into the collection. That pointer will move forward on reading and you can place it wherever you want. The class ReadStream should be used to read elements from collections.

Methods next and next: are used to retrieve one or more elements from the collection.

```
stream := ReadStream on: #(1 (a b c) false).
stream next    ⟶    1
stream next    ⟶    #(#a #b #c)
stream next    ⟶    false
```

```
stream := ReadStream on: 'abcdef'.
stream next: 0    ⟶    ''
stream next: 1    ⟶    'a'
stream next: 3    ⟶    'bcd'
stream next: 2    ⟶    'ef'
```

The message peek is used when you want to know what is the next element in the stream without going forward.

```
stream := ReadStream on: '-143'.
negative := stream peek = $-.    "look at the first element without reading it"
negative    ⟶    true
negative ifTrue: [stream next].    "ignores the minus character"
number := stream upToEnd.
number    ⟶    '143'
```

This code sets the boolean variable negative according to the sign of the number in the stream and number to its absolute value. The method upToEnd returns everything from the current position to the end of the stream and sets the stream to its end. This code can be simplified using peekFor:, which moves forward if the following element equals the parameter and doesn't move otherwise.

```
stream := '-143' readStream.
stream peekFor: $-    ⟶    true
stream upToEnd          ⟶    '143'
```

peekFor: also returns a boolean indicating if the parameter equals the element.

You might have noticed a new way of constructing a stream in the above example: One can simply send readStream to a sequenceable collection to get a reading stream on that particular collection.

Positioning. There are also methods to position the stream pointer directly. If you have the index, you can go directly to it using position:. You can request the current position using position. Please remember that a stream is not positioned on an element, but between two elements. The index corresponding to the beginning of the stream is 0.

You can obtain the state of the stream depicted in Figure 11.4 with the following code:

```
stream := 'abcde' readStream.
stream position: 2.
stream peek    ⟶    $c
```

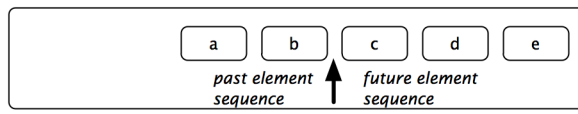


Figure 11.4: A stream at position 2

To position the stream at the beginning or the end, you can use reset or setToEnd. skip: and skipTo: are used to go forward to a location relative to the current position: skip: accepts a number as argument and skips that number of elements whereas skipTo: skips all elements in the stream until it finds an element equal to its parameter. Note that it positions the stream after the matched element.

```
stream := 'abcdef' readStream.
stream next          ⟶    $a    "stream is now positioned just after the a"
stream skip: 3.                  "stream is now after the d"
stream position      ⟶     4
stream skip: -2.                 "stream is after the b"
stream position      ⟶    2
stream reset.
stream position      ⟶    0
stream skipTo: $e.               "stream is just after the e now"
stream next          ⟶    $f
stream contents      ⟶    'abcdef'
```

As you can see, the letter e has been skipped. The method contents always returns a copy of the entire stream.

Testing. Some methods allow you to test the state of the current stream: atEnd returns true if and only if no more elements can be read whereas isEmpty returns true if and only if there is no element at all in the collection.

Here is a possible implementation of an algorithm using atEnd that takes two sorted collections as parameters and merges those collections into another sorted collection:

```
stream1 := #(1 4 9 11 12 13) readStream.
stream2 := #(1 2 3 4 5 10 13 14 15) readStream.

"The variable result will contain the sorted collection."
result := OrderedCollection new.
[stream1 atEnd not and: [stream2 atEnd not]]
   whileTrue: [stream1 peek < stream2 peek
      "Remove the smallest element from either stream and add it to the result."
      ifTrue: [result add: stream1 next]
      ifFalse: [result add: stream2 next]].

"One of the two streams might not be at its end. Copy whatever remains."
result
   addAll: stream1 upToEnd;
   addAll: stream2 upToEnd.

result.   ⟶   an OrderedCollection(1 1 2 3 4 4 5 9 10 11 12 13 13 14 15)
```

Writing to collections

We have already seen how to read a collection by iterating over its elements using a ReadStream. We'll now learn how to create collections using WriteStreams.

WriteStreams are useful for appending a lot of data to a collection at various locations. They are often used to construct strings that are based on static and dynamic parts as in this example:

```
stream := String new writeStream.
stream
   nextPutAll: 'This Smalltalk image contains: ';
   print: Smalltalk allClasses size;
   nextPutAll: ' classes.';
   cr;
   nextPutAll: 'This is really a lot.'.

stream contents.   ⟶   'This Smalltalk image contains: 2814 classes.
This is really a lot.'
```

This technique is used in the different implementations of the method printOn: for example. There is a simpler and more efficient way of creating streams if you are only interested in the content of the stream:

```
string := String streamContents: [:stream |
        stream
            print: #(1 2 3);
```

```
            space;
            nextPutAll: 'size';
            space;
            nextPut: $=;
            space;
            print: 3].
string    ⟶    '#(1 2 3) size = 3'
```

The method streamContents: creates a collection and a stream on that collection for you. It then executes the block you gave passing the stream as a parameter. When the block ends, streamContents: returns the content of the collection.

The following WriteStream methods are especially useful in this context:

nextPut: adds the parameter to the stream;

nextPutAll: adds each element of the collection, passed as a parameter, to the stream;

print: adds the textual representation of the parameter to the stream.

There are also methods useful for printing different kinds of characters to the stream like space, tab, and cr (carriage return). Another useful method is ensureASpace which ensures that the last character in the stream is a space; if the last character isn't a space it adds one.

About Concatenation. Using nextPut: and nextPutAll: on a WriteStream is often the best way to concatenate characters. Using the comma concatenation selector (,) is far less efficient:

```
[| temp |
  temp := String new.
  (1 to: 100000) do: [:i |
    temp := temp, i asString, ' ']] timeToRun.    ⟶    386381 "(milliseconds)"

[| temp |
  temp := WriteStream on: String new.
  (1 to: 100000) do: [:i |
    temp nextPutAll: i asString; space].
  temp contents] timeToRun.    ⟶    25 "(milliseconds)"
```

The reason that using a stream can be much more efficient is that the comma creates a new string containing the concatenation of the receiver and the argument, so it must copy both of them. When you repeatedly concatenate onto the same receiver, it gets longer and longer each time, so

that the number of characters that must be copied goes up exponentially. This also creates a lot of garbage, which must be collected. Using a stream instead of string concatenation is a well-known optimization. In fact, you can use streamContents: (mentioned on page 233) to help you do this:

```
String streamContents: [:tempStream |
  (1 to: 100000) do: [:i |
    tempStream nextPutAll: i asString; space]].
```

Reading and writing at the same time

It's possible to use a stream to access a collection for reading and writing at the same time. Imagine you want to create a SBEHistory class which will manage backward and forward buttons in a web browser. A history would react as in figures from 11.5 to 11.11.



Figure 11.5: A new history is empty. Nothing is displayed in the web browser.

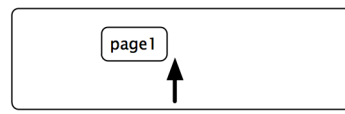


Figure 11.6: The user opens page 1.

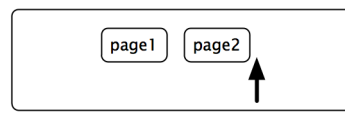


Figure 11.7: The user clicks on a link to page 2.

This behavior can be implemented using a ReadWriteStream.

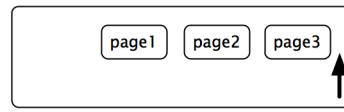


Figure 11.8: The user clicks on a link to page 3.

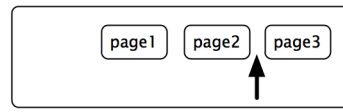


Figure 11.9: The user clicks on the back button. He is now viewing page 2 again.

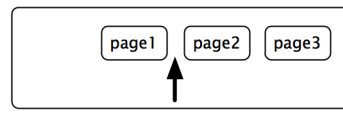


Figure 11.10: The user clicks again the back button. Page 1 is now displayed.

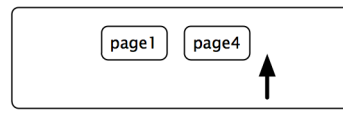


Figure 11.11: From page 1, the user clicks on a link to page 4. The history forgets pages 2 and 3.

```
Object subclass: #SBEHistory
    instanceVariableNames: 'stream'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-Streams'
```

Method 11.1: *Initialization of* SBEHistory *instances.*

```
SBEHistory»initialize

    super initialize.
    stream := ReadWriteStream on: Array new.
```

Nothing really difficult here, we define a new class which contains a stream. The stream is created during the initialize method.

We need methods to go backward and forward:

Method 11.2: *Method* goBackward *in* SBEHistory.

```
SBEHistory»goBackward

    self canGoBackward ifFalse: [self error: 'Already on the first element'].
    stream skip: -2.
    ^ stream next
```

Method 11.3: *Method* goForward *in* SBEHistory.

```
SBEHistory»goForward

    self canGoForward ifFalse: [self error: 'Already on the last element'].
    ^ stream next
```

Until then, the code was pretty straightforward. Now, we have to deal with the goTo: method which should be activated when the user clicks on a link. A possible solution is:

```
SBEHistory>>goTo: aPage

    stream nextPut: aPage.
```

This version is incomplete however, as when the user clicks on the link, there should be no more future pages to go to, *i.e.*, the forward button must be deactivated. To represent this, the simplest solution is to write **nil** after adding the page to indicate the end of the history:

Method 11.4: *Method* goTo: *in* SBEHistory.

```
SBEHistory»goTo: anObject

    stream
        nextPut: anObject;
        nextPut: nil;
        back.
```

Now, only methods canGoBackward and canGoForward have to be implemented.

A stream is always positioned between two elements. To go backward, there must be two pages before the current position: One page is the current page, and the other one is the page we want to go to.

Method 11.5: *Method* canGoBackward *in* SBEHistory.

```
1 SBEHistory»canGoBackward
2
3   ^ stream position > 1
```

To go forward, the stream should not be at its end, and the next element should not be **nil**, which we used to represent the end of the history.

Method 11.6: *Method* canGoForward *in* SBEHistory.

```
1 SBEHistory»canGoForward
2
3   ^ stream atEnd not and: [stream peek notNil]
```

Let us add a method to peek at the contents of the stream:

```
SBEHistory>>contents

  ^ stream contents
```

And the history works as advertised:

```
SBEHistory new
  goTo: #page1;
  goTo: #page2;
  goTo: #page3;
  goBackward;
  goBackward;
  goTo: #page4;
  contents.   ⟶   #(#page1 #page4 nil nil)
```

11.4 Using streams for file access

You have already seen how to stream over collections of elements. It's also possible to stream over files on your hard disk. Once created, a stream on a file is really like a stream on a collection: You will be able to use the same protocol to read, write, or position the stream. The main difference appears in the creation of the stream. There are several different ways to create file streams, as we shall now see.

Creating file streams

To create file streams, you will have to use one of the following instance creation methods offered by the class FileStream:

fileNamed: Open a file with the given name for reading and writing. If the file already exists, its prior contents may be modified or replaced, but the file will not be truncated on close. If the name has no directory part, then the file will be created in the default directory.

newFileNamed: Create a new file with the given name and answer a stream opened for writing on that file. If the file already exists, ask the user what to do.

forceNewFileNamed: Create a new file with the given name and answer a stream opened for writing on that file. If the file already exists, delete it without asking before creating the new file.

oldFileNamed: Open an existing file with the given name for reading and writing. If the file already exists, its prior contents may be modified or replaced, but the file will not be truncated on close. If the name has no directory part, then the file will be created in the default directory.

readOnlyFileNamed: Open an existing file with the given name for reading.

You have to remember that each time you open a stream on a file, you have to close it too. This is done through the close method.

```
stream := FileStream forceNewFileNamed: 'test.txt'.
stream
   nextPutAll: 'This text is written in a file named ';
   print: stream localName.
stream close.

stream := FileStream readOnlyFileNamed: 'test.txt'.
stream contents.    ⟶    'This text is written in a file named "test.txt"'
stream close.
```

The method localName answers the last component of the name of the file. You can also access the full pathname using the method fullName.

You will soon notice that manually closing the file stream is painful and error-prone. That's why FileStream offers varieties of the above messages, that take care of opening and closing the FileStream before and after you work with it. You pass the code that works with the stream as a block to these methods, *e.g.*, the second argument of forceNewFileNamed:do:.

```
FileStream
   forceNewFileNamed: 'test.txt'
   do: [:stream |
      stream
         nextPutAll: 'This text is written in a file named ';
         print: stream localName].
string := FileStream
         readOnlyFileNamed: 'test.txt'
         do: [:stream | stream contents].
string    ⟶    'This text is written in a file named "test.txt"'
```

The stream-creation methods that take a block as an argument first create a stream on a file, then execute the block with the stream as an argument, and finally, close the stream. These methods return what is returned by the block, which is to say, the value of the last expression in the block. This is used in the previous example to get the content of the file and put it in the variable string.

Binary streams

By default, file streams are text-based which means you will read and write characters. If your stream must be binary, you have to send the message binary to your stream.

When your stream is in binary mode, you can only write numbers from 0 to 255 (1 Byte). If you want to use nextPutAll: to write more than one number at a time, you have to pass a ByteArray as an argument.

```
FileStream
 forceNewFileNamed: 'test.bin'
 do: [:stream |
       stream
         binary;
         nextPutAll: #[145 250 139 98]].

FileStream
 readOnlyFileNamed: 'test.bin'
 do: [:stream |
       stream binary.
       stream size.          ⟶    4
       stream next.          ⟶    145
       stream upToEnd    ⟶    #[250 139 98]
      ].
```

Here is another example which creates a picture in a file named “test.pgm”

(portable graymap file format). You can open this file with your favorite drawing program. If you do not have a relevant program, you can also drag the file into your Squeak image to watch it.

Method 11.7: *Writing a checkerboard file.*

```
SBEFileStreams»exampleCheckerboard
    "SBEFileStreams exampleCheckerboard"

    FileStream
        forceNewFileNamed: 'test.pgm'
        do: [:stream |
            stream
                nextPutAll: 'P5'; cr;
                nextPutAll: '4 4'; cr;
                nextPutAll: '255'; cr;
                binary;
                nextPutAll: #[255 0 255 0];
                nextPutAll: #[0 255 0 255];
                nextPutAll: #[255 0 255 0];
                nextPutAll: #[0 255 0 255]].
```

This creates a 4x4 checkerboard as shown in Figure 11.12.

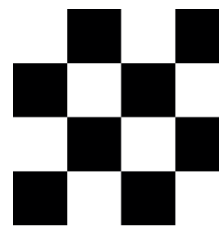


Figure 11.12: A 4x4 checkerboard you can draw using binary streams.

11.5 Chapter summary

Streams offer a better way than collections to incrementally read and write a sequence of elements. There are easy ways to convert back and forth between streams and collections.

- Streams may be either readable, writeable, or both readable and writeable.

- To get a stream for a collection, create a stream "on" a collection, *e.g.*, `ReadStream on: (1 to: 1000)`, or send the messages `readStream` etc. to the collection.
- To get the collection of a stream, send the message `contents`.
- To concatenate large collections, instead of using the concatenation operator, it is more efficient to create a stream, append the collections to the stream with `nextPutAll:`, and extract the result by sending `contents`.
- File streams are by default character-based. Send `binary` to explicitly make them binary.

Chapter 12

Morphic

Morphic is the name given to Squeak's graphical interface. Morphic is written in Smalltalk, so it is fully portable between operating systems; as a consequence, Squeak looks exactly the same on Unix, macOS, and Windows. What distinguishes Morphic from most other user interface toolkits is that it does not have separate modes for "composing" and "running" the interface: All the graphical elements can be assembled and disassembled by the user, at any time.[1]

12.1 The history of Morphic

Morphic was developed by John Maloney and Randy Smith for the Self language, starting around 1993. Maloney later wrote a Morphic for Squeak with the basic ideas of the Self version still alive and well: *directness* and *liveness*. Directness means that the shapes on the screen are objects that can be examined or changed directly, that is, by pointing at them using the mouse. Liveness means that the user interface is always able to respond to user actions: Information on the screen is continuously updated as the world that it describes changes. A simple example of this is that you can detach a menu item and keep it as a button.

 *Bring up the world menu. Blue-click on the world menu to bring up its morphic halo, then blue-click again on a menu item to bring up its halo. Now drag that item elsewhere on the screen by grabbing the black handle*  *, as shown in Figure 12.1.*

[1] We thank Hilaire Fernandes for permission to base this chapter on his original article in French.

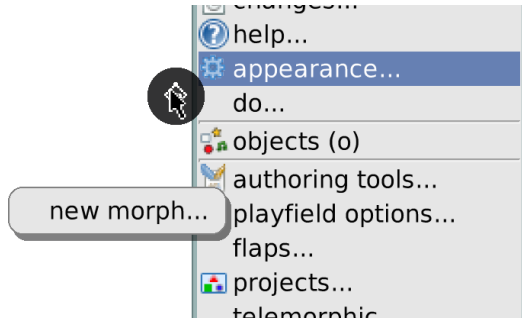


Figure 12.1: Detaching a morph, here the new morph menu item, to make it an independent button.

All of the objects that you see on the screen when you run Squeak are *Morphs*, that is, they are instances of subclasses of class Morph. Morph itself is a large class with many methods; this makes it possible for subclasses to implement interesting behavior with little code. You can create a morph to represent any object, although how good a representation you get depends on whether the creators of the class of the object provided the necessary methods.

To create a morph to represent a string object, execute the following code in a workspace, one line at a time.

```
s := 'Morph' asMorph openInWorld.
s explore.
```

The first line creates a Morph to represent the string 'Morph', and then opens it (that is, displays it) in the "world", which is the name that Squeak gives to the screen. You should obtain a graphical element — a Morph — which you can manipulate by blue-clicking. The second line opens an explorer that shows you attributes of this Morph, such as its fullBounds and color.

Of course, it is possible to define morphs that are more interesting graphical representations than the one that you have just seen. The method asMorph has a default implementation in class Object class that just creates a StringMorph. For many objects, that is not a very useful graphical representation and thus the override it with a more interesting mapping.

Open a browser on the Color *class and browse the following method.*

Method 12.1: *Getting a morph for an instance of* Color.

```
Color»asMorph

    ^ (RectangleMorph new)
        fillStyle: self;
        borderWidth: 0;
        yourself
```

When you try this method by executing Color orange asMorph openInWorld in a workspace, you get an orange rectangle, instead of a plain StringMorph!

12.2 Manipulating morphs

Morphs are objects, so we can manipulate them like any other object in Smalltalk: By sending messages, we can change their properties, create new subclasses of Morph, and so on.

Every morph, even if it is not currently open on the screen, has a position and a size. For convenience, all morphs are considered to occupy a rectangular region of the screen; if they are irregularly shaped, their position and size are those of the smallest rectangular "box" that surrounds them, which is known as the morph's bounding box, or just its "bounds". The position method returns a Point that describes the location of the morph's upper left corner (which is also the upper left corner of its bounding box). The origin of the coordinate system is the screen's upper left corner, with y coordinates increasing *down* the screen and x coordinates increasing to the *right*. The extent method also returns a point, but this point specifies the width and height of the morph rather than a location.



Type the following code into a workspace and do it*:*

```
joe := Morph new color: Color blue.
joe openInWorld.
bill := Morph new color: Color red .
bill openInWorld.
```

Then type joe position and print it. To move joe, execute joe position: joe position + (10 @ 3) repeatedly.

It is possible to do a similar thing with size. joe extent answers joe's size; to have joe grow, execute joe extent: joe extent * 1.1. To change the color of a morph, send it the color: message with the desired Color object as argument, for instance, joe color: Color orange. To add transparency, try joe color: (Color orange alpha: 0.5).

To make bill follow joe, you can repeatedly execute this code:

```
bill position: joe position + (100 @ 0).
```

If you move joe using the mouse and then execute this code, bill will move so that it is 100 pixels to the right of joe.

12.3 Composing morphs

One way of creating new graphical representations is by placing one morph inside another. This is called *composition*; morphs can be composed to any depth. You can place a morph inside another by sending the message addMorph: to the container morph.

Try adding a morph to another one:

```
star := StarMorph new color: Color yellow.
joe addMorph: star.
star position: joe position.
```

The last line positions the star at the same coordinates as joe. Notice that the coordinates of the contained morph are still relative to the screen, not to the containing morph. There are many methods available to position a morph; browse the *geometry* protocol of class Morph to see for yourself. For example, to center the star inside joe, execute star center: joe center.

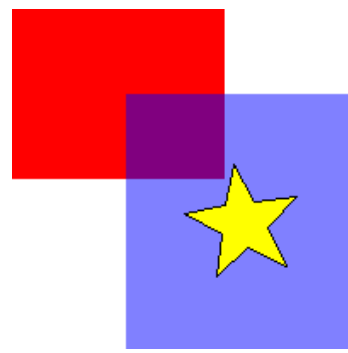


Figure 12.2: The star is contained inside joe, the translucent blue morph.

If you now try to grab the star with the mouse, you will find that you actually grab joe, and the two morphs move together: The star is *embedded* inside joe. It is possible to embed more morphs inside joe. In addition

to doing this programmatically, you can also embed morphs by direct manipulation.

From the world menu select "objects" and click on the button labeled "Graphics". Drag an ellipse and a star from the Supplies *flap. Place the star over the ellipse, and yellow-click the star to bring up a menu*[2]*. Select* embed into ▷ Ellipse*. Now the star and the ellipse move together.*

To remove a sub-morph, do joe removeMorph: star or star delete. This can also be done by direct manipulation:

Blue click twice on the star. Drag the star away from the ellipse using the grab handle  .

The first click brings up the morphic halo on the ellipse; the second click the halo on the star. Each click moves the focus down one level of embedding.

12.4 Creating and drawing your own morphs

While it is possible to make many interesting and useful graphical representations by composing morphs, sometimes you will need to create something completely different. To do this you define a subclass of Morph and override the drawOn: method to change its appearance.

The morphic framework sends the message drawOn: to a morph when it needs to redisplay the morph on the screen. The parameter to drawOn: is a kind of Canvas; the expected behavior is that the morph will draw itself on that canvas, inside its bounds. Let's use this knowledge to create a cross-shaped morph.

Using the class browser, define a new class SBECrossMorph *inheriting from* Morph*:*

Class 12.2: *Defining* SBECrossMorph.

```
1 Morph subclass: #SBECrossMorph
2    instanceVariableNames: ''
3    classVariableNames: ''
4    poolDictionaries: ''
5    category: 'SBE-Morphic'
```

We can define the drawOn: method like this:

[2]You can also blue-click the star to bring up the morphic halo, and then click the red menu handle.

Method 12.3: *Drawing a* SBECrossMorph.

```
drawOn: aCanvas

    | crossHeight crossWidth horizontalBar verticalBar |
    crossHeight := self height / 3.0 .
    crossWidth := self width / 3.0 .
    horizontalBar := self bounds insetBy: 0@crossHeight.
    verticalBar := self bounds insetBy: crossWidth@0.
    aCanvas fillRectangle: horizontalBar color: self color.
    aCanvas fillRectangle: verticalBar color: self color.
```

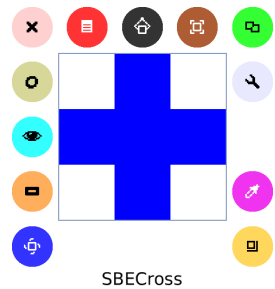


Figure 12.3: A SBECrossMorph with its halo; you can resize it as you wish.

Sending the bounds message to a morph answers its bounding box, which is an instance of Rectangle. Rectangles understand many messages that create other rectangles of related geometry; here we use the insetBy: message with a point as its argument to create first a rectangle with reduced height, and then another rectangle with reduced width.

To test your new morph, execute SBECrossMorph new openInWorld.

The result should look something like Figure 12.3. However, you will notice that the sensitive zone — where you can click to grab the morph — is still the whole bounding box. Let's fix this.

When the Morphic framework needs to find out which Morphs lie under the cursor, it sends the message containsPoint: to all the morphs whose bounding boxes lie under the mouse pointer. So, to limit the sensitive zone of the morph to the cross shape, we need to override the containsPoint: method.

Define the following method in class SBECrossMorph*:*

Method 12.4: *Shaping the sensitive zone of the* SBECrossMorph.

```
containsPoint: aPoint

    | crossHeight crossWidth horizontalBar verticalBar |
    crossHeight := self height / 3.0.
    crossWidth := self width / 3.0.
    horizontalBar := self bounds insetBy: 0@crossHeight.
    verticalBar := self bounds insetBy: crossWidth@0.
    ^ (horizontalBar containsPoint: aPoint)
       or: [verticalBar containsPoint: aPoint]
```

This method uses the same logic as drawOn:, so we can be confident that the points for which containsPoint: answers **true** are the same ones that will be colored in by drawOn. Notice how we leverage the containsPoint: method in class Rectangle to do the hard work.

There are two problems with the code in methods 12.3 and 12.4. The most obvious is that we have duplicated code. This is a cardinal error: if we find that we need to change the way that horizontalBar or verticalBar are calculated, we are quite likely to forget to change one of the two occurrences. The solution is to factor out these calculations into two new methods, which we put in the private protocol:

Method 12.5: horizontalBar.

```
SBECrossMorph»horizontalBar

    | crossHeight |
    crossHeight := self height / 3.0.
    ^ self bounds insetBy: 0 @ crossHeight
```

Method 12.6: verticalBar.

```
SBECrossMorph»verticalBar

    | crossWidth |
    crossWidth := self width / 3.0.
    ^ self bounds insetBy: crossWidth @ 0
```

We can then define both drawOn: and containsPoint: using these methods:

Method 12.7: *Refactored* SBECrossMorph»drawOn:.

```
SBECrossMorph»drawOn: aCanvas

    aCanvas fillRectangle: self horizontalBar color: self color.
    aCanvas fillRectangle: self verticalBar color: self color.
```



Figure 12.4: The center of the cross is filled twice with the color.



Figure 12.5: The cross-shaped morph, showing a row of unfilled pixels.

Method 12.8: *Refactored* SBECrossMorph»containsPoint:.

```
1 SBECrossMorph»containsPoint: aPoint
2
3   ^ (self horizontalBar containsPoint: aPoint)
4     or: [self verticalBar containsPoint: aPoint]
```

This code is much simpler to understand, largely because we have given meaningful names to the private methods. In fact, it is so simple that you may have noticed the second problem: The area in the center of the cross, which is under both the horizontal and the vertical bars, is drawn twice. This doesn't matter when we fill the cross with an opaque color, but the bug becomes apparent immediately if we draw a semi-transparent cross, as shown in Figure 12.4.

 *Execute the following code in a workspace, line by line:*

```
m := SBECrossMorph newBounds: (0 @ 0 corner: 300 @ 300).
m openInWorld.
m color: (Color blue alpha: 0.3).
```

The fix is to divide the vertical bar into three pieces, and to fill only the top and bottom. Once again we find a method in class Rectangle that does the hard work for us: r1 areasOutside: r2 answers an array of rectangles comprising the parts of r1 outside r2. Here is the revised code:

Method 12.9: *The revised* drawOn: *method, which fills the center of the cross once.*

```
1 SBECrossMorph»drawOn: aCanvas
2
3   | topAndBottom |
4   aCanvas fillRectangle: self horizontalBar color: self color.
5   topAndBottom := self verticalBar areasOutside: self horizontalBar.
6   topAndBottom do: [:each | aCanvas fillRectangle: each color: self color].
```

This code seems to work, but if you try it on some crosses and resize them, you may notice that at some sizes, a one-pixel wide line separates

the bottom of the cross from the remainder, as shown in Figure 12.5. This is due to rounding: when the size of the rectangle to be filled is not an integer, fillRectangle: color: seems to round inconsistently, leaving one row of pixels unfilled. We can work around this by rounding explicitly when we calculate the sizes of the bars.

Method 12.10: SBECrossMorph»horizontalBar *with explicit rounding.*

```
1 SBECrossMorph»horizontalBar
2
3     | crossHeight |
4     crossHeight := (self height / 3.0) rounded.
5     ^ self bounds insetBy: 0 @ crossHeight
```

Method 12.11: SBECrossMorph»verticalBar *with explicit rounding.*

```
1 SBECrossMorph»verticalBar
2
3     | crossWidth |
4     crossWidth := (self width / 3.0) rounded.
5     ^ self bounds insetBy: crossWidth @ 0
```

12.5 Interaction and animation

To build live user-interfaces using morphs, we need to be able to interact with them using the mouse and the keyboard. Moreover, the morphs need to be able to respond to user input by changing their appearance and position — that is, by animating themselves.

Mouse events

When a mouse button is pressed, Morphic sends each morph under the mouse pointer the message handlesMouseDown:. If a morph answers **true**, then Morphic immediately sends it the mouseDown: message; it also sends the mouseUp: message when the user releases the mouse button. If all morphs answer **false**, then Morphic initiates a drag-and-drop operation. As we will discuss below, the mouseDown: and mouseUp: messages are sent with an argument — a MouseEvent object — that encodes the details of the mouse action.

Let's extend SBECrossMorph to handle mouse events. We start by ensuring that all crossMorphs answer **true** to the handlesMouseDown: message.



Add this method to SBECrossMorph*:*

Method 12.12: *Declaring that* SBECrossMorph *will react to mouse clicks.*

```
SBECrossMorph»handlesMouseDown: anEvent

    ^ true
```

Suppose that when the red mouse button is clicked, we want to change the color of the cross to red, and when the yellow button is clicked we want to change the color to yellow. This can be accomplished by method 12.13.

Method 12.13: *Reacting to mouse clicks by changing the morph's color.*

```
SBECrossMorph»mouseDown: anEvent

    anEvent redButtonPressed
        ifTrue: [self color: Color red].
    anEvent yellowButtonPressed
        ifTrue: [self color: Color yellow].
    self changed.
```

Notice that in addition to changing the color of the morph, this method also sends **self** changed. This makes sure that morphic sends drawOn: in a timely fashion. Note also that once the morph handles mouse events, you can no longer grab it with the mouse and move it. Instead you have to use the halo: blue-click on the morph to make the halo appear and grab either the brown move handle or the black pickup handle at the top of the morph.

The anEvent argument of mouseDown: is an instance of MouseEvent, which is a subclass of MorphicEvent. MouseEvent defines the redButtonPressed and yellowButtonPressed methods. Browse this class to see what other methods it provides to interrogate the mouse event.

Keyboard events

To catch keyboard events, we need to take three steps.

1. Give the "keyboard focus" to a specific morph: For instance, we can give focus to our morph when the mouse is over it.

2. Handle the keyboard event itself with the handleKeystroke: method: This message is sent to the morph that has keyboard focus when the user presses a key.

3. Release the keyboard focus when the mouse is no longer over our morph.

Let's extend SBECrossMorph so that it reacts to keystrokes. First, we need to arrange to be notified when the mouse is over the morph. This will happen if our morph answers **true** to the handlesMouseOver: message.

 *Declare that* SBECrossMorph *will react when it is under the mouse pointer.*

Method 12.14: *We want to handle "mouse over" events.*

```
1 SBECrossMorph»handlesMouseOver: anEvent
2
3     ^ true
```

This message is the equivalent of handlesMouseDown: for the mouse position. When the mouse pointer enters or leaves the morph, the mouseEnter: and mouseLeave: messages are sent to it.

 *Define two methods so that* SBECrossMorph *catches and releases the keyboard focus, and a third method to actually handle the keystrokes.*

Method 12.15: *Getting the keyboard focus when the mouse enters the morph.*

```
1 SBECrossMorph»mouseEnter: anEvent
2
3     anEvent hand newKeyboardFocus: self.
```

Method 12.16: *Handing back the focus when the pointer goes away.*

```
1 SBECrossMorph»mouseLeave: anEvent
2
3     anEvent hand releaseKeyboardFocus: self.
```

Method 12.17: *Receiving and handling keyboard events.*

```
1  SBECrossMorph»handleKeystroke: anEvent
2
3      | keyValue |
4      keyValue := anEvent keyValue.
5      keyValue = 30    "up arrow"
6          ifTrue: [self position: self position - (0 @ 1)].
7      keyValue = 31    "down arrow"
8          ifTrue: [self position: self position + (0 @ 1)].
9      keyValue = 29    "right arrow"
10         ifTrue: [self position: self position + (1 @ 0)].
11     keyValue = 28    "left arrow"
12         ifTrue: [self position: self position - (1 @ 0)].
```

We have written this method so that you can move the morph using the arrow keys. Note that when the mouse is no longer over the morph,

the handleKeystroke: message is not sent, so the morph stops responding to keyboard commands. To discover the key values, you can open a Transcript window and add Transcript show: anEvent keyValue to method 12.17. The anEvent argument of handleKeystroke: is an instance of KeyboardEvent, another subclass of MorphicEvent. Browse this class to learn more about keyboard events.

Global accessors for UI objects

In complex applications, one may need to refer to the currently processed event without having it available in the current method. While we should avoid such situations in general, there is an accessor message defined on Object, currentEvent, that allows us to retrieve the event currently being handled again from any method in any object. To do so, we can simply write **self** currentEvent in an arbitrary method. Analogously, there are currentHand and currentWorld to retrieve the current HandMorph (which represents your cursor) or PasteUpMorph (which contains all other morphs displayed at your screen).

In some cases, one may also want to directly control the user interaction without following the usual event handling control flow. As an example, we may want to track the mouse cursor and find out which point the user clicks next. Implementing this via the usual event-handling protocol would be complicated because we want to handle click events for morphs whose methods we do not control.[3] For cases like this, there is the class EventSensor an instance of which you can retrieve by evaluating EventSensor default. To implement our example, we can do the following:

Method 12.18: *Retrieving information about the cursor from* EventSensor *without using events.*

```
1 getPositionFromClick
2
3     | position |
4     [EventSensor default anyButtonPressed] whileFalse: [
5         position := EventSensor default cursorPoint].
6     ^ position
```

However, keep in mind that all these approaches use global state which can lead to problems (just imagine a Squeak image that deals with multiple cursors[4], which cursor's position are we retrieving now?). So whenever pos-

[3] Actually, there is an advanced mechanism called *event-bubbling and -capturing filters* that would be applicable to this problem. However, this approach requires a bit more code and thus is not optimally suited for our simple use case.

[4] This is actually the case in some multi-touch implementations for Squeak.

sible, try to avoid these accessors and stick with the usual event-handling protocol instead.

Morphic animations

Morphic provides a simple animation system with two main methods: step is sent to a morph at regular intervals of time, while stepTime specifies the time in milliseconds between steps.[5] In addition, startStepping turns on the stepping mechanism, while stopStepping turns it off again; isStepping can be used to find out whether a morph is currently being stepped.



Make SBECrossMorph *blink by defining these methods as follows:*

Method 12.19: *Defining the animation time interval.*

```
SBECrossMorph»stepTime

    ^ 100
```

Method 12.20: *Making a step in the animation.*

```
SBECrossMorph»step

    (self color diff: Color black) < 0.1
        ifTrue: [self color: Color red]
        ifFalse: [self color: self color darker].
```

To start things off, you can open an inspector on a SBECrossMorph (using the debug handle in the morphic halo), type **self** startStepping in the small workspace pane at the bottom, and do it. Alternatively, you can modify the handleKeystroke: method so that you can use the + and − keys to start and stop stepping.



Add the following code to method 12.17:

```
keyValue = $+ asciiValue
    ifTrue: [self startStepping].
keyValue = $- asciiValue
    ifTrue: [self stopStepping].
```

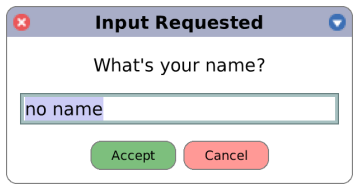


Figure 12.6: Dialog displayed by UIManager»request:initialAnswer:.

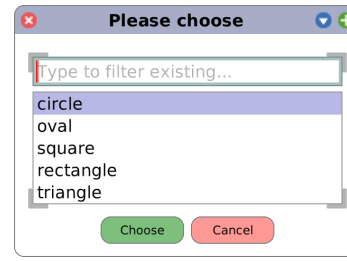


Figure 12.7: Dialog displayed by UIManager»chooseFrom:.

12.6 Dialog windows

To prompt the user for input, the UIManager class provides a few ready-to-use dialog boxes. For instance, the request:initialAnswer: method returns the string entered by the user (Figure 12.6). However, UIManager is an abstract class that has several subclasses such as MorphicUIManager, MVCUIManager, and others. What's the reason for this?

Squeak supports a range of different UI systems, called , of which Morphic is only one of many. For instance, another project type in Squeak is (named after its tripartite Model-View-Controller architecture) which was the only available UI system in classic Smalltalk-80. To allow developers to write code that works in any kind of project, Squeak provides the UIManager class hierarchy that defines a consistent interface to invoke user dialogs for any UI system.

You can retrieve an appropriate subclass instance of UIManager for the current project by sending Project uiManager.

For example, to display the request dialog in Figure 12.6, you can send the message request:initialAnswer::

```
Project uiManager
   request: 'What''s your name?'
   initialAnswer: 'no name'
```

To display the request dialog in Figure 12.7, send a chooseFrom::

```
Project uiManager
   chooseFrom: #('circle' 'oval' 'square' 'rectangle' 'triangle')
```

[5] stepTime is actually the *minimum* time between steps. If you ask for a stepTime of 1 ms, don't be surprised if Squeak is too busy to step your morph that often.

12.7 Drag-and-drop

Morphic also supports drag-and-drop. Let's examine a simple example with two morphs, a receiver morph and a dropped morph. The receiver will accept a morph only if the dropped morph matches a given condition: In our example, the morph should be blue. If it is rejected, the dropped morph decides what to do.

 *Let's first define the receiver morph:*

Class 12.21: *Defining a morph on which we can drop other morphs.*

```
1 Morph subclass: #ReceiverMorph
2    instanceVariableNames: ''
3    classVariableNames: ''
4    poolDictionaries: ''
5    category: 'SBE-Morphic'
```

 *Now define the initialization method in the usual way:*

Method 12.22: *Initializing* ReceiverMorph.

```
1 ReceiverMorph»initialize
2
3    super initialize.
4    color := Color red.
5    bounds := 0 @ 0 extent: 200 @ 200.
```

How do we decide if the receiver morph will accept or repel the dropped morph? In general, both of the morphs will have to agree to the interaction. The receiver does this by responding to wantsDroppedMorph:event:; the first argument is the dropped morph, and the second the mouse event, so that the receiver can, for example, see if any modifier keys were held down at the time of the drop. The dropped morph is also given the opportunity to check and see if it likes the morph onto which it is being dropped; it is sent the message wantsToBeDroppedInto:. The default implementation of this method (in the class Morph) answers **true**.

Method 12.23: *Accept dropped morphs based on their color.*

```
1 ReceiverMorph»wantsDroppedMorph: aMorph event: anEvent
2
3    ^ aMorph color = Color blue
```

What happens to the dropped morph if the receiving morph doesn't want it? The default behavior is for it to do nothing, that is, to sit on top of

the receiving morph, but without interacting with it. A more intuitive behavior is for the dropped morph to go back to its original position. This can be achieved by the receiver answering **true** to the message repelsMorph:event: when it doesn't want the dropped morph:

Method 12.24: *Changing the behavior of the dropped morph when it is rejected.*

```
ReceiverMorph»repelsMorph: aMorph event: anEvent
    ^ (self wantsDroppedMorph: aMorph event: anEvent) not
```

That's all we need as far as the receiver is concerned.

 *Create instances of* ReceiverMorph *and* EllipseMorph *in a workspace:*

```
ReceiverMorph new openInWorld.
EllipseMorph new openInWorld.
```

Try to drag-and-drop the yellow EllipseMorph onto the receiver. It will be rejected and sent back to its initial position.

 *To change this behavior, change the color of the ellipse morph to* Color blue *using an inspector. Blue morphs should be accepted by the* ReceiverMorph.

Let's create a specific subclass of Morph, named DroppedMorph, so we can experiment a bit more:

Class 12.25: *Defining a morph we can drag-and-drop onto* ReceiverMorph.

```
Morph subclass: #DroppedMorph
    instanceVariableNames: ''
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-Morphic'
```

Method 12.26: *Initializing* DroppedMorph.

```
DroppedMorph»initialize

    super initialize.
    self
        color: Color blue;
        position: 250 @ 100.
```

Now we can specify what the dropped morph should do when it is rejected by the receiver; here it will stay attached to the mouse pointer:

Method 12.27: *Reacting when the morph was dropped but rejected.*

```
DroppedMorph»rejectDropMorphEvent: anEvent

    | h |
    h := anEvent hand.
    WorldState addDeferredUIMessage: [h grabMorph: self].
    anEvent wasHandled: true.
```

Sending the hand message to an event answers the *hand*, an instance of HandMorph that represents the mouse pointer and whatever it holds. Here we tell the World that the hand should grab **self**, the rejected morph.

Create two instances of DroppedMorph*, and then drag-and-drop them onto the receiver.*

```
ReceiverMorph new openInWorld.
(DroppedMorph new color: Color blue) openInWorld.
(DroppedMorph new color: Color green) openInWorld.
```

The green morph is rejected and therefore stays attached to the mouse pointer.

12.8 A complete example

Let's design a morph to roll a die. Clicking on it will display the values of all sides of the die in a quick loop, and another click will stop the animation.

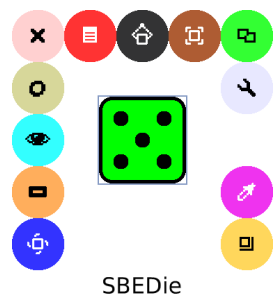


Figure 12.8: The die in Morphic.

Define the die as a subclass of BorderedMorph *instead of* Morph *because we will make use of the border.*

Class 12.28: *Defining the die morph.*

```
BorderedMorph subclass: #SBEDieMorph
    instanceVariableNames: 'faces dieValue isStopped'
    classVariableNames: ''
    poolDictionaries: ''
    category: 'SBE-Morphic'
```

The instance variable faces records the number of faces on the die; we allow dice with up to 9 faces! dieValue records the value of the face that is currently displayed, and isStopped is true if the die animation has stopped running. To create a die instance, we define the faces: n method on the *class* side of SBEDieMorph to create a new die with n faces.

Method 12.29: *Creating a new die with the number of faces we like.*

```
SBEDieMorph class»faces: aNumber

    ^ self new faces: aNumber
```

The initialize method is defined on the instance side in the usual way; remember that new sends initialize to the newly-created instance.

Method 12.30: *Initializing instances of* SBEDieMorph.

```
SBEDieMorph»initialize

    super initialize.
    self
        extent: 50 @ 50;
        useGradientFill; borderWidth: 2; useRoundedCorners;
        setBorderStyle: #complexRaised;
        color: Color green.
    dieValue := 1.
    faces := 6.
    isStopped := false.
```

We use a few methods of BorderedMorph to give a nice appearance to the die: a thick border with a raised effect, rounded corners, and a color gradient on the visible face. We define the instance method faces: to check for a valid parameter as follows:

Method 12.31: *Setting the number of faces of the die.*

```
SBEDieMorph»faces: aNumber
    "Set the number of faces"

    self assert: (aNumber isInteger and: [aNumber > 0] and: [aNumber <= 9]).
    faces := aNumber.
```

Now that we have created an acessor method for this instance variable, we can also use it in the initialize method to assure that all changes to the variable go through the accessor method.

It may be good to review the order in which the messages are sent when a die is created. For instance, if we start by evaluating SBEDieMorph faces: 9:

1. The class method SBEDieMorph class»faces: sends new to SBEDieMorph class.
2. The method for new (inherited by SBEDieMorph class from Behavior) creates the new instance and sends it the initialize message.
3. The initialize method in SBEDieMorph sets faces to an initial value of 6.
4. SBEDieMorph class»new returns to the class method SBEDieMorph class» faces:, which then sends the message faces: 9 to the new instance.
5. The instance method SBEDieMorph»faces: now executes, setting the faces instance variable to 9.

Before defining drawOn:, we need a few methods to place the dots on the displayed face:

Methods 12.32: *Nine methods for placing points on the faces of the die.*

```
SBEDieMorph»face1
    ^ {0.5 @ 0.5}
SBEDieMorph»face2
    ^ {0.25 @ 0.25 . 0.75 @ 0.75}
SBEDieMorph»face3
    ^ {0.25 @ 0.25 . 0.75 @ 0.75 . 0.5 @ 0.5}
SBEDieMorph»face4
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75}
SBEDieMorph»face5
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75 . 0.5 @ 0.5}
SBEDieMorph»face6
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75 . 0.25 @ 0.5 . 0.75 @
       0.5}
SBEDieMorph»face7
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75 . 0.25 @ 0.5 . 0.75 @
       0.5 . 0.5 @ 0.5}
SBEDieMorph »face8
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75 . 0.25 @ 0.5 . 0.75 @
       0.5 . 0.5 @ 0.5 . 0.5 @ 0.25}
SBEDieMorph »face9
    ^ {0.25 @ 0.25 . 0.75 @ 0.25 . 0.75 @ 0.75 . 0.25 @ 0.75 . 0.25 @ 0.5 . 0.75 @
       0.5 . 0.5 @ 0.5 . 0.5 @ 0.25 . 0.5 @ 0.75}
```

These methods define collections of the coordinates of dots for each face. The coordinates are in a square of size 1×1; we will simply need to scale them to place the actual dots.

The drawOn: method does two things: It draws the die background with the **super**-send, and then draws the dots.

Method 12.33: *Drawing the die morph.*

```
SBEDieMorph»drawOn: aCanvas

    super drawOn: aCanvas.
    (self perform: ('face' , self dieValue) asSymbol) do: [:aPoint |
        self drawDotOn: aCanvas at: aPoint].
```

The second part of this method uses the reflective capacities of Smalltalk. Drawing the dots of a face is a simple matter of iterating over the collection given by the faceX method for that face, sending the drawDotOn:at: message for each coordinate. To call the correct faceX method, we use the perform: method which sends a message built from a string, here ('face', dieValue asString) asSymbol.

Method 12.34: *Drawing a single dot on a face.*

```
SBEDieMorph»drawDotOn: aCanvas at: aPoint

    aCanvas
        fillOval: (Rectangle
            center: self position + (self extent * aPoint)
            extent: self extent / 6)
        color: Color black.
```

Since the coordinates are normalized to the $[0{:}1]$ interval, we scale them to the dimensions of our die: **self** extent * aPoint.



We can already create a die instance from a workspace:

```
(SBEDieMorph faces: 6) openInWorld.
```

To change the displayed face, we create an accessor that we can use as myDie dieValue: 4:

Method 12.35: *Setting the current value of the die.*

```
SBEDieMorph»dieValue: aNumber

    self assert: (aNumber isInteger and: [aNumber between: 1 and: faces]).
    dieValue := aNumber.
    self changed.
```

Now we will use the animation system to show quickly all the faces:

Methods 12.36: *Animating the die.*

```
SBEDieMorph»stepTime

    ^ 100


SBEDieMorph»step

    isStopped ifFalse: [self dieValue: (1 to: self faces) atRandom].
```

Now the die is rolling!

To start or stop the animation by clicking, we will use what we learned previously about mouse events. First, activate the reception of mouse events:

Methods 12.37: *Handling mouse clicks to start and stop the animation.*

```
SBEDieMorph»handlesMouseDown: anEvent

    ^ true


SBEDieMorph»mouseDown: anEvent

    anEvent redButtonPressed ifTrue: [
        isStopped := isStopped not].
```

Now the die will roll or stop rolling when we click on it.

12.9 More about the canvas

The drawOn: method has an instance of Canvas as its sole argument; the canvas is the area on which the morph draws itself. By using the graphics methods of the canvas you are free to give the appearance you want to a morph. If you browse the inheritance hierarchy of the Canvas class, you will see that it has several variants. The default variant of Canvas is FormCanvas; you will find the key graphics methods in Canvas and FormCanvas. These methods can draw points, lines, polygons, rectangles, ellipses, text, and images with rotation and scaling.

It is also possible to use other kinds of canvas, to obtain transparent morphs, more graphics methods, antialiasing, and so on. To use these features you will need an AlphaBlendingCanvas or a BalloonCanvas. But how

can you obtain such a canvas in a drawOn: method, when drawOn: receives an instance of FormCanvas as its argument? Fortunately, you can transform one kind of canvas into another.

To use a canvas with a 0.5 alpha-transparency in SBEDieMorph, *redefine* drawOn: *like this:*

Method 12.38: *Drawing a translucent die.*

```
SBEDieMorph»drawOn: aCanvas

    | theCanvas |
    theCanvas := aCanvas asAlphaBlendingCanvas: 0.5.
    super drawOn: theCanvas.
    (self perform: ('face' , dieValue asString) asSymbol) do: [:aPoint |
        self drawDotOn: theCanvas at: aPoint].
```

That's all you need to do!

If you're curious, have a look at the asAlphaBlendingCanvas: method. You can also get antialiasing by using BalloonCanvas and transforming the die drawing methods as shown in methods 12.39.

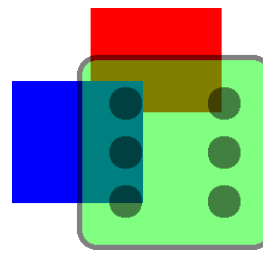


Figure 12.9: The die displayed with alpha-transparency.

Methods 12.39: *Drawing an antialiased die.*

```
SBEDieMorph»drawOn: aCanvas

    | theCanvas |
    theCanvas := aCanvas asBalloonCanvas aaLevel: 3.
    super drawOn: aCanvas.
    (self perform: ('face' , dieValue asString) asSymbol) do: [:aPoint |
        self drawDotOn: theCanvas at: aPoint].


SBEDieMorph»drawDotOn: aCanvas at: aPoint

    aCanvas
```

```
13      drawOval: (Rectangle
14          center: self position + (self extent * aPoint)
15          extent: self extent / 6)
16      color: Color black
17      borderWidth: 0
18      borderColor: Color transparent.
```

12.10 Chapter summary

Morphic is a graphical framework in which graphical interface elements can be dynamically composed.

- You can convert an object into a morph and display that morph on the screen by sending it the messages asMorph openInWorld.
- You can manipulate a morph by blue-clicking on it and using the handles that appear. (Handles have help balloons that explain what they do.)
- You can compose morphs by sending the message addMorph: or by using the embed into feature, either through the menu or via drag and drop.
- You can subclass an existing morph class and redefine key methods, like initialize and drawOn:.
- You can control how a morph reacts to mouse and keyboard events by redefining the methods handlesMouseDown:, handlesMouseOver:, etc.
- You can animate a morph by defining the methods step (what to do) and stepTime (the number of milliseconds between steps).
- Various pre-defined morphs, like PopUpMenu and FillInTheBlank, are available for interacting with users.
- Canvas objects provide basic drawing operations, advanced features such as antialiasing support are provided by Canvas subclasses.

Part III

Advanced Squeak

Chapter 13

Classes and metaclasses

As we saw in Chapter 5, in Smalltalk, everything is an object, and every object is an instance of a class. Classes are no exception: Classes are objects, and class objects are instances of other classes. This object model captures the essence of object-oriented programming: It is lean, simple, elegant, and uniform. However, the implications of this uniformity may confuse newcomers. The goal of this chapter is to show that there is nothing complex, "magic" or special here: just simple rules applied uniformly. By following these rules you can always understand why the situation is the way that it is.

13.1 Rules for classes and metaclasses

The Smalltalk object model is based on a limited number of concepts applied uniformly. Smalltalk's designers applied Occam's razor: Any consideration leading to a model more complex than necessary was discarded.

To refresh your memory, here are the rules of the object model that we explored in Chapter 5.

Rule 1. Everything is an object.

Rule 2. Every object is an instance of a class.

Rule 3. Every class has a superclass.

Rule 4. Everything happens by message sends.

Rule 5. Method lookup follows the inheritance chain.

As we mentioned in the introduction to this chapter, a consequence of Rule 1 is that *classes are objects too*, so Rule 2 tells us that classes must also be instances of classes. The class of a class is called a *metaclass*. A metaclass is created automatically for you whenever you create a class. Most of the time you do not need to care or think about metaclasses. However, every time that you use the system browser to browse the "class side" of a class, it is helpful to recall that you are actually browsing a different class. A class and its metaclass are two separate classes, even though the former is an instance of the latter.

To properly explain classes and metaclasses, we need to extend the rules from Chapter 5 with the following additional rules.

Rule 6. Every class is an instance of a metaclass.

Rule 7. The metaclass hierarchy parallels the class hierarchy.

Rule 8. Every metaclass inherits from Class and therefore from Behavior.

Rule 9. Every metaclass is an instance of Metaclass.

Rule 10. The metaclass of Metaclass is an instance of Metaclass.

Together, these 10 rules complete Smalltalk's object model.

We will first briefly revisit the 5 rules from Chapter 5 with a small example. Then we will take a closer look at the new rules, using the same example.

13.2 Revisiting the Smalltalk object model

Since everything is an object, the color blue in Smalltalk is also an object.

```
Color blue  ⟶  Color blue
```

Every object is an instance of a class. The class of the color blue is the class Color:

```
Color blue class  ⟶  Color
```

Interestingly, if we set the *alpha* value of a color, we get an instance of a different class, namely TranslucentColor:

```
(Color blue alpha: 0.4) class  ⟶  TranslucentColor
```

We can create a morph and set its color to this translucent color:

```
EllipseMorph new color: (Color blue alpha: 0.4); openInWorld
```

You can see the effect in Figure 13.1.

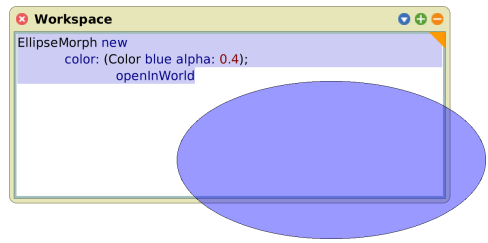


Figure 13.1: A translucent ellipse.

By Rule 3, every class has a superclass. The superclass of TranslucentColor is Color, and the superclass of Color is Object:

```
TranslucentColor superclass  ⟶  Color
Color superclass             ⟶  Object
```

Everything happens by message sends (Rule 4), so we can deduce that blue is a message to Color, class and alpha: are messages to the color blue, openInWorld is a message to an ellipse morph, and superclass is a message to TranslucentColor and Color. The receiver in each case is an object since everything is an object, but some of these objects are also classes.

Method lookup follows the inheritance chain (Rule 5), so when we send the message class to the result of Color blue alpha: 0.4, the message is handled when the corresponding method is found in the class Object, as shown in Figure 13.2.

The figure captures the essence of the *is-a* relationship. Our translucent blue object *is a* TranslucentColor instance, but we can also say that it *is a* Color and that it *is an* Object since it responds to the messages defined in all of these classes. In fact, there is a message, isKindOf:, that you can send to any object to find out if it is in an *is a* relationship with a given class:

```
translucentBlue := Color blue alpha: 0.4.
translucentBlue isKindOf: TranslucentColor  ⟶  true
```

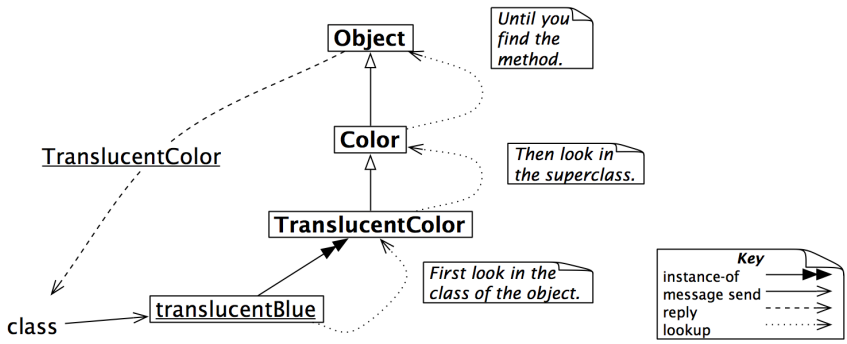


Figure 13.2: Sending a message to a translucent color.

```
translucentBlue isKindOf: Color      ⟶  true
translucentBlue isKindOf: Object     ⟶  true
translucentBlue isKindOf: Collection ⟶  false
```

13.3 Every class is an instance of a metaclass

As we mentioned in Section 13.1, classes whose instances are themselves classes are called metaclasses.

Metaclasses are implicit. Metaclasses are automatically created when you define a class. We say that they are *implicit* since as a programmer you never have to worry about them. An implicit metaclass is created for each class you create, so each metaclass has only a single instance.

Whereas ordinary classes are named by global variables, metaclasses are anonymous. However, we can always refer to them through the class that is their instance. The class of Color, for instance, is Color class, and the class of Object is Object class:

```
Color class  ⟶  Color class
Object class ⟶  Object class
```

Figure 13.3 shows how each class is an instance of its (anonymous) metaclass.

The fact that classes are also objects makes it easy for us to query them by sending messages. Let's have a look:

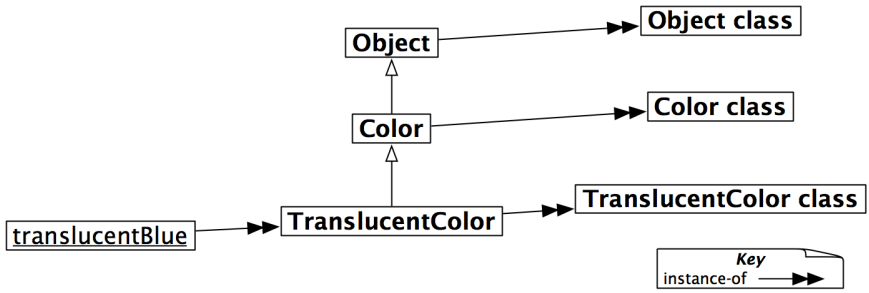


Figure 13.3: The metaclasses of Translucent and its superclasses.

```
Color subclasses                    ⟶  {TranslucentColor}
TranslucentColor subclasses         ⟶  #()
TranslucentColor allSuperclasses    ⟶  an OrderedCollection(Color Object
    ProtoObject)
TranslucentColor instVarNames       ⟶  #('alpha')
TranslucentColor allInstVarNames    ⟶  #('rgb' 'cachedDepth' 'cachedBitPattern' '
    alpha')
TranslucentColor selectors sorted   ⟶  #(#addName: #alpha #alpha:
    #asHTMLColor #asNontranslucentColor #balancedPatternForDepth:
    #bitPatternForDepth: #convertToCurrentVersion:refStream: #hash #isOpaque
    #isTranslucent #isTranslucentColor #isTransparent #name #pixelValueForDepth:
    #pixelWordForDepth: #printOn: #privateAlpha #scaledPixelValue32 #setAlpha:
    #setRgb:alpha: #storeArrayValuesOn: #storeOn:)
```

13.4 The metaclass hierarchy parallels the class hierarchy

Rule 7 says that the superclass of a metaclass cannot be an arbitrary class: It is constrained to be the metaclass of the superclass of the metaclass's unique instance.

```
TranslucentColor class superclass   ⟶  Color class
TranslucentColor superclass class   ⟶  Color class
```

This is what we mean by the metaclass hierarchy being parallel to the class hierarchy; Figure 13.4 shows how this works in the TranslucentColor hierarchy.

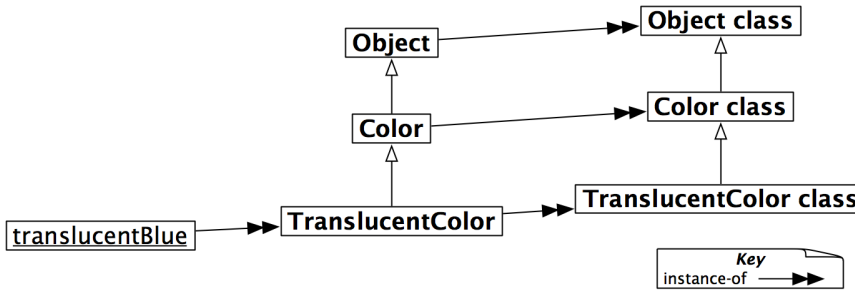


Figure 13.4: The metaclass hierarchy parallels the class hierarchy.

```
TranslucentColor class                          ⟶  TranslucentColor class
TranslucentColor class superclass               ⟶  Color class
TranslucentColor class superclass superclass    ⟶  Object class
```

Uniformity between classes and objects. It is interesting to step back a moment and realize that there is no difference between sending a message to an object and a class. In both cases, the search for the corresponding method starts in the class of the receiver and proceeds up the inheritance chain.

Thus, messages sent to classes must follow the metaclass inheritance chain. Consider, for example, the method blue, which is implemented on the class side of Color. If we send the message blue to TranslucentColor, then it will be looked-up the same way as any other message. The lookup starts in the class of the object named TranslucentColor, which is TranslucentColor class, and proceeds up the metaclass hierarchy until it is found in Color class (see Figure 13.5).

```
TranslucentColor blue   ⟶  Color blue
```

Note that we get as a result an ordinary Color blue and not a translucent one — there is no magic!

Thus we see that there is one uniform kind of method lookup in Smalltalk. Classes are just objects, and behave like any other objects. Classes have the power to create new instances only because classes happen to respond to the message new, and because the method for new knows how to create new instances. Normally, non-class objects do not understand this message, but

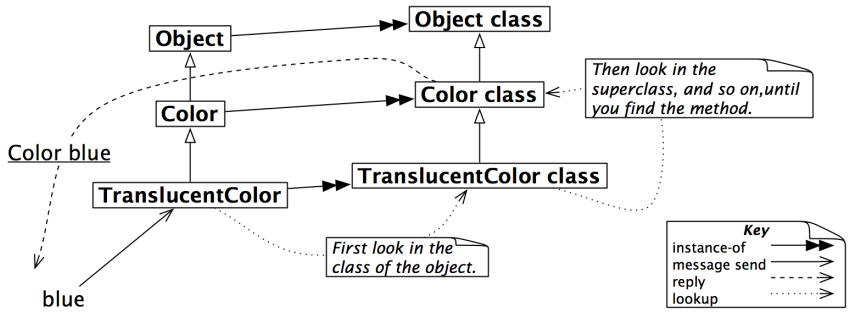


Figure 13.5: Message lookup for classes is the same as for ordinary objects.

if you have a good reason to do so, nothing is stopping you from adding a new method to a non-metaclass.

Since classes are objects, we can also inspect them.

Inspect Color blue *and* Color.

Notice that in one case you are inspecting an instance of Color and in the other case the Color class itself. This can be a bit confusing because the title bar of the inspector names the *class* of the object being inspected.

The inspector on Color allows you to see the superclass, instance variables, method dictionary, and so on, of the Color class, as shown in Figure 13.6.

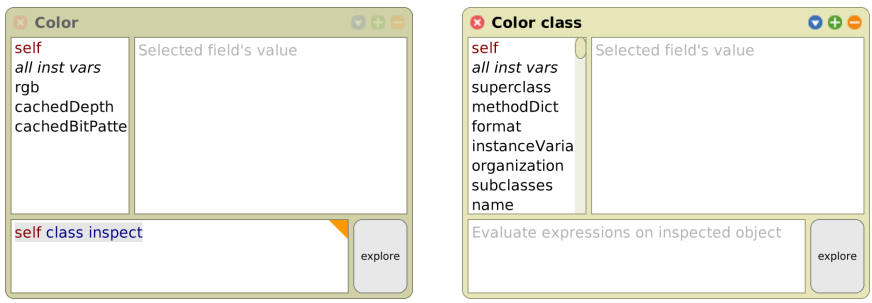


Figure 13.6: Classes are objects too.

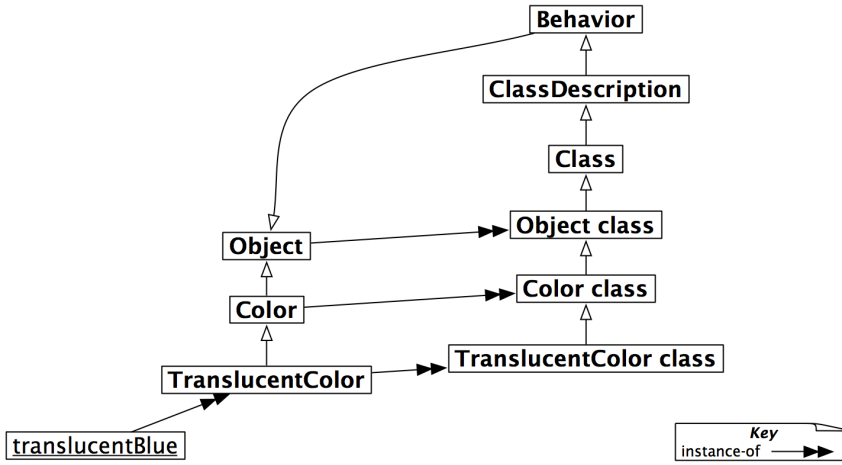


Figure 13.7: Metaclasses inherit from Class and Behavior.

13.5 Every metaclass inherits from Class and Behavior

Every metaclass *is-a* class, hence inherits from Class. Class in turn inherits from its superclasses, ClassDescription and Behavior. Since everything in Smalltalk *is-an* object, these classes all inherit eventually from Object. We can see the complete picture in Figure 13.7.

Where is new defined? To understand the importance of the fact that metaclasses inherit from Class and Behavior, it helps to ask where new is defined and how it is found. When the message new is sent to a class it is looked up in its metaclass chain and ultimately in its superclasses Class, ClassDescription, and Behavior as shown in Figure 13.8.

The question *"Where* new *is defined?"* is crucial. new is first defined in the class Behavior, and it can be redefined in its subclasses, including any of the metaclasses of the classes we define, when this is necessary. Now when a message new is sent to a class it is looked up, as usual, in the metaclass of this class, continuing up the superclass chain right up to the class Behavior, if it has not been redefined along the way.

Note that the result of sending TranslucentColor new is an instance of TranslucentColor and *not* of Behavior, even though the method is looked-up

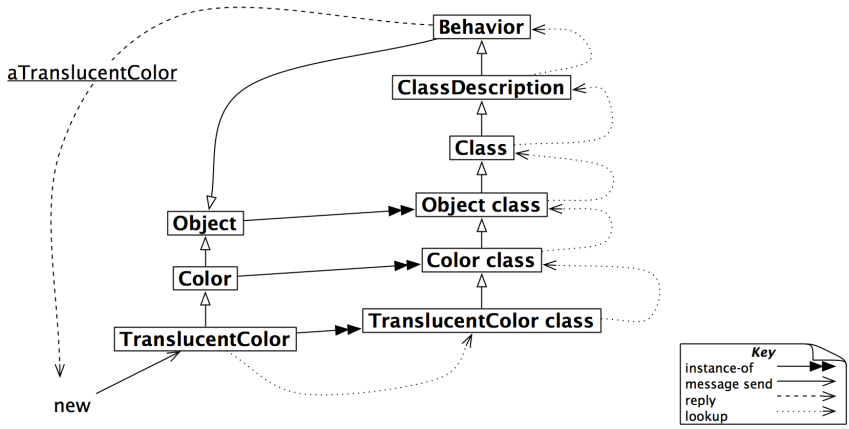


Figure 13.8: new is an ordinary message looked up in the metaclass chain.

in the class Behavior! new always returns an instance of self, the class that receives the message, even if it is implemented in another class.

```
TranslucentColor new class  ⟶  TranslucentColor  "not Behavior"
```

A common mistake is to look for new in the superclass of the receiving class. The same holds for new:, the standard message to create an object of a given size. For example, Array new: 4 creates an array of 4 elements. You will not find this method defined in Array or any of its superclasses. Instead, you should look in Array class and its superclasses since that is where the lookup will start.

Responsibilities of Behavior, ClassDescription, and Class. Behavior provides the minimum state necessary for objects that have instances: this includes a superclass link, a method dictionary, and a description of the instances (*i.e.*, representation and number). Behavior inherits from Object, so it, and all of its subclasses, can behave like objects.

Behavior is also the basic interface to the compiler. It provides methods for creating a method dictionary, compiling methods, creating instances (*i.e.*, new, basicNew, new:, and basicNew:), manipulating the class hierarchy (*i.e.*, superclass:, addSubclass:), accessing methods (*i.e.*, selectors, allSelectors, compiledMethodAt:), accessing instances and variables (*i.e.*, allInstances, instVarNames ...), accessing the class hierarchy (*i.e.*, superclass, subclasses),

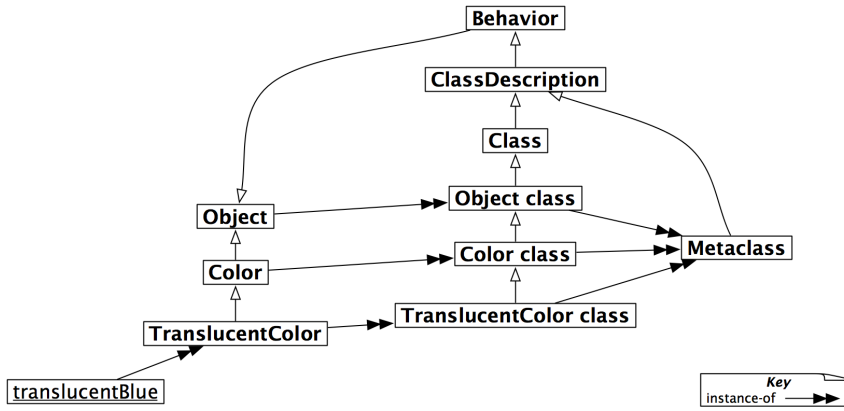


Figure 13.9: Every metaclass is a Metaclass.

and querying (*i.e.*, hasMethods, includesSelector, canUnderstand:, inheritsFrom:, isVariable).

ClassDescription is an abstract class that provides facilities needed by its two direct subclasses, Class and Metaclass. ClassDescription adds a number of facilities to the basis provided by Behavior: named instance variables, the categorization of methods into protocols, the notion of a name (abstract), the maintenance of change sets and the logging of changes, and most of the mechanisms needed for filing-out changes.

Class represents the common behavior of all classes. It provides a class name, compilation methods, method storage, and instance variables. It provides a concrete representation for class variable names and shared pool variables (addClassVarName:, addSharedPool:, initialize). Class knows how to create instances, so all metaclasses should inherit ultimately from Class.

13.6 Every metaclass is an instance of Metaclass

Metaclasses are objects too; they are instances of the class Metaclass as shown in Figure 13.9. The instances of class Metaclass are the anonymous metaclasses, each of which has exactly one instance, which is a class.

Metaclass represents common metaclass behavior. It provides methods for instance creation (subclassOf:) creating initialized instances of the metaclass's sole instance, initialization of class variables, metaclass instance,

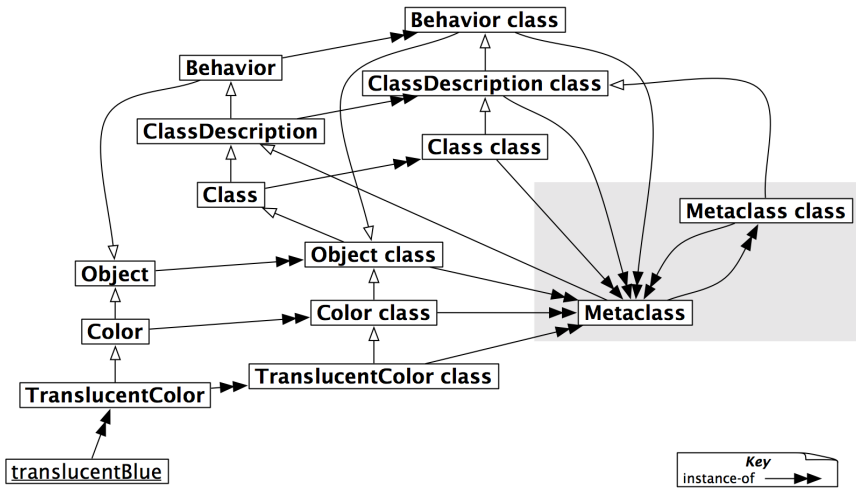


Figure 13.10: All metaclasses are instances of the class Metaclass, even the metaclass of Metaclass.

method compilation, and class information (inheritance links, instance variables, etc.).

13.7 The metaclass of Metaclass is an instance of Metaclass

The final question to be answered is: What is the class of Metaclass class?

The answer is simple: It is a metaclass, so it must be an instance of Metaclass, just like all the other metaclasses in the system (see Figure 13.10).

The figure shows how all metaclasses are instances of Metaclass, including the metaclass of Metaclass itself. If you compare Figures Figure 13.9 and Figure 13.10 you will see how the metaclass hierarchy perfectly mirrors the class hierarchy, all the way up to Object class.

The following examples show us how we can query the class hierarchy to demonstrate that Figure 13.10 is correct. (Actually, you will see that we told a white lie — Object class superclass $\longrightarrow$ ProtoObject class, not Class. In Squeak, we must go one superclass higher to reach Class.)

Example 13.1: *The class hierarchy.*

```
TranslucentColor superclass  ⟶  Color
Color superclass             ⟶  Object
```

Example 13.2: *The parallel metaclass hierarchy.*

```
TranslucentColor class superclass          ⟶  Color class
Color class superclass                     ⟶  Object class
Object class superclass superclass         ⟶  Class    "NB: skip ProtoObject class"
Class superclass                           ⟶  ClassDescription
ClassDescription superclass                ⟶  Behavior
Behavior superclass                        ⟶  Object
```

Example 13.3: *Instances of Metaclass.*

```
TranslucentColor class class  ⟶  Metaclass
Color class class             ⟶  Metaclass
Object class class            ⟶  Metaclass
Behavior class class          ⟶  Metaclass
```

Example 13.4: *Metaclass class is a Metaclass.*

```
Metaclass class class  ⟶  Metaclass
Metaclass superclass   ⟶  ClassDescription
```

13.8 Chapter summary

Now you should understand better how classes are organized and the impact of a uniform object model. If you get lost or confused, you should always remember that message passing is the key: You look for the method in the class of the receiver. This works on *any* receiver. If the method is not found in the class of the receiver, it is looked up in its superclasses.

- Every class is an instance of a metaclass. Metaclasses are implicit. A Metaclass is created automatically when you create the class that is its sole instance.
- The metaclass hierarchy parallels the class hierarchy. Method lookup for classes parallels method lookup for ordinary objects, and follows the metaclass's superclass chain.
- Every metaclass inherits from Class and Behavior. Every class *is a* Class. Since metaclasses are classes too, they must also inherit from Class. Behavior provides behavior common to all entities that have instances.

- Every metaclass is an instance of Metaclass. ClassDescription provides everything that is common to Class and Metaclass.
- The metaclass of Metaclass is an instance of Metaclass. The *instance-of* relation forms a closed loop, so Metaclass class class $\longrightarrow$ Metaclass.

Part IV

Appendices

Appendix A

Frequently asked questions

A.1 Getting started

FAQ 1 *Where do I get the latest Squeak?*

Answer https://squeak.org/downloads/

FAQ 2 *Which Squeak image should I use with this book?*

Answer We recommend that you use the standard image provided on https://squeak.org/downloads/. You should also be able to use most other images, but you may find that the hands-on exercises behave differently in surprising ways.

FAQ 3 *I have destroyed my world docking bar at the top of the world in some way. How do I get it back?*

Answer Open the yellow button menu of the world (by clicking the yellow button in the world). There you find the menu item show main docking bar. Click it once to deactivate it completely, and then click it a second time to re-activate it.

FAQ 4 *How do I install more than what is already in the image?*

Answer There are several ways to install additional packages. First, you can always download sar- or mcz-files and simply drop them into Squeak. Second, you can use the class Installer, which provides methods for installing several kinds of packages from different sources, *e.g.*, Squeaksource. To start using the Installer you can take a look at the class comment. Finally, you can currently find an increasing number of Squeak projects on GitHub, many of them to be installed via Metacello. Metacello is a configuration management project for specifying dependencies between Smalltalk projects which installs itself on first usage.

FAQ 5 *Some code triggered a lot of debuggers and my whole screen is filled with them. How can I get rid of them?*

Answer You can close all windows that look similar at once by selecting World docking bar ▷ Windows ▷ Bad Window ▷ Close all like this.

A.2 Collections

FAQ 6 *How do I sort a* Collection*?*

Answer If you want a sorted copy, you can use sorted. If you want to have a collection that keeps the elements sorted, even when you add or remove elements, send the message asSortedCollection. Finally, some collections can be sorted in-place as they understand sort, which will not return a copy, but re-order the collection.

```
collection := {7 . 2 . 6 . 1}.
collection sorted.    ⟶    #(1 2 6 7)
collection sorted == collection.    ⟶    false

collection asSortedCollection.    ⟶    a SortedCollection(1 2 6 7)

collection sort.    ⟶    #(1 2 6 7)
collection.    ⟶    #(1 2 6 7)
collection sort == collection.    ⟶    true
```

To define custom sort criteria, use a sort block or a function (see Section 10.6):

```
collection := #('Foo' 'Bar' 'baz').
collection sorted.    ⟶    #('Bar' 'Foo' 'baz')
```

```
collection sorted: [:a :b | a caseInsensitiveLessOrEqual: b].  ⟶  #('Bar' 'baz' 'Foo'
    )
collection sorted: [:a | a asUppercase] ascending.  ⟶  #('Bar' 'baz' 'Foo')
collection sorted: [:a | a asUppercase] descending.  ⟶  #('Foo' 'baz' 'Bar')

(#(1 0 2) @ #(2 0 1)) sorted: #r ascending , #x descending.  ⟶  {0@0 . 2@1 . 1
    @2}
```

FAQ 7 *How do I convert a collection of characters to a* String*?*

Answer

```
collection := 'hello' asSet.
String newFrom: collection asArray.  ⟶  'oelh'
```

A.3 Browsing the system

FAQ 8 *How do I search for a class?*

Answer CMD−b (browse) on the class name, CMD−f in the category pane of the class browser, or the global search on the top right of the world.

FAQ 9 *How do I browse all references to a given class?*

Answer Select the class name and press CMD−N, or open the yellow button menu and select more ▷ references to it.

FAQ 10 *How do I find/browse all sends to super?*

Answer The second solution is much faster:

```
self systemNavigation browseMethodsWithSourceString: 'super'.
self systemNavigation browseAllSelect: [:method | method sendsToSuper ].
```

FAQ 11 *How do I browse all super sends within a hierarchy?*

Answer

```
browseSuperSends := [:aClass | self systemNavigation
   browseMessageList: (aClass withAllSubclasses gather: [ :each |
      (each methodDict associations
         select: [ :assoc | assoc value sendsToSuper ])
            collect: [ :assoc | MethodReference class: each selector: assoc key ] ])
   name: 'Supersends of ' , aClass name , ' and its subclasses'].
browseSuperSends value: OrderedCollection.
```

FAQ 12 *How do I find out which new methods are introduced by a class?* (i.e., *not including overridden methods.)*

Answer Here we ask which methods are introduced by TranslucentColor:

```
newMethods := [:aClass| aClass methodDict keys select:
   [:aMethod | (aClass superclass canUnderstand: aMethod) not ]].
newMethods value: TranslucentColor    ⟶    #(#setAlpha: #setRgb:alpha:)
```

FAQ 13 *How do I tell which methods of a class are abstract?*

Answer

```
abstractMethods :=
   [:aClass | aClass methodDict keys select:
      [:aMethod | (aClass>>aMethod) isAbstract ]].
abstractMethods value: Collection    ⟶    #(#remove:ifAbsent: #add: #do:)
```

FAQ 14 *How do I generate a view of the AST of an expression?*

Answer Load AST from squeaksource.com. Then evaluate:

```
(RBParser parseExpression: '3 + 4') explore
```

Alternatively you can also directly *explore it*.

FAQ 15 *How do I find all the Traits in the system?*

Answer

```
Smalltalk allTraits
```

FAQ 16 *How do I find which classes use traits?*

Answer

```
Smalltalk allClasses select: [:each | each hasTraitComposition]
```

A.4 Morphic

FAQ 17 *How can I show images in Morphic?*

Answer A first and easy way to get an image into Squeak is by dragging the image file into the environment and dropping it directly within the world and selecting open graphic in a window. This will create an ImageMorph in the world.

Alternatively, you can also create an ImageMorph through code, for example using the following expression, if the image file is in the default folder:

```
ImageMorph new
   image: (Form fromFileNamed: 'anImage.jpg');
   openInHand.
```

FAQ 18 *How do I center text in a* TextMorph*?*

Answer The following expression will do this:

```
TextMorph new
   contents: 'Hello';
   hResizing: #rigid; "States that the morph should not change
                  its width to match the width of the content"
   width: 300;
   centered;
   openInWorld.
```

FAQ 19 *How do I add formatting to text?*

Answer The relevant classes are Text and TextAttribute. You can add TextAttributes to section of text by using the messages in the emphasis protocol of Text, for example:

```
text := 'yellow and green' asText.
text
   addAttribute: TextColor yellow from: 1 to: 6;
   addAttribute: TextColor green from: 12 to: 16;
   addAllAttributes: {TextEmphasis italic. TextEmphasis bold}.
text asMorph openInHand.
```

A.5 System

FAQ 20 *How can I achieve concurrency?*

Answer There are a number of classes to create and control concurrency. You can find most of them, such as Process, Mutex, or Promise in the system category Kernel-Processes. Starting new processes is done via the messages in the scheduling protocol of BlockClosure. Note, that, like most interpreter-based languages to date, the Squeak environment runs in a single operating system process, so you will not benefit from parallel hardware. Writing concurrent code might still be useful for background processing of long computations or I/O.

FAQ 21 *I have created concurrent behavior, but lost track of some of the processes I started. What can I do?*

Answer There is a tool called Process Browser which you can either open from World menu ▷ open or World docking bar ▷ Tools. The process browser lists all processes and the yellow button menu allows you to control the process, *e.g.*, terminate it.

FAQ 22 *How do I express OS-independent file paths?*

Answer A very manual way to achieve this is to construct paths using OS-specific path delimiters by using slash. FileDirectory instances can be created with any kind of path by sending the message uri:. Finally, if you already have a FileDirectory instance you can send the message \ to navigate it, for example:

```
(FileDirectory default / 'release-notes' / '5.0') readStream contents lines first.
```

A.6 Using Monticello and SqueakSource

FAQ 23 *How do I load a Squeaksource project?*

Answer

1. Find the project you want in squeaksource.com
2. Copy the registration code snippet
3. Select World docking bar ▷ Tools ▷ Monticello browser
4. Select +Repository ▷ HTTP
5. Paste and accept the Registration code snippet; enter your password
6. Select the new repository and Open it
7. Select and load the latest version

FAQ 24 *How do I create a SqueakSource project?*

Answer

1. Go to squeaksource.com
2. Register yourself as a new member
3. Register a project (name = category)
4. Copy the Registration code snippet
5. open ▷ Monticello browser
6. +Package to add the category
7. Select the package
8. +Repository ▷ HTTP
9. Paste and accept the Registration code snippet; enter your password
10. Save to save the first version

FAQ 25 *How do I extend* Number *with* Number»chf *but have Monticello recognize it as being part of my* Money *project?*

Answer Put it in a method-category named *Money. Monticello gathers all methods that are in other categories named like *package and includes them in your package.

A.7 Tools

FAQ 26 *Doing everything with the mouse take so much time. Which keyboard shortcuts are there?*

Answer You can find a list of available keyboard shortcuts directly in the environment at World docking bar ▷ Help ▷ Keyboard Shortcuts.

FAQ 27 *Some menus are quite large and I struggle to find what I am looking for.*

Answer Menus, like all panes, can be filtered by simply typing on your keyboard. If you type the complete string of the menu item you can also directly activate it by pressing return.

FAQ 28 *Some message sends seem slow to me. How do I find out what makes them slow?*

Answer You can use the MessageTally tool that allows you to profile your code. To profile all processes currently running in the process, you can open the tool from World docking bar ▷ Extras ▷ Start Profiler. If you only want to profile a specific message send, you can use spyOn:, for example:

```
MessageTally spyOn: [10000 timesRepeat: [1.23 printString]]
```

You can achieve the same by selecting an expression and select spy on it from the yellow button menu.

FAQ 29 *How do I programmatically trigger any UI logic in Morphic?*

Answer Imagine you want to write a method that opens a test runner, reverts an earlier version of a method, or saves a new Monticello version. For tasks like this, you need to figure out the implementation behind certain graphical tools in the UI. Luckily, Morphic provides some tools to facilitate such exploration.

To introspect on the object behind any morph (e.g., a window or a menu item), make a blue-button click on it to open a halo, click on the debug handle , and select inspect morph or explore morph. You can also choose browse morph class to jump to the class definition of the morph directly. For many windows and their contents, the underlying *model* is more interesting, so you can also select inspect model or explore model. In the case of a test runner, the model is a TestRunner instance, which you can use to programmatically

open another test runner window. Open a halo on the existing window, select browse model class from the debug menu, and browse the class side of TestRunner to find the message open.

For some widgets such as buttons, you can also observe its specific *action code*. For instance, to find out what happens when you click on the Save button in the Monticello browser, open a halo on the button and select browse action code from the debug menu. You can also start a debugger on the action code by selecting debug action code, which allows you to follow the button's implementation in context and step-by-step.

For more complex UI automation tasks and UI testing purposes, you can also use a UI automation framework such as the MorphicTestingFramework (load it from https://github.com/hpi-swa-teaching/Morphic-Testing-Framework).

FAQ 30 *Where can I find the Refactoring Browser?*

Answer From the docking bar, select Apps ▷Preference Wizard, then click on Configure, Done, tick "Refactoring support in code browsers", and confirm via Yes, install selected packages.

FAQ 31 *How do I register the browser that I want to be the default?*

Answer Click the menu icon in the top left of the Browser window.

A.8 Additional packages

FAQ 32 *Where is the documentation for the Regex package?*

Answer Look at the DOCUMENTATION protocol of RxParser class in the Regex-Core category.

FAQ 33 *How can I make web requests or start a web server from Squeak?*

Answer The most convenient way for handling web connections is to use the WebClient packages. You can use the class WebClient to make HTTP/HTTPS requests and the class WebServer to start a web server. Comprehensive documentation is available in the tutorials in the package WebClient-Help. To read these tutorials, go to the docking bar, select Help ▷ Help Browser, and choose WebClient or WebServer from the pane on the left.

FAQ 34 *Are there tools for writing parsers?*

Answer You can use SmaCC — the Smalltalk Compiler Compiler (load it from https://www.squeaksource.com/SmaccDevelopment.html). Or you can use Ohm/S, a Smalltalk implementation of Ohm/JS (load it from https://www.github.com/hpi-swa/Ohm-S).

Bibliography

Sherman R. Alpert, Kyle Brown and Bobby Woolf: The Design Patterns Smalltalk Companion. Addison Wesley, 1998, ISBN 0–201–18462–1

Kent Beck: Smalltalk Best Practice Patterns. Prentice-Hall, 1997

Kent Beck: Test Driven Development: By Example. Addison-Wesley, 2003, ISBN 0321146530

Erich Gamma et al.: Design Patterns: Elements of Reusable Object-Oriented Software. Reading, Mass.: Addison Wesley Professional, 1995, ISBN 978–0201633610

Adele Goldberg and David Robson: Smalltalk 80: the Language and its Implementation. Reading, Mass.: Addison Wesley, May 1983 ⟨URL: https://dl.acm.org/doi/pdf/10.5555/273⟩, ISBN 0–201–13688–0

Edward J. Klimas, Suzanne Skublics and David A. Thomas: Smalltalk with Style. Prentice-Hall, 1996, ISBN 0–13–165549–3

Wilf LaLonde and John Pugh: Inside Smalltalk: Volume 1. Prentice Hall, 1990, ISBN 0–13–468414–1

Alec Sharp: Smalltalk by Example. McGraw-Hill, 1997

Bobby Woolf: Null Object. In **Robert Martin, Dirk Riehle and Frank Buschmann, editors:** Pattern Languages of Program Design 3. Addison Wesley, 1998, 5–18

Index

" ", *see* comment
| |, *see* temporary variable declaration
*, *see* package, dirty
,, *see* Collection, ,
:=, *see* assignment
;, *see* cascade
< >, *see* < >
←, *see* assignment
[ ], *see* block
#( ), *see* Array, literal
#[ ], *see* ByteArray, literal
#, *see* literal symbol
do restore display, 82
_, *see* assignment
., *see* statement separator
==, *see* Object, identity
=, *see* Object, equality
>>, *see* Behavior, >>
$, *see* literal, character
' ', *see* literal, string
{ }, *see* Array, brace construct
↑, *see* return
^, *see* return

abstract class, *see* class, abstract
abstract method, *see* method, abstract
accept it, *see* keyboard shortcuts, accept
accessing (protocol), 38, 80, 203
accessing untypeable characters (protocol), 192
accessor, 38, 79
adding (protocol), 203
agile software development, 159
all (protocol), 29, 35, 108, 117
AlphaBlendingCanvas
    (class), 263
Array
    (class), 204, 205, 207, 208
    at:, 208, 209
    at:put:, 208, 209
    brace construct, 209
    copy, 209
    literal, 50, 179, 208, 209
Array class
    new:, 208
    with:, 208
Array, brace construct, 49
ArrayCollection
    sort:, 224
ArrayedCollection
    (class), 205
    sort, 223
as yet unclassified (protocol), 35
AssertionFailure
    (class), 185
assignment, 50, 90
association, *see* Object, ->
AST, 288
at:, *see* Collection, at:
at:put:, *see* Collection, at:put:
attribute, *see* instance variable

Bag
    (class), 204–206, 213
BalloonCanvas
    (class), 263, 264
Beck, Kent, 93, 159
Behavior
    >>, 21, 36
    (class), 270, 276, 277

addSubclass:, 277
allInstances, 277
allInstVarNames, 273
allSelectors, 277
allSuperclasses, 273
basicNew, 277
basicNew:, 277
canUnderstand:, 278
compiledMethodAt:, 277
hasMethods, 278
includesSelector, 278
inheritsFrom:, 278
instVarNames, 273, 277
isVariable, 278
new, 57, 187, 276, 277
new:, 57, 277
selectors, 273, 277
subclasses, 277
superclass, 271, 277
superclass:, 277
Behavior Driven Development, *see* Test Driven Development
benchmarks, 137
binary message, *see* message, binary
Bitmap
(class), 204
block, 47, 50, 54, 72, 166, 185
BlockClosure
(class), 55, 290
ascending, 225
bench, 137
benchFor:, 137
descending, 225
ensure:, 197
on:do:, 197
on:do:on:do:, 197
timeProfile, 137
timeToRun, 137
value, 54
value:, 54, 219
value:value:, 54
valueSupplyingAnswers:, 199
valueSuppressingMessages:, 199
valueWithArguments:, 55
whileFalse:, 56
whileTrue:, 56
Blue Book, 204
blue button, 5, 6, 243
Boolean
&, 196
(class), 16, 47, 51, 183, 186, 194
and:, 196
ifFalse:, 56, 195
ifFalse:ifTrue, 195
ifTrue:, 56, 195
ifTrue:ifFalse:, 56, 194
BorderedMorph
(class), 32, 259
fullPrintOn:, 95
breakpoint, *see* Object, halt
Browser
(class), 178
browser, *see* system browser, 194
browsing programmatically, 118, 287
ByteArray
(class), 50, 239
literal, 50, 209
ByteString
(class), 125, 192, 214

C++, 79, 82, 84
camelCase, 47
Canvas
(class), 247, 263
asAlphaBlendingCanvas:, 264
draw:, 112, 113
caret, *see* return
cascade, 51, 53, 73, 222
Categorizer
(class), 119
category
creating, 26, 109
filing in, *see* file, filing in
filing out, *see* file, filing out
message, *see* mthod16
method, 16
system, 15
change set, *see* file, filing out
change set browser, 151
changes, 3, 4, 8, 156
Character
(class), 19, 49, 88, 183, 188, 191, 214
asString, 192
asUppercase, 19

isAlphaNumeric, 192
isCharacter, 192
isDigit, 192
isLowercase, 192
isVowel, 192
printOn:, 192
printString, 192
Character class
backspace, 192
cr, 192
escape, 192
euro, 192
space, 49, 192
tab, 49, 101, 192
value:, 191
CharacterArray
(class), 205
Class
(class), 270, 276, 278
addClassVarName:, 278
addSharedPool:, 278
initialize, 278
subclasses, 273
class
abstract, 87, 186, 194
comment, 16, 28
creation, 27, 110
filing in, *see* file, filing in
filing out, *see* file, filing out
finding, 16, 117
initialization, 101
instance variable, 80, 82
invariant, 187
method, 80, 82, 85
recent, 117
variable, 48, 97, 99
class browser, *see* system browser
class category, *see* system category
class side, *see* system browser, class side
ClassDescription
(class), 276, 278
ClassListBrowser class
browseClassesSatisfying:title:, 119
ClassOrganizer
(class), 119
closure, *see* block
Collection
,, 214, 233
(class), 204, 211, 286
add:, 206
allSatisfy:, 222
anySatisfy:, 222
asOrderedCollection, 210
asSet, 212
asSortedCollection, 213, 286
at:, 206
at:put:, 206, 210
collect:, 56, 204, 207, 218
collect:as:, 219
common errors, 222
concatenation, *see* Collection, ,
count:, 221
detect:, 56, 220
detect:ifFound:, 220
detect:ifNone:, 204, 220
do:, 56, 204, 207, 217, 219
do:separatedBy:, 217
do:without:, 217
fold:, 221
gather:, 222
include:, 207
includes:, 204, 213, 221
inject:into:, 56, 204, 221
intersection:, 213
isEmpty, 204, 207
iteration, 217
noneSatisfy:, 222
occurrencesOf:, 204
reduce:, 221
reject:, 56, 204, 220
remove:, 186, 206, 222
select:, 56, 204, 207, 220
size, 206
sorted, 223, 286
sorted:, 224
sorting, *see* Collection, asSortedCollection
union:, 213
weak, 206
Collection class
new:, 207
newFrom:, 207
with:, 207
with:with:, 207

withAll:, 207
Collections-SortFunctions (category), 225
Collections-Strings (category), 191, 192
Color
(class), 81, 99, 205, 270–272
alpha:, 271
name, 100
Color class
(class), 272
blue, 82, 271, 274
colorNames, 101
initialize, 101
initializeNames, 100
showColorCube, 82
ColorArray
(class), 205
ColorNames (class variable), 100
comma, *see* Collection, concatenation
comment, 50
comparing (protocol), 193
CompiledMethod
(class), 204
Complex
=, 180
(class), 188, 189
hash, 181
concurrency, 290
constant methods, 36
Context
(class), 51
control constructs, *see* iteration
converting (protocol), 203
copy, *see* Object, copy
copying (protocol), 203
creation (protocol), 86, 203
CrLf (global), 101

debugger, 21, 39, 106, 124, 185, 293
declaration, *see* temporary variable declaration
deep copy, *see* Object, deepCopy
dependency browser, 136
dependents (protocol), 108
deprecation, 185
design by contract, 185
Dictionary
(class), 98, 181, 186, 204–207
associationsDo:, 218
at:, 211
at:ifAbsent:, 211
at:put:, 211
do:, 218
keys, 211
keysDo:, 218
overriding = and hash, 211, 223
removeKey:, 186
values, 211
valuesDo:, 218
Dictionary class
newFrom:, 208
withAll:, 208
dirty package, *see* package, dirty
do:, *see* Collection, do:
download, 3, 4, 285
Duration
(class), 121, 188, 189

EllipseMorph
(class), 258
defaultColor, 92
encapsulation boundary, 79
enumerating (protocol), 203, 218
enumeration, *see* iteration
Environment
(class), 98
equality, *see* Object, equality
Error
(class), 196, 199
EventSensor
(class), 254
Exception
,, 197
(class), 196
defaultAction, 196, 198
pass, 198
resume, 198
resume:, 198
retry, 199
return, 199
return:, 199
signal, 196, 198
signal:, 196, 198
execution context, 39
explorer, 14, 122

exponent, 49
extension, *see* method, extension
extension method, *see* package, extension
extension methods, 143
eXtreme Programming, 159, 161

False
    (class), 51, 194
    ifTrue:, 195
false (pseudo variable), 47, 51
Feathers, Michael, 174
field, *see* instance variable
file
    browsing, *see* file list browser
    change set, 151
    filing in, 42, 117
    filing out, 42, 117, 141, 144, 153
file list browser, 154
FileDirectory
    slash, 290
FileStream
    (class), 238
    binary, 239
    close, 238
    localName, 238
FileStream class
    fileNamed:, 238
    fileNamed:do:, 229
    forceNewFileNamed:, 238
    forceNewFileNamed:do:, 238
    newFileNamed:, 238
    oldFileNamed:, 238
    readOnlyFileNamed:, 238
fixture, *see* SUnit, fixture
flat map, *see* Collection, gather:
Float
    (class), 49, 187, 189, 205
Float class
    e, 190
    infinity, 190
    nan, 190
    pi, 190
FloatArray
    (class), 205
floating point number, 49, 189
Form
    (class), 184
    postCopy, 184
FormCanvas
    (class), 263
Fraction
    (class), 182, 187, 190
    numerator:denominator:, 190
full stop, *see* statement separator

geometry (protocol), 246
getter method, *see* accessor
Git, 42, 142, 157
gitless, 157
global method search, 136
global variable, *see* variable, global, *see* variable, global

halt, *see* Object, halt
HandMorph
    (class), 254
HashedCollection
    rehash, 223
help browser, xiii, 293
hierarchy browser, 16, 115
hook method, 187
HTTP, 293
HTTPS, 293

identity, *see* Object, identity
IdentityDictionary
    (class), 211
IdentitySet
    (class), 213
image, 3, 4, 8
ImageMorph
    (class), 289
inheritance, 86, 91
    canceling, 186
initialization, 30, 84, 94
initialization (protocol), 35, 80
inspector, 13, 31, 79, 120, 275
Installer
    (class), 286
instance browser, 123
instance side, *see* system browser, instance side
instance variable, 31, 79, 90
instance variable definition, 34

Integer
(class), 49, 187, 190
atRandom, 190
bitAnd:, 57
bitOr:, 57
bitShift:, 57
bitXor:, 223
factorial, 189, 190
gcd:, 190
isPrime, 190
timesRepeat:, 56, 191
Interval
(class), 56, 205–207, 210
at:, 205
printOn:, 179
Interval class
from:to:, 210
from:to:by:, 210
printString, 210
is-a, 271, 276
iteration, *see also* Collection, iteration, 56

Java, 79, 82, 84, 92, 159

Kernel-Classes (category), 86
Kernel-Numbers (category), 187
Kernel-Objects (category), 15, 16, 177
keyboard events, 252
keyboard shortcuts, xiii, 12, 24, 292
accept, 20, 28
browse it, 16, 111, 287
cancel, 127
do it, 12
explore it, 14, 122, 127
find ..., 17, 287
inspect it, 120, 127
print it, 13
search bar, 136
KeyboardEvent
(class), 254
keys, *see* Dictionary, keys
keyword message, *see* message, keyword
Knight, Alan, xii

lambda expression, 201
LargeNegativeInteger
(class), 187, 190
LargePositiveInteger
(class), 187, 190
launching Squeak, 4
lexical closure, *see* block
lexicon, 136
LinkedList
(class), 204–206
Lisp, 201, 204
literal, 49
array, *see* Array, literal
character, 49
number, 49
string, 49
symbol, 49
literal objects, 47
loops, *see* iteration

Mac OS X Finder, 108
Magnitude
<, 87, 188
<=, 224
=, 188
>, 188
>=, 87
(class), 87, 186–188, 191, 193
Matrix
(class), 34, 37
Matrix class
new:tabulate:, 34
rows:columns:, 35
Mercurial, 42
message
binary, 50, 52, 61
evaluation order, 66
keyword, 51, 52, 61
not understood, 96
receiver, 62
selector, 49, 51, 62, 220
send, 62, 89
unary, 50, 52, 61
message name finder, 106
message names browser, 135
message send, 271
message tally, 137
MessageCatcher
(class), 86
MessageSend

(class), 220
value:, 220
MessageTally class
spyOn:, 292
Metacello, 286
Metaclass
(class), 270, 278
metaclass, 79–81, 270, 272, 276
anonymous, 272
hierarchy, 270, 273, 279
implicit, 272
Metaclass class
(class), 279
method
abstract, 87
byte code, 117
categorize, 36, 38
creation, 29, 110
decompile, 117
dictionary, 275
extension, 93
filing in, *see* file, filing in
filing out, *see* file, filing out
finding, 17, 117
lookup, 91, 274
overriding, 93, 115
pretty-print, 117
public, 80
returning self, 40
selector, 79
version, 113
method category, 15
method finder, 17, 106
method lookup, 94
Model
myDependents, 108
ModificationForbidden
(class), 209
Monticello, 25, 42, 141, 142, 145, 153, 290
Morph
(class), 32
addMorph:, 246
bounds, 248
center:, 246
color:, 245
composing, 246
defaultColor, 114
delete, 247
drawOn:, 247
extent, 245
handleKeystroke:, 252, 254
handlesMouseDown:, 251, 253
handlesMouseOver:, 253
initString, 95
mouseDown:, 251
mouseEnter:, 253
mouseLeave:, 253
mouseUp:, 251
openInWorld, 92, 244, 245, 248, 271
position, 245
removeMorph:, 247
repelsMorph:event:, 258
subclassing, 247
wantsDroppedMorph:event:, 257
Morphic, 8, 27, 243
animation, 255, 263
halo, 6, 32, 41, 243
isStepping, 255
startStepping, 255
step, 255
stepTime, 255
stopStepping, 255
morphic halo, *see* Morphic
MorphicEvent
(class), 254
hand, 259
MorphicTestingFramework, 293
MorphicUIManager
(class), 256
mouse events, 252
MouseEvent
(class), 251, 252
redButtonPressed, 252
yellowButtonPressed, 252
Mutex
(class), 290
MVC, 256
MVCUIManager
(class), 256
MyTestCase class
buildSuiteFromSelectors, 173

.Net, 159
new, *see* Behavior»new

NeXTstep, 107
nil (pseudo variable), 47, 51
Notification
    (class), 196, 199
notifier, 39
Null Object (pattern), 186
Number
    *, 188
    +, 188
    -, 188
    /, 188
    (class), 30, 182, 186–188
    asFloat, 189
    asInteger, 189
    asScaledDecimal:, 191
    ceiling, 189
    day, 189
    even, 189
    floor, 189
    fractionPart, 189
    hour, 189
    i, 189
    integerPart, 189
    isInfinite, 189
    log, 189
    negative, 189
    odd, 189
    positive, 189
    printOn:, 189
    printOn:base:, 189
    raiseTo:, 189
    rounded, 189
    roundTo:, 189
    roundUpTo:, 189
    sin, 189
    sqrt, 189
    squared, 189
    to:, 210
    to:by:, 210
    to:do:, 56
    week, 189

Object
    ->, 211
    =, 180, 223
    (class), 14, 15, 27, 86, 96, 177, 271
    ~=, 181
    ~=, 181
    as:, 214
    asMorph, 244
    assert:, 185
    asString, 217
    at:, 57
    at:put:, 57
    class, 181, 271
    copy, 184
    copyTwoLevel, 183
    currentEvent, 254
    currentHand, 254
    currentWorld, 254
    deepCopy, 183
    deprecated:, 185
    doesNotUnderstand:, 96, 185
    equality, 180
    error, 185
    error:, 185
    halt, 184
    haltIf:, 133
    haltOnce, 133
    hash, 181, 223
    identity, 180
    initialization, *see* initialization
    inspectOnce, 133
    isArray, 186
    isBlock, 186
    isBoolean, 186
    isCollection, 186
    isComplex, 186
    isKindOf:, 182, 271
    isMemberOf:, 181
    notNil, 186
    perform:, 262
    postCopy, 184
    printOn:, 178
    printString, 16, 178, 217
    respondsTo:, 182
    shallow copy, 182
    shallowCopy, 183
    shouldNotImplement, 186
    storeOn:, 179
    subclassResponsibility, 87, 88, 186
    veryDeepCopy, 184
    yourself, 208, 222
Object class

(class), 272
Object, halt, 133
Ohm/S, 294
on the fly variable definition, 34
OrderedCollection
(class), 204–207, 209, 228
add:, 209, 222
addAll:, 209
addFirst:, 205, 209
addLast:, 205, 209
at:, 205
at:put:, 205
do:, 228
remove:, 210
remove:ifAbsent:, 210
removeAt:, 118
reverseDo:, 217
override method, *see* package, extension, overriding
overriding, *see* method, overriding
overriding method, 143

package, 25, 42, 118, 142
cache, 146, 149
dirty, 43
extension, 143
overriding, 143
package browser, *see* Monticello
package cache, 43
package pane browser, 118
PackageInfo
printOn:, 178
parentheses, 61, 65, 68
PasteUpMorph
(class), 254
Pelrine, Joseph, 90, 159
period, *see* statement separator
Pluggable collections, 205
PluggableListMorph
(class), 123
Point
(class), 30
dist:, 79
printOn:, 179
pool dictionary, *see* variable, pool
PositionableStream
(class), 227
atEnd, 231
contents, 231
isEmpty, 231
peek, 230
peekFor:, 230
position, 230
position:, 230
reset, 227, 231
setToEnd, 231
skip:, 231
skipTo:, 231
PositionableStream class
on:, 230
pragma, 50
pre-debugger, 133
predebugger, 125
preference browser, 7, 185
primitive, 50, 57, 90
primitive., 50
printing (protocol), 15
private (protocol), 80
Process
(class), 290
process
browser, 134
interrupting, 106, 133
Process Browser, 290
process browser, 106
Profiler, 292
profiling tools, 137
Project
uiManager, 256
projects, 256
Promise
(class), 290
PropertySortFunction
(class), 224
protocol, 15, 35
creating, 109
ProtoObject
==, 211
(class), 86, 96, 177, 181
initialize, 187
isNil, 186
ProtoObject class
(class), 279
ProvideAnswerNotification

(class), 198, 199
pseudo-variable, *see* variable, pseudo
Python, 159

Quinto, 25

radix notation, 49
RawBitsArray
(class), 205
RBParser
(class), 288
read-only object, 209
ReadStream
(class), 228, 229
next, 230
next:, 230
upToEnd, 230
ReadWriteStream
(class), 228, 234
ReceiverMorph
(class), 258
Recover changes, 114
Recover method versions, 114
Rectangle
(class), 30, 248
containsPoint:, 249
RectangleMorph
(class), 116
defaultColor, 114
red button, 5, 7, 24, 123
refactoring, 29, 144
Refactoring Browser, 293
reflection, 79, 160, 178, 262
Regex, 216, 293
regular expressions, 194, 216, 293
removing (protocol), 203
resource, *see* test, resource
return, 51, 54, 90, 92
implicit, 54

saving code, *see* categories
SBECell
(class), 27
initialize, 30
mouseUp:, 38
SBECrossMorph
(class), 247
SBEDieMorph
(class), 260
SBEGame
(class), 32
cellsPerSide, 36
initialize, 32, 40
newCellAt:at:, 36, 41
toggleNeighboursOfCellAt:at:, 37
SBEWebServer
(class), 85
scaled decimal, 49, 191
ScaledDecimal
(class), 49, 191
ScaleMorph
(class), 116
checkExtent:, 111
defaultColor, 114
Self, 243
self
send, 94
self (pseudo variable), 30, 47, 48, 51, 54, 91
self-evaluating objects, 179
SequenceableCollection
(class), 204
first, 204
last, 204
readStream, 230
withIndexDo:, 217
withoutDuplicates, 213
SequenceableCollection class
streamContents:, 233, 234
Set
(class), 204–207, 212
add:, 212
intersection, *see* Collection, intersection:
membership, *see* Collection, includes:
union, *see* Collection, union:
Set class
newFrom:, 212
setter method, *see* accessor
shallow copy, *see* Object, shallowCopy
Sharp, Alex, xi
shortcut constructor methods, 189, 194
SimpleSwitchMorph
(class), 27

Singleton (pattern), 194
Singleton pattern, 84
slot, *see* instance variable
SmaCC, 294
SmallInteger
    +, 50, 57
    (class), 13, 78, 183, 187, 190, 205
    maxVal, 190
    minVal, 190
Smalltalk (global), 97, 98
SmalltalkImage
    (class), 97, 98
sort function, 224
SortedCollection
    (class), 204, 206, 207, 213
SortedCollection class
    sortBlock:, 214
SortFunction
    ., 225
    reversed, 225
    undefinedFirst, 225
    undefinedLast, 225
SortFunction, class, 224
sources, 4
space tally, 138
SqueakSource, 43, 150
Squeaksource, 290
Stack
    pop, 185
stack trace, 125
StandardFileStream
    fullName, 238
statement, 52
    separator, 51, 52, 73
Stream
    (class), 178, 227
    next, 228
    nextPut:, 228
    print:, 233
String
    ., 23, 215
    (class), 19, 20, 22, 49, 53, 178, 192, 205, 214, 217, 287
    anySatisfy:, 216
    asDate, 194
    asFileName, 194
    asLowercase, 194, 216
    asUppercase, 19, 216
    at:put:, 215
    capitalized, 194, 216
    concatenation, *see* String, ,
    copyReplaceAll:, 215
    expandMacros, 216
    expandMacrosWith:, 216
    format:, 216
    includes:, 216
    isEmpty, 216
    lineCount, 53
    match:, 193, 215
    pattern matching, 215
    replaceAll:with:, 215
    replaceFrom:to:with:, 215
    templating, 216
String class
    newFrom:, 214
    withAll:, 214
StringTest
    (class), 20, 132
    testShout, 21
SUnit, 20, 21, 106, 131, 159
    fixture, 163
    set up method, 162
super
    initialize, 94
    send, 94, 113, 287
super (pseudo variable), 47, 51, 91
superclass, 86, 91
Symbol
    (class), 49, 183, 192, 205, 211, 217, 220, 225
    ascending, 225
    descending, 225
    value:, 220
    value:value:, 221
symbol, 27
syntax, 47
system browser, 14, 16, 26, 81, 106
    browse button, 111
    button bar, 110
    class side, 81, 82, 84, 85, 101, 270, 274
    defining a class, 27, 110
    defining a method, 29, 110
    finding classes, *see* class, finding

finding methods, *see* method, finding
hierarchy button, 113, 116
implementors button, 111
inheritance button, 115
instance side, 81, 84
references, 116
senders button, 111, 113
source button, 117
variables button, 116
versions button, 113
system category, *see* category
SystemNavigation
(class), 118, 287
browseAllCallsOn:, 119
browseAllImplementorsOf:, 119
browseAllSelect:, 119
SystemOrganizer
(class), 119
classesIn:, 119

Tab (global), 101
tearDown, 163
template method, 178
Test Driven Development, 19, 159
Test Runner, 106, 164
TestCase
(class), 162, 168
assert:, 165, 185
assert:description:, 166, 171
deny:, 164
deny:description:, 166, 171
failureLog, 171
isLogging, 171
run, 171
run:, 171
runCase, 172
setUp, 162, 168, 173
should:description:, 171
should:raise:, 166
shouldnt:description:, 171
shouldnt:raise:, 166
tearDown, 168, 173
TestFailure
(class), 185
testing, 20, *see* SUnit
testing (protocol), 186, 203, 204
TestResource
(class), 168, 170, 174
setUp, 174
TestResource class
current, 174
isAvailable, 174
TestResult
(class), 168, 169, 171
runCase:, 171
TestResult class
error, 167
TestRunner, 21
(class), 292
TestRunner class
open, 293
TestSuite
(class), 167–169
run, 173
run:, 173
Text
(class), 101, 289
TextAttribute
(class), 289
TextMorph
(class), 289
thisContext (pseudo variable), 47, 51
three button mouse, 5
time profile browser, 137
Timespan
(class), 188
TimeStamp
(class), 120
Tools flap, 21, 24, 167
Trait
(class), 88
trait, 86, 88
Transcript (global), 53, 98, 106, 254
transcript, 11
TranscriptStream
(class), 98
TranslucentColor
(class), 100, 270, 271, 273, 276
True
(class), 51, 194
ifTrue:, 195
not, 195
true (pseudo variable), 47, 51, 196

UI automation, 293
UI testing, 293
UIManager
    (class), 256
    request:initialAnswer:, 256
unary message, *see* message, unary
UndefinedObject
    (class), 51, 125, 183

value, *see* BlockClosure
values, *see* Dictionary, values
variable
    class, *see* class, variable
    class instance, *see* class, instance variable
    declaration, 50, 55, 90
    global, 48, 97
    instance, *see* instance variable
    pool, 48, 97, 101
    pseudo, 48, 51, 92
    shared, 97
variable declaration, *see* temporary variable declaration
versions browser, 114
virtual machine, 3, 10, 50, 57, 90, 91, 96
VM, 4

weak collections, 206
WebClient, 293
    (class), 293
WebServer
    (class), 293
WideString
    (class), 192, 214
workspace, 11, 106
world menu, 5, 8
WriteStream
    (class), 228, 232
    cr, 233
    ensureASpace, 233
    nextPut:, 233
    nextPutAll:, 233
    space, 233
    tab, 233

xUnit, 159

yellow button, 5–7, 28, 35, 108, 111, 114, 116, 120, 135